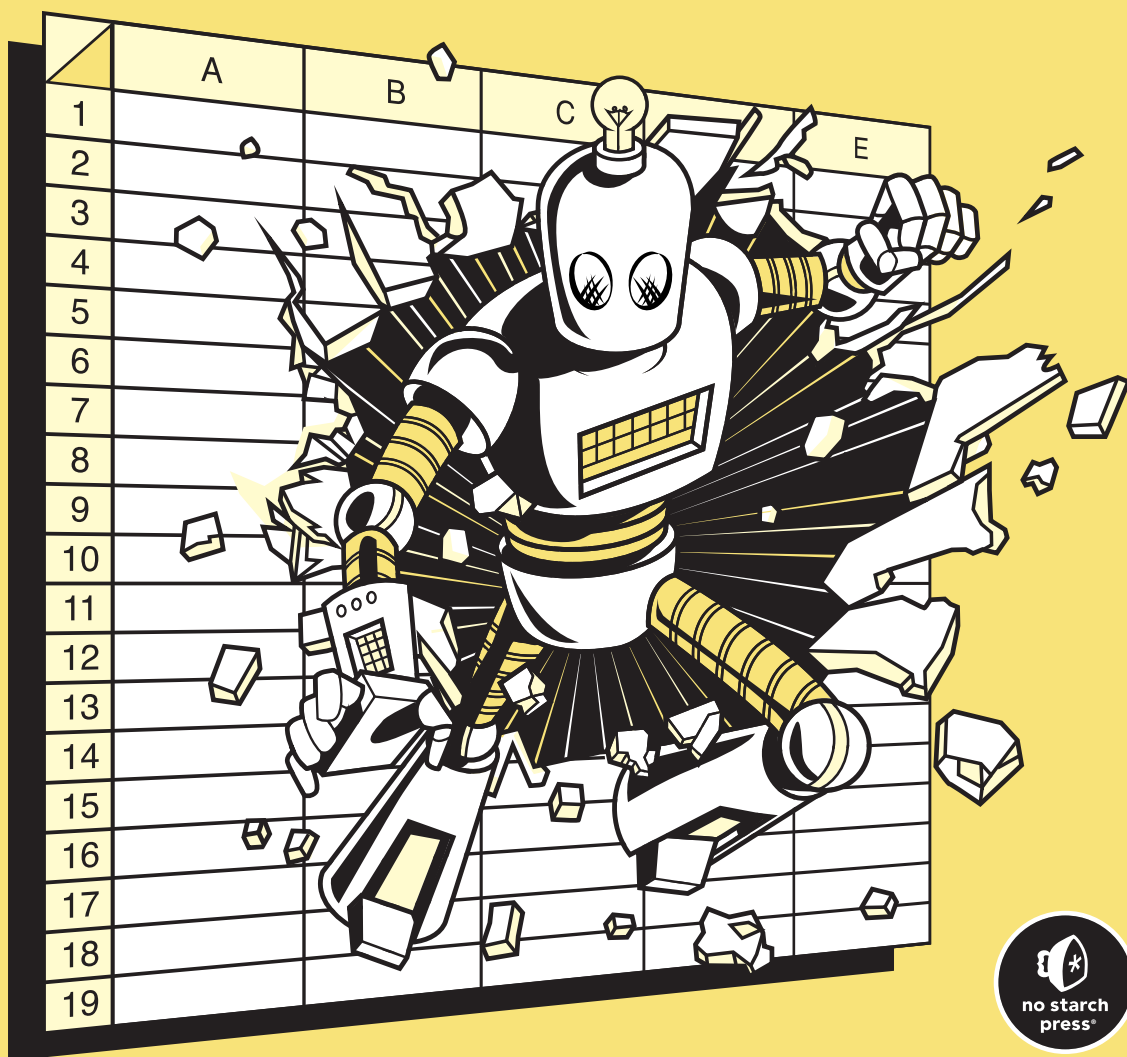


PYTHON FOR EXCEL USERS

KNOW EXCEL? YOU CAN LEARN PYTHON

TRACY STEPHENS



PYTHON FOR EXCEL USERS

PYTHON FOR EXCEL USERS

**Know Excel?
You Can Learn Python**

by Tracy Stephens



**no starch
press®**

San Francisco

PYTHON FOR EXCEL USERS. Copyright © 2025 by Tracy Stephens.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

First printing

29 28 27 26 25 1 2 3 4 5

ISBN-13: 978-1-7185-0398-4 (print)

ISBN-13: 978-1-7185-0399-1 (ebook)



Published by No Starch Press®, Inc.
245 8th Street, San Francisco, CA 94103
phone: +1.415.863.9900
www.nostarch.com; info@nostarch.com

Publisher: William Pollock
Managing Editor: Jill Franklin
Production Manager: Sabrina Plomitallo-González
Production Editor: Allison Felus
Developmental Editor: Nathan Heidelberger
Cover Illustrator: Joshua Kemble
Interior Design: Octopod Studios
Technical Reviewer: Chris Lam
Copyeditor: Sharon Wilkey
Proofreader: James Brook
Indexer: BIM Creatives, LLC

Library of Congress Control Number: 2025937129

For customer service inquiries, please contact info@nostarch.com. For information on distribution, bulk sales, corporate sales, or translations: sales@nostarch.com. For permission to translate this work: rights@nostarch.com. To report counterfeit copies or piracy: counterfeit@nostarch.com. The authorized representative in the EU for product safety and compliance is EU Compliance Partner, Pärnu mnt. 139b-14, 11317 Tallinn, Estonia, hello@eucompliancepartner.com, +3375690241.

No Starch Press and the No Starch Press iron logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this work, neither the author nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.

[E]

Start where you are. Use what you have. Do what you can.

—Arthur Ashe

About the Author

Tracy Stephens is a quantitative developer based in New York City. Her experience includes building systematic trading strategies at some of the world's top financial institutions. A long-time Python evangelist, she focuses on designing quantitative infrastructure that's flexible, explainable, and efficient—when she can successfully keep her one-eyed tuxedo cat off her keyboard.

About the Technical Reviewer

Chris Lam is a software engineer based in Las Vegas with over 20 years of engineering experience in various industries such as fintech, healthtech, and gaming. He specializes in distributed systems, serverless architecture, and streaming data. He holds a bachelor's degree in computer science from the University of California, Los Angeles.

BRIEF CONTENTS

Introduction	xix
--------------------	-----

PART I: GETTING STARTED WITH PYTHON

Chapter 1: Setting Up Your Coding Environment	3
Chapter 2: Coding Fundamentals Explained Through Excel	25
Chapter 3: Automating Tasks with Python Scripts	51
Chapter 4: Tracking Changes with Version Control	75

PART II: WORKING WITH DATA

Chapter 5: Data Analysis Made Interactive	97
Chapter 6: Analyzing and Transforming Data	109
Chapter 7: Working with Databases	133
Chapter 8: Retrieving Data from the Internet	161
Chapter 9: Creating Charts and Visuals	185
Chapter 10: Building Interactive Reports	211

PART III: WRITING BETTER CODE

Chapter 11: Organizing Your Code with Classes	235
Chapter 12: Finding and Fixing Errors	257
Chapter 13: Three Good Coding Habits	273
Afterword	285
Index	287

CONTENTS IN DETAIL

INTRODUCTION	xix
Who This Book Is For	xx
How Spreadsheets Can Hold You Back	xxi
Detecting Errors	xxii
Documenting Your Work	xxiii
Collaborating with Others	xxiv
Integrating Modern AI Tools	xxv
The Benefits of Using Python vs. Excel	xxv
Speed	xxvi
Modularity	xxviii
Automation	xxviii
Interoperability	xxx
Security	xxxi
Alternatives to Python	xxxii
VBA	xxxii
Python in Excel	xxxiii
What's in This Book	xxxiv

PART I GETTING STARTED WITH PYTHON

1	
SETTING UP YOUR CODING ENVIRONMENT	3
Why a Well-Configured Installation Matters	3
Using the Command Line	4
Navigating Directories and Subdirectories	5
Manipulating Files	9
Customizing and Storing System Settings	10
Working Within Corporate IT Policies	13
Installing Python	14
Running the Installer	14
Verifying the Installation	16
Running Some Code	17
Getting Started with Python Libraries	18

Tools to Simplify Writing Code	19
Important Features	20
Popular Python Editors	21
VS Code Installation	21
Conclusion	23

2 CODING FUNDAMENTALS EXPLAINED THROUGH EXCEL 25

What Is Code?	26
Objects and Variables	27
Rules of Python Syntax	28
Case Sensitivity	28
Indentation	28
Comments	29
Statements and Assignments	29
Testing Examples in VS Code	30
Basic Data Types	31
Boolean Variables	31
Integers and Floats	31
Strings and Characters	32
None	34
Operators	34
Arithmetic Operators	34
Comparison Operators	35
Logical Operators	35
Composite Data Types	36
Lists	36
Tuples	37
Dictionaries	38
Functions and Methods	39
Common String Methods	40
Common List Methods	40
Common Dict Methods	41
Conditionals	41
if Statements	42
else Clauses	43
elif Clauses	43
Nested Conditionals	44
Representing Dates and Times	45
Internal Representation of Dates in Excel	45
The datetime Library	46
Conclusion	50

3

AUTOMATING TASKS WITH PYTHON SCRIPTS

51

What Are Scripts?	51
When to Use a Script	52
Parts of a Script	53
Loops	54
for Loops	54
List Comprehensions	56
Loop Controls	57
while Loops	59
User-Defined Functions	59
Arguments	60
Scope	62
Returns	62
Modifier Functions	64
Functionalizing Logic	65
Building and Running a Complete Script	66
Running a Script on the Command Line	69
Importing a Script as a Module	70
Using a Main Block	71
Passing System Arguments	72
Conclusion	73

4

TRACKING CHANGES WITH VERSION CONTROL

75

Why Use Version Control?	76
What Is Git?	77
Why Git Works Best with Code	77
Git vs. GitHub	78
Working Within Git Repositories	78
Centralizing Your Work in a Single Folder	78
Navigating Local and Remote Repositories	80
Maintaining a .gitignore File	81
Setting Up a Git Repository	82
Installing Git	82
Configuring Git	82
Creating a New Git Repository	83
Tracking Changes to Files	84
Git's Staging Area	84
Commits	85
Syncing Local and Remote Repositories	86
fetch	86
pull	86
push	86

Managing Histories	87
The Main Branch	87
Feature Branches	87
Merges	88
Conflicts	88
Best Practices for Using Git	90
Troubleshooting with Status Messages	91
Integrating Git with VS Code	92
How Git Works	92
Directed Acyclic Graphs	92
Hashing	93
Conclusion	93

PART II WORKING WITH DATA

5 DATA ANALYSIS MADE INTERACTIVE 97

What Are Jupyter Notebooks?	98
Code Cells	98
Markdown Cells	99
Raw Text Cells	100
The JupyterLab Code Editor	101
Installing JupyterLab	101
Launching JupyterLab	101
Creating a Jupyter Notebook	102
Closing a Jupyter Notebook	103
Building a Jupyter Notebook	103
Scripts and Notebooks Working Together	105
Version Control with Jupyter Notebooks	107
Conclusion	107

6 ANALYZING AND TRANSFORMING DATA 109

Efficient Data Analysis with pandas	110
Series and DataFrames	110
pandas and Jupyter Notebooks	111
Getting Started with pandas	111
Creating a DataFrame from Scratch	112
Reading Data from a File into a DataFrame	114
Working with pandas Data Types	115
Moving Between Excel and pandas	116
Using the Clipboard	117
Using openpyxl	118

Manipulating and Transforming Data	118
Accessing Values	118
Updating Values	120
Sorting Datasets	121
Manipulating Data	121
Handling Missing Data	124
Combining DataFrames	125
Aggregating Data	128
Easily Reshaping Data with pandas	129
Grouping	129
Pivoting	130
Conclusion	131

7

WORKING WITH DATABASES 133

Why Use a Database?	134
How Databases Work	135
Incorporating Databases into Your Workflow	135
Where Databases Live	135
How to Access Corporate Databases	136
The Costs of a Database Connection	136
Relational Databases and SQL Workflows	137
Database Management Systems	137
Sandbox Environments	138
Creating a Practice Database	138
Installing PostgreSQL	138
Managing Users and Databases	140
Using PostgreSQL in VS Code	141
Creating and Editing Tables	142
Creating a Table	143
Altering a Table's Structure	144
Dropping a Table	145
Adding Rows	145
Updating Records	145
Deleting Data	146
Retrieving Database Data	146
Selections	146
Filters	148
Joins	150
Column Transformations	154
Aggregations	154
Groupings	155
Structuring Complex Queries	156
Automating Database Tasks with Python and SQL	157
Installing Psycopg 2	158
Connecting to PostgreSQL from Python	158

Running a Query from Python	159
Closing the Database Connection.....	160
Conclusion	160

8 RETRIEVING DATA FROM THE INTERNET 161

What Is an API?	162
REST APIs	162
Other Types of APIs	163
Making Requests and Handling Responses	163
Internet Communication Protocols	163
Request Types	165
Response Types	166
Simplifying Requests with the Requests Library	169
Making an API Request	169
Adding Arguments	170
Building in Error Handling	171
try...except Blocks	172
Common Errors	173
Breaking Large Requests into Smaller Parts	174
Handling Authentication.....	176
API Keys	177
OAuth	179
Streamlining Your Requests with Reusable Functions	180
Conclusion	183

9 CREATING CHARTS AND VISUALS 185

Charting with Excel vs. Python	186
Integration into a Broader Workflow	186
Reusability	186
Version Control	187
Customization	187
Standardization.....	187
Charting Libraries in Python	188
Using Plotly in Jupyter Notebooks	189
Installing and Importing Plotly	189
Creating Simple Charts	189
An Example Dataset.....	190
A Basic Line Chart	191
A Multi-Trace Line Chart	193
A Basic Bar Chart.....	194
A Multi-Trace Bar Chart.....	196
Saving Charts as Images	198
Charting Multidimensional Data.....	198

Elevating Charts with Custom Styling	200
Layout Properties	201
Themes	205
Using Plotly's Interactivity Features	209
Conclusion	210

10 BUILDING INTERACTIVE REPORTS 211

Why Use Interactive Reports	211
Creating Interactive Reports with Dash	212
When Interactive Reports Make Sense	212
How to Choose the Right Framework	213
Dash Basics	214
Installing Dash	215
Setting Up a Dashboard	215
Deploying a Dashboard Locally	216
Web Development Concepts and Dash	216
HTML Basics	217
HTML in Dash	219
CSS Basics	221
CSS in Dash	222
Integrating Plotly and Dash	223
Functionalizing Plotly Charts	223
Embedding a Chart in a Dashboard	225
Enabling User Selections	226
Updating Charts with Callbacks	226
Putting It All Together	228
Sharing Your Reports with Others	230
Conclusion	231

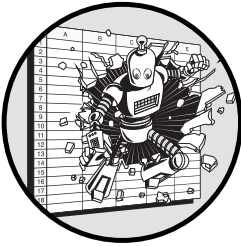
PART III WRITING BETTER CODE

11 ORGANIZING YOUR CODE WITH CLASSES 235

From Rows and Columns to Classes	236
Examples of Classes in Common Python Libraries	237
Custom Classes	237
Programming with Objects to Stay Organized	239
Reusability	243
Extendability	244
Encapsulation	245
Inheritance	249
Conclusion	255

12	
FINDING AND FIXING ERRORS	257
The Art of Debugging	257
Debugging Inside the VS Code Editor	258
Breakpoints	259
The Debug Panel	260
The Debug Console	262
Debugging Code with Errors	263
The Step Out Command	263
The Step Into Command	264
Pausing Code at the Right Moment	265
Debugging with Different Inputs	266
The Process of Testing Code	267
Assertions	268
The unittest Module	269
Identifying Test Cases	271
Conclusion	272
13	
THREE GOOD CODING HABITS	273
Simpler Code Is Better Code	274
Too Many Conditions	274
Mixed Scopes	276
Inconsistency	277
Confusing Variable Names	277
Repetition	278
Making Code Manageable	279
The Right Amount of Modularization	280
Too Much Modularization	281
First Make It Run, Then Make It Better	282
Conclusion	283
AFTERWORD	285
INDEX	287

INTRODUCTION



When I started my first job in finance, Microsoft Excel was revered as a productivity tool, as it is in many other industries.

I worked with many incredibly smart, highly skilled people who had spent decades-long careers performing high-stakes and highly complex work in Excel. Having spent hundreds or perhaps even thousands of hours wrangling data in Excel, these seasoned professionals knew virtually all there was to know about the software. They were familiar with all of Excel's quirks, and they knew how to stretch Excel to its limits of size and complexity.

At some point, however, I started to notice that many of these brilliant, highly technical people were as much confined by Excel as they were empowered by it. Much of their time, energy, and skill was spent finding ways to circumvent Excel's limitations. They would need to add layers of extra effort to keep large files from crashing, work around the static nature of cells, and improve upon features that were built with simpler tasks in mind.

Wanting to avoid a career of unnecessarily long workdays, I decided to focus on using Python to solve some of the same kinds of problems my colleagues were tackling with Excel. My hope was that this would eventually free me from many of Excel's limitations. The investment paid off: Over time, the tasks I had built in Python proved to be far more reliable and efficient than ones built in Excel. Processes that would take hours of manual effort could be reduced to mere minutes, and common errors became much rarer. After some time, I began hosting seminars with my spreadsheet-based colleagues demonstrating how they could transition their work from Excel to Python. Eventually, seeing the benefits of Python in action, even the most die-hard Excel evangelists subscribed to this new way of working.

This book distills the lessons I've learned and benefits I've reaped from applying Python and other related coding tools to traditionally Excel-based workflows. The chapters that follow illustrate not just Python's tremendous effect on productivity but also how accessible it can be, especially to those who already know and love Excel.

Who This Book Is For

If your default move when you have a problem to solve is to open Excel, but you've at times neared its limits (think crashing files, cells containing long and complex formulas, hidden errors that you discover only long after the fact), then this book is for you. Whether you've never tried coding before, started and gotten stuck a few times, or learned some of the basics but never quite figured out how to apply them to your real-world work, this book will help you grasp not just the *how* of Python but also the *why* behind learning Python.

We'll use Excel as a starting point for learning to code in Python. As we explore each coding concept, we'll pinpoint what feels familiar from your Excel experience and what's new, helping you begin to picture how to solve problems without relying on spreadsheets. As you'll hopefully see, learning Python isn't as out of reach as you might think, especially since, knowing Excel already, you're not exactly starting from scratch.

If you've already taken the time to become comfortable building processes in Excel, you've probably become intuitively familiar with the intricate details and nuances of those processes. Shifting those processes to Python will therefore come naturally. As an example, if you can create pivot tables to perform data aggregation or grouping by multiple dimensions, you can learn to perform the exact same function more safely and efficiently, and on much larger and more complex datasets, with a few lines of Python code. If you can build a `VLOOKUP` formula to retrieve information from one sheet into another, you know enough about the basics of relationships between tables to start using a database. Working in spreadsheets means you're already doing much of what coders do, but you're leaving your knowledge and valuable work locked inside self-contained workbooks, where it will never be built upon, shared, or scaled.

The goal of this book isn't to get you to drop everything and become a professional software developer. Rather, the aim is to teach you enough Python so you can apply it to the work you're already doing. With the knowledge base you'll establish in this book, you'll also be ready to use other resources to expand your skills as your coding needs evolve.

How Spreadsheets Can Hold You Back

Excel's grid is essentially "paint-by-numbers" data storage and analysis. Each cell acts as a predefined slot for data, formulas, and functions. This framework forces you to apply structure to your work, making data manipulation accessible and allowing you to reach a finished result without needing to think much about the structure ahead of time.

However, this convenience comes at a cost. The structure Excel imposes substantially limits what you can do and how efficiently you can do it. When you don't need to think about the underlying design of your solution and can let the grid do the work for you, there's a good chance that, at best, your implementation won't be as optimal as it could be and that, at worst, serious errors could creep into your work unnoticed.

It's not that there's no need for Excel in the workplace. It's that people rely upon Excel for too many things. Spreadsheets are unstable, especially when they're punching above their weight. We have so many examples of crucial business processes failing because they remained in Excel long after their scale or sensitivity far surpassed Excel's practical limitations.

In 2012, the investment bank JPMorgan Chase suffered a trading loss of \$6 billion on account of Excel. The bank used a combination of multiple spreadsheets to do the complex financial modeling required to understand the risk of loss in its synthetic credit portfolio. As noted in an internal report, the spreadsheet "required time-consuming manual inputs to entries and formulas, which increased the potential for errors." In other words, relying on spreadsheets for such a critical, intricate task led to a high risk of blunders, and unsurprisingly, a blunder occurred.

In another example, two Harvard University economists, Carmen Reinhart and Kenneth Rogoff, published a paper in 2010 claiming that countries with public debt over 90 percent of gross domestic product (GDP) tend to have relatively low rates of economic growth. In the years following, as the world was emerging from the global financial crisis, the paper became a touchstone in the debate around austerity measures and even became the basis for the US Republican Party's budget proposal in 2013. In the midst of all this attention, another study brought to light that Reinhart and Rogoff's original study excluded three countries that experienced both outsized debt and outsized growth: Australia, New Zealand, and Canada. Later it was revealed that the spreadsheet Reinhart and Rogoff used had mistakenly left off the first five cells of a column in a formula. Factoring in the excluded data effectively invalidated their argument.

More recently, during the height of the COVID-19 pandemic in October 2020, England's public health agency underreported its case counts because

of Excel. Evidently, a CSV file containing COVID test results was opened in Excel prior to being loaded into a database. The file exceeded Excel's strict row limit, so several thousand tests were left off and never made it into the agency's updates on case counts and tests performed. More than 15,000 positive cases were left out of the official COVID figures, which meant many potentially infectious people weren't contacted and told to stay home, causing who knows how many additional cases of COVID-19.

As these stories illustrate, Excel is so accessible an analytical tool that it finds its way into processes requiring more stability than it is prepared to offer. If JPMorgan Chase, Reinhart and Rogoff, and Public Health England hadn't put so much blind faith in their spreadsheets, costly mistakes could have been avoided. And these are far from isolated incidents. Let's look more generally at some of the important tasks that are surprisingly challenging in Excel. Then we'll see how Python makes these and other tasks much easier.

Detecting Errors

Spreadsheets are error prone. We could write off the stories just discussed as cases of human error and say the people involved should have been looking more closely at their spreadsheets, but the truth is that end users often aren't entirely at fault. Excel certainly doesn't make it easy to find and fix errors. While it's good practice to double- and triple-check your work, you'll be much more likely to do so—and do so correctly—if those checks are easy to perform. This ability for a user to ensure the accuracy and reliability of a process is known as *testability*.

In Excel, the relationships among the data in individual cells or columns aren't explicitly stated or formalized, which gets in the way of testability. If C3 depends on B2, and B2 depends on A2, determining the cause of an error in C3 means tracing this long chain of dependencies, any one of which could be wrong or, worse, accidentally right. Errors can easily remain undetected until they cause significant issues, and by then, as it was for JPMorgan, it could be too late. The absence of clear, automatic linkages between formulas creates a hospitable environment for inaccuracies to fester, like a disease permeating through a vulnerable population.

One argument for Excel is that someone with no technical knowledge can audit a spreadsheet. Someone without a deep understanding of how spreadsheets work could look at one, dig into the formulas, and understand exactly what's happening in every cell. For smaller tasks, that's definitely true, but when you consider a sheet with hundreds of cells, each with its own formula and countless dependencies, it's questionable that anyone could audit it effectively, no matter how much background knowledge they have. The larger the spreadsheet, the longer the formulas, and the more connections between them, the less testable it is. Scale increases complexity, and complexity obscures errors.

In Python, code can be organized into isolated components, which simplifies testing. You can test each piece on its own to ensure that it works correctly. Many frameworks leverage this modularity to facilitate automated

testing and reporting, including Python's `unittest` module, which we'll cover in Chapter 12. With these frameworks, you can not only determine the accuracy of your code but also do it in a systematic and automated way, ensuring that you've built each component correctly and will be able to maintain its accuracy going forward.

Excel's lack of explicit error handling is another factor that influences testability. Errors like `#REF!` and `DIV/0!` are easy enough to see if you don't have too many cells, but other errors can be difficult to find. Excel makes lots of assumptions about what's intended in formulas, some of which may differ from what you as the user are looking to do. For example, different formulas handle blank cells differently. Some treat them as zero, like `IF`, `SUM`, and `PRODUCT`; others ignore them, like `MEDIAN` and `STDEV`; and some will return errors if you use one as an input, like `VLOOKUP`. If instead, for example, you meant for your `MEDIAN` function to count blanks as zero, that could cause an error that might be difficult to notice. Without built-in tools to systematically track down and manage errors, identifying the root cause of an issue often involves a painful venture deep into the nitty-gritty.

Documenting Your Work

Poor documentation is ubiquitous in organizations. If there's a task to be done, chances are somebody is out there complaining about how poorly it's documented. This complaint is usually coupled with the sentiment that documentation could have easily been created at some point and that the reason it wasn't done was sheer negligence or maybe just spite.

In fact, good documentation is hard to find not because people are negligent, self-serving, or malicious, but because, for many tasks, good documentation is hard to write. Explaining a process often requires a more robust understanding of its concepts than doing the process. Tasks that are intuitive to you may not be intuitive to others, and bridging that gap can be difficult without knowing where someone else's knowledge begins and ends.

For example, when you brew your morning coffee, you may have an internalized understanding of the right water temperature, brew time, and number of scoops of coffee grounds needed for the perfect cup. If you needed to explain this process to someone else, however, you'd need to go beyond just knowing how *you* perform it; you'd also need to consider how *they* would approach it. To do that, you must first understand their starting point. For instance, they might have different assumptions about scoop size or the ideal water-to-coffee ratio. Translating your actions into instructions therefore requires you to know more about the coffee-making process than is necessary to simply be able to perform it yourself.

That said, some processes are inherently harder to document than others. For example, while making a simple cup of coffee ultimately follows a linear set of steps, making espresso is more of a nonlinear process, meaning it involves details and adjustments that have complex relationships with one another. Factors like grind size, coffee amount, and tamping pressure all impact how a cup of espresso turns out, and the optimal settings for each depend on multiple factors, both internal and external to the process. Perfecting the

process demands experimentation and intuition rather than just following a linear set of instructions.

Code, by nature, isn't like making espresso. It's a linear medium, at least in its textual form. It follows a logical sequence of commands that are executed one after the other and can be read step-by-step. Every decision and branch is explicitly defined, and outcomes are predictable because they're based on a predetermined flow. This makes code easy to document; in fact, when code is clearly written, it tends to document itself.

By contrast, Excel's grid encourages nonlinearity. In Excel, you can jump from one cell to another, adjust the values in individual cells on the fly, and repeat (or not repeat) formulas as much as you want. This flexibility offers room for exploration and experimentation, but at the cost of linearity and therefore ease of documentation. Even with a clear understanding and great communication, the nonlinear nature of Excel will make the documentation process difficult.

Collaborating with Others

Working with others on the same Excel file is always less simple than it seems it should be. With Excel, collaboration is mostly handled in one of two ways. First, it could be done without the intervention of any technological intermediary to reconcile changes: Multiple users have multiple copies of the same file, they edit them independently, and then they manually reconcile the versions. Second, it could be done with too much intervention: Different users make changes to the same file that exists on the internet, and those changes are synced automatically, without any user input. Microsoft calls this *co-authoring*.

With the first method, it's easy to see how you might have issues remembering what you changed and agreeing on how to combine multiple people's changes into one file. And while co-authoring might appear seamless, intuitive, and efficient, it's less than ideal. For one, tracking individual changes or understanding the timeline of edits is challenging. This can lead to confusion about when and why certain adjustments were made, especially when files have many authors or have a long, convoluted history. Excel's nonlinearity compounds this issue, as simultaneous changes can occur in faraway parts of a spreadsheet that may or may not depend on one another. On top of all this, Excel doesn't have a convenient place to make comments with detailed annotations or rationales for edits, which means you'll need to guess at the reasoning for a change every time you're unsure.

By contrast, Python facilitates collaboration through the use of a common coding tool called *version control*. As we'll explore in Chapter 4, this is a method of tracking and managing the entire history of a file, beginning with its creation and following its evolution line by line, change by change. With version control, multiple people working on the same file can review one another's contributions, identify where and how changes are made, and effectively coordinate their divergent efforts without fear of losing previous work. This makes combining multiple people's work much safer and simpler over the long term, even as the files themselves grow increasingly complex.

Integrating Modern AI Tools

In the past few years, artificial intelligence (AI) tools have transformed the way we work. These tools can easily do most of the tedious, repetitive tasks, and as we've recently found, large language models (LLMs) can even do some of the complicated tasks too. Unlike humans, LLMs have read virtually everything available on the internet and have become fairly good at pulling out the right information in a useful format for a wide variety of use cases.

While tools exist to allow AI to assist in spreadsheet work, they aren't nearly as useful as they've become for code, and the progress to improve them has been slower. For comparison, consider a tool like GitHub Copilot, an AI pair programmer that uses machine learning models trained on vast code repositories. With Copilot and Python, you can create a basic framework of what you want to build, and AI can fill in the rest. The tool integrates seamlessly into your code editor, so there's no need to write out an explanation of exactly what you need to do for the AI to understand the problem. Microsoft has launched a Copilot for Excel, but it still relies heavily on your explicitly telling the model what you need it to do and doesn't come close to being as deeply entrenched in a spreadsheet-based workflow as AI programming tools have.

The gap in the reliance on AI between spreadsheet work and code isn't because it's not technologically possible to use AI to help you build spreadsheets. It's because AI works much better with clearer, more linear structure, allowing it to understand the entire context and make accurate predictions. Code follows a strict set of predefined syntax rules and programming constructs. Spreadsheets, on the other hand, are only somewhat structured. While they operate with a grid layout and use formula syntax, the way data is entered and manipulated can vary widely. With a spreadsheet, you can input data in any cell, create formulas that reference seemingly unrelated cells, and use the sheet in ways that might not be immediately obvious without context. Just as nonlinearity makes writing documentation more difficult, this variability makes it more challenging for AI to predict or automate actions without a deep understanding of what's going on. So regardless of how much Copilot for Excel improves, it will always lag behind a similar tool applied to Python code.

Not being able to make the best use of modern tools inhibits productivity. The recent developments in AI have proven to be a huge step forward, drastically reducing the time it takes to complete tasks. These models can do things most of us thought would never be possible for computers as recently as just a few years ago. When you use Python instead of Excel, you can determine the degree to which you benefit from this advancement.

The Benefits of Using Python vs. Excel

Just about anything you can do in Excel, including storing, organizing, and manipulating data, you can also do in Python. The opposite isn't true; Python does a lot that Excel can't. More important, Python offers greater capacity, efficiency, and control, which allows you to handle complexity

with more ease than you ever could in Excel. Python also works well with many other tools, including many you may already be using, like Microsoft Outlook, Gmail, and Slack. You can connect your Python code seamlessly to these other applications, which opens up a range of possibilities for automating tasks.

Not only is Python useful in lots of areas, but the range of things it can do is also growing all the time thanks to contributions from Python developers all over the world. Learning Python makes you a part of a vibrant community of coders who can provide you with support, new ideas, and open source code that you can build upon. Python has grown so much in recent years that it's become the default general-purpose programming language and a go-to tool for data analysis and automation. And hopefully, soon it will be a go-to tool for you as well.

Unlike Excel, Python doesn't give you a predefined visual grid to work from. It's more like a blank canvas. When you're tied to an unnecessary visual interface, you're limited in where you can go and what you can do. Without those restrictions, you're free to do more and build upon work that's already been done. You can connect to external applications, borrow from your past projects, or leverage libraries and frameworks built by your teammates or members of the broader Python community. This means that the journey from scratch to finished work will always set you up better for next time. In Python, time is spent building reusable computational assets rather than doing computations.

Let's take a closer look at some of the tangible benefits you'll reap when you shift your workflow from Excel to Python.

Speed

On the most basic level, Python can process larger datasets than Excel can, and faster. Python holds data in structures that can both allocate memory efficiently and take better advantage of more of your computer's memory.

To illustrate this, I ran an experiment. I opened a blank Excel file and added the `=RAND()` function to the first million rows in just the first column. Then I turned off Excel's automatic calculation option so that the formulas would update only when I told them to. To test how long it takes Excel to calculate these formulas, I wrote this simple Python program by using a library called `xlwings` to connect to the Excel application, open the workbook, and force a full-sheet recalculation, timing Excel in the process:

```
import xlwings as xw
import time

app = xw.App(visible=True)
wb = app.books.open('calc_time.xlsx')

excel_start_time = time.time()
wb.app.calculate()
excel_end_time = time.time()
```



```
wb.close()
app.quit()

print("Excel Calculation Time:")
print(excel_end_time - excel_start_time, " seconds")
```

This script opens an instance of the Excel application, opens the workbook, recalculates the sheet, and prints the time it took for the calculation to happen. (Don't worry if you can't follow the precise details of the code; you'll learn to write scripts like this in Chapter 2.) On my MacBook, the recalculation took 0.944 seconds. Not bad for 1 million formulas!

Next, I wrote another Python program that simply generates 1 million random numbers, independent of Excel:

```
import numpy as np
import time

python_start_time = time.time()
random_numbers = np.random.rand(1000000)
python_end_time = time.time()
print("Python Calculation Time:")
print(python_end_time - python_start_time, " seconds")
```

Using just Python, the calculation took only 0.016 seconds on my MacBook, 98 percent faster than the Excel calculation. And note that I was measuring only the time it took Excel to come up with a list of 1 million random numbers. This doesn't include the time it took to open Excel, create the sample file, and save it.

Of course, both random-number generators took less than 1 second to run, so maybe the difference sounds trivial, but imagine that you were working with not 1 million but 1 billion data points. Or imagine that the computations were much more involved than generating random numbers, such as performing complex filtering or querying, or running a machine learning algorithm. Or what about computations that depend on one another, like a time series-based forecasting model? The difference in computational time becomes infinitely more impactful.

No matter how many cells are involved, the Excel calculation will nearly always take longer than its Python equivalent. Because of the visual nature of the Excel application, calculating a formula involves the additional overhead of managing the worksheet's state and interface updates, even if they aren't visible. By contrast, the Python version is purely computational. No extra effort is allocated to handling the numbers in a cell-based environment so that you can interact with them individually. You aren't dealing with a resource-guzzling interactive dashboard—just your numbers, unadulterated by unnecessary complexity.

Modularity

Python allows for greater *modularity* than Excel, making it straightforward to divide work into smaller, more manageable units. Modularity is difficult to achieve in Excel since functions and computations are tied to the sheets they live in.

Consider a process that Alice, a human resources analyst, might do in Excel, like creating a chart based on company employee data including title, tenure, and salary. You can probably imagine the Excel file that Alice might build to accomplish this task. She might have a sheet called Employee Data where she pastes the data from *employee_data.csv* each time she runs the process. Then she might have a sheet, or possibly a pivot table, that analyzes and extracts metrics on the data in Employee Data. And she might have another sheet called Charts for creating visualizations of the metrics and analysis.

All the data, charts, and computations here are inherently tied to the Excel file they live in. In some ways, this is convenient. Alice can scroll through all the data, click into any cell, investigate the individual formulas and references, and interact directly with the charts.

The problem is it's difficult to separate specific components of Alice's workflow so she can reuse them elsewhere. If she ever needs to repeat the same computations for a different task, her only option will be to copy and paste them, which is time-consuming and error prone. She also can't scale her process. Since Excel's performance degrades quickly as datasets grow larger, Alice may have difficulty adapting her process when her company hires more employees, adds new data to the dataset, or requires new kinds of analysis.

How would a Pythonized version of Alice's process be better? Using Python, Alice can divide her workflow into separate functions that handle specific tasks. As we'll discuss in Chapter 3, Python *functions* provide a way to encapsulate logic into a contained piece of code so you can reuse it later. In this case, Alice might define functions such as `load_employee_data()`, `calculate_department_summary()`, `create_chart()`, and `save_chart()`, each of which performs its own individual piece of the process and can be reused in other contexts if necessary.

This modular approach facilitates easy updates and the recycling of code across projects without the need to copy and paste. Something like calculating a department summary goes from a task that needs to be completed to a useful asset (a Python function) that Alice can draw on when needed. If she improves her functions (for example, by changing `load_employee_data()` to pull from a database instead of a CSV file), those changes will take effect in every code file, or *script*, where the functions are used. With tasks divided into separate, independent processes, maintaining, testing, and improving each of these processes is much less complicated. That facilitates scalability.

Automation

One of the key business advantages of Python is that it can automate some of the spreadsheet work you used to do manually in Excel. This may sound

scary: When many people hear the word *automation*, they immediately envision the job-eliminating kind. They think of all the switchboard operators, typists, milkmen, cashiers, tollbooth operators, bank tellers, and other careers that have largely fallen victim to technological obsolescence. But in fact, most real-world automation is job enhancing, not job eliminating. It's the subtle upgrades that refine your tasks incrementally. Think of a trick that reduces 10 clicks to 7—that's automation.

To evaluate whether automation is appropriate for a given task, you might make two estimations:

Manual execution time Estimate the amount of time it takes you to do the task without automation, then multiply that by the number of times you expect to perform this task over the relevant period of time (perhaps a month or a year). This gives you the cumulative time you'd spend doing the task manually.

Automation time Estimate the amount of time needed to set up the automation. This should include not only the time to create the automated process but also any necessary testing and troubleshooting to ensure that it works correctly.

If the task's automation time is less than the task's manual execution time, automation is appropriate. Conversely, if automation time is greater than manual execution time, automating the task might not be worthwhile.

For a lot of spreadsheet work, the manual execution time is grossly underestimated. Operations like sorting, filtering, and formatting data can be nonintuitive and error prone. Complex formulas can take time to understand, and common errors like cutting instead of copying or referencing the wrong cells in formulas can take time to identify and fix. All these little things can turn a seemingly quick task into a time-consuming one.

On top of this, because the process to be automated was built inside the Excel application, and without an explicit structure around the data and computations, the automation time is higher than it would be otherwise. Because Excel is centered around a user who visually interacts with the application, automation is unnecessarily cumbersome, even if you know your way around Visual Basic for Applications (VBA), Microsoft's own programming language for automation. On the flip side, Python is designed with automation in mind: Scripts are intended to run without a human to babysit them, so you become free to focus on less mundane things.

Even better, because connecting Python to Excel is easy through libraries like *xlwings*, you have the option to incorporate Python-based automation gradually rather than all at once. You can start small and scale your automation efforts as you become more comfortable. Returning to the example of Alice from HR, she could start with automating just the `load_employee_data()` function and leave the rest of the process for manual work in Excel. That would set the groundwork for moving toward another, more robust method of sourcing employee data in the future, like pulling it from a database.

The beauty of this approach is its immediacy. Full-blown automation can sometimes be a long-term endeavor, while marginal gains can be implemented

and felt almost immediately. A few seconds saved here and there might seem negligible in isolation, but over a year they can equate to days of reclaimed time. Moreover, these small automation steps can serve as building blocks. As you get more comfortable with these incremental improvements, they can pave the way for more substantial automation projects in the future. Your journey starts with a single step—or in this case, a single script.

Interoperability

In Excel, you're limited in what kinds of data you can use as inputs and how you can automate processes with other applications. For example, you may want to create an automation that streamlines a process involving another application, like Gmail, Slack, Salesforce, or a Bloomberg Terminal, but there aren't straightforward ways to work with these tools from within Excel. It's not that connections with these external applications would be technologically impossible; it's just that Excel is designed and maintained by Microsoft, and you as the user aren't allowed to add the desired functionality yourself. As a result, you'll likely be stuck manually transferring data between the applications.

By contrast, many applications widely used in business already have Python libraries built around them that you can harness in your Python scripts. These libraries simplify integration and automation with these applications. Here's a list of popular Python libraries for working with commonly used business applications:

google-api-python-client Accesses Google Drive and other Google services like Sheets, Docs, and Calendar

O365 Accesses Microsoft Office 365 services like Outlook mail, calendar, contacts, and more through the Microsoft Graph application programming interface (API)

Boto3 Integrates with Amazon Web Services (AWS) features like Amazon Simple Storage Service (S3) and Elastic Compute Cloud (EC2)

openpyxl Reads from, writes to, and manipulates Excel files

simple-salesforce Accesses Salesforce data and functionality

slackclient Interacts with Slack workspaces

openai Integrates with OpenAI's API for accessing AI models like GPT and DALL-E

zoomus Manages Zoom sessions and integrates Zoom functionalities

dropbox-sdk-python Allows Python applications to communicate with Dropbox user accounts

asana Automates and integrates with Asana for project management tasks

py-trello Automates tasks on Trello boards

hubspot-api-client Interacts with HubSpot’s marketing, sales, and service APIs

Zenpy Manages customer service operations through the Zendesk API

xbbg Accesses Bloomberg financial market data through the Bloomberg API

Because Python is open source, these libraries come from a variety of developers. Some were created on behalf of the applications themselves, while others were created because someone needed the library’s functionality for their own purposes and then decided to package the code and make it freely available to the entire Python community.

On top of all this, Python gives you the ability to seamlessly connect to all sorts of databases (discussed in Chapter 7) and APIs (discussed in Chapter 8), allowing you to integrate a wide range of data sources into your work. These types of solutions make Python scripts *interoperable*, or able to communicate with external technologies, services, and applications. Smooth interoperability is an essential ingredient for scalability and automation.

Security

Once installed, Python lives entirely on your own computer; it doesn’t inherently require a connection to the internet. This means you can customize your installation based on your specific security constraints and exert a greater level of control over your system’s operational environment and what happens to your data.

By contrast, Microsoft has increasingly relied on the cloud to add features to Office. Gone are the days when you would buy a disc containing the Excel software to install on your local machine. Instead, to use many of Excel’s new features, you need to create a Microsoft account and allow Microsoft to store your data online. For example, in 2023 Microsoft introduced Python in Excel, which allows you to execute Python functions within a spreadsheet. However, at least in its initial form, this capability involved interacting with a Python instance that ran in the cloud. This would give most corporate IT professionals pause, as cloud data may be vulnerable to cyberattacks, data breaches, or unauthorized access.

In July 2024, the fragility of cloud infrastructure became front and center in the public consciousness when the prominent cybersecurity company CrowdStrike, which provides security services to Microsoft, released a faulty software update that trapped millions of Windows-based computers worldwide in reboot loops, which the internet deemed the *blue screen of death*, rendering the machines unusable for several hours. This caused widespread issues, from hospitals being unable to access patients’ medical records to entire airports being shut down. The event and the dramatic impacts that followed highlighted the risks inherent in relying on highly interconnected cloud-based systems, and more specifically in having such strong dependency on a single company.

With a local installation of Python, you and your organization can be entirely aware of what software is allowed to run on your system, and you aren't subject to the whims of an outside corporation. This autonomy ensures that your data remains within your control, improving security and facilitating compliance with internal policies.

Alternatives to Python

While I would argue that transitioning to Python is the best solution to Excel's shortcomings, it certainly isn't the only one. Excel already offers a few tools that aim to solve some of the deficiencies we've discussed. Let's take a look at these tools to see their advantages and disadvantages.

VBA

VBA is a language developed by Microsoft that allows you to interact with Excel via code. Like Excel, VBA is accessible because it's close by: You can add VBA code to a spreadsheet by using a code editor embedded in Excel. VBA allows you to write functions and subprocedures rather than rely on cells and sheets to apply structure to your data and logic, so it provides some of the benefits of using Python, like improved modularity and more-accessible automation.

In the end, though, VBA forces you to work in a language that's proprietary to Microsoft and inherently tied to Microsoft applications. Therefore, you don't get to enjoy many of Python's benefits, such as its readability and its vast, community-maintained selection of external, open source libraries. On top of all this, because Python is easier to read and has a broader community supporting it, it's easier to learn in the first place.

Python and VBA were both initially developed in the early 1990s, but in Python's case, you wouldn't know it. Python is open source, so the language has benefited from the contributions of a global community of developers. VBA's evolution fell into the hands of a smaller, corporate team at Microsoft, which would never be able to incorporate user feedback as well as a community-driven project like Python could. Because of this difference, VBA's language structure and syntax still reflects the programming conventions and user interface design principles of the 1990s, whereas Python is decidedly modern.

To illustrate, here's a VBA function to sum all the numbers that are greater than 5 from a list of numbers:

```
Function SumNumbersGreaterThanOrEqualToFive(numbers As Range) As Long
    Dim sum As Long
    Dim i As Integer
    Dim arr As Variant

    arr = numbers.Value
    sum = 0
```

```

If Not IsArray(arr) Then
    If IsNumeric(arr) And arr > 5 Then
        sum = arr
    End If
Else
    For i = LBound(arr, 1) To UBound(arr, 1)
        For j = LBound(arr, 2) To UBound(arr, 2)
            If IsNumeric(arr(i, j)) And arr(i, j) > 5 Then
                sum = sum + arr(i, j)
            End If
        Next j
    Next i
End If

SumNumbersGreaterThanOrEqualToFive = sum
End Function

```

This function uses a for loop to iterate through each number in the list, check whether it's greater than 5, and if so, add it to a running total. The code is simple enough, but verbose. Some parts require more in-depth knowledge of what's going on behind the scenes, like needing to use `Dim` to specify the data type and allocate storage space when defining a variable. A newcomer to the language probably could tease out what's going on here, but that process wouldn't be terribly enjoyable.

Here's an equivalent Python function:

```

def sum_numbers_greater_than_five(numbers):
    return sum(num for num in numbers if num > 5)

```

This function filters and sums a list of numbers in a single elegant line of code. Python's readable syntax makes the operation clear and easy to follow: Sum only those numbers in the list that are greater than 5. The code is so readable, it's almost English.

Python's concise, human-friendly approach means it's easy to write and, perhaps more important, easy to read. No matter what you use your code for, you're going to spend a significant amount of time staring at it. Wouldn't you rather stare at a modern language with a modern syntax that has benefited from decades of user feedback rather than having to decipher convoluted code that's stuck in the 1990s?

Python in Excel

Python in Excel is a new feature that integrates Python directly into spreadsheets and allows you to replace traditional Excel formulas in cells with Python code. This means you can simplify operations that are unnecessarily convoluted in Excel and take advantage of some areas of data analysis where Python shines, like time-series analysis and visualization.

To understand how this works, imagine you have a dataset in Excel. You want to group the data based on one column and calculate summaries, like

totals or averages, for another column within each of these groups. In regular Excel, this would typically require a clunky mix of lengthy formulas or a pivot table, which can take tedious manual work to generate and update. As we'll discuss in Chapter 6, Python allows you to achieve the same results more efficiently using concise functions. Python in Excel brings those functions right into your spreadsheets.

At first glance, this seems incredibly convenient, but Python in Excel gives you only some of the advantages of using Python while retaining most of Excel's limitations. For example, because your dataset still resides within Excel, you're constrained to the import and export options that Microsoft has built into the application, meaning you miss out on Python's interoperability. You also won't be able to take advantage of version control to collaborate on your code, and automation will be more difficult too.

All this raises the question: If you already understand how to represent and communicate concepts in Python well enough to use these functions in Excel, why not take the next step and Pythonize the entire process? Doing so would give you the full benefits of Python, including its accuracy, scalability, and seamless interoperability with other tools. Defaulting to Python in Excel is like cutting an onion with a butter knife that's right in front of you on your kitchen counter, even though you know a freshly sharpened chef's knife is in a drawer just a few steps away.

What's in This Book

Each chapter of this book explores a Python or related coding concept that can in some way improve upon what you already do in Excel. Every concept comes with enough background information for you to get started, as well as an explanation of why it's useful.

Part I covers the basics of writing and working with Python scripts:

Chapter 1: Setting Up Your Coding Environment Walks you step-by-step through installing Python and the popular Visual Studio Code (VS Code) editor so you're all set to code. You'll also learn skills like using a command line interface and editing environment variables that are necessary to ensure that your setup is ready to go.

Chapter 2: Coding Fundamentals Explained Through Excel Translates basic programming concepts, like variables, data types, functions, and conditionals, into terms familiar to Excel users. Viewing these concepts through the lens of Excel will make them more familiar and approachable, easing your transition into a code-based workflow.

Chapter 3: Automating Tasks with Python Scripts Expands upon the basics from Chapter 2, synthesizing them into complete Python programs. You'll learn how to package logic into loops and custom functions so you don't need to repeat yourself. By the end of the chapter, you'll know enough to craft a practical script to automate a simple task.

Chapter 4: Tracking Changes with Version Control Outlines how to use the Git version-control system to track your code file changes over time. Version control isn't covered in most introductory "learn to code" books, but it's essential to any coding workflow, especially when you're collaborating with others. We'll articulate why version control is useful and how it differs from traditional methods for managing files.

Part II focuses on using Python and related tools to work with datasets:

Chapter 5: Data Analysis Made Interactive Introduces Jupyter Notebook, an environment for writing and executing Python code in a more dynamic, interactive way. As you'll see, working with Jupyter Notebook can be preferable to writing traditional Python scripts when you're doing exploratory data analysis, as notebooks make it easier to play around with your data and immediately see the results.

Chapter 6: Analyzing and Transforming Data Covers the basics of the pandas library, the most effective and widely used tool for data analysis in Python. You'll practice working with pandas DataFrames, tabular structures similar to spreadsheets. You'll see how DataFrames make Excel-like tasks such as grouping, aggregating, and pivoting much easier.

Chapter 7: Working with Databases Begins the journey into manipulating databases with Structured Query Language (SQL). You'll create, access, and edit database tables, and see how to run SQL queries from within your Python scripts. Learning about databases will allow you to manage larger datasets than you ever could in Excel.

Chapter 8: Retrieving Data from the Internet Shows you the basics of using application programming interfaces (APIs) to pull data from external sources into your Python scripts. You'll use a Python library called Requests to communicate with APIs and see how to handle common errors that arise. We'll also discuss how APIs work so you'll understand what your Python code is actually doing.

Chapter 9: Creating Charts and Visuals Explores data visualization, including when to do it inside and outside Excel. We'll walk through creating various kinds of charts with Python's popular Plotly library, and we'll customize and standardize those charts with themes.

Chapter 10: Building Interactive Reports Covers how to build simple dashboard-style applications around your Plotly charts via the Dash framework. You'll use hypertext markup language (HTML) and Cascading Style Sheets (CSS) elements in conjunction with Python code to structure and customize your dashboards. With this skill, you'll be able to make your visualizations shareable and interactive so that data analysis can become a collaborative experience.

Finally, Part III covers a few techniques that make the process of writing code more organized and reliable so you can get the most out of your scripts:

Chapter 11: Organizing Your Code with Classes Introduces object-oriented programming (OOP), a coding style that allows you to manage complex projects with clearer and cleaner code. We'll frame the discussion in terms of OOP's practical benefits so you can understand when using this technique makes sense.

Chapter 12: Finding and Fixing Errors Focuses on finding, fixing, and preventing code errors so you can write and run code more efficiently. We'll go over the ins and outs of debugging so you'll know what to do when your code doesn't work. Then we'll cover the unit-testing process so you can make sure errors don't occur in the first place.

Chapter 13: Three Good Coding Habits Discusses a few habits that promote quality, robust, and maintainable code. We'll highlight simplicity and modularity as guiding principles and will discuss the benefits of making incremental improvements to your code after getting it working.

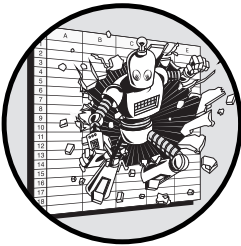
By the end of the book, you should be able to incorporate all of the tools and concepts discussed into the work you're already doing in Excel. As you read, consider times when you've perhaps leaned too heavily on Excel, and think about the situations where incorporating Python could make your work more efficient and productive. I hope you'll find that transitioning to Python will lead to a better, faster, and simpler workflow. Who knows, maybe it'll even save you \$6 billion.

PART I

GETTING STARTED WITH PYTHON

1

SETTING UP YOUR CODING ENVIRONMENT



When you start working with Python, it's important to correctly configure your coding environment. In this chapter, we'll make sure the basic tools you need are installed in the right places and, more importantly, that you know where those places are. We'll walk through installing Python and Visual Studio Code (VS Code), a popular code editor. We'll also cover the basics of interacting with your computer via a command line interface, an essential skill for troubleshooting your setup if something goes wrong.

Why a Well-Configured Installation Matters

If you don't take the time to correctly configure your coding environment, there's a good chance unexpected issues will sneak up on you later. And this can become a huge disruption to your productivity when you start coding.

To understand why this matters, imagine you're getting ready to cook dinner and need something from your kitchen cabinet. You open the cabinet door and look inside, only to find a disorganized mess of precariously placed containers, all jammed on top of one another. When you reach your hand inside the chaos for the ingredient you need, all of a sudden everything comes tumbling out.

In this acutely frustrating situation, you're confronted with two harsh realities at once. First, a mess of jars, bottles, and cans are now all over the floor, and you have to clean them up. Second, and perhaps more disconcerting, the disaster serves as a reminder of past oversights: Had you previously taken the time to organize the cabinet, everything wouldn't have come cascading down in your face.

Installing Python, a code editor, and other necessary software is akin to stocking your kitchen. If these tools aren't organized logically or familiarly, you'll face unforeseen challenges down the road. Because new coders aren't used to being so close to the insides of their computers, they often rush through this setup process only to find themselves with a development environment that fails at unexpected and sometimes inopportune times. Like a messy kitchen cabinet, things aren't set up where and how they're assumed to be: Paths don't point to where they're meant to go, and important tools are buried in the digital clutter, leading to unnecessary frustration and lost time.

Python isn't a monolithic application like Excel. When you download Python, you're getting multiple pieces that work together to help you write and run code. This includes the *Python interpreter*, the piece of software that runs Python files; a set of prewritten pieces of code called the Python Standard Library; and a package manager, which allows you to easily install additional code libraries.

To make sure all these pieces end up in the right place and that your computer knows where to look for them, you'll have to do a bit more than just click Download on the Python website. That's why we'll go over the basics of working with the command line before we turn to installing Python. Understanding the command line first will enable you to view the file structure of your operating system in order to identify where and how Python is installed.

Using the Command Line

A *command line interface (CLI)* is a tool that allows you to use text commands to interact with the inner workings of your computer. Unlike the graphical user interface (GUI) of your computer's desktop, a CLI doesn't confine you to predefined buttons and visual layouts. Instead, it provides a more direct and versatile way to control your computer and execute a wide range of operations.

In this section, we'll go over the fundamentals of working with a CLI and the basic commands you'll need to install Python and verify that it's working correctly. Becoming comfortable with the CLI and its text commands is a

relatively small undertaking that can yield significant benefits. While a CLI can't organize your actual kitchen cabinets (yet), getting used to working with your CLI will enable you to identify and disentangle the messes that will inevitably arise in the proverbial kitchen cabinet of your computer setup.

To begin, let's clarify five key terms that are essential to working with a CLI:

Command An instruction given to the computer through the CLI. Commands tell the computer to perform a specific task, such as moving a file (`mv` or `move`) or copying files (`cp` or `copy`).

Argument An additional piece of information provided to a command that influences or specifies the action to be performed. Arguments are essentially modifiers to the command. For example, if you were using the `mv` or `move` command to move a file, you'd include arguments specifying the file to be moved and its intended destination.

Shell The program that interprets your commands and communicates them to your computer's operating system for execution. Different operating systems have different shells. All versions of Windows include Command Prompt (CMD), while newer versions also offer an alternative called PowerShell. Older versions of macOS use `bash`, while newer versions use Z shell (`zsh`).

Console The application where you conduct command line sessions. The console acts as the user interface for entering commands, while the shell provides the functionality to actually process the commands. On Windows, the console, like the shell, is called Command Prompt (CMD), while on macOS it's called Terminal.

Prompt A symbol or set of symbols displayed in the console that indicates the shell is ready to receive the next command. The prompt usually provides contextual information, like the current user's name and location in the filesystem. For example, a typical `zsh` prompt on macOS might look like `YourUsername@YourMac: ~ %`, while a Windows CMD prompt might look like `C:\Users\Username>`. When you run programs in the command line, they may have their own prompts.

These five key terms provide a solid foundation for using the CLI.

Navigating Directories and Subdirectories

One common use for a CLI is to navigate the directories and subdirectories of your computer's filesystem, accessing various locations where you've stored scripts or files. In the context of the command line, you'll notice the term *directories* being used to refer to what are more colloquially known as *folders*, but both terms define the same concept: containers in the filesystem that can hold files and other directories.

Table 1-1 lists some basic filesystem navigation commands for macOS and Windows.

Table 1-1: Filesystem Navigation Commands for macOS and Windows

Action	macOS command	Windows command
Change directory	<code>cd <i>directory</i></code>	<code>cd <i>directory</i></code>
List directory contents	<code>ls</code>	<code>dir</code>
Display current directory	<code>pwd</code>	<code>cd</code>
Move up one directory	<code>cd ..</code>	<code>cd ..</code>
Create a new directory	<code>mkdir <i>directory</i></code>	<code>mkdir <i>directory</i></code>
Delete a directory	<code>rmdir <i>directory</i></code>	<code>rmdir <i>directory</i></code>

Every file and directory has a *path*, which specifies its address within the filesystem. Some commands require a path as an argument, indicated in Table 1-1 by the italicized *directory* placeholder. For example, to make a directory with `mkdir`, you must provide the desired path of the new directory you’re trying to create. At each stopping point between commands, the CLI prompt will indicate the path of the directory you’re currently navigated into, so you can always tell your location in the filesystem.

Before we explore how to use filesystem navigation commands in a CLI, it’s important to distinguish between the user level and the system level of a filesystem. Both macOS and Windows are multiuser operating systems, meaning they’re set up to accommodate multiple accounts on a single system. The *user level* provides a personal workspace for a single user account, where that individual can store their personal files, application settings, and preferences. Files and configurations stored at the user level typically affect only that particular user account. By contrast, files stored at the *system level* affect all users. This has implications for setting up firewalls, installing programs, and modifying configurations or settings.

At the user level, every user has a home directory, the top-level location in that user’s portion of the filesystem. On macOS, that directory is `/Users/<YourUsername>`, and on Windows, it’s `C:\Users\<YourUsername>`. When you start a CLI session, you begin in your user account’s home directory by default. At the system level, the system root directory is the base of the entire filesystem hierarchy. This directory is denoted with a forward slash (`/`) on macOS and with `C:\` on Windows.

Notice that Windows and macOS use different types of slashes in filepaths. In macOS, a forward slash separates each directory; for example, the path to your user’s *Documents* folder would typically be `/Users/<YourUsername>/Documents`. Windows, on the other hand, uses backslashes, so its equivalent path would be `C:\Users\<YourUsername>\Documents`. When you reference paths in code that are shared among people with different operating systems, keep this discrepancy in mind. You should write paths in a way that’s compatible with both operating systems. This usually means using forward slashes (`/`) universally, since most Windows versions do accept them in the context of the command line, or replacing them with a system-aware method.

We’ll now look at how to use the most common CLI commands for directory navigation. We’ll walk through the same simple example on macOS

and Windows: navigating from the user home directory to the *Desktop* directory, creating a new directory called *my_new_dir*, checking to see whether the new directory exists, and then deleting it.

NOTE

As you begin using the command line, you may forget the syntax or format for particular commands. A good resource to help you reacquaint yourself is <https://ss64.com>, which gives short explanations of the common commands for macOS and Windows.

Directory Navigation on macOS

If you're using macOS, first open the Terminal application: Use the ⌘-spacebar keyboard shortcut to open Spotlight Search, then type **Terminal** into the search bar. Press ENTER when the Terminal icon is highlighted in the search results.

When the terminal loads, you should see a prompt that gives your username (we'll call it *YourUsername*), the computer you're on (we'll call it *YourMac*), and the current directory that you're navigated to. As mentioned, a CLI defaults to your user account's home directory, which Terminal represents with a tilde (~). The prompt ends with a percent symbol (%) or a dollar sign (\$), indicating it's ready to accept input. In all, the prompt should look something like this:

```
YourUsername@YourMac ~ %
```

Or it may look like this:

```
YourUsername@YourMac:~$
```

In either case, the prompt indicates you're in the home directory and ready to type commands.

For reassurance, let's check the path of the current directory. To do this, type **pwd** (short for *print working directory*) and press ENTER to execute the command. This should give you the path to your user home:

```
YourUsername@YourMac ~ % pwd
/Users/YourUsername
```

Now let's navigate to the Desktop by using the **cd** (change directory) command. Run **cd Desktop** from the user home:

```
YourUsername@YourMac ~ % cd Desktop
```

To make sure it worked, stop here and run **pwd** again. It should show your current location as */Users/YourUsername/Desktop*:

```
YourUsername@YourMac Desktop % pwd
/Users/YourUsername/Desktop
```

If all is good, create the *my_new_dir* directory within *Desktop* by using the **mkdir** (make directory) command:

```
YourUsername@YourMac Desktop % mkdir my_new_dir
```

Now let's check whether the new directory exists. Enter **ls** to list all the files and subdirectories in the current directory:

```
YourUsername@YourMac Desktop % ls
```

This command should result in a list of file and directory names inside *Desktop*, including *my_new_dir*.

Now that we've created the directory, let's delete it. Use the **rmdir** (remove directory) command as follows:

```
YourUsername@YourMac Desktop % rmdir my_new_dir
```

Run the **ls** command again. You should see that *my_new_dir* has been deleted.

Directory Navigation on Windows

On Windows, open the CMD application: Open the Start menu and type **cmd** into the search bar. From the search results, click either **Command Prompt** or **CMD**. You should then see a prompt that usually starts with the drive letter (typically *C*), followed by the path of the current directory. In most cases, that will be the home directory for your user account, which we'll call *YourUsername*. The prompt ends with an angle bracket (*>*), indicating you can enter a command. All together, the prompt should look something like this:

```
C:\Users\YourUsername>
```

First, we'll navigate from the user home to the *Desktop*. Type **cd** (change directory) and press ENTER to run the command:

```
C:\Users\YourUsername> cd Desktop
```

To verify you're now in the *Desktop* directory, enter **cd** again, this time without specifying a directory afterward. Used this way, the command prints the path of the current directory, which should be *Desktop*:

```
C:\Users\YourUsername\Desktop> cd
C:\Users\YourUsername\Desktop
```

Now create a new directory within *Desktop* named *my_new_dir* by using the **mkdir** (make directory) command:

```
C:\Users\YourUsername\Desktop> mkdir my_new_dir
```

To check whether the command succeeded, enter **dir** to list all the files and directories in the current directory:

```
C:\Users\YourUsername\Desktop> dir
```

You should see a list of file and directory names, *my_new_dir* among them.

Finally, to delete the *my_new_dir* directory, use the **rmdir** (remove directory) command:

```
C:\Users\YourUsername\Desktop> rmdir my_new_dir
```

If you run **dir** again without changing directories, *my_new_dir* should no longer appear in the resulting list.

Manipulating Files

Another common command line task is to move, copy, delete, create, or otherwise manipulate individual files. Table 1-2 lists common file-manipulation commands for macOS and Windows.

Table 1-2: File-Manipulation Commands for macOS and Windows

Action	macOS command	Windows command
Move (or rename) a file	<code>mv source destination</code>	<code>move source destination</code>
Copy a file	<code>cp source destination</code>	<code>copy source destination</code>
Delete a file	<code>rm file</code>	<code>del file</code>
Create a file	<code>touch file</code>	<code>echo. > file</code>

To illustrate how some of these file-manipulation commands work, let's walk through the process of creating a text file on the Desktop, verifying its existence, and then deleting it in both macOS and Windows. As you perform these file manipulations, you may also want to follow along in your computer's file manager GUI to visually confirm the changes. On Windows, this is File Explorer, and on macOS, it's Finder. After you create the file, you should be able to see it appear in the *Desktop* directory within your file manager. Once you delete the file with the command line, you'll also notice it disappear from the *Desktop* folder.

File Manipulation on macOS

Open Terminal and once again navigate to the *Desktop* directory as we did in the previous section. Once you're in the directory, use the **touch** command to create a new text file called *example.txt*:

```
Yourusername@YourMac Desktop % touch example.txt
```

To confirm the file was created, list the contents of the *Desktop* directory by using **ls**:

```
Yourusername@YourMac Desktop % ls
```

You should see *example.txt* listed among the files.

Now remove the file by using the **rm** command:

```
Yourusername@YourMac Desktop % rm example.txt
```

To verify that the file was removed, enter **ls** to again see the contents of *Desktop*.

File Manipulation on Windows

Open CMD and navigate to the *Desktop* directory by using **cd Desktop** as before. From there, we'll use the **echo** command to create a new file named *example.txt*. If we just wanted to create an empty file, as we did in the macOS example, we'd use **echo. > example.txt**, where the period after **echo** indicates the file should be empty. However, the **echo** command also includes an argument to specify the contents of the new file. Enter the following to see how it works:

```
C:\Users\YourUsername\Desktop> echo Some Text > example.txt
```

The **>** symbol indicates that the text *Some Text* should be directed to the *example.txt* file.

Once the file is created, run the **dir** command to ensure that it exists:

```
C:\Users\YourUsername\Desktop> dir
```

You should see *example.txt* in the list of files in the *Desktop* directory.

To remove the file, run the **del** command:

```
C:\Users\YourUsername\Desktop> del example.txt
```

To verify the file was removed, run the **dir** command again.

NOTE

In general, it's best not to include spaces in the names of directories or files. On the command line, the space character is used as a delimiter to separate commands and their arguments, so unnecessary spaces can cause confusion. To avoid this problem while still creating readable file and directory names, use underscores instead to separate individual words. We did this earlier with the `my_new_dir` directory.

Customizing and Storing System Settings

You'll often want to store information in your operating system for programs to use. This could include file locations, user preferences, system configurations, and sensitive data such as passwords. *Environment variables* allow you to store and update this kind of information in a way that's accessible through a CLI.

Python and other coding tools also use environment variables behind the scenes to work with your computer. When something's wrong with your setup, often an incorrect or missing environment variable is to blame. Therefore, being able to manipulate environment variables is an important troubleshooting skill.

Environment variables function as key-value pairs in an operating system: The *key* is the name of the variable, and the *value* is the data or settings associated with that name. Each environment variable has a *scope*, or a range of influence. *System environment variables* apply to the system as a whole, while *user environment variables* apply to the individual user's session. *Application-specific environment variables* can be used only by individual applications.

To access the value stored in an environment variable, you need to ask for its key. Table 1-3 gives basic commands for handling environment variables through a CLI.

Table 1-3: Environment Variable Commands for macOS and Windows

Action	macOS command	Windows command
Access an environment variable	echo \$VAR_NAME	echo %VAR_NAME%
View all environment variables	printenv	set
Set an environment variable	export VAR_NAME=VALUE	set VAR_NAME=VALUE

Viewing environment variables on the command line is generally straightforward. However, setting them, especially permanently, can require a few more steps and depends on your operating system. This is because the scope of an environment variable determines how it can be modified and how permanent it is.

System environment variables require administrative privileges to alter. Any changes to one of these variables are permanent and remain in effect until the variable is explicitly altered or removed. User-level environment variables allow individual users to modify them without affecting other users or the system as a whole. These modifications are typically persistent across a user's sessions but don't extend beyond the user's own environment. Environment variables can also be set temporarily for the duration of a specific session or process. Whenever you set an environment variable, it's helpful to understand its scope and whether it's permanent.

One of the most common setup mistakes new Python users make during installation is accidentally setting environment variables temporarily instead of permanently. As you'll see in "Verifying the Installation" on page 16, Python relies on certain key environment variables to function correctly. If these variables are set only temporarily, they will reset when you close your terminal or restart your computer, potentially causing issues later. Understanding the difference between temporary and permanent environment variables can save you a lot of troubleshooting down the road.

Modifying Environment Variables on macOS

Temporarily modifying environment variables on macOS is easy. Here, we use the `export` command to create an environment variable called `VAR_NAME` with a value of `VALUE`:

```
YourUsername@YourMac ~ % export VAR_NAME=VALUE
```

This stores the new environment variable for the duration of the current command line session, but it won't be available in future sessions. To prove it, use `echo` to access the variable, adding a dollar sign before the key:

```
YourUsername@YourMac ~ % echo $VAR_NAME  
VALUE
```

This should output the variable's value, but if you close and reopen the terminal and try the same command again, it will no longer work. The environment variable didn't persist from one session to the next.

To permanently modify environment variables, as you may need to do when troubleshooting your Python installation, you need to create a *profile*, or a default configuration, for your shell. This is a set of commands and settings that customize the shell's behavior every time you open it. To do this, you first need to check whether your macOS shell is bash or zsh. Open the terminal and print the value of the SHELL environment variable:

```
YourUsername@YourMac ~ % echo $SHELL
```

If the console prints `/bin/bash`, your shell is bash; if it prints `/bin/zsh`, your shell is zsh. Knowing this, you can create your profile.

Apple computers come preinstalled with a simple file editor called nano, which you can use to create your profile. If your shell is zsh, run this:

```
YourUsername@YourMac ~ % nano ~/.zshrc
```

If your shell is bash, use the following command:

```
YourUsername@YourMac ~ % nano ~/.bash_profile
```

The terminal should now turn into an editor for the new empty profile file. Add the following line:

```
export VAR_NAME="VALUE"
```

Save and close the file by pressing CTRL-X, then Y to confirm, and ENTER to save. Then restart the terminal session to apply the changes. After that, test whether your environment variable is still there:

```
YourUsername@YourMac ~ % echo $VAR_NAME  
VALUE
```

If everything worked, the console should print `VALUE`.

To delete the environment variable, use nano to reopen your shell profile and remove the export line that you added. To do this, first use `nano ~/.zshrc` or `nano ~/.bash_profile` to open the saved profile file. Then delete the export line, and press CTRL-X and then Y to exit and save the file.

Modifying Environment Variables on Windows

You can temporarily set a Windows environment variable with the `set` command. Here's how to create a temporary environment variable called `VAR_NAME` and give it the value `VALUE`:

```
C:\Users\YourUsername> set VAR_NAME=VALUE
```

Next, confirm that the new environment variable was saved, like so:

```
C:\Users\YourUsername> echo %VAR_NAME%
```

Notice that the key name must be surrounded by percent symbols. This should return `VALUE`, the value of the environment variable.

This change will last only until you close the Command Prompt session. To prove that the environment variable hasn't been permanently saved, close your existing CMD session and open a new one. Now use `echo` to check for the environment variable again. Since it wasn't saved permanently, the console should print an empty line.

For permanent changes to a Windows environment variable, which you might need to make when troubleshooting your Python installation, you need to access the System Properties. Right-click **This PC** or **My Computer** on your desktop or in File Explorer, then choose **Properties ► Advanced System Settings ► Environment Variables**. Under the User Variables or System Variables section, click **New** to add your new variable. Enter `VAR_NAME` as the variable name and `VALUE` as the variable value and apply the changes.

After you set the system environment variable, you'll need to restart your computer to ensure that the changes take effect. Then use the `echo` command in CMD to check whether the new environment variable exists:

```
C:\Users\YourUsername> echo %VAR_NAME%  
VALUE
```

If the variable has been set correctly, the console should display `VALUE`.

Working Within Corporate IT Policies

In the previous sections, we discussed how to modify environment variables, and in the next section, we'll walk through the process of installing Python. If you work on a computer owned by a larger organization or corporation, however, you likely don't have unlimited control over your system. A fire-wall may be restricting internet use, and you may not be able to install new software or change configurations like environment variables as you please. This is for good reason, as real security risks come with allowing anyone on a network to install software off the internet and change configurations.

Ideally, these boundaries won't hinder your quest to boost your productivity and efficiency through code. To prevent that from happening, you need to understand your organization's specific policies and learn to work within them. Some important questions to find out about your organization's boundaries might be as follows:

- Can I install programs at the system level?
- Can I change system environment variables?
- Can I download software off the internet?
- Can I edit system configurations?
- Am I required to use a virtual private network (VPN)?

Depending on the answers to these questions, you may need to get permission from someone on your organization's IT team to do something

like install Python on your system. You'll also need to articulate in specific terms what you need. If you just say, "I want to install Python," your IT team may not know what that entails. You can use the terms and concepts detailed in this chapter to be more specific. For example, if your corporate firewall prevents you from changing settings or files on the system level of your computer, you should ask specifically which rights or approvals you need to make those changes.

The best-case scenario is to develop a close, mutually trusting relationship with someone from your IT department or someone who knows about the specific restrictions at your company. Work with that person to understand the limits of your organization's policies and develop strategies to ensure that your setup works for both you and your company.

Installing Python

Now that you have some facility working at the command line, let's walk through the steps to install and test Python. The process is a bit more involved than installing an ordinary application like Excel. To get Excel working, you don't really need to understand the quirks of your specific operating system. You don't need to know where system files are located or whether you have permission to permanently update environment variables. By contrast, installing Python sometimes forces you to get closer to the inner workings of your machine.

For this reason, some people opt to bypass the process altogether and use a web-based coding tool like Google Colab that doesn't require local installation. Going in that direction greatly limits the amount of functionality available, however. It removes several of the benefits of learning to code in the first place, like increased interoperability and scalability of your work. Therefore, going through the sometimes painful process of installing Python locally makes sense.

To make the process a little easier, we'll be installing the standard Python setup. Other installation options can be helpful for various reasons, like using virtual environments or Docker images. These options can ensure that everyone working on a team has a similar configuration or can allow multiple versions of code libraries to be installed at the same time. But the goal of this book is to make code useful for you, not to check unnecessary boxes. If and when you need virtual environments or Docker images, you should absolutely embrace them, but for now, a vanilla installation is all you need to get started.

Running the Installer

The first step in the process is to download and run the Python installer from <https://www.python.org>, the official website of the Python Software Foundation. The site's Downloads section includes the latest version of Python, as

well as all previous versions. In most cases, you'll want to install the latest version, unless colleagues you're sharing code with use a different one or you intend to work with a tool or library that's compatible with only certain versions. For example, as of this writing, the machine learning library TensorFlow is compatible with only Python 3.6 to 3.9. (See the "Python Version Numbering" box below for more on understanding version numbers.)

Select the Python version you want, find the installation file for your operating system, and download the installer. When you run the installer, a window will pop up to guide you through the installation process.

One of the screens in the installer will show the default path where Python will be installed. On Windows, this will probably look something like `Users\<YourUsername>\AppData\Local\Programs\Python\Python3X`. On macOS, it will probably look like `/Library/Frameworks/Python.framework/Versions/3.x`, but it could also be something like `/usr/local/bin/python3`. Copy this path and save it somewhere close by. You may need it later.

You'll be offered the chance to customize your installation. If you've been given guidance by your IT department to veer away from the standard installation, you can skip this step. On Windows, you may be given the option to add Python to the PATH during the installation process. If so, you should select this option, which automatically stores the relevant path to Python in the PATH environment variable, if your administrator privileges give you the right to change it. On macOS, this is part of the standard installation process as of this writing. When you click Install, you may be prompted to input administrator credentials. If you have restrictions on how you can modify your system, you may require IT assistance for this step.

PYTHON VERSION NUMBERING

A Python version is denoted by three numbers—for example, 3.11.7. These numbers specify the major version (3), minor version (11), and micro version (7). *Micro versions* consist of incremental changes like bug fixes and security patches. *Minor versions* include larger changes like new features but generally don't contain changes that are incompatible with the previous minor version. For example, code written for version 3.10 should typically work in 3.11 without modifications. *Major versions* include significant changes that may not be backward compatible. This is the case with Python 2 and Python 3.

For context, minor version upgrades are released fairly often (every 18 months or so), but the last major version upgrade occurred when Python 3 was released in December 2008. As of this writing, the Python Software Foundation has not given any indication that a Python 4 is in the works.

Once your installation is ready, it's good practice to locate the new Python files and ensure that they are where you think they are. You can see this by navigating to the installation directory via the command line as we discussed

earlier (or using your computer's file manager). On Windows, the directory's contents might look like this:

```
Python3x\  
├── DLLs\  
├── Lib\  
├── Scripts\  
├── tcl\  
├── Tools\  
├── include\  
├── libs\  
├── python.exe\  
├── pythonw.exe\  
└── vcruntime140.dll
```

On macOS, the contents might look like this:

```
3.x/  
├── bin/  
│   ├── python3.exe/  
│   └── pip3.exe/  
├── include/  
├── lib/  
├── Resources/  
└── share/
```

Once you know that these files exist and are organized in the right way in the right place, that removes a lot of the guesswork typically associated with setting up your Python environment.

Verifying the Installation

After you've run the installer, you should verify that the installation was successful and confirm which version of Python has been installed. To do this, open your CLI and enter the following command:

```
python --version
```

Or, depending on your setup, you may need to use this command instead:

```
python3 --version
```

Which of these two commands works for you depends on the name and location of the Python file that was installed. On macOS, you'll typically use the `python3` command to reference a binary file called *python3*. On Windows, you'll usually use the `python` command to reference a *.exe* file called *python.exe*, although exceptions occur.

If Python was installed successfully and the `PATH` variable was set correctly during the installation process, you should see the Python version you downloaded displayed in the console. However, if you receive an error or

if Python isn't recognized, most often that's because Python wasn't added to the PATH during installation. This step is crucial as it tells your operating system where to find the Python interpreter. If you have this problem, you'll need to manually add your Python installation path to the PATH environment variable, just as we outlined in "Customizing and Storing System Settings" on page 10.

macOS Troubleshooting

Say you're on macOS, and the path to your Python installation is */Library/Frameworks/Python.framework/Versions/3.x*. You'll need to store that in your bash or zsh profile, as we outlined earlier. Run either `nano ~/.zshrc` or `nano ~/.bash_profile` in Terminal, then add a new line to your profile that looks like this:

```
export PATH="/Library/Frameworks/Python.framework/Versions/3.x/bin:$PATH"
```

The `:$PATH` at the end of this line appends what's already in your PATH variable to the new Python path that you're adding. Anything you already added to the PATH environment variable remains accessible.

Note also that some distributions of macOS come with a version of Python preinstalled. If this is the case for you, you'll want to double-check that the Python version running in your command line terminal is the one you installed yourself. If not, you may need to alter your PATH environment variable to point to the correct version. This is important as the preinstalled Python version will likely be older and could cause compatibility issues.

Windows Troubleshooting

Say you're on Windows, and the path to your Python installation is *Users\<YourUsername>\AppData\Local\Programs\Python\Python3X*. You'll need to open **This PC** or **My Computer**, choose **Properties ► Advanced System Settings ► Environment Variables**, and look at the User or System Variables. An environment variable called PATH should already exist by default. If so, click it, add a semicolon (;) at the end of its existing content as a separator, and then add the path to your Python installation to the list of paths. If a PATH variable doesn't exist, create one and set the Python path as the value.

Running Some Code

Now that you've verified the installation, let's run a simple Python command to make sure the Python interpreter works. To do this, create an instance of Python within your command line terminal by entering `python` or `python3` as appropriate. You should see a message including information like the Python version and build date, followed by the `>>>` symbol, which is the Python interactive shell prompt. This prompt indicates that Python is waiting for you to enter a command. As long as you see the `>>>` prompt, you'll know you've entered the Python interpreter. Reaching this point means that

you've installed Python correctly and you're able to access and run the version of Python you installed.

To test a command, try entering a simple `print()` statement into the interpreter to make Python display some text:

```
>>> print("Hello, world!")
Hello, world!
```

The interpreter should respond by outputting the text `Hello, world!`, which indicates that Python understood your request.

Now that you know everything is working, you can exit the Python interpreter and go back into the regular command line shell by entering the `exit()` command:

```
>>> exit()
```

You should now be back to seeing your usual macOS or Windows command line prompt, instead of the `>>>` prompt for the Python interpreter.

Getting Started with Python Libraries

Python's vast ecosystem of open source external libraries is a large part of its appeal. *Libraries* are collections of prewritten code that you can use to extend the functionality of the language. You wouldn't want to use Python without being able to use any libraries.

To use libraries, you need the pip package-management tool. With pip (short for *pip installs packages*), you can easily install, update, remove, and otherwise manage Python software packages.

Most Python installations automatically include pip. To verify it's installed, enter the following on the command line:

```
pip --version
```

You may need to use this command instead:

```
pip3 --version
```

Whether you use the `pip` or `pip3` command depends on the name of the pip executable file in Python's directory.

The console should print the version of pip that you're using, and then a description of which version of Python it goes with, which should match the Python version you installed. If the pip version doesn't print in the console as expected, that likely means your system doesn't recognize pip. To fix this, you must add the path to pip to the `PATH` environment variable.

First, find the path to pip by navigating through the command line into the directory of your path to Python, which you took note of during the installation process. For example, if your path to Python is `C:\Program Files\Python\Python39`, you would run the following:

```
cd C:\Program Files\Python\Python39
```

The *pip* file is usually located in the *Scripts* or *bin* subdirectory of your Python installation directory. Run `ls` to find the file. Then add the location of the *pip* file to your `PATH` environment variable.

Alternatively, *pip* could have not been installed with Python in the first place. In this case, you'll need to add it. Any modern version of Python (versions 3.4 and later) will include a module called *ensurepip* that will download *pip* for you if necessary. Run the module on the command line as follows:

```
python -m ensurepip
```

You may need to enter this instead:

```
python3 -m ensurepip
```

This installs *pip* if you don't already have it, or does nothing if you do.

HOW LIBRARIES END UP IN PIP

For a package to be available through *pip*, it must be contained in the Python Package Index (PyPI). This collection of Python libraries follows a standardized format. Anyone can submit Python libraries to PyPI as long as the code is packaged correctly.

Because of the open nature of Python and PyPI, code that's available through *pip* hasn't necessarily undergone a rigorous approval process. Therefore, taking regular precautions to keep nefarious software out of your system is generally prudent. Don't install libraries from unknown creators, stick to well-known and widely used libraries when possible, and be wary of any libraries that require extensive system permissions.

Tools to Simplify Writing Code

A *code editor* is an application that makes coding more manageable and less error prone. It's like a text editor but with enhanced features to facilitate writing, editing, and running scripts. You'll likely be familiar with some of the benefits of using a code editor if you've ever found yourself staring down an elaborate Excel formula like this:

```
=IFERROR(VLOOKUP(A1, Sheet1!B:C, 2, FALSE), "Not Found") & " - " &  
TEXT(SUMIFS(Sheet2!D:D, Sheet2!A:A, ">=" & DATE(YEAR(TODAY()), 1, 1),  
Sheet2!A:A, "<=" & DATE(YEAR(TODAY()), 12, 31), Sheet2!B:B, A1), "$#,##0.00") &  
" - " & IF(AND(COUNTIF(Sheet3!B:B, A1) > 0,  
SUMPRODUCT(--(Sheet3!B:B = A1), --(YEAR(Sheet3!C:C) = YEAR(TODAY()))),  
--(Sheet3!D:D > 1000)) > 3), "High Value", "Standard")
```

Depending on the version of Excel you're using, the software might give you some assistance in writing and understanding this formula, turning it from a jumbled bowlful of noodles into something more comprehensible and digestible. The formula bar may highlight certain parts of the formula

red, blue, or green, or it may show you which part of the formula you're currently editing.

A code editor does something similar, using highlighting and other features to simplify the process of reading, writing, and debugging code. This kind of functionality isn't available in simple text editors like Notepad or nano that come preinstalled with your operating system. In this section, we'll walk through the benefits of using a code editor and show you how to get started with the primary editor we'll use in this book, VS Code.

Important Features

Code editors provide a wide variety of features. Some of the most common are text highlighting, error detection, code completion, advanced navigation, and integrated debugging.

Text Highlighting

Text highlighting helps distinguish between different elements of code, such as variables, keywords, operators, and comments. This visual differentiation improves readability and allows you to see the overarching code structure. This is akin to Excel highlighting different parts of the convoluted formula in our earlier example.

Error Detection

Code editors use *error detection* to point out mistakes as you make them. Excel does this too. For example, when you have a range of cells with similar formulas and one of them deviates from the pattern, Excel automatically generates a warning flag in the form of a small green triangle that displays the message *Inconsistent Formula*. The immediate feedback provided by error-detection tools allows you to correct issues on the fly so they don't get in your way later.

Code Completion

Code editors use *code completion* to predict the ends of partially typed words or code sections. This can be anything from completing variable or function names to filling in entire code snippets. The predictions are tailored to the syntax and libraries of the programming language you're using.

This area has seen dramatic innovation in recent years with the growth of large language models (LLMs). LLMs can now make suggestions based on the entire context of code that you're using as well as what others have written in the past. Being able to leverage these models through tools like GitHub Copilot can be a significant boon to productivity.

Advanced Navigation

Excel workbooks can quickly grow complex as you add more sheets, formulas, visualizations, and pivot tables. Code can similarly grow in complexity. *Advanced navigation* in code editors simplifies the way you move through and manage complex codebases. These functions make it easy to jump between

sections of code and follow nonlinear logic. In VS Code, for example, you can see where a particular piece of code is being used across a project by clicking Find All References.

Integrated Debugging

A *debugger* is a tool in a code editor that allows you to pause code as it's running in order to understand exactly what it's doing. This way, you can pinpoint the cause of errors or unexpected behavior.

In Excel, you may have used the F9 key to isolate and calculate specific parts of a formula instead of the whole thing. This allows you to determine whether each part is doing what you think it's doing so you can more easily identify errors. A debugger works similarly, but instead of highlighting and calculating specific portions of the code, it allows you to insert *breakpoints*, which pause the code on a specific line before it continues to run. We'll see how this works in Chapter 12.

NOTE

An integrated development environment (IDE) goes a step further than a code editor. An IDE includes a code editor but also comes with a suite of other tools like a debugger, compiler, and build automation tools. VS Code is often described as a middle ground between a code editor and an IDE. Out of the box, it's an editor, but you can install extensions that essentially make it an IDE.

Popular Python Editors

Many Python code editors are available, but we'll primarily use VS Code in this book for a few reasons. First, VS Code is free to use, including for commercial purposes. We all love saving money, so this is a nice feature. Second, VS Code is built and maintained by Microsoft, which continues to actively develop and support the software. We wouldn't want to spend time learning how to use an editor that will stop being maintained in the near future, so this is comforting. Third, VS Code is general-purpose, meaning it was built for a wide variety of use cases. It works well for everything from web development to data science. And it isn't just that you *can* use VS Code for all these tasks; it's fair to say that VS Code is the tool that you *should* use for them.

VS Code isn't the simplest editor for newcomers to coding. It's highly customizable, which can be overwhelming at first, but this allows you greater potential for growth in your skill set over time. Just as you don't want to pigeonhole yourself to Excel, you don't want to pigeonhole yourself to an editor that holds you back. VS Code offers support for a variety of coding languages, as well as a library of thousands of extensions that make coding easier.

VS Code Installation

To install VS Code, go to the application's official website (<https://code.visualstudio.com>) and use the download link for your operating system (Windows or macOS), as well as the appropriate version (32-bit or 64-bit, if you're on Windows). Run the downloaded installer and follow the instructions on

your screen. Once VS Code is installed, you can launch it just like any other application on your computer.

Adding the Python Extension

When you open VS Code, you should see a list of icons on the left side of the screen. One of these icons is labeled Extensions when you hover over it. Click this icon and search for **Python** to find and install the Python extension. This extra installation includes features like a Python formatter and debugger, which will make it easier to write Python in VS Code.

Testing the Installation

To make sure VS Code is working with Python, let's create a new Python file. We'll write a script that retrieves a joke from an external source and prints it out. In VS Code, choose **File ► New File**. Enter *joke.py* as the filename (the *.py* extension indicates the file contains Python code) and choose a location to save it. You'll then be taken to an editor window for the new file. Fill in the following code:

```
import requests

url = "https://joke-api.pythonforexcelusers.com"
response = requests.get(url).json()
setup = response["setup"]
punchline = response["punchline"]
print(setup)
print(punchline)
```

This script uses a Python library called Requests to retrieve a random joke from the URL <https://joke-api.pythonforexcelusers.com>. Then it prints out the setup and the punchline of the joke separately. You don't need to understand exactly what this code is doing, but it's useful to test your setup if you can get to the point where it runs as expected in VS Code.

As you type, you'll notice that VS Code highlights the code in different colors to help show its syntax. The editor will also flag any problematic code, using a squiggly underline. For example, VS Code will warn you if you reference a library in your code that you haven't already installed. In this case, since you probably haven't installed the Requests library yet, you should see `requests` underlined in the first line of the script. You'll need to use `pip` to install this package before your *joke.py* script will run. To do that, enter the following on the command line (replacing `pip` with `pip3` if necessary):

```
pip install requests
```

Now you should be ready to run your script, and for that VS Code needs to use your Python interpreter (which we installed previously). Upon installation of the Python extension, VS Code tries to automatically detect the available Python interpreters on your system. If your version of Python is

included in the PATH environment variable, this should work automatically. However, it's common to have issues at this step.

If VS Code found the Python interpreter on its own, you should see something like *Python 3.x.x 64-bit* in the lower right of the screen, with your specified Python version number filled in. If you don't see this, you'll need to point the Python extension to the location of your interpreter.

To do this, first open the VS Code settings window by pressing CTRL-, on Windows or ⌘, on macOS. Open the **Extensions** drop-down menu and click **Python**. Then scroll down to the Default Interpreter Path field. In it, put the path to the executable file for your Python interpreter. The exact path will depend on where Python is stored in your computer. For example, on Windows this could be *Users\<YourUsername>\AppData\Local\Programs\Python\Python3X\python3.exe*, or on macOS it could be */Library/Frameworks/Python.framework/Versions/3.x/bin/python3.exe*. After you change this field, VS Code should automatically sync the changes, and the Python extension will use the specified interpreter.

Once VS Code can see your interpreter, run your script. To do this, with your script open in the editor, click the triangle icon in the upper right that looks like a play symbol. The Terminal window should now print a random joke, like this:

```
Why did the worker get fired from the orange juice factory?  
Lack of concentration.
```

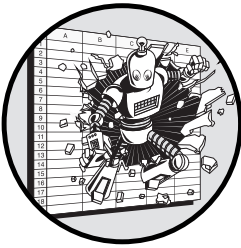
The exact joke you get back may vary.

Conclusion

This chapter introduced the basics of working with a CLI, walked you through the process of installing Python and VS Code, and showed you how to use your CLI to troubleshoot any problems with the installation. Now that you've properly set up your Python environment, you can focus on learning to use code to solve problems. It's as if you've now organized your kitchen cabinet and are ready to make dinner without having to worry about any cans of beans falling in your face.

2

CODING FUNDAMENTALS EXPLAINED THROUGH EXCEL



In this chapter, we'll explore several fundamental Python concepts by explaining how they relate to Excel. We'll start with the basics of writing and formatting code, and then we'll cover the most common data types you'll find in Python, including numbers, strings, dictionaries, lists, and dates. We'll begin working with operators and functions and start using conditionals to make decisions in your code.

Learning to code can be intimidating. You need to understand many unfamiliar concepts, and this can be hard for people starting from scratch because they don't have an existing mental framework for how to think about them. But fortunately, you aren't starting from scratch: Excel has already given you some experience with many concepts from the world of programming. We'll leverage that experience in this chapter to get you writing real code quickly.

What Is Code?

Writing code amounts to taking the intangible requirements of a problem you're trying to solve and converting them into a tangible series of steps that a computer can follow. Even for humans, it's much easier to follow specific, tangible instructions like "stand up straighter" than abstract, intangible ones like "be more confident." Tangible instructions are already actionable, while intangible concepts require an extra step of interpretation. When you write code, you're doing that interpretation so you can communicate actionable instructions for the computer. You're really just filling in the gaps between "be more confident" and "stand up straighter."

Your instructions for the computer are created using a *programming language*, a vocabulary and set of grammatical rules for writing code. Developers and computer scientists have designed a wide variety of programming languages over the years that are suitable in different contexts and for different purposes. Python is just one of them.

Every programming language falls somewhere on a spectrum from *low level* to *high level*. The lowest level is *machine code*, which uses a binary system consisting of just two symbols, 0 and 1. These binary digits, or *bits*, are typically grouped into larger units known as *bytes* (which usually include eight bits). Each byte represents a specific instruction that a computer's central processing unit (CPU) can understand and execute. These instructions cover a wide range of actions, but they generally involve some variation of storing, moving, or updating the information in the computer's memory. At the most basic level, all the tasks we give our computers are made up of these 0s and 1s. Every click and keystroke can be broken into combinations of tens, hundreds, or millions of those two digits. In this way, binary machine code forms the basis of all computing, from the simplest operations to the most complex algorithms.

Higher-level programming languages abstract away the 0s and 1s of machine instructions into a more human-readable format, allowing coders to think more intuitively about the problems they're trying to solve. Behind the scenes, the language has tools to translate the higher-level code into the necessary binary instructions representing the specific actions the computer must take. Different languages offer different degrees of abstraction, making them more or less "high level." The following list ranks types of programming languages from most primitive to most abstract:

1. *Machine code* is the most primitive (least abstract) level of code, consisting of binary instructions that are directly executed by the computer's CPU.
2. *Assembly language* is the next level of abstraction above machine code. It's specific to the computer's architecture, so it uses symbolic representations to make the code more readable, while still being closely related to machine code.
3. *Intermediate-level languages* don't provide direct access to the computer's architecture like assembly language, but they do allow for

detailed control over memory and system processes. Some examples are C and C++.

4. *High-level languages* are more abstracted from the hardware and designed around readability. They automatically handle much of the memory management and other low-level tasks. Python is a high-level language, and so are JavaScript, Java, C#, and PHP.
5. *Interactive applications* are software tools that provide a user-friendly interface for specific tasks. These tools abstract away the underlying programming complexity altogether, focusing instead on task efficiency and usability. Excel falls in this category.

Although Excel is an interactive application, it still heavily features many of the concepts found in high-level languages like Python, albeit with a bit more abstraction. When you're learning Python, the gap that you'll need to bridge therefore might not be as large as it seems at first. We'll leverage this connection in the following sections, de-abstracting some of the coding concepts found in Excel and translating them to Python so you can start learning to code through Excel's familiar lens.

Objects and Variables

On the most basic level, code instructs a program to make decisions based on data. That data consists of *objects*, which are essentially any entities that hold information. When you code, all you're doing is manipulating objects in different ways to achieve your desired outcome. In Excel, you're essentially doing the same thing: manipulating information stored in cells.

In Excel, each cell has a name indicating where it lives within a worksheet, like A1, B2, and so on. These names make it easy to identify and reference a cell and the information within it. Similarly, when you're working with objects in Python, it's easiest to give them names. Those names are *variables*, which act as references to data (objects) stored on your computer.

Variables are containers for data. The concept of a *container* is common in coding and works much as it does in the physical world: A container is a place to store something, whether it's a box, a bag, or in Excel's case, a cell. Just as physical containers come in various types (soft and hard, large and small), and Excel cells can contain various types of information (numbers, text, dates, formulas), Python variables can represent a wide range of types, from simple numbers and text to more complex structures like lists or dictionaries (which we'll cover in this chapter), and even custom types of objects (which we'll explore in Chapter 11).

Each object in Python, much like an Excel cell, has certain attributes. *Attributes* are specific properties or characteristics of an object that can store additional information or provide functionalities. For example, an object representing a date might have separate attributes that store the year, month, and day. You can think of attributes as the details or traits that describe the object and allow you to interact with it in more nuanced ways. By accessing

and manipulating these attributes, you can perform more complex operations on the objects in your code, much as you might use different functions and formulas to manipulate data within an Excel cell.

Rules of Python Syntax

Just like any language, Python has rules around its syntax that govern how the Python interpreter understands what you're telling it to do. These rules ensure that your code executes and that there isn't any ambiguity in the instructions you're looking to communicate to your computer. Let's go over some of these rules.

Case Sensitivity

When you're writing formulas in Excel, it doesn't matter whether you write in uppercase or lowercase; Excel will treat SUM and sum the same. In Python, however, case matters. For instance, consider the following lines of code:

```
x = 7
X = 5
```

These two lines might look the same at first glance, but in Python, lowercase `x` and uppercase `X` are considered completely different identifiers. This means that `x` holds the value 7, while `X` holds the value 5. This case sensitivity allows for more flexibility in the names you can use to refer to things, but it also requires more attention to detail, since using the wrong case will result in an error.

Indentation

Code in Python is organized into *blocks*, which are groups of statements that are executed together as a unit. Code within the block is treated as a single logical section that runs under a specific condition or in response to a particular command.

To differentiate between blocks of code, Python relies on the indentation of the blocks to be consistent. Each time you start a new block, you indent the code that belongs to it. The standard practice is to use four spaces per indentation level. Here's an example with an indented block:

```
if x > 0:
    print("x is positive")
    print("This line is part of the if block")
```

Both lines beginning with `print` are indented, signaling that they're part of the block starting with `if`. If you didn't indent them, Python wouldn't know that they're grouped together or when to run them. (We'll discuss `if` statements, like the code in this example, in "Conditionals" on page 41.)

Other than the indentation at the start of a line, the interpreter ignores additional whitespace between pieces of code, like spaces, tabs, or line breaks.

This means that you can strategically use blank lines and spaces to make your code more readable.

Comments

Another way to make your code more readable is to use *comments*, which in Python are marked with a hash mark (#). Comments allow you to leave notes in your code that Python will ignore when running the program. Here's an example:

```
# This is a single-line comment explaining the code below
x = 3
y = 2 # Another comment explaining this specific line
```

Here, `x = 3` and `y = 2` are pieces of executable code. The first line of the listing, and everything after the # on the last line, is a comment and will be ignored.

For longer explanations, you can use multiline comments by enclosing the text in triple quotes (`"""`). These kinds of comments are called *docstrings*:

```
"""
This is a multiline comment, or a docstring.
It's often used to describe what a block of code does.
"""

def add(x, y):
    return x + y
```

Using comments effectively can make your code much easier to understand, both for others who might read it and for yourself when you come back to it later.

Statements and Assignments

A line of Python code that performs an action is called a *statement*. Here's a simple statement:

```
print("Print this line!")
```

This statement tells Python to output the text `Print this line!` in the console. It's a single action contained in one line of code.

One specific kind of statement is an *assignment statement*, which is used to set the value of a variable. Assignment statements will always have an equal sign (=), like so:

```
x = 10
y = x + 5
```

In these examples, `x = 10` and `y = x + 5` are assignment statements. The `=` symbol tells Python to store the value on the right side of the symbol in the variable named on the left side. You can then use the variable to represent

the value later in the code. After the two assignment statements shown here, the variable `x` holds the value 5, and `y` holds the value 15.

Testing Examples in VS Code

You can (and should) test each of the examples in this chapter by using VS Code, which you installed in Chapter 1. For each example, create an empty `.py` file, add the code snippet you'd like to test, and run it using VS Code's integrated terminal.

When learning to code, it's tempting to copy and paste whole examples and then try to understand what they're doing after the fact, but this isn't the best approach. Manually typing out each example helps many people intuit the logic more efficiently. In each case, write and test the code on your own, and maybe even tweak it a little to help it better fit your personal use cases for Python.

It's easy to get lost without a visual interface showing the data and variables you're working with, but print statements are an easy tool for pulling back the curtain on your scripts. Anytime a script isn't giving you the expected result, use `print()` with the code you want to understand inside the parentheses to see what's going on. For example, in this code snippet we find the average of a list of numbers by taking the sum and dividing by the length of the list:

```
numbers = [1, 2, 3, 4, 5]
total = sum(numbers)
count = len(numbers)
average = total / count
```

To dissect what's going on at each point in the script, let's add some print statements to display the value of each variable (`total`, `count`, and `average`) in the VS Code terminal:

```
numbers = [1, 2, 3, 4, 5]
total = sum(numbers)
print(total)
count = len(numbers)
print(count)
average = total / count
print(average)
```

When you run this in VS Code, your console will display the values inside the parentheses of each of your print statements in order, enabling you to see what Python sees inside each variable:

```
15
5
3.0
```

In this simple code snippet, the print statements may seem like overkill, but they become an invaluable tool to get you back on track when the code gets more complicated and you get a little lost. When in doubt, print!

Basic Data Types

Each type of object in Python is designed to hold a specific kind of data. This is known as the object's *data type*. Using the right type ensures that your data is stored and manipulated correctly; just as you wouldn't store hot coffee in a suitcase, you don't want to store information in an object of the wrong type.

In this section, we'll consider some of Python's basic data types: Booleans, numbers, strings, and None, which represents the absence of a value. In each of these cases, the object's type determines how it can be used and manipulated and how much space it consumes on your machine.

Boolean Variables

Booleans are variables that can take on only two values: True or False. They're useful in situations that are entirely black-and-white, where only two values are possible. Here are some examples of Boolean variable assignment in Python:

```
is_alive = True
is_married = False
did_vote = True
went_to_gym_today = True
```

Because Booleans have only two possible values, they take up very little space in memory (only a single bit). Just as you don't need a large suitcase when all you're packing is a pair of sunglasses, Python doesn't need to allocate lots of bits of space when you need only one.

You've likely used Booleans in Excel before. Therefore, you may have seen them treated as integers, with 1 for TRUE and 0 for False. For example, to count the number of times TRUE occurs in a column, you can simply take the sum of the column's entries, without having to explicitly convert the column's values to numbers.

Python handles Booleans similarly. The Boolean values True and False are treated as 1 and 0, respectively, when they're used in arithmetic operations or evaluated in any situation that requires a numeric value. This implicit conversion allows Booleans to be used interchangeably with integers in most cases.

Integers and Floats

Python has two primary numeric data types: *integers*, which represent whole numbers, and *floats*, which represent numbers that include fractions or decimals. Integers are used primarily for counting discrete items, like the number

of visitors to a website, the score of a soccer game, or the relative position of an item in a list. None of these would ever be a fraction. Here are some examples of integer variables in Python:

```
count = 42
temperature = -5
year = 1987
```

Floats, on the other hand, are useful when precise measurement or computation is required, so it's likely that you'll need to store fractional values. Here are some examples of float variables in Python:

```
interest_rate = 3.75
average_temperature = 23.5
pi_approximation = 3.14159
earth_radius_km = 6371.0
apple_stock_price = 199.99
```

As you can see in these examples, to create a numeric variable in Python, you don't need to specify which type you intend to use. Python automatically detects whether you're working with an integer or a float based on the way you write the number. For example, if you write `x = 5`, Python recognizes `x` as an integer. If you write `y = 5.0`, Python knows that `y` is a float. Excel works similarly; when you enter a number in a cell, Excel automatically determines whether it's an integer or a float based on whether you include a decimal point.

Even though Python is automatically able to differentiate between these two types, being aware that they exist is still important because in Python, as with Excel, certain operations behave differently depending on whether you're working with an integer or a float.

Some programming languages, like C, have limits on the size of integers. In Python, however, the size of integers is dynamically managed, meaning Python allows for the storage of integers as large as the available memory in the system permits. This flexibility comes with trade-offs in terms of memory usage and performance. On the other hand, because a computer's memory isn't infinite, floats have restrictions on their precision, meaning there's a limit to how accurately a fractional number can be represented. Some numbers, like $1/3$, can never be accurately represented in decimal form, no matter how many digits you include. Floats in Python will always approximate these values.

Strings and Characters

Strings are representations of text, made up of individual characters. You might use strings to record names, addresses, or messages. Some examples of strings in Python are as follows:

```
my_favorite_condiment = "Peanut Butter"
full_name = "Bill Gates"
url = "www.pythonforexcelusers.com"
```

In Excel, when you input letters into a cell, they'll automatically be treated as a string. In Python, however, the characters in a string must be enclosed in quotation marks. You can use either single quotes (') or double quotes (") to define strings. The choice is mostly a matter of personal preference.

It's common to want to integrate the value of a variable or expression into a string. Python provides a convenient tool for this called *f-strings*. Any code enclosed in curly brackets ({}) inside an f-string will be evaluated as a Python expression and the result inserted into the string. The letter *f* before the opening quotation mark indicates that the string is an f-string. Consider this example:

```
full_name = "Bill Gates"
greeting = f"My name is {full_name}."
print(greeting)
```

For the f-string stored in the greeting variable, the contents inside the curly brackets will be replaced with the value of the `full_name` variable, leading to the following greeting:

```
My name is Bill Gates.
```

The individual characters that make up each string are themselves another Python data type. Characters have a limited number of values; the range of options is based on the *encoding*, or character mapping system, that you're using.

Encodings are necessary because computers inherently understand only numbers, not letters. For a computer to be able to interpret what exactly an *A* is, there must be a standardized system to translate it into a unique numerical code and ensure that text is consistently represented and readable across different programs. Common character encodings include American Standard Code for Information Interchange (ASCII), which covers only English characters and symbols, and Unicode, a more comprehensive encoding standard that supports a vast array of characters from multiple languages and scripts around the world.

Python by default uses Unicode. This allows Python programs to handle a wide variety of text, including symbols, emojis, and non-English writing systems. Excel, on the other hand, uses system-specific character encodings, depending on the operating system's default settings. This can cause issues when you're collaborating with users from all over the world with different default settings. For example, if you share a spreadsheet on a system that uses a Chinese encoding (like GB2312) and includes Chinese characters, these characters may not display correctly when the spreadsheet is opened on a system with a different default encoding.

None

In Excel, a blank cell is like an empty container or a variable that hasn't been assigned a value. It's essentially a placeholder that can store data but currently doesn't contain anything. Python has a similar concept called *None*, a data type that represents the absence of a value. For example, in Python, you might assign a variable to *None* like this:

```
x = None
```

This assignment indicates that *x* currently holds no value, much like an empty cell in Excel. You can later assign a value to this variable, just as you would enter data into an empty cell.

Like blank cells in Excel, *None* serves an important role. It helps manage situations where a value is intentionally absent, ensuring that your code (or spreadsheet) handles these cases appropriately without causing errors.

Operators

An *operator* is a symbol representing a specific action to perform on one or more objects. You've probably known about operators since long before you learned Excel: In school, for example, you were introduced to addition (+), subtraction (−), multiplication (×), and division (÷), and also less than (<), greater than (>), and equals (=). Languages like Python have operators for more tasks than just simple arithmetic. This section covers the types of operators, how they're used in Python, and their Excel equivalents.

Arithmetic Operators

Arithmetic operators are used for mathematical operations like addition and subtraction. Table 2-1 shows common arithmetic operations, an example Excel formula using that operation, and the equivalent expression in Python.

Table 2-1: Arithmetic Operations

Operation	Excel formula	Python equivalent
Addition	=A1 + A2	a + b
Subtraction	=A1 - A2	a - b
Multiplication	=A1 * A2	a * b
Division	=A1 / A2	a / b
Exponentiation	=POWER(A1, A2)	a ** b
Modulus	=MOD(A1, A2)	a % b

In math, established rules govern how to use arithmetic operators to manipulate numbers. In Python, the rules around these operators are modified so that they work intuitively with different data types. For example, in math, 5 + 3 simply adds two numbers and yields 8. However, when you apply Python's + operator to two strings, like "Peanut" + "Butter", the operator's

meaning changes. It no longer means “sum two values” but instead “concatenate two strings.” Excel takes a more rigid approach. When you try to apply the + operator in Excel on two cells containing text, you’ll get an error.

In programming, allowing the same operator to behave differently depending on the context and the type of operands it’s working with is called *polymorphism*. Polymorphism can make coding more intuitive and efficient, since you need to remember only one operator, not different operators for every data type. This makes it more straightforward to perform operations that are intuitively the same but are technically implemented differently across various types, as is the case when Python’s + operator is applied to numbers (for summing) versus strings (for appending the second string to the first).

Comparison Operators

Comparison operators assess the relationship between two values. You’ve probably used comparison operators like > and < in Excel as part of conditional formulas like IF or COUNTIF. These sorts of comparisons all result in Boolean true/false values, making them useful when you need to treat data differently based on certain criteria. Table 2-2 gives an overview of the various comparison operators in Excel and their Python equivalents.

Table 2-2: Comparison Operations in Excel and Python

Operation	Excel formula	Python equivalent
Equal to	=A1 = A2	a == b
Not equal to	=A1 <> A2	a != b
Greater than	=A1 > A2	a > b
Less than	=A1 < A2	a < b
Greater than or equal to	=A1 >= A2	a >= b
Less than or equal to	=A1 <= A2	a <= b

Notice that Python uses two equal signs (==) for its equality operator. This is to disambiguate between *assigning* a value to a variable (=) and *comparing* two values (==). Without this distinction, it may not be clear whether something like x = 7 is meant as a question or a statement. Are you asking the computer to determine whether x is 7, or are you telling it to set the variable x to 7? It’s the difference between “The sky is blue?” and “The sky is blue.” By contrast, Excel uses a single equal sign for both comparison and assignment. The placement of the equal sign indicates its meaning: An equal sign at the beginning of a formula always signifies assignment, whereas any equal sign within the formula signifies comparison.

Logical Operators

Logical operators take in one or more True/False as input, combine or modify them, and output a single True/False result—for example, determining if

both one comparison *and* another are true, or if either one comparison *or* another is true. Table 2-3 shows equivalent logical operations in Excel and Python.

Table 2-3: Logical Operations in Excel and Python

Operation	Excel formula	Python equivalent
And	=AND(A1>0, B1<5)	a > 10 and b < 5
Or	=OR(A1>10, B1<5)	a > 10 or b < 5
Not	=NOT(A1=5)	not a == 5

The Python expression `a > 10 and b < 5` in the table can be broken into the `and` logical operator and its two sides, `a > 10` and `b < 5`. Both of these sides evaluate to either `True` or `False`. In this way, the logical operator takes in two Boolean expressions and evaluates whether both are true (`and`) or one or the other is true (`or`).

Composite Data Types

Unlike basic data types like Booleans, integers, floats, and strings, *composite data types* are designed to organize and manage collections of multiple values. Python's composite data types include lists, tuples, and dictionaries. Each of these types of objects can contain a group of data points, but each has slightly different rules governing how the data is stored and how your code interacts with it.

In Excel, the analogy for these composite data types would be ranges of cells. Just as a list or dictionary can hold multiple values in Python, a range in Excel can encompass multiple cells, each containing its own data point, and all organized in a sequence. Keeping cells that hold similar information within the same range, like a column, makes it efficient to perform the same calculation on all the cells in the range at once. Python's composite structures enable the same efficiency.

Lists

A *list* is an ordered collection of elements. Having an order makes this data structure useful when you might want to access specific elements by their position in the list, or when the relative placement of each item carries importance, such as a list of steps in a recipe or a ranking of athletes' performance after a sporting event. In Python, the most basic implementation of a list is the aptly named list structure. Lists are defined within square brackets, with the list's values separated by commas. Here are some examples of lists created in Python:

```
my_numbers_list = [1, 2, 3, 4]
my_strings_list = ["Peanut Butter", "Jelly", "Sandwich"]
my_mixed_list = [1, 2.5, "Peanut Butter", True, [3, 4, 5]]
```

As you can see in `my_mixed_list`, Python lists don't necessarily need to contain elements of all the same data type. You can include a mix of integers, strings, Booleans, and other data types within a single list. However, in many cases, keeping the data type consistent over the entire list is preferable. When the data type is consistent, you can predict the behavior of operations or functions applied to each element in the list. For example, if you have a list of only numbers, you can safely perform arithmetic operations on its elements without worrying about data-type conflicts.

Each element in a list has an *index*, an integer corresponding to its position in the list. Python indexes lists starting from 0, so the index 0 refers to the first element in the list, the index 1 refers to the second element, and so on. To retrieve an element from a list, place its index in square brackets after the name of the list. For example, here's how to retrieve the first element from `my_strings_list` created in the previous snippet:

```
my_strings_list[0]
```

This returns the string `Peanut Butter`. Here, we retrieve the second element:

```
print(my_strings_list[1])
```

This returns `Jelly`. We can retrieve the final element, `Sandwich`, like so:

```
print(my_strings_list[2])
```

Lists also support *negative indexing*, which uses negative numbers to count from the end of the list. Index -1 indicates the last element in the list, -2 indicates the second-to-last element, and so on. To retrieve the last element, the string `Sandwich`, from `my_strings_list`, you could therefore use this:

```
print(my_strings_list[-1])
```

Indexing from 0 may take some getting used to, since unlike Python, Excel indexes rows and columns starting from 1. For example, the formula `=INDEX(A1:B5, 1, 1)` will return the value in the first row of the first column of the range A1 to B5. Zero-based indexing is common in programming languages, however, and becomes more natural the more you use it.

Tuples

A *tuple* is an ordered collection of elements, similar to a list. However, unlike lists, tuples are *immutable*: changing the values in a tuple is impossible without re-creating it entirely. This immutability makes tuples useful when you don't plan on adding elements, removing elements, or updating elements after the object is created. Tuples provide a sense of security when you want to guarantee that the information stored within the tuple remains unchanged.

Tuples are defined by placing the elements inside parentheses, separated by commas. Here's an example:

```
sandwich_contents = ("Peanut Butter", "Jelly")
```

Just like lists, tuples can contain elements of different data types, including integers, strings, Booleans, and even other lists or tuples. Each element has an index, which starts at 0, also as in a list. You can access elements in a tuple by index via the same square-bracket notation that applies to lists:

```
sandwich_contents[0]
```

This returns the string `Peanut Butter`, the first element in the `sandwich_contents` tuple.

Dictionaries

Dictionaries (*dicts* for short) are collections of named values. They consist of *key-value pairs*, where each key is a unique name assigned to a value. Unlike lists, whose elements are accessed by their numeric index, values in dicts are accessed using their unique keys. Dicts are one expression of a broader programming concept called a *hash table*, a data structure that maps keys to values. Dicts and other hash tables are useful in three situations:

- When each value in a collection has a unique name or identifier
- When the order of the elements in the collection doesn't matter
- When quick access to the data is necessary

The efficiency of hash tables stems from their ability to perform operations like insertion, deletion, and retrieval of values near instantly. However, unlike lists, hash tables don't maintain an order among their elements. Therefore, if you retrieve the elements of a dict one by one, you may not necessarily retrieve them in the same order every time.

In Python, dicts are demarcated using curly brackets. These brackets contain a comma-separated series of key-value pairs, where each key is linked to a value with a colon. For example, to create a dict called `my_dict` with the keys `a`, `b`, and `c` mapped to the values `1`, `2`, and `3`, you would write:

```
my_dict = {  
    "a": 1,  
    "b": 2,  
    "c": 3  
}
```

Now that the dict has been created, accessing one of the elements is straightforward: Place the element's key in square brackets after the dict name. For example, here's how to access the value at key `c`:

```
my_dict["c"]
```

This returns `3`.

Because dicts don't have a fixed order to their key-value pairs as the elements in lists and tuples do, you can't reference keys or values based just on their location in the dict when you created it.

Functions and Methods

Just like formulas in Excel, a *function* is a self-contained, reusable piece of code that accomplishes a particular task. This should be a familiar concept, as Excel provides lots of built-in functions like SUM, MAX, and COUNT.

Functions in Excel operate by taking in information (often as cell references or ranges) and returning a result, such as a sum, average, or maximum value. Similarly, functions in Python take in information—called *arguments*—and return a result based on the operations defined within the function. Each built-in function in Python can be used to perform a specific task, such as printing text, calculating the length of an object, or converting between data types. Table 2-4 gives some of the most common functions in Python.

Table 2-4: Common Functions in Python

Function	Description	Example
print()	Outputs text or variables to the console	print("Hello!")
len()	Returns the length (number of items) of a composite object	len([1, 2, 3])
type()	Returns the data type of an object	type(42)
int()	Converts a value to an integer	int("123")
str()	Converts a value to a string	str(123)
range()	Generates a sequence of numbers	range(0, 10)
sum()	Returns the sum of all items in a list	sum([1, 2, 3, 4])
max()	Returns the largest item in list	max(10, 20, 5)

Each of these functions can take arguments within the parentheses after the function name and use those arguments to perform its task. Sometimes the arguments have default values and don't need to be explicitly specified. For example, the `sum()` function can be called with just a single argument, such as a list of numbers, like `sum([1, 2, 3, 4])`, which will return 10. Or it can be called with an optional second argument that specifies a starting value for the sum. For example, `sum([1, 2, 3, 4], 10)` would return 20 because it adds the starting value 10 to the sum of the list. When this second argument isn't provided, `sum()` defaults to using 0 as the starting value, so `sum([1, 2, 3, 4])` (which returns 10) is effectively the same as `sum([1, 2, 3, 4], 0)`.

Keeping track of the details of every Python function can get tedious, but the language's online documentation can help. For any given function, look to the documentation for information about the specific arguments it accepts and their default values, if any.

The functions listed in Table 2-4 are stand-alone: They exist independently of any objects or variables in your code. Other functions, known as *methods*, are associated with a specific type of object and apply to variables of that type. For example, there are methods associated with strings, methods associated with lists, methods associated with dicts, and so on. In Python, methods are called via *dot notation*: you start with an object or variable, followed by a dot (`.`), followed by the name of a method associated with the

specified object. For example, if you have a string object called `text` and you call `text.upper()`, you're applying the `upper()` method to the string stored in the `text` variable. This method capitalizes all the letters in the string.

Common String Methods

String methods are functions that operate on string objects. Table 2-5 lists some of the most common methods for manipulating strings.

Table 2-5: Common String Manipulations in Python

Method	Description	Example
<code>replace()</code>	Replaces part of a string with another string	<code>"Peanut Butter".replace("Butter", "Jelly")</code>
<code>split()</code>	Splits a string into a list of substrings	<code>"Peanut,Butter".split(",")</code>
<code>strip()</code>	Removes leading and trailing whitespace	<code>" Peanut Butter ".strip()</code>
<code>upper()</code>	Converts a string to uppercase	<code>"Peanut Butter".upper()</code>
<code>lower()</code>	Converts a string to lowercase	<code>"Peanut Butter".lower()</code>

Suppose you have a string that contains the phrase `Good Morning`, and you want to change it to `Good Evening`. You can use the `replace()` method as follows:

```
greeting = "Good Morning"
new_greeting = greeting.replace("Morning", "Evening")
```

Now you have a new variable called `new_greeting` that contains the string `Good Evening` instead.

Common List Methods

List methods are functions that operate on list objects. Table 2-6 shows some of the most common methods for manipulating lists.

Table 2-6: Common List Operations in Python

Method	Description	Example
<code>append()</code>	Appends elements to the end of the list	<code>my_list.append(4)</code>
<code>insert()</code>	Inserts elements at a specific position	<code>my_list.insert(1, 5)</code>
<code>remove()</code>	Removes elements by their value	<code>my_list.remove(3)</code>

Suppose you have a list of numbers `[1, 2, 3]` and want to add the number 4 to the end of the list. You can use `append()` like so:

```
my_list = [1, 2, 3]
my_list.append(4)
```

After calling `.append()`, `my_list` will contain the elements `[1, 2, 3, 4]`.

Common Dict Methods

Dictionary methods are functions that operate on dict objects. Table 2-7 gives some of the most common methods for working with dicts.

Table 2-7: Common Dictionary Operations in Python

Method	Description	Example
keys()	Returns all the keys in the dictionary	keys = my_dict.keys()
values()	Returns all the values in the dictionary	values = my_dict.values()
items()	Returns all key-value pairs in the dictionary	items = my_dict.items()
get()	Retrieves the value for a given key	value = my_dict.get("a")
update()	Updates the dictionary with new key-value pairs	my_dict.update({"d": 4})
pop()	Removes and returns the value for a specified key	value = my_dict.pop("a")

Suppose you have the following dictionary:

```
my_dict = {"a": 1, "b": 2, "c": 3}
```

You want to add {"d": 4} as a new key-value pair to the dictionary. You can use the `update()` method as follows:

```
my_dict.update({"d": 4})
```

After this, `my_dict` will include the new key-value pair.

Conditionals

Conditionals are statements that give alternative paths forward in a program based on whether a test is found to be true or false. They're a fundamental component of any computer program, since they enable the program to respond dynamically to different inputs or situations.

Conditional statements are an element of *control flow*, the determination of when and how a script executes instructions. Other elements of control flow include loops, functions, and exceptions, which we'll discuss in later chapters.

This concept of control flow is central to coding but foreign to Excel. In Excel, you use your cursor to select specific cells for editing. You can move it up, down, left, or right, select a range of cells, or jump to a different worksheet. The position of the cursor indicates the location on your spreadsheet where you're currently working.

By contrast, when a Python script runs, you have no cursor to explain what to do when. Therefore, you write the script so that each command executes when and how you intend it to. In other words, control flow refers to the path the Python interpreter takes through your program as it is guided by code elements like conditional statements.

By default, without any changes to control flow, the interpreter executes code in a script sequentially. Consider this simple example:

```
print("First line of code")
print("Second line of code")
print("Third line of code")
```

The Python interpreter will execute these lines in the order in which they appear, resulting in the following output:

```
First line of code
Second line of code
Third line of code
```

As we'll see, conditionals allow you to override the normal sequential execution, creating more-complex paths through the code.

if Statements

An if statement in Python modifies control flow based on a condition: When the interpreter comes across an if, it determines whether the condition is true and executes the code inside the if block only if so. Otherwise, the code gets skipped. To demonstrate, let's enhance the previous example by adding an if statement:

```
print("First line of code")
print("Second line of code")

x = 5
if x > 3:
    print("x is greater than 3")

print("Third line of code")
```

With this example, after printing the first two lines, the script checks whether x (which has been assigned the value 5) is greater than 3. Since this condition is true, the script executes the print statement inside the if block, printing x is greater than 3. Then the code continues with the sequential execution of the remainder of the script, yielding the following output:

```
First line of code
Second line of code
x is greater than 3
Third line of code
```

Notice that the if statement's condition, $x > 3$, is followed by a colon. The contents of the if block—that is, the code that should be executed if the condition is true—must be indented.

The Excel IF function works similarly to a Python if statement. For example, the Excel equivalent of the if statement in the previous example would be something like this:

```
=IF(A1>3, "x is greater than 3", "")
```

This sets the current cell to `x is greater than 3` if the value in cell A1 is greater than 3, or leaves the cell empty (""), otherwise. The primary difference here is that Excel's IF function operates within only a single cell, making it poorly equipped for constructing more complex conditions or triggering a more elaborate series of actions if a condition is true.

Python, on the other hand, allows for more-complex conditional structures with clearer syntax (as the coming examples will show), and it enables the execution of larger code blocks within each condition: Just add as many indented lines of code after the if statement as you need. In this way, Python provides a more readable format for complex decision-making processes, making it suitable for a greater variety of tasks.

else Clauses

In Python, an else clause can be added to an if statement to define an action that should be executed when the if condition is not met. This clause is optional and provides an alternate path for the control flow. For example:

```
x = 2
if x > 3:
    print("x is greater than 3")
else:
    print("x is not greater than 3")
```

In this case, since `x` isn't greater than 3, the else block is executed, and `x is not greater than 3` will be printed. Like the if block, the else block is set off by a colon and indented.

elif Clauses

Python elif clauses, short for *else if*, allow for multiple, sequential conditions to be checked. One or more elif clauses can sit between an initial if statement and a final else clause. Each elif tests an additional condition if the preceding if (or another elif) condition was false. Once a true condition is found, that block executes, and any remaining elif blocks are skipped (along with the final else). Here's an example:

```
x = 4
if x > 5:
    print("x is greater than 5")
```

```
elif x > 3:
    print("x is greater than 3 but not greater than 5")
else:
    print("x is 3 or less")
```

Here, *x* is greater than 3 but not greater than 5, so the first condition (*if x > 5*) is false, but the second condition (*elif x > 3*) is true. Therefore, *x* is greater than 3 but not greater than 5 is printed.

Nested Conditionals

When working with conditionals in Python, you often need to check multiple conditions that depend on one another. Known as *nested conditionals*, these allow you to create complex decision-making logic in a clean and organized way. Nested conditionals in Python are much more readable and manageable compared to similar logic in Excel's environment. If you were to use a nested conditional with Excel's IF function, where you're confined to the context of an individual spreadsheet cell, you'd end up with a cumbersome and hard-to-follow formula.

Say you're writing code to determine whether a person shopping at a store is eligible for a discount based on their age and membership status. The conditions are as follows:

- If the person is under 18, they are eligible for a discount.
- If the person is 65 or older, they are also eligible for a discount.
- If the person is a member, they are eligible for a discount regardless of age.
- If the person is between the ages of 25 and 30, they also receive a special promotional discount designed to attract new shoppers of that demographic.

In Python, you can express this logic with nested conditionals:

```
age = 28
is_member = False

if is_member:
    eligible_for_discount = True
else:
    if age < 18 or age >= 65:
        eligible_for_discount = True
    else:
        if 25 <= age <= 30:
            eligible_for_discount = True
        else:
            eligible_for_discount = False
```

In this example, the code clearly shows the decision-making process. It's easy to follow which decisions the code will make based on which conditions are met. Now let's try to implement this same logic in Excel:

```
=IF(OR(A1<18, A1>=65), TRUE, IF(B1=TRUE, TRUE, IF(AND(A1>=25, A1<=30), TRUE, FALSE)))
```

While Excel handles these nested conditions, the formula becomes increasingly hard to read and maintain as complexity grows. Python's nested conditionals, in contrast, allow you to structure your logic in a more intuitive and readable way, making it easier to understand and identify the source of any errors that might arise.

Representing Dates and Times

Dates and times are a nearly universally important data type. Almost any process requires some sort of handling of dates and times, whether it's reading them, reformatting them, comparing them, grouping them, or otherwise manipulating them. On the surface, they seem simple, but handling them effectively requires consideration of complex concepts like days of the week, leap years, time zones, daylight saving time, holidays, and the nuances of different formats. Let's consider how Excel handles temporal information and then look at how Python does it better.

Internal Representation of Dates in Excel

Excel handles dates by representing them as a whole number of days since December 31, 1899. For example, January 1, 1900, is represented as 1; January 2, 1900, is 2; and January 2, 2000, is 36,527. Time is represented as fractions of dates. For example, 6:00 AM is represented as 0.25 because it's one-quarter of the way through a 24-hour day, so 6:00 AM on January 2, 2000, becomes 36,527.25.

Luckily, Excel reformats these numbers to be more readable for the user. This system allows for readable dates while also facilitating date arithmetic, such as adding days or calculating the difference between dates: You simply do addition or subtraction with the internal numeric representations of the dates. That said, Excel's system also has its limitations.

For one, since there's no default time-zone awareness in Excel, conversions between time zones and accounting for daylight saving can be error prone. As an example, because the East Coast of the United States observes daylight saving, the second Sunday in March actually has 23 hours, not 24, while the first Sunday in November has 25. Excel doesn't handle these anomalies by default. It would take additional logic to overcome them if needed.

Unfamiliar formatting can also cause issues for parsing dates. Often you need Excel to extract information about a date from an irregular format. For example, if you input 1987-Nov-19, Tuesday into a cell, you would need to use a combination of several functions (LEFT, MID, RIGHT, FIND, and DATEVALUE) to extract the date's numerical representation. What's more, differences in geolocation between systems can create ambiguous date formats, especially

when you're working with international datasets. Does 01/02/1987 mean January 2 or February 1? Depending on the system's location settings, Excel may misinterpret the date.

By providing more-robust representations of dates and times, Python helps overcome these issues efficiently and accurately. Once you're familiar with Python's way of doing things, the common inconveniences you may have experienced while working in Excel, like time-zone conversions, daylight saving, and irregular date formats, may become nonissues.

The datetime Library

Python has several libraries with the functionality to represent and manipulate dates and times. We'll focus on `datetime` from the Python Standard Library, which is one of the most widely used. To facilitate more-complex functionality, `datetime` represents dates and times primarily through four types of objects:

date Represents a date without time information. For example, `my_date = date(1987, 11, 19)` creates a date object representing November 19, 1987. Takes arguments for the year, month, and day.

time Represents a time without date information. For example, `my_time = time(14, 30)` creates a time object representing 2:30 PM. Takes arguments for the hour, minute, second, microsecond, and time zone (`tzinfo`).

datetime Represents a date with time information. For example, `my_datetime = datetime(1987, 11, 19, 14, 30)` creates a `datetime` object representing November 19, 1987, at 2:30 PM. Takes arguments for the year, month, day, hour, minute, second, microsecond, and time zone.

timedelta Represents a length of time, which can be used to add or subtract time from a date, time, or `datetime` object. For example, `my_timedelta = timedelta(days=5)` creates a `timedelta` object representing five days. Takes arguments for days, seconds, microseconds, milliseconds, minutes, hours, and weeks.

Since `datetime` is part of the Python Standard Library, it comes preinstalled with Python. You therefore don't need to install it separately using `pip`, but you still need to *import* it into your Python script before you can use it. This gives the Python interpreter access to the underlying source code where the library's objects are defined. You can then reference and use these objects in your script.

Imports can be done generally (the whole library of code at once). However, if you know you need to use only certain objects from the library (for example, just date objects), importing only those specific elements is considered the best practice. Here's how to import the date data type from the `datetime` library:

```
from datetime import date
```

To import multiple data types, separate them with commas at the end of the import statement:

```
from datetime import date, time, datetime, timedelta
```

Once the data types are imported at the start of a script, you can use them. For example:

```
from datetime import date, time, datetime, timedelta
```

```
my_date = date(1987, 11, 19)
my_time = time(14, 30)
my_datetime = datetime(1987, 11, 19, 14, 30)
my_timedelta = timedelta(days=5)
```

Now you can perform operations with these objects. For example, you can add a `timedelta` to a `datetime`:

```
print(my_datetime + my_timedelta)
```

This outputs a new `datetime` representing the date and time five days after the original `my_datetime`, which is November 24, 1987, at 14:30 (2:30 PM).

Converting datetime Objects to Strings

In Excel, you generally reformat a date by changing the format of the cell containing the date; in Python, you do it by converting the date or `datetime` object into a string with the intended formatting. To specify the format, use one of the library's format codes. These differ from the formatting options used in Excel. You can find a complete list of available format codes in the `datetime` documentation, but Table 2-8 lists a few of the common ones.

Table 2-8: `datetime` Format Codes and Their Excel Counterparts

Format code	Description	Excel counterpart
%Y	Four-digit year	"yyyy"
%y	Two-digit year	"yy"
%m	Zero-padded month number	"mm"
%B	Full month name	"mmmm"
%b	Abbreviated month name	"mmm"
%d	Zero-padded day of month	"dd"
%A	Full weekday name	"dddd"
%a	Abbreviated weekday name	"ddd"
%H	Zero-padded hour (24-hour format)	"hh" (with 24-hour clock)
%I	Zero-padded hour (12-hour format)	"hh" (with 12-hour clock)
%p	AM or PM	"AM/PM"
%M	Zero-padded minute	"mm"
%S	Zero-padded second	"ss"

With the date object `my_date` created in the previous example, you can extract the date in the format `1987-11-19` by applying the `strftime()` method to the object, using the format code `%Y-%m-%d`:

```
date_str = my_date.strftime("%Y-%m-%d")
print(date_str)
```

This code stores the string `1987-11-19` in the `date_str` variable and then prints it. The same pattern works for `datetime` objects, which allows you to include information about the time as well as the date:

```
datetime_str = my_datetime.strftime("%Y-%m-%d %H:%M")
print(datetime_str)
```

Using the `my_datetime` object created earlier, this example stores and prints the string `1987-11-19 14:30`, with the hour component represented in 24-hour time.

Extracting datetime Objects from Strings

To convert a string like `1987-11-19 14:30` into a `datetime` object, use the `strptime()` method. While `strftime()` is automatically imported with the `date` or `datetime` type, `strptime()` can only be imported with the `datetime` type. Call `strptime()` to convert from a string, passing the string and the appropriate format code as arguments:

```
datetime_str = "1987-11-19 14:30"
my_new_datetime = datetime.strptime(datetime_str, "%Y-%m-%d %H:%M")
```

Now you have a `datetime` object representing 2:30 PM on November 19, 1987.

You can use `strptime()` and `strftime()` together to convert a string from one format to another, with a `datetime` object serving as an intermediary. First, parse the original string into a `datetime` object by using `strptime()`. Then convert the `datetime` object back into a string with the new format by using `strftime()`. Here's how to do it:

```
from datetime import datetime

first_string = "1987-Nov-19, Thursday"
datetime_obj = datetime.strptime(first_string, "%Y-%b-%d, %A")
second_string = datetime_obj.strftime("%d-%b-%Y")

print(second_string)
```

This code will print `19-Nov-87`, the reformatted string. Since Python automatically understands which parts of the string refer to which characteristics of the date, thanks to the format codes, no complex parsing is needed. By contrast, for Excel to correctly interpret `1987-Nov-19, Thursday` as a date, you would need to write a complex formula to parse each portion of the date individually:

```
=TEXT(DATE(LEFT(A1, 4), MATCH(MID(A1, 6, 3), {"Jan","Feb","Mar","Apr","May",  
"Jun","Jul","Aug","Sep","Oct","Nov","Dec"}, 0), MID(A1, 10, 2)), "yyyy/mm/  
dd") & " - " & RIGHT(A1, LEN(A1) - FIND(" ", A1) - 1)
```

This formula uses several functions to extract and reassemble the date components, so it's much less intuitive and concise.

Working with Time Zones

To work with time zones by using `datetime` objects, you need to import the `timezone` data type from the `pytz` library. This allows you to make your dates *time-zone aware*, meaning they become associated with a specific time zone. Since `pytz` isn't part of the Python Standard Library, you need to install it like so (replace `pip` with `pip3` as needed):

pip install pytz

One way to incorporate time-zone information is to use the `localize()` method from `pytz`, as shown here:

```
from datetime import datetime  
from pytz import timezone  
  
naive_datetime = datetime(1987, 11, 19, 12, 0)  
ny_datetime = timezone("America/New_York").localize(naive_datetime)
```

This code creates a `datetime` object for November 19, 1987, at noon, specifically set to the New York time zone ("America/New_York").

To see a list of available time-zone settings, you can use the `all_timezones` variable from the `pytz` library:

```
from pytz import all_timezones  
  
print(all_timezones)
```

This prints a list of all the time-zone strings that the library supports.

You can add a time zone to an existing time-zone-unaware object by using the `localize()` method, which is tied to the `timezone` type. For example:

```
from datetime import datetime  
from pytz import timezone  
  
timezone_unaware_datetime = datetime(1987, 11, 19, 12, 0)  
ny_timezone = timezone("America/New_York")  
timezone_aware_datetime = ny_timezone.localize(timezone_unaware_datetime)
```

To convert between time zones, apply the `astimezone()` method to a time zone-aware datetime object. This method allows you to specify the new time zone you want to convert to. For example:

```
from datetime import datetime
from pytz import timezone

timezone_unaware_datetime = datetime(1987, 11, 19, 12, 0)

ny_timezone = timezone("America/New_York")
ny_timezone_datetime = ny_timezone.localize(timezone_unaware_datetime)

london_timezone = timezone("Europe/London")
london_timezone_datetime = ny_timezone_datetime.astimezone(london_timezone)

print(ny_timezone_datetime)
print(london_timezone_datetime)
```

The `pytz` library automatically handles adjustments for daylight saving time as appropriate, based on the location setting and the date. This makes converting between time zones and calculating time differences much simpler and more accurate than in Excel.

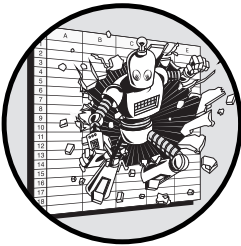
Conclusion

Before you even started this chapter, you probably had an intuitive understanding of many of the fundamental concepts required to code in Python, thanks to your experience with Excel. And, this chapter has shown that Python concepts like variables, data structures, and conditionals have clear Excel analogues.

Often the Python representations of these concepts offer more power and flexibility than Excel. While these additional features require you to think a little more about what you're doing, they open up a world of possibilities in terms of added features and efficiency.

3

AUTOMATING TASKS WITH PYTHON SCRIPTS



In this chapter, we'll explore how to write Python scripts, a first step to automation and an essential skill to free your work from the confines of Excel. We'll also cover two related programming concepts to help you write code that's both useful and reusable: performing iteration with loops and creating user-defined functions. With the basics covered in this chapter, you'll be able to build simple Python scripts from scratch, without needing to rely on a visual interface to assist you with developing logic.

What Are Scripts?

In Python, *scripts* are files with the `.py` extension that contain code for the Python interpreter to execute. When the interpreter executes a script, it runs the entire file from top to bottom, which is useful when you want to use Python to automate tasks. Scripts are the simplest way to integrate Python

into your workflow, but they have additional features that make them valuable for more than just simple automation and calculation.

Scripts in Python are unique in that they can stand on their own or be imported as modules into other scripts, allowing them to do double duty as self-contained tools and building blocks for other tools. Thanks to this flexibility, a simple script that automates a small portion of your workflow can turn into a jumping-off point for a much larger or more sophisticated project.

We'll see an example of this as we explore how to turn your code into a script that can run on its own or live in a library of other code that you can easily reuse in other projects. By the end of this chapter, we'll have worked through the steps necessary to construct an example script that allows you to automatically manipulate Excel files through Python.

When to Use a Script

Knowing when to use a Python script for a task versus when to use Excel isn't straightforward and depends on your goals. To get started, let's go over a few situations where Python scripts might be more accessible and effective tools for the job than spreadsheets.

In the first situation, say you want to automate a task. This task could be something like creating a report that you need to run daily or weekly. Or it could also be a task that occurs in response to an event, like updating the relevant records when your company takes on a new customer or hires a new employee. In each of these cases, creating a Python script that can run without too much manual intervention can save you valuable time and effort.

Second, say you are working on a task that requires data and calculations that are too large or complex for Excel. Excel has a hard limit of 1,048,576 rows, but spreadsheet performance can start to deteriorate with datasets much smaller than that. Workbooks of mere thousands of rows can be enough to crash the application if they contain enough intricate cell references or calculations. Python, on the other hand, is limited only by the amount of memory available on your system. When a task involves working with large datasets, like aggregating data from several large files or creating summary statistics from a database table with millions of entries, a Python script might be the better option.

And third, say your task requires you to interact with multiple different applications. Many tasks that you might need to perform involve other applications like shared drives, email, messaging platforms like Slack or Microsoft Teams, cloud storage services like Microsoft OneDrive or iCloud, databases, or APIs. With a Python script, you can programmatically connect to these applications. For example, you could set up an automated email alert by using the Google API client library. In these contexts, scripts can programmatically make connections and exchange data in a way Excel never could.

In the final situation, say you want to reuse your logic. While working in Excel, you've probably found yourself repeating the same operations over and over, perhaps with slight modifications each time, like copying over a function from one column to another and changing a single argument. By writing reusable functions or creating your own libraries, you can consistently apply the same logic without having to rewrite code. For example, you might create a utility function to clean and preprocess data, a custom module for generating standardized reports, or a script that handles common database operations. These reusable components save time, reduce errors, and ensure consistency across your projects.

Technically, Excel can be used to solve many of these problems, but that isn't how it was intended to be used. Python scripts are flexible enough to work well in all these contexts, so learning how to design them will expand both what you can do in your work and how well you can do it.

Parts of a Script

Whatever their purpose, Python scripts tend to have the same basic elements. To keep their *.py* files as neat and tidy as possible, most coders put those elements in a specific order:

1. *Imports*, which bring in external libraries or modules that your script needs. These are typically placed at the very top of the file to clearly indicate the script's dependencies.
2. *Function definitions*, including user-defined functions, which we'll cover in "User-Defined Functions" on page 59. These outline general logic that can be called upon within the script. *Classes*, a concept we'll introduce in Chapter 11, also fit in this section, after your user-defined functions.
3. An `if __name__ == "__main__"` block, which we'll discuss in "Building and Running a Complete Script" on page 66. This block serves as the entry point of the script. It's placed after function definitions so it can use those functions if needed.

You'll notice that this structure has a clear separation of tasks based on where code lives and where it's used. The imports section brings in code from outside the script. The function definitions include new code for the script (which can also be used elsewhere). And the `if __name__ == "__main__"` block contains the main logic that will run only when the script is executed directly, rather than being imported as a module elsewhere.

The order also follows a clear logical flow: First you set up the tools and libraries you need, then you define the functions you'll be using, and finally the main logic runs. With this structure, you aren't likely to end up in a situation where you need something that hasn't yet been defined in the script.

GLOBAL VARIABLES

One other relatively common element of scripts that you might encounter is *global variables*. These are variables that need to be available anywhere within your script. Using global variables isn't regarded as good programming practice, because they can make your code more complicated to maintain. However, they can be an efficient way to store certain kinds of information when used sparingly. Typical global variables might hold configuration values or settings that will be used in multiple places within your script, such as the following:

```
FILE_SAVE_PATH = "/user/data/"
SUPPORTED_FILE_TYPES = ["jpg", "png", "gif"]
API_VERSION = "3.0"
```

When you do use global variables, it's customary to define them in all capital letters to differentiate them from other variables. They typically come after the imports section of your script.

In “Building and Running a Complete Script” on page 66, we'll put the various elements of a script together in a simple *.py* file that manipulates an Excel file via Python code. First, though, we'll cover two concepts that are essential to writing scripts: iteration and user-defined functions.

Loops

In programming, *iteration* is the process of executing a block of code repeatedly, often with small, clearly defined variations between each repetition. This is typically accomplished using a *loop*, where the code moves through each item in a queue and performs the same series of actions each time.

Loops are a foreign concept to Excel, since each spreadsheet cell operates entirely separately from the others. The closest analogy is dragging a formula across multiple cells, but that just creates multiple copies of the same formula, which has its downsides. Having multiple copies of the same information means the copies could get out of sync with one another. It also means that to make a change, you'll need to re-create all the copies again, potentially across thousands of cells. By contrast, Python loops make it simple to apply the same block of code to multiple elements, reducing errors and decreasing the amount of code you need to write.

In Chapter 2, we discussed conditional statements as an element of control flow, in that they dictate when and how a script executes instructions. Loops are another element of control flow. Python provides a few kinds of loops, and their structures differ depending on how the code knows when to stop.

***for* Loops**

One form of loop is the *for* loop, which iterates over a sequence of a fixed length. In Python, *for* loops look like this:

```
for value in sequence:
    # Block of code to execute
```

Here, *sequence* is an object containing pieces of data. The `for` loop extracts those pieces of data one by one, stores them in a temporary variable (*value* in this example), and applies the indented block of code to them. You can create `for` loops that iterate over a variety of types of objects, including lists, ranges, and strings. Let's look at an example of each.

Using Lists

First, we'll demonstrate iterating over the elements in a Python list object with a `for` loop. Here's a simple example, using a loop to sum the values in a list:

```
list_of_numbers = [1, 2, 3, 4, 5]
sum_of_numbers = 0
for number in list_of_numbers:
    sum_of_numbers = sum_of_numbers + number
```

We create a list containing the numbers 1 through 5. Then we create a variable called `sum_of_numbers` and assign it the value 0. Next, inside a `for` loop, we iterate over each element in `list_of_numbers` and add the current element (`number`) to `sum_of_numbers`. After the loop has processed all the elements, `sum_of_numbers` will contain the total sum of the numbers in the list, which in this case is 15.

Using Ranges

Another common method for constructing a `for` loop is to use a *range*, a built-in Python data type representing a sequence of numbers. You create a range with the `range()` function, which operates differently depending on which arguments you pass into it.

If you input a single number, like `range(3)`, the function will return the numbers from 0 up to but excluding 3: (0, 1, 2). If you give the function two numbers, like `range(1, 3)`, it will return the numbers between the two numbers, including the first but excluding the last: (1, 2). If you give the function three inputs, like `range(1, 10, 3)`, it will return the numbers between the first two numbers in increments of the third: (1, 4, 7).

This example uses all three `range()` function arguments to create a range with the odd numbers from 1 to 10:

```
range(1, 10, 2)
```

The resulting range will include the numbers 1, 3, 5, 7, and 9.

Here's how to iterate over a range by using a `for` loop:

```
n = 10
sum_of_numbers = 0
for number in range(1, n + 1):
    sum_of_numbers = sum_of_numbers + number
```

This code snippet loops through the numbers from 1 to 10. (We set the upper bound of the range to $n + 1$, or 11, since the upper bound is excluded.) On each cycle through the loop, we add the current number, represented by the `number` variable, to the value of `sum_of_numbers`. In the end, `sum_of_numbers` is the sum of the entire range (55).

Using Strings

When we introduced strings in Chapter 2, we talked about how they're essentially just lists of characters. Therefore, it's possible to pass a string to a `for` loop and iterate over its individual characters, like so:

```
my_string = "hello"
for character in my_string:
    print(character)
```

This `for` loop will perform one iteration for each of the five letters in `hello` to print out the string's characters, one by one.

List Comprehensions

To make simple loops cleaner and more manageable, Python provides *list comprehension* syntax. It lets you write a `for` loop with just one line of code, provided the loop results in a list or dict. The list comprehension iterates over the values in an object and applies a condition or operation to each element. Here's an example:

```
numbers = [1, 2, 3, 4, 5]
doubled = [x * 2 for x in numbers]
```

The list comprehension `[x * 2 for x in numbers]` takes each number in the `numbers` list, multiplies it by 2, and puts the results in a new list, which we store in the `doubled` variable. This approach is a more concise way of doing what you could do in a regular `for` loop:

```
numbers = [1, 2, 3, 4, 5]
doubled = []

for x in numbers:
    doubled.append(x * 2)
```

You can incorporate conditional statements into list comprehensions to selectively apply logic to a list. There are two ways to do this: attaching the conditional to each individual item in the sequence you're iterating over, or attaching it to the sequence itself. Which approach you use depends on how you want the conditional to be applied in the loop.

To apply a conditional to each item, include an `if` statement before the `for` keyword. In our example of doubling numbers, we might use this approach if we want to double only the numbers that are larger than 2, while leaving the other numbers unchanged in the resulting list:

```
doubled = [x * 2 if x > 2 else x for x in numbers]
```

The conditional clause `x * 2 if x > 2 else x` indicates that we want the list to include `x * 2` only if `x > 2` and that we want just the original `x` otherwise. This results in `[1, 2, 6, 8, 10]`. Conditional clauses that apply to every element in the list comprehension must have an `else` included so that the resulting list has no missing elements.

Alternatively, if you place the conditional clause after the `for` clause, elements that don't meet the condition are excluded from the resulting list:

```
doubled = [x * 2 for x in numbers if x > 2]
```

Here, the conditional `if x > 2` filters the elements of the `numbers` list, ensuring that only numbers greater than 2 are doubled and included in the resulting `doubled` list. No `else` is needed here because we intend to exclude the items not meeting the condition from the final list. This means the output list will be shorter than the input list: `[6, 8, 10]`.

Many `for` loops can be turned into one line in Python by using a list comprehension, but that may not always be preferable. Just as more-complex formulas in Excel can become difficult to read, list comprehensions can become unwieldy if the logic is too complex.

Loop Controls

Python allows you to fine-tune the operation of a loop by using loop-control keywords like `break` and `continue`. These are examples of Python *reserved words*, words that are built into the language and thus can't be used as variable names or other identifiers. (Other reserved words include the names of built-in data types like `int`, `string`, and `list`, as well as values like `True` and `None`.)

break

The `break` keyword in Python exits a loop prematurely, before it completes all its iterations. Usually, `break` is applied after a conditional, so it's useful when you're searching for something, or when something happens that would make continuing the loop unnecessary or incorrect, like reaching a limit or encountering an error. For example, this `for` loop searches a list of email subject lines (called `messages`) and stops the loop via `break` when a message labeled *URGENT* is found:

```
messages = [
    "Meeting at 10",
    "Project deadline extended",
    "URGENT: Server downtime",
    "Lunch with Bob"
]

for message in messages:
    if "URGENT" in message:
```

```
        print(f"Action required: {message}")
        break
    else:
        print(f"Message reviewed: {message}")
```

Within the `for` loop, the `if` statement checks for the string `URGENT` in the current message, prints that message with the label `Action required`, and then halts the loop with `break`. Until the first urgent message is found, the `else` statement will run each time through the loop, printing messages with the label `Message reviewed`. Here's the result:

```
Message reviewed: Meeting at 10
Message reviewed: Project deadline extended
Action required: URGENT: Server downtime
```

Notice that the final message in the list isn't printed, since the `break` statement halts the loop before it can be processed.

continue

The `continue` keyword skips the rest of the current iteration of a loop and proceeds to the next iteration. Therefore, unlike `break`, `continue` doesn't terminate the entire loop. For example, this `for` loop processes a similar list of email subject lines and skips over any message containing the word *SPAM* by using `continue`:

```
messages = [
    "Meeting at 10",
    "Project deadline extended",
    "SPAM: Buy now!",
    "Lunch with Bob",
    "SPAM: Free Python course"
]

for message in messages:
    if "SPAM" in message:
        print(f"Skipping: {message}")
        continue
    print(f"Message reviewed: {message}")
```

In this version, when the loop comes across the string *SPAM*, it skips the message and moves on rather than stopping. Here's the output:

```
Message reviewed: Meeting at 10
Message reviewed: Project deadline extended
Skipping: SPAM: Buy now!
Message reviewed: Lunch with Bob
Skipping: SPAM: Free Python course
```

Notice that the two messages containing *SPAM* are skipped, but the loop continues with the next message.

while Loops

Another kind of loop in Python is a while loop, which iterates indefinitely while a condition remains true. Unlike a for loop, a while loop doesn't require a collection of items to iterate over. A Python while loop looks like this:

```
while condition:
    # Block of code to execute
```

Here, *condition* is an expression that evaluates to True or False, often a comparison. This condition is checked before each repetition of the loop.

In some cases, you could use either a while loop or a for loop, but using for is generally preferable, since while loops have several downsides. First, the loop's length is dynamic instead of fixed, so they're often less intuitive and harder to read than for loops. Second, while loops can continue looping indefinitely if the halting condition is never met, which can cause programs to unexpectedly use excessive time and memory or crash entirely. To see an example of an infinite while loop, run this Python code:

```
is_true = True
while is_true:
    is_true = True
```

This loop will repeat as long as the value of the `is_true` variable is True. Since `is_true` never becomes False within the body of the loop, the while loop continues looping forever, until you close your Python instance.

In practice, while loops are typically used when the script has to wait for an external action to occur, such as waiting for data to post on an API or waiting for a user to input information. In these kinds of cases, there's no way to know how many repetitions are necessary here, so a for loop would not be appropriate. We'll cover examples of this type of loop in Chapter 8.

User-Defined Functions

We discussed in Chapter 2 that a *function* is a stand-alone piece of code that performs a specific task. While Python has many built-in functions, you can go further by creating your own functions that are custom designed to break complex tasks into smaller, more manageable pieces. These *user-defined functions* allow you to organize your code into distinct reusable sections that can be easily understood, tested, and maintained.

In Excel, the formulas that live in cells and generate values aren't reusable in the same way Python functions are. To apply the same logic across multiple cells, you typically enter a formula in one cell and then copy or drag it to other cells where it's needed. For instance, let's say you want to calculate the average of the values in each row across columns B through D and display the result in column E. You would enter a formula like `=AVERAGE(B2:D2)` in cell E2, and then drag the fill handle down to apply this formula to the rest of the rows in column E. If you do this for 20 rows, you end up with 20 cells, each containing its own copy of the formula.

Now imagine you need to update the formula to include a new column—say, column F. You’d have to manually update the formula in all 20 cells. Forgetting to update one or accidentally changing the formula in just one cell could lead to inconsistent results across your spreadsheet, and these errors might not be immediately noticeable.

In contrast, when you write a function in Python, you define the logic once, inside the function, and then you can call that function wherever you need it and apply that same logic over and over again. When you *call* a function, you’re telling the program to execute the specific set of instructions encapsulated within that function according to a particular set of inputs. Because you’re executing the same code each time the function is called, you don’t risk the sorts of inconsistencies and errors that may sneak up on you in Excel.

Functions have several other benefits as well:

Simplification Just as breaking a complex task into smaller steps makes it easier to manage, functions break complex coding problems into smaller, more manageable pieces. This means functions make your code easier to write, read, and maintain.

Modularity Using functions allows you to organize your code into distinct sections, each performing a specific role. This modularity ensures that your code is more organized and easier to fix when problems arise. If you run across an issue, you can pinpoint the problem by examining just the relevant function, not the entire script.

Reusability Functions facilitate reuse of code. Once you’ve defined a function, you can use it as many times as you like. This reusability saves time, ensures consistency, and reduces the risk of introducing errors.

When you put code inside a function, you’re taking the solution to a specific problem and generalizing it. This abstraction simplifies the problem-solving process and allows you to focus on the higher-level logic of your algorithm.

A user-defined function definition begins with the `def` keyword, followed by your desired name for the function, and then a set of parentheses:

```
def function_name():  
    # The code inside your function
```

This creates a new function called `function_name()` that will do whatever you specify inside the indented code block.

In the following sections, we’ll go over the other key components you need to build a function.

Arguments

Arguments are variables that are used as inputs to a function. They allow you to pass information into your function to help it accomplish whatever it’s

trying to do. When you define a function, list the arguments in parentheses after the function name.

Functions in Python can take in two types of arguments: positional and keyword. *Positional arguments* are identified by the order in which they're passed into the function. They don't have a default value, so a value for each positional argument must be provided for the function to run. Here's an example function definition with positional arguments:

```
def greet(first_name, last_name):  
    print(f"Hello, {first_name} {last_name}!")
```

This `greet()` function uses two arguments, `first_name` and `last_name`, to construct and print a greeting to the user. When you call the function, you need to provide values for the `first_name` and `last_name` arguments, in that order. For example, say you call `greet("Bill", "Gates")`. The function will assign `Bill` to the `first_name` variable and `Gates` to the `last_name` variable based on their positions in the function call, and so the message `Hello, Bill Gates!` will print to the console.

By contrast, *keyword arguments* are identified by the name used in the function definition and can be passed in any order when calling the function. Keyword arguments can be given default values in the function definition, which in effect makes it optional to provide values for those arguments when calling the function. Here, we reframe the `greet()` function to use keyword arguments with default values:

```
def greet(first_name="Bill", last_name="Gates"):  
    print(f"Hello, {first_name} {last_name}!")
```

In this function definition, we assign `Bill` and `Gates` as default values for the `first_name` and `last_name` arguments, respectively. Thanks to these default values, we can call the function without providing any arguments, like this:

```
greet()
```

With this call, the console will print `Hello, Bill Gates!` because the function uses the default values provided in the definition. Any keyword arguments you omit when you call a function will revert to their default, even if it's `None`. However, you can override the default argument values by passing your own arguments using their keyword names:

```
greet(last_name="Jobs", first_name="Steve")
```

In this case, the console will print `Hello, Steve Jobs!` because we've explicitly provided the values for `first_name` and `last_name`, which replace the defaults. It doesn't matter that the arguments are passed to the function out of order, since the function matches the values to the correct argument based on their keyword names, not their position in the argument list.

Scope

Variables defined within the functions you create fall within that function's *scope*, meaning those variables can be used only inside the function, not outside of it. For example, say we define a function called `my_function()` with a variable called `inside_variable` in its body:

```
def my_function():
    inside_variable = "I am inside the function."
    print(inside_variable)
```

If we try to access `inside_variable` from code that isn't part of the function block, we'll get an error:

```
NameError: name 'inside_variable' is not defined
```

This error occurs because `inside_variable` is out of scope, so Python doesn't recognize it as a valid variable.

Scope creates isolation between different parts of your code, ensuring that the code inside the function doesn't interfere with the code outside of it. This also means that any value calculated or modified inside a function will potentially be lost after the function finishes executing. Fortunately, when you do need to pass information from inside a function's scope to the outside, we have a way around this: `return` statements.

Returns

A *return* is a value that a function outputs for use elsewhere in a script. By adding a `return` statement to a function definition, you can send a value from inside the function back to the part of the script that called the function, allowing that value to be printed, stored in a variable, or used however you need. Returns provide a mechanism for using the work done inside a function, instead of losing that work forever after the function finishes executing.

Here's how to include a return value in a function definition:

```
def create_greeting(first_name, last_name):
    greeting = f"Hello, {first_name} {last_name}!"
    return greeting
```

This `create_greeting()` function takes in `first_name` and `last_name` as arguments and constructs a greeting message, just like our earlier `greet()` function. Instead of printing that greeting to the console, however, the function outputs it to the calling script with the `return` keyword. When we call the function, we should therefore store the result in a variable:

```
message = create_greeting("Steve", "Jobs")
```

Now we have a variable called `message` that contains the `Hello, Steve Jobs!` greeting constructed by the function. We can print it to the console with `print(message)` or continue to use the variable elsewhere in our code.

Functions that don't explicitly have a return statement, like our original `greet()` function, have an implicit return value of `None`. You can see this if you try to assign the function call to a variable and then print that variable's value:

```
def greet(first_name, last_name):  
    print(f"Hello, {first_name} {last_name}!")  
  
result = greet("Steve", "Jobs")  
print(result)
```

When you run this code, you should see the following in the terminal:

```
Hello, Steve Jobs!  
None
```

The first line in the terminal comes from the `print` statement within the function body. The second is the result of `print(result)` and illustrates that `greet()` implicitly returns `None`.

Having an implicit return value of `None` is equivalent to using the `return` keyword in a function definition without including a value after it. For example, we could have defined the `greet()` function like this and achieved the same result:

```
def greet(first_name, last_name):  
    print(f"Hello, {first_name} {last_name}!")  
    return
```

In this context, the `return` keyword operates in the function like a period operates in a sentence; it tells Python to exit the function body and go back to the point in the script where the function was called. In fact, no matter where you put the `return` keyword, it brings an end to the function. This means you can use `return` to halt a function early under certain conditions. For example, the following function searches a word for vowels and returns the first one:

```
def find_first_vowel(word):  
    for char in word:  
        if char in "aeiouAEIOU":  
            return char  
    print("No vowel found.")  
    return None
```

This function takes in a string through the `word` argument and uses a `for` loop to examine that string character by character. Notice that we include two `return` statements, only one of which will be triggered during any given function call. If a character is found to be a vowel, we return that character and stop looking. If the `for` loop ends without finding a vowel, we return `None` instead. Therefore, if we call the function and pass in `word = "hello"`, it will return `e`, but if we pass in `word = "sky"`, it will return `None`.

Modifier Functions

So far we've considered functions that return a new variable, but you can also write a *modifier function* that changes an existing variable. This is common with lists and dicts, which are *mutable* data structures in Python, meaning they can be changed after they're created. Other data types like floats, strings, and tuples are immutable, so while you can use a modifier function to change the elements in a list or dict, you can't, for example, use one to change the characters in a string.

The benefit of updating lists and dicts with modifier functions is that it allows you to perform operations in place rather than creating a new variable. Your script won't require as much memory to run, because it's modifying the original data instead of creating additional copies of it. Here's a function that modifies a list:

```
def add_to_list(existing_list, element, unique=False):
    if unique and element in existing_list:
        print("Element is already in the list.")
        return
    existing_list.append(element)

my_list = [1, 2, 3]
my_element = 2
add_to_list(my_list, my_element, unique=True)
```

When the `unique` argument is set to `True`, the `add_to_list()` function adds the provided element to `existing_list` only if it isn't already present in the list. If the element is already in `existing_list`, the function prints a message and exits without adding it again. We don't need to return `existing_list`, because we've modified it inside the function.

Modifier functions operate similarly on dicts:

```
def add_to_dict(existing_dict, key, value, overwrite=False):
    if not overwrite and key in existing_dict:
        print("Key already exists in dictionary.")
        return
    existing_dict[key] = value

my_dict = {"a": 1, "b": 2}
my_key = "b"
my_value = 3
add_to_dict(my_dict, my_key, my_value, overwrite=True)
```

This function checks whether the provided key exists in `existing_dict` and whether `overwrite` is set to `False`. If so, the function prints a message and exits without changing the dictionary. Otherwise, if `overwrite` is `True` or the key doesn't already exist in the dictionary, the function adds or updates the key with the given value. Again, there's no need to return the dictionary at the end of the function.

Functionalizing Logic

Scripts are cleanest and easiest to maintain when you can clearly see what each piece of code does. The code that performs one task shouldn't be intertwined with the code that performs another. To accomplish this in Python, you can “functionalize” your logic by encapsulating it inside functions.

For example, say we have the following list comprehension that doubles the numbers from 1 to 5:

```
[i * 2 for i in range(1, 6)]
```

We can functionalize this logic by putting the list comprehension inside a `double_numbers()` function and turning the start and stop arguments for the range into generic arguments for the function:

```
def double_numbers(start, stop):  
    return [i * 2 for i in range(start, stop)]
```

Now we can double the numbers from 1 to 5 by using one simple, readable function call:

```
doubled = double_numbers(1, 6)
```

Even better, we've gained the flexibility to reuse the `double_numbers()` function to double other ranges of numbers without having to duplicate and tweak the original list comprehension:

```
more_doubled = double_numbers(1, 100)  
even_more_doubled = double_numbers(1, 1000000)
```

Organizing your code into functions like this makes it *modular*; it's broken into separate, smaller sections (modules) that can operate independently from one another. Increasing the modularity of your code has a variety of benefits, as we've already discussed, but modularity can also be taken too far. If your code gets fragmented into too many small functions, each doing a trivial task, the code can become harder to follow and maintain, despite the intention to increase clarity and reusability. Here's an example of over-modularized code:

```
def get_first_half(lst):  
    length = len(lst)  
    return lst[:length // 2]  
  
def get_last_half(lst):  
    length = len(lst)  
    return lst[length // 2:]  
  
my_list = [1, 2, 3, 4]  
first_half = get_first_half(my_list)  
last_half = get_last_half(my_list)  
list_sum = sum(first_half + last_half)
```

The goal of this code is to add up the numbers in a list. But rather than use the concise `sum(my_list)`, we define separate functions that retrieve just the first and second halves of the list, and then we add the halves together. This is an extreme example, but it illustrates the possibility of taking modularization too far. When used appropriately, however, functionalization and modularity are powerful tools for writing code that you can build upon and reuse in many contexts.

Building and Running a Complete Script

Let's synthesize what we've discussed in this chapter with a practical example of creating a complete script. In the process, we'll also look at ways of working with `.py` files.

For our example, Alice is an analyst on the finance team at the Gridlock Grill, a popular Excel-themed chain of hamburger restaurants. Her team is looking to convert some of its Excel-driven processes to more automated Python scripts. One of the team's processes aggregates data from the sales reports of several restaurants in the chain into separate sheets in an Excel file called *SalesReports.xlsx*. To store this data in a way that doesn't rely on the Excel application and is more easily accessible in Python, Alice decides to write a script that converts each sheet to a separate CSV file.

To accomplish this task, Alice will need to import two modules, `os` and `csv`, from the Python Standard Library into her script, as well as the third-party `xlwings` library. The `os` module contains functions for viewing and modifying your operating system's file structure. This will allow Alice to manipulate filepaths. For example, she can use the following code to save the path of the sales report Excel file in her script:

```
import os

home_dir = os.path.expanduser("~")
sales_report_dir = os.path.join(home_dir, "Documents", "SalesReports")
excel_file_path = os.path.join(sales_report_dir, "SalesReports.xlsx")
```

Here, `os.path.expanduser("~")` Alice's home directory (the equivalent of `~` at the command line). This is preferable to manually writing out the filepath because it will work on both Windows and macOS, which use different path separators. Next is the `path.join()` function, which joins the elements in filepaths, to store the parent directory and filepath of *SalesReports.xlsx*.

Another useful feature of `os` is the ability to check whether a directory exists and to create it if it doesn't. To do this, Alice can use `path.exists()` and `makedirs()`:

```
output_dir = os.path.join(sales_report_dir, "SalesReportFiles")
if not os.path.exists(output_dir):
    os.makedirs(output_dir)
```

Now that Alice has ensured the correct directories are in place, she can move on to opening the Excel file. The `xlwings` library allows Alice to interact with Excel workbooks directly from Python. To follow along with the example, install the library with `pip install xlwings` or `pip3 install xlwings` as appropriate.

We won't go into too much detail on how to use `xlwings` to manipulate Excel workbooks, but you'll need to know two things for this example. First, to interact with Excel, Alice needs to start an `xlwings.App` instance of Excel:

```
import xlwings
```

```
app = xlwings.App()
```

Second, here's how Alice can open the workbook containing the sales reports:

```
wb = app.books.open(excel_file_path)
```

Our last element is the `csv` module, which helps read and write CSV files. For example, Alice can write the data from the first sheet of the Excel workbook to a CSV file with the following code:

```
import csv
```

```
sheet = wb.sheets[0]
csv_file_path = os.path.join(output_dir, "output.csv")
```

```
with open("csv_file_path", "w") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows(sheet.used_range.value)
```

In this example, Alice writes the entire used range of the first sheet into a CSV file called *output.csv*. The function `open("csv_file_path", "w")` opens the file in write mode ("w") so it can be edited. The `writerows()` method of the `writer` object takes the data from `sheet.used_range.value` and writes each row of the Excel sheet to the CSV file row by row, all at once.

This operation takes place inside a `with` block, a structure that makes working with files in Python simpler and safer by ensuring that the file is automatically closed after the block finishes, even if an error occurs while writing to the file. Without the `with` block, Alice would need to explicitly close the file with the `close()` method after writing to it, like so:

```
csv_file = open("output_file_path", "w")
writer = csv.writer(csv_file)
writer.writerows(sheet.used_range.value)
csv_file.close()
```

This is much less safe than the version using `with` because if an error occurs before the file is closed, it could become corrupted.

To save all the sheets as separate CSV files, Alice needs to use a for loop to iterate through each sheet in the workbook. The loop will generate a CSV file for each sheet in the Excel file:

```
import os
import csv
import xlwings

home_dir = os.path.expanduser("~")
sales_report_dir = os.path.join(home_dir, "Documents", "SalesReports")
excel_file_path = os.path.join(sales_report_dir, "SalesReports.xlsx")

output_dir = os.path.join(sales_report_dir, "SalesReportFiles")
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

with xlwings.App(visible=False) as app:
    wb = app.books.open(excel_file_path)

    for sheet in wb.sheets:
        csv_file_path = os.path.join(output_dir, f"{sheet.name}.csv")
        with open(csv_file_path, "w") as csvfile:
            writer = csv.writer(csvfile)
            writer.writerows(sheet.used_range.value)
    wb.close()
```

When Alice runs this code, Python will direct her computer to open her Excel file, iterate through each of the sheets, and save them as CSV files.

Alice now has the code to accomplish her task, but to keep her script organized and to make the code reusable in the future, she should function-
alize the logic—that is, put it inside a user-defined function. The function
should take in two arguments: a path to an Excel workbook and an out-
put directory where the CSV files extracted from that workbook should be
saved. We'll call the function `save_sheets_as_csv()`:

```
def save_sheets_as_csv(excel_file_path, output_dir):

    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    with xlwings.App(visible=False) as app:
        wb = app.books.open(excel_file_path)

        for sheet in wb.sheets:
            csv_file_path = os.path.join(output_dir, f"{sheet.name}.csv")
            with open(csv_file_path, "w") as csvfile:
                writer = csv.writer(csvfile)
                writer.writerows(sheet.used_range.value)
        wb.close()
```

Now Alice has the list of libraries she needs to import and a function that performs her task. At this point, she has a few options for finishing her script and executing the code to generate the CSV files. Let's consider those options.

Running a Script on the Command Line

Running a `.py` file as a stand-alone Python script involves calling the Python interpreter (using the `python` or `python3` command) and passing the script file as an argument to it. This will execute all the code in the file. You can do this either through your main system command line (CMD or Terminal) or within the embedded terminal in VS Code, which you can open from inside the application via View ► Terminal. Regardless of the method you choose, you'll need to be aware of the current directory of your command line relative to the location of the script you want to run.

For example, say Alice puts her code to convert Excel sheets to CSV files in a script called `run_save_sheets.py`, which she stores at `/Users/Alice/python_projects/scripts`. The file contains the necessary imports, the `save_sheets_as_csv()` function, and some code to call the function with the appropriate arguments:

```
import os
import csv
import xlwings

def save_sheets_as_csv(excel_file_path, output_dir):
    # The function logic goes here

home_dir = os.path.expanduser("~")
sales_report_dir = os.path.join(home_dir, "Documents", "SalesReports")
excel_file_path = os.path.join(sales_report_dir, "SalesReports.xlsx")
output_dir = os.path.join(sales_report_dir, "SalesReportFiles")

save_sheets_as_csv(excel_file_path, output_dir)
```

On the command line, Alice can run this script as follows:

```
$ python /Users/Alice/python_projects/scripts/run_save_sheets.py
```

This will work no matter where the command line is navigated to, since Alice provides the full path to the script as an argument. Alternately, if she first navigates the CLI to `/Users/Alice/python_projects/scripts/`, she can simply run the script like this:

```
$ python run_save_sheets.py
```

In both cases, the script will execute, calling the `save_sheets_as_csv()` function and saving the CSV files extracted from *SalesReports.xlsx* in the location specified in the script, the *SalesReportFiles* directory.

Importing a Script as a Module

Python files can import other Python files as modules, allowing you to reuse code across different scripts. As a coder, you can leverage this by functionalizing code that you may want to reuse and putting it in a module, separate from the code you want to run on demand when you execute the script. This creates a distinction between the aspects of the script that actually perform tasks and the functions defining how those tasks should be carried out.

In Alice's scenario, she can separate the `save_sheets_as_csv()` function from `run_save_sheets.py` and store the function in a file named `excel_utils.py`. This file contains the function but not the code that calls the function:

```
import os
import csv
import xlwings

def save_sheets_as_csv(excel_file_path, output_dir):

    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    with xlwings.App(visible=False) as app:
        wb = app.books.open(excel_file_path)

        for sheet in wb.sheets:
            csv_file_path = os.path.join(output_dir, f"{sheet.name}.csv")
            with open(csv_file_path, "w") as csvfile:
                writer = csv.writer(csvfile)
                writer.writerows(sheet.used_range.value)

        wb.close()
```

Now Alice can import `excel_utils.py` as a module into `run_save_sheets.py` to use the function. Since this will import the `save_sheets_as_csv()` function, Alice can remove its definition from her script, leaving just the declaration of the filepath and the call to the function. Since we imported the module generally instead of importing specific elements from it, we use dot notation, as in `excel_utils.save_sheets_as_csv()`, to access the function:

```
import excel_utils

home_dir = os.path.expanduser("~")
sales_report_dir = os.path.join(home_dir, "Documents", "SalesReports")
excel_file_path = os.path.join(sales_report_dir, "SalesReports.xlsx")
output_dir = os.path.join(sales_report_dir, "SalesReportFiles")

excel_utils.save_sheets_as_csv(excel_file_path, output_dir)
```

For Python to be able to read the imported file, a few conditions need to be met. First, Python must be able to find the file. The file must be either

in the same directory as the script that's trying to import it or in a directory specified as part of the PYTHONPATH environment variable. You can add any directory containing Python modules to your PYTHONPATH variable. For example, Alice can add her *scripts* directory by using the process for appending to an environment variable outlined in Chapter 1.

After adding the directory to the PYTHONPATH environment variable, Alice has one more step. She'll need to create an empty file in *scripts* called `__init__.py`. This file tells Python that *scripts* is a package containing modules.

Using a Main Block

One unique aspect of Python files is that they can be imported into other files as modules without running all the code within them. This flexibility allows a single Python file to serve dual purposes: It can be imported as a module or executed as a stand-alone script. To achieve this, you need to separate the code that should run only when the file is executed as a script from the code that's intended to be imported.

To include extra code in a module file that will run only if the module itself is executed (rather than imported), you put it inside a block of code that begins with `if __name__ == "__main__":`, like so:

```
if __name__ == "__main__":  
    # Code to run only when the script is executed directly
```

The indented code after this `if` conditional is considered the *main block* of the script. It serves as the script's entry point, where execution of the logic begins. The Python interpreter knows whether the file is being run as a stand-alone script or is being imported into another script, and it will run the code within the `if __name__ == "__main__":` block only if it's being run directly, not just imported. The special variable `__name__`, which is automatically set by Python, stores the context in which the file is being used. When `__name__` is set to the string `"__main__"`, the Python interpreter knows to run the code inside the block, and it will skip over the code otherwise.

Returning to Alice's example script, she could include code inside *excel_utils.py* that will execute only when the file is run directly as a script and not imported as a module into another script. The extra code invokes the `save_sheets_as_csv()` function for the desired Excel file:

```
import os  
import csv  
import xlwings  
  
def save_sheets_as_csv(excel_file_path, output_dir):  
  
    if not os.path.exists(output_dir):  
        os.makedirs(output_dir)  
  
    with xlwings.App(visible=False) as app:
```

```

wb = app.books.open(excel_file_path)

for sheet in wb.sheets:
    csv_file_path = os.path.join(output_dir, f"{sheet.name}.csv")
    with open(csv_file_path, "w") as csvfile:
        writer = csv.writer(csvfile)
        writer.writerows(sheet.used_range.value)
wb.close()

if __name__ == "__main__":
    home_dir = os.path.expanduser("~")
    sales_report_dir = os.path.join(home_dir, "Documents", "SalesReports")
    excel_file_path = os.path.join(sales_report_dir, "SalesReports.xlsx")
    output_dir = os.path.join(sales_report_dir, "SalesReportFiles")

    save_sheets_as_csv(excel_file_path, output_dir)

```

The code within the main block will execute only when the script is run directly, not when it's imported as a module. Practically, this mechanism is often used for embedding test cases and examples of the functions in the module. It serves as a demonstration of how the module's functions can be used.

Passing System Arguments

Sometimes you want to pass arguments to a script as a whole when you run it on the command line. The script can use these arguments in its main block or can pass them on to other functions. These arguments are called *system arguments*, and Python's `sys` module, which is in the standard library, allows you to read them in through a list called `sys.argv`.

Alice can update `run_save_sheets.py` to take in the Excel file she wants to convert and the CSV destination directory as system arguments. This way, she won't have to update the script itself every time she wants to convert a different Excel workbook to individual CSV files. To do this, she uses `sys.argv` to obtain command line arguments specifying the Excel filepath and destination directory for the `save_sheets_as_csv()` function to work with:

```

import sys
from excel_utils import save_sheets_as_csv

if __name__ == "__main__":
    excel_file_path = sys.argv[1]
    output_directory = sys.argv[2]
    save_sheets_as_csv(excel_file_path, output_directory)

```

In this example, `sys.argv[1]` and `sys.argv[2]` retrieve the first and second arguments passed to the script, assuming the script is called with an Excel filepath and output directory as arguments. These values are then used as the arguments for the `save_sheets_as_csv()` function. To run this script,

Alice must add the path to the sales report Excel file and the intended output `_directory` following the name of the script:

```
$ python run_save_sheets.py \  
    /Users/Alice/Documents/SalesReports.xlsx \  
    /Users/Alice/Documents/SalesReportFiles
```

This will execute *run_save_sheets.py* for the filepaths specified in the arguments. (Note that a backslash allows you to continue a long command on a new line within a CLI.) Importantly, the script would also work for any other Excel file and destination directory that Alice wants to apply it to. You can verify that for yourself by running *run_save_sheets.py*, using one of your own Excel files and an output directory of your choice in your filesystem.

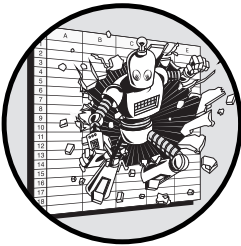
Conclusion

This chapter covered the basics of iteration, functions, and scripting, all of which allow you to take advantage of the benefits of a world beyond visual interfaces. With iteration, you can clearly and concisely apply logic across many elements without worrying about the hazards of copying and pasting. By functionalizing your code, you can take advantage of modularity to encapsulate specific tasks into clean, organized, and reusable pieces. And through scripting, you can begin to put your functions to work in automating the tasks you may have previously done manually with Excel. This could mean taking spreadsheets out of the equation for an entire, end-to-end process, or it could mean a marginal automation, as Alice did with her sales reports. Either way, you'd be making progress toward a faster and more adaptable workflow.

Now that you've learned to write Python scripts in *.py* files, the next chapter will show you how to keep track of these files in an organized, manageable, and collaborative way.

4

TRACKING CHANGES WITH VERSION CONTROL



In this chapter, we'll explore *version control*, a way of automatically tracking and managing changes to code files over time. We'll focus on the most popular tool for version control, Git: what it is, why it exists, and what it can do for you. We'll detail how to structure your Git workspaces, known as *repositories*, and then go over step-by-step instructions for getting started. You'll learn basic skills like commits, merges, and working with local and remote repositories. By the chapter's conclusion, you'll be equipped with the knowledge and confidence you need to use Git to keep track of files and collaborate with others on code.

Why Use Version Control?

In your work, you've probably seen how the files you work on follow a chronological journey. From its inception, every file undergoes additions, subtractions, and other modifications, with each change marking a point in the file's evolution. Version control captures the file's exact state at each of these points, providing an organized, systematic way to keep track of the file's development. Without it, you'll likely find yourself facing down unwieldy directories full of filenames like these:

ProjectProposal.docx

ProjectProposal_v2.docx

ProjectProposal_v2-Dave Comments.docx

ProjectProposal_v3.docx

ProjectProposal_final.docx

ProjectProposal_final_updated.docx

This list of names tells a story. A document is created, then revised, shared among peers for input, and revised again. And again. Eventually, the document arrives at its supposed final destination, *ProjectProposal_final.docx*. Until, that is, it's inevitably updated once more. With every modification, the author ensures that the earlier version is preserved, perhaps just for reference or in case they need to revert to a prior draft. But while the filenames may offer an illusion of organization, this approach is haphazard at best.

Version control provides a better way to cleanly and methodically log and traverse the changes to files. Using version control, each version of your document is neatly archived and labeled automatically, with everyone's contributions categorized appropriately.

Picture the evolution of a file as a train ride—not from one location to another but between points in time. The train stops at various stations, with each stop representing a distinct change or event in the file's history. Version control is the conductor of a file's temporal train journey. It lays the tracks (*branches*), constructs the stations (*commits*), and orchestrates connections with other lines (*merges*). When used correctly, version control can transform file management from an unlabeled linear path to a dynamic and intentional journey through time, complete with stops, reroutes, and combined paths, ensuring you always reach your desired destination with clarity and efficiency.

What Is Git?

The most common version-control system among coders, and the one I recommend you learn and adopt, is Git. This free and open source software is widely used, has an active community, and is full of helpful features.

Git's defining characteristic is that it allows projects to become *distributed*: Each version-controlled project, called a *repository*, can be spread across multiple computers rather than confined to one central location. This aspect will come into play in “Navigating Local and Remote Repositories” on page 80 when we discuss the components of the Git version-control system and see how to use them to work collaboratively with others.

Why Git Works Best with Code

Git is designed primarily to track the changes in code files. This is possible because, on the most basic level, code is just text, and alterations made to code amount to nothing more than additions, deletions, and modifications to lines of text. Version-control systems like Git track and document all the changes within each line of code in a file. For coders, this translates to a clear road map of their code's evolution, enabling them to trace back every alteration, understand its context, and ensure consistent, informed progress throughout the development life cycle.

Text is a relatively easy medium for version-control systems to keep track of for a few reasons. First of all, text is uniform, transparent, and universally understood. It doesn't require specialized software to view or interpret, and there isn't any hidden metadata or required configurations that might cause some aspect of the file to be misinterpreted. Text is also linear and sequential, with a clear start and end to each line. Each of these points can serve as a reference for keeping track of changes.

By contrast, Git isn't well suited to managing more-complex file types like Microsoft Word documents or Excel workbooks. This is because, unlike plaintext files such as Python scripts, something like an Excel workbook is an intricately woven mosaic of symbols and codes that can't easily be broken into comprehensible lines of text. Behind the scenes, an Excel file is an amalgamation of several files and directories of different types and structures. You can see this for yourself if you open a *.xlsx* file in a text editor like VS Code. The contents don't appear as a neat grid of rows and columns like those you'd see when viewing the file in Excel. Instead, the majority of the content will appear as gibberish or a sequence of strange characters and symbols, nothing like the simple and coherent lines of text in a Python script.

When you try to track the evolution of a large, complex file structure like a *.xlsx* file, Git struggles to determine and represent the exact changes made. An Excel file's convoluted labyrinth of data and metadata just isn't conducive to Git's line-by-line methodology of tracking changes. For this reason, there's no version-control solution for Excel that works as well as Git does for code. Microsoft has added the capability to track changes and collaborate on files, but this generally can't match Git's efficiency and robustness in handling changes to code files, especially when it comes to files with long, varying histories and multiple collaborators.

Git vs. GitHub

Many new coders conflate Git with GitHub, but they aren't the same thing. *Git* is free software that resides on your computer, enabling version control for your projects. *GitHub* is an online space to host and share Git repositories. Posting Git repositories to GitHub allows you to collaborate with others on the same code in a synchronized manner.

To put this in Microsoft Office terms, Git is analogous to a local application, like Excel or Word, that lives on your computer. GitHub, on the other hand, is akin to a service like OneDrive, Dropbox, or Google Drive where you can store, share, and collaboratively work on files over the internet. The inherent difference between GitHub and these other services is that it has unique features that specifically leverage Git's version-control features.

GitHub isn't the only hosting site for Git repositories. Some other names you may hear are Bitbucket, GitLab, and SourceForge. All these platforms host Git repositories, but each has unique features that cater to specific users with different needs. As of now, GitHub is by far the most widely used of these platforms, which is why we'll use it in this chapter.

Working Within Git Repositories

A Git repository (*repo* for short) is a structured workspace that holds and tracks all the files pertaining to a project. From Git's perspective, where you store those files matters. Therefore, to integrate Git within your workflow, you need to organize your work in a way that Git understands. This means saving the files for a project in a centralized place, with every file intended for version control residing inside the Git repository's parent directory.

Centralizing Your Work in a Single Folder

In an Excel-based workflow, the location where you put your files doesn't necessarily matter. You can always link or reference a file from another part of the filesystem if necessary. As a result, Excel-based teams typically don't put much thought into how they organize their files. Most teams I've seen have a haphazard file structure that looks something like this:


```

A-Team Shared Drive/
├── Alice/
│   ├── Project A/
│   │   ├── data.xlsx
│   │   └── report.docx
│   ├── Project B/
│   └── random_notes.txt
├── Bob/
│   ├── Project A analysis.xlsx
│   ├── Project C/
│   └── draft.pptx
├── Project A/
│   ├── Charlie/
│   │   ├── draft_v2.docx
│   └── final_report.docx
├── Project B/
│   ├── Alice-feedback.docx
│   └── Bob-notes.txt
└── Random/
    ├── random_file1.docx
    └── random_file2.xlsx

```

You can tell that this file structure grew organically over time. Some files are organized by project, some by person, and some were simply put in *Random*, probably because they didn't fit cleanly into any other folder.

While this file structure is less than ideal, you can't really blame Alice and Bob. The team has no compelling reason to prioritize having a clean file structure. Locating and referencing data isn't necessarily more difficult this way than it would be if the file structure were organized and decluttered. If anyone needs to reference data that's stored in a file in a far-off directory, they can just put the filepath into the Excel formula where they need the data. For example, *random_file2.xlsx* can sum the values in column B of *Alice/Project A/data.xlsx* in a simple formula:

```
=SUM('C:A Team Shared Drive\Alice\Project A\[data.xlsx]Sheet1'!B:B)
```

Likewise, if Alice, Bob, or Charlie is looking for a file and doesn't know which directory it lives in, they can always search for the file by name. Yes, the haphazard structure is ugly, but it really isn't that big of a deal.

When you're using Git, however, haphazard file structuring becomes a problem. Git operates by keeping constant track of the changes that occur in the files within a repository. If you place files outside of the main folder structure of a Git repository, those files become undetectable to Git.

Here's an example of the file structure of a simple Python project that makes appropriate use of Git:

```
project_a/  
├── .git/  
├── .gitignore  
├── README.txt  
├── python_script1.py  
├── python_script2.py  
└── data/  
    ├── dataset1.csv  
    └── dataset2.csv
```

The *project_a* folder represents the overall repository. For the repository's version-control system to apply to a file, it must be inside this folder (or one of its subfolders). In this case, the repository consists primarily of two Python files, *python_script1.py* and *python_script2.py*, and two CSV files stored in the *data* subfolder. The *README.txt* file contains a description of the project and its guidelines; coders typically include an explanatory file with this name as part of their projects.

The remaining two items in the repository serve purposes unique to Git. The *.git* folder is Git's internal directory for tracking and storing data about all the changes to the repository. It exists for every Git repository and is automatically created when you create the repository. The *.gitignore* file contains a list of files that shouldn't be tracked by version control—more on this in a moment.

Both of these Git-specific filepaths start with a period. This indicates that they're hidden. They won't be displayed in standard directory listings unless specifically requested. If you were to open the *project_a* root directory in your operating system's file browser, *.git* and *.gitignore* won't automatically be visible. You'll have to modify your settings to make them show up. In Windows, open the Control Panel and select **File Explorer ► Show Hidden Files**. In macOS, open any directory in Finder and press ⌘-SHIFT-**.** (period) to reveal the hidden files. You can also list the hidden files in a directory from the command line by entering `dir /a` in Windows or `ls -a` in macOS.

Navigating Local and Remote Repositories

Earlier in this chapter, we mentioned that Git repositories allow multiple coders to work on the same project in a distributed way and that platforms like GitHub can host repositories to facilitate that distributed workflow. The way this works is that each individual coder has their own *local repository*, which lives on their own computer. Meanwhile, these local repositories are linked to a centralized *remote repository* that lives on the GitHub website.

A local repository is your working draft of a project. Think of it like saving a draft when you're writing an email. The draft version serves as a sandbox for experimentation, where you can test, refine, and discard ideas without worrying about unintended consequences. When you decide to send the

finalized version of your draft, the email becomes available to others. Similarly, *pushing* changes from your local version of the repository to the remote version makes those changes available to your collaborators. In this way, the remote repository serves as the “official,” or public-facing, version of the project that includes the edits everyone made to their local repositories.

Note that you should never keep a local Git repository on a shared drive like a network drive, iCloud, Microsoft SharePoint, or OneDrive. In fact, there isn’t any need to store files in a shared drive when you’re using Git. Instead, everyone can access the files and collaborate through the remote repository. We’ll discuss the Git features that facilitate this in “Syncing Local and Remote Repositories” on page 86.

Maintaining a .gitignore File

When you *clone* a remote Git repository into your local environment, or vice versa, you copy the *.git/* directory in its entirety. It contains the complete history of the repo, including all versions of every file. You probably don’t want or need that kind of treatment for every file, however. This is where the *.gitignore* file comes in: It lists files that can be ignored for version-control purposes and that won’t be included when a repository is duplicated.

Tracking unnecessary files can pose several challenges and risks. For example, some files are really, really large. These can be cumbersome and might fail to upload or download reliably. Other files contain sensitive information that you wouldn’t want shared. If files containing protected data, like passwords or API keys, inadvertently make their way onto an online platform like GitHub, they could be exposed to unintended audiences. Also, some files contain information used by each person’s individual system that doesn’t have relevance outside their own environment, such as logs or configuration files.

Knowing this, it’s crucial to be discerning about the files that are tracked and stored in Git, and to maintain your *.gitignore* file accordingly. Here’s what the *.gitignore* file might look like for the example *project_a* repository shown earlier:

```
# .gitignore content
data/
```

This indicates that the contents of the *data/* folder should be excluded from being tracked by Git.

When considering which files can safely be ignored, keep in mind that Git uses an “all-or-nothing” approach to managing information about the files in a repository. When you make changes to a file in Git, these changes apply to the repository as a whole—but only the specific version of the repository that you’re working on. You’re not just saving a new version of a particular file but effectively snapshotting the state of the entire project at that moment in time. This holistic approach ensures that all aspects of your project are synchronized in every version, but it also underscores the importance of being judicious about which files are included in the repository. A thoughtful

.gitignore file can ensure that the repository remains clean, organized, and manageable for everyone involved.

Setting Up a Git Repository

The exact process for setting up a Git repository will differ depending on the code editor you're using, whether you're working in an existing repository or creating a new one, and whether your remote repository is public or private. In this section, we'll take a high-level look at the process. If you get stuck, try searching Medium, Stack Overflow, or Reddit for details and the exact steps and commands relevant to your specific situation.

Installing Git

To set up a repository, you first have to install Git on your machine if you haven't already. Git is open source software, so this is free to do. While there are lots of ways to install Git, the easiest and safest is to download it from <https://git-scm.com>.

Once you've finished the installation process, make sure Git is working by asking at the command line which version of Git your system sees:

```
git --version
```

If you get an error, you'll know something isn't set up correctly. One common error that you may encounter is due to your system not being able to locate the *git.exe* file. To fix this, add the necessary filepath to your PATH environment variable, as we discussed in "Customizing and Storing System Settings" on page 10.

Configuring Git

To work with remote repositories, you'll need to establish a connection between the Git software and your profile on GitHub (or another remote hosting platform). To do this, first establish an account on the platform's website. Be sure to consult with your company's IT team to determine whether it has an existing corporate account or guidelines you should follow.

Once you've created your account, you need to connect it to Git on your local machine. You can do this in two ways. For simple operations that don't require authentication, like using only existing publicly available repositories, you can enter your GitHub username and email address at the command line like so:

```
git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"
```

Most of the time that you're using a remote-hosted repository, extra authentication is required. To do this, you'll first need to create a *personal access token* (PAT) on the hosting platform. This PAT acts as a secure key for your account and takes the place of a password. Depending on the platform you're

using, the specific instructions for creating the PAT may differ slightly, so refer to the hosting platform's documentation if you get stuck. Once you have your PAT, you can use it when Git prompts for your password during operations like cloning, pushing, and pulling from remote repositories.

Creating a New Git Repository

There are two methods of establishing a repository. You can either create one from scratch or clone an existing one. Using either method, you end up with two versions of your repository that are synced up with each other: the local one that lives on your computer and the remote one that lives on the internet.

Creating a Repository from Scratch

To establish a local repository at the command line, use the `cd` command to navigate to the folder where you want your files stored. As we discussed earlier, Git works best when all the tracked files are in a central location. Make sure that the folder you're using is on a local drive and not a shared or network drive.

Once you're inside the folder that you want to turn into a repository, run this command to initialize the repository:

```
git init
```

This will create the `.git` directory containing the files that hold all the information about commits and branches. After this is done, your folder is now a local Git repository.

Next, you'll want to connect your local repository to a corresponding remote repository on the hosting platform you're using so you can share the project and collaborate with others. To do this, follow the instructions to create a new, empty repository on your chosen remote hosting platform. Then find the address of the remote repository. It may look something like `https://github.com/<your-username>/<repository-name>.git`. To link your local repository to this remote repository, run this command from your local repo, passing along the remote repository address:

```
git remote add origin https://github.com/your-username/repository-name.git
```

After executing this command, any files in the local repository will automatically be copied to the remote repository.

Cloning an Existing Repository

Instead of creating a local repository and then copying it to a remote repository hosted on the internet, you can do the reverse by *cloning*, or copying, an existing remote repository into your local system. To do this, run the following command:

```
git clone https://github.com/your-username/repository-name.git
```

This creates copies of the remote directory and all its files on your local machine. It also establishes a link between the local and remote repositories so future pushes and pulls will go to the right place.

Generally, when you collaborate on code with others, one person will create the repository locally and set up the corresponding remote repository, and then everyone else will clone it.

Tracking Changes to Files

In this section, we'll walk through how Git is able to track basic changes to the files in a repository. To follow along, create a Git repository called *my_repo* by following these steps:

1. Create a new *my_repo* directory and navigate into it by using `mkdir my_repo` and `cd my_repo`.
2. Initialize the directory as a Git repository by using `git init`.
3. Create a corresponding remote repository on GitHub, then link your local repository to the remote one by using `git remote add origin Remote_Repository_URL`.

Once this is complete, you can begin adding and changing files.

Git's Staging Area

Git doesn't automatically track every file that lives in the repository folder. It tracks changes to a file only if the file is part of the repository's *staging area*. To see this, let's create a new text file in the *my_repo* directory called *some_text.txt* with one line of text:

```
This is the first line.
```

Git won't be paying attention to *some_text.txt* yet, since we haven't added it to the staging area. You can do this via the `git add` command:

```
git add some_text.txt
```

When you add a file to the staging area, you've marked it for inclusion in the next commit.

If you now run `git status`, you should see output indicating that the file has been added to the staging area and that there are changes to be committed:

```
git status
On branch main

No commits yet
```

```
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   some_text.txt
```

This indicates there's a new file, *some_text.txt*, in the repository.

Commits

Git doesn't track the changes to a repository constantly, at every moment in time. You have to tell it when to save the state of the project and to take note of changes that have occurred by making a *commit*. Commits are the stations in our train analogy. You wouldn't just jump off a train in the middle of the wilderness; you'd wait for the train to reach a station. Similarly, you can't access a version of the file from an unmarked point in time.

Each commit is essentially a checkpoint that's available for you to return to in the future. It contains a snapshot of what each file in the repository looked like at that point in time, what changed since the previous commit, and who changed it. You can also think of a commit as the \$25,000 mark in *Who Wants to Be A Millionaire*, or like saving your progress in a video game. If any errors or questions about your previous work arise later, you can always return to a previous commit. Let's commit the staged file *some_text.txt* to save the initial version we created:

```
git commit -m "Add some_text.txt with initial content"
```

You should always include a descriptive message with your commits by using the `m` argument so you can look back later and read about what changed. The `git commit` command itself also outputs a summary of the changes being committed:

```
1 file changed, 1 insertion(+)
```

To see how commits function as checkpoints in the repository's history, change the contents of *some_text.txt* as follows and save the changes:

```
This is the UPDATED first line.
```

Now that you've made changes, you need to add them to the staging area again and make a new commit:

```
git add some_text.txt
git commit -m "Update first line in some_text.txt"
```

Git's history now shows two commits: your initial commit and your update to the first line.

Syncing Local and Remote Repositories

Your local and remote repositories should remain mostly in sync as you work on a project so that changes are safely backed up and available for others who may be collaborating with you. The remote version will inevitably lag the local version as you're making live updates, but you should never let it get too far behind. To synchronize your local repository with its remote counterpart, use the `fetch`, `pull`, and `push` commands.

fetch

The `git fetch` command allows you to download updates to files from the remote repository to your local repository, but it doesn't make any changes to your local files. In other words, fetching doesn't change your local repository at all; it simply makes you aware of what's been updated remotely without actually applying those updates. It's like taking a sneak peek at what's new without committing to any changes.

The `fetch` command by default fetches changes from the *origin*, which in Git terms is the default name for your remote connection. This is just a convention, and Git doesn't require this name, but it's widely adopted in the Git community. When you're using `fetch`, you can also specify the name of a branch, a concept we'll cover more in depth in the next section, and specifically fetch changes from there. Here's how it works:

```
git fetch origin main
```

This fetches the changes for a branch called *main* from *origin*, the remote repository.

pull

If you want to commit to the changes you see in the remote repository, you need to *pull* rather than *fetch* with the `git pull` command. Pulling doesn't just tell you about updates in the remote repository; it actively incorporates them into your local repository. This ensures that your local version is aligned with the latest modifications to the remote version. This is a combination of `git fetch` and `git merge`, which we'll discuss further in the next section.

Similarly to `fetch`, you can specify a particular remote branch to pull from if you'd like:

```
git pull origin main
```

This pulls the local repository with any changes from *main*.

push

The opposite of `pull` is `push`. The `git push` command uploads the content from your local repository to a remote repository. This makes the changes

you've made locally available in the remote repository. Think of this as the act of sharing your work, akin to finally sending your draft email.

In most cases, you'll use `push` without specifying a branch name, but just as with `fetch` and `pull`, you can add a branch name if need be:

```
git push origin main
```

This pushes your changes specifically to *main* on the remote repository.

Managing Histories

Git allows you to manage multiple, parallel histories of the same file through branches. A *branch* is a series of commits that are tied together to form a single, shared history. Branches are kept separate from one another, which makes it possible to have multiple, divergent histories of the same file. Branching allows multiple people to work independently on a project without interfering with one another, or for you to freely experiment with different ways of implementing new code.

Going back to the train analogy, branches are the different lines on the train system. In New York City, for example, *avenues* run north-south, and *streets* run east-west, and north-south subway lines run along several of the avenues, including 7th Avenue, 8th Avenue, Lexington Avenue, and 2nd Avenue. All four of these subway lines have a station called 86th Street, since these avenues traverse that same east-west street. Branches in a Git repository are like these different north-south lines. Two branches may both have commits on the same file that include conflicting changes, but the two parallel histories may have diverged.

The Main Branch

To help manage divergent branches, Git repos have one branch that's considered the default. So far, the changes you've made to *some_text.txt* have been on the default branch, *main*. To see this, run the `git branch` command, which lists all the branches in your local repository and the one you're currently using.

NOTE

In prior years, the primary branch of a repo was called master. More-recent versions of Git have switched to main.

Feature Branches

To keep the main branch unadulterated and free from the bugs that come with new code, you can move your untested and experimental code to offshoots from *main* called *feature branches*. It's customary to give descriptive names to feature that accurately describe what you're going to be doing with them.

While on the main branch of *my_repo*, run the checkout command to create a new feature branch called *my-feature-branch* and start using it:

```
git checkout -b my-feature-branch
```

At the time of its creation, a feature branch inherits all the files and changes of the main branch up to that point, so *my-feature-branch* starts off with all the history included in *main*. From there, however, the feature branch can diverge. To illustrate, make some changes to your feature branch's *some_text.txt* file:

```
This is the UPDATED AGAIN first line.  
This is the second line.
```

Use **git add** and **git commit** to stage and commit these changes to the file, and run **git status** to see a summary of what's been committed.

Merges

To integrate changes from one branch into another in Git, you use *merging*. This process takes the commits from one branch and applies them to another. This can involve merging in the changes from a feature branch back to *main*, or vice versa, or merging one feature branch into another.

To merge the commits from *my-feature-branch* into *main*, you need to first switch back to the main branch from the feature branch:

```
git checkout main
```

With the main branch checked out, you can now merge your feature branch into it with the **merge** command:

```
git merge my-feature-branch
```

This command tells Git to take all the changes that have been made in *my-feature-branch* and merge them into the main branch. If the changes in the two branches don't conflict, Git will automatically create a new *merge commit* that incorporates all the changes.

Once all the changes have been made and the code is ready to become part of the default version of the repo, the feature branch is joined with the main branch, along with its history of commits. After that happens, the feature branch no longer serves a purpose. It can either live out the rest of its existence as a redundant relic of the past or be deleted.

Conflicts

You can use merges to incorporate changes that others have made to files in the repository. Because you're dealing with divergent histories of files, this can lead to *merge conflicts*, which occur when Git can't automatically reconcile differences in code between two commits. This usually happens when

multiple people have made changes to the same lines of code, or when one branch deleted a file that the other branch modified.

To explain this, let's say that while you were editing *some_text.txt* on my *-feature-branch*, your esteemed colleague Alice was making edits to the same file, committing them to her feature branch called *alice-feature-branch* and then pushing them to the remote repository. Here's what Alice's *some_text.txt* looks like:

```
This is Alice's first line.
```

This differs from your current version of the file:

```
This is the UPDATED AGAIN first line.  
This is the second line.
```

To merge in Alice's changes, you first need to make your local repository aware of them by using the `fetch` command:

```
git fetch
```

If you now run `git branch`, you should be able to see Alice's branch. To see her changes to *some_text.txt*, you can run the `diff` command:

```
git diff alice-feature-branch
```

The output will give a summary of the differences between the two files. It should look something like this:

```
diff --git a/some_text.txt b/some_text.txt  
index 1234567..89abcde 100644  
--- a/some_text.txt  
+++ b/some_text.txt  
@@ -1,2 +1,1 @@  
-This is the UPDATED AGAIN first line.  
+This is Alice's first line.  
  This is the second line.
```

Now that you've seen Alice's changes, you can merge them into your feature branch. This will allow you to keep the second line that you added, which doesn't conflict with Alice's version, and gives you the option of keeping Alice's first line or yours, or changing it to something else entirely:

```
git merge alice-feature-branch
```

Since a conflict exists, Git will indicate that a manual merge is necessary for *some_text.txt*. When you open the file in your text editor, you'll see something like this:

```
<<<<<< HEAD  
This is the UPDATED AGAIN first line.  
=====
```

```
This is Alice's first line.  
>>>>> alice-feature-branch  
This is the second line.
```

Now you can decide which line to keep or can edit the line to incorporate elements from both versions, then remove the conflict markers (<<<<<<, =====, and >>>>>>). To combine both lines, you may change the file to something like this:

```
This is the combined first line from both Alice and me.  
This is the second line.
```

For the changes to be reflected in the remote repository, you'll need to go through the steps to stage them, commit them, merge them into *main*, and then push them to the remote version of the repository:

```
git add some_text.txt  
git commit -m "Resolve merge conflict by combining Alice's and my changes"  
git checkout main  
git merge my-feature-branch  
git push
```

After you run `git push`, the merge conflict will be resolved in both your local and remote repositories.

Best Practices for Using Git

Git was created to facilitate keeping track of changes to code, so it works best when files, branches, and commits are clearly labeled and defined. Imagine that instead of using easy-to-remember letters, the New York City subway gave its trains vague, haphazard names like *train-test* and *updated-train-test*. It would be difficult to plan your route, give people directions, or identify lines on a map. Similarly, effective version control with Git requires making full use of branches, merges, commits, and local and remote repositories with proper labeling.

Here are a few good habits I recommend falling into to keep your repositories clean and organized, and to get the most out of Git:

Commit often More stations on the time train means more places can be reached. By consistently committing your changes, you create a detailed history of your project's evolution, which gives you more options to go back to when necessary. This makes it easier to track progress and pinpoint when an issue first appeared.

Write clear commit messages Clear messages enable both you and other contributors to understand the rationale and impact of each change. For example, calling a commit adding `summary()` function to `script.py` is much easier to understand than the hopelessly general `code updates`.

Use feature branches Segregating your work based on specific features or improvements keeps the main branch stable and bug-free for everyone using it. This approach ensures that any potential issues in the new code won't disrupt the main project until they've been thoroughly tested and vetted.

Name branches in a standardized way Using a consistent naming convention for your branches makes it easier to keep track of them. You could use prefixes to denote the type of branch, such as *feature/* or *bugfix/* followed by a brief description. For instance, *feature/sales-report-automation*.

Keep your feature branches in sync with your main branch Regularly merging changes from *main* into your feature branches helps prevent your feature branches from diverging too far from the main line of development, which causes fewer conflicts and makes merging smoother.

Push and pull between local and remote repositories frequently Regular interactions with the remote repository ensure that you're always working with the latest version and that your contributions are safely stored online. If your local repository starts to diverge too much from its remote counterpart, merging can become tedious.

Adopting these good habits ensures not only that you are making the most of Git and version control but also that your hard work will be available for others to learn from and add to.

Troubleshooting with Status Messages

New Git users tend to get stuck when they make assumptions about their Git repository that aren't true. For example, you may accidentally establish the local repository in an unintended directory and later realize it's not in the right location. Or you could forget which branch you're currently working on and make changes intended for another branch. You might merge branches prematurely, resulting in unexpected changes in your code. These are common experiences, and virtually everyone who uses Git has had them.

If this happens to you, you can use Git's diagnostic commands to help you identify the issue and get unstuck. Table 4-1 explains some of these commands, what they do, and when to use them.

Table 4-1: Git Diagnostic Commands

Command	Use for troubleshooting
git status	Identify unstaged changes, staged changes, and untracked files
git diff	View differences between commits, branches, or the working directory and index
git log	Review the commit history to trace when changes were made
git blame	Determine who last modified each line of a file and in which commit
git show	Inspect changes in a specific commit
git branch	List, create, or delete branches to manage different lines of development

If these commands don't help identify and solve the issue, you can always make use of the plethora of information about Git that's available publicly on the internet, but understanding the fundamentals discussed in this chapter will help immensely in sifting through that information. A solid grasp of the underlying concepts and architecture behind Git commands empowers you to frame precise questions for Google, ChatGPT, or any future resources that might emerge. This understanding not only aids in problem-solving but also streamlines your learning process.

Integrating Git with VS Code

Most popular code editors, including VS Code, have features that make Git more user-friendly by providing built-in support or extensions for Git operations. These features allow you to visually see important information like which files have been staged or committed, what branch you're on, and who changed each line and in which commit.

To integrate Git with VS Code, you can start by utilizing the built-in Git support that VS Code offers. This includes viewing changed files, staging changes, committing them, and viewing the Git history directly within the editor. You can find more detailed explanations of how this works in the VS Code source-control documentation at <https://code.visualstudio.com/docs/sourcecontrol/overview>. You can also download several extensions to VS Code in order to expand this functionality for different purposes.

How Git Works

Behind the scenes, the inner workings of Git are quite complex. On a basic level, Git needs to be able to keep track of all changes in the repository, but it also can't take up too much space on your computer. Keeping a full copy of every file in the repository at every commit would take up a large amount of memory, but the actual size of the *.git* folder is usually no more than a few kilobytes, smaller than most simple scripts. How does Git do it?

A few foundational computer science concepts come into play. These structures enable Git to efficiently store and retrieve data. Understanding these inner workings isn't entirely necessary for getting started with Git, but it will make things easier when Git becomes integrated into your workflow and you require more-precise control over the versions and histories of your files.

Directed Acyclic Graphs

One tool Git uses to do its job is a *directed acyclic graph* (DAG). A DAG can be thought of as a series of points connected by arrows, like a family tree with arrows pointing from children to their parents. Based on the direction of the arrow, you can easily identify which is the parent and which is the child in the tree. These arrows are the *directed* part of the DAG. *Acyclic* simply means the graph has no loops, or points referencing each other in a circular fashion. In

our family tree example, someone wouldn't be both a parent and a grandchild of the same person; that would create a cycle, or loop.

Every commit in a Git repo is a point, or *node*, in a DAG that points to its parent commit. These interconnected nodes create a treelike structure that facilitates the tracing of the lineage of each piece of code. Since the commits are directionally connected, it's far easier for Git to search for specific points in its history than it would be if the commits were stored separately or with directionless connections.

As the merge conflict example earlier in this chapter outlined, when two branches have edits on the same lines, Git relies on its internal structures, like the DAG, to track the lineage of both sets of changes. By understanding which commits led to the differing versions, Git can highlight the differences and request manual intervention. This ensures the integrity of the project's history while giving you control over reconciling discrepancies.

Hashing

Git uses another computer science concept called a *hash table* to quickly reference and locate commit data. Think of hash tables as a sort of index, allowing Git to instantly access any given commit without searching through every single one.

Git also uses the cryptographic technique of *hashing*, a different but related concept, to ensure data integrity with all the information that it stores. When a commit is created, Git produces a unique identifier of the commit's content and metadata. This identifier, called a *hash*, acts as a unique key and serves as a verification mechanism to ensure that the data remains unaltered.

Conclusion

This chapter provided all the information you need to create a version-controlled project using Git. We discussed when and why to use version control, and how Git can make the typically chaotic and awkward ordeal of managing divergent histories of files into an elegant, harmonious, intuitive process. We then covered the basics of setting up a repository, making changes to files, and managing those changes through branches and merges. And finally, we reviewed some best practices for using Git and showed how Git works under the hood.

Taking control of the history of your work has myriad wide-reaching benefits beyond just helping you manage your Python code. Knowing who has done what, and how, can lead to increased precision, better teamwork, and greater efficiency, provided this knowledge is utilized effectively. For Excel-based teams of people who are used to working on their own files without much insight into what they have to offer their colleagues or what their colleagues have to offer them, the transition to Git often means a move toward true collaboration and collective growth.

Git offers transparency not only into the individual contributions of lines of code but also into the broader vision of the project as a whole. Grasping

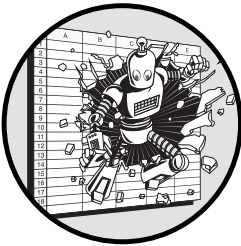
this bigger picture cultivates a shared ethos of building something together. Work no longer means getting lost in a maze of obscurely named files tucked away in forgotten folders. Instead, each contribution emerges as a vital link in a series of interconnected threads, weaving together a tapestry of collective achievement and progress.

PART II

WORKING WITH DATA

5

DATA ANALYSIS MADE INTERACTIVE



This chapter covers Jupyter Notebooks, an interactive alternative to scripts for writing and running Python code. We'll discuss the basics of working with notebooks and will consider when it makes sense to use them rather than scripts. In the end, you'll learn to use Jupyter Notebooks to improve the process of writing more exploratory and analytical code.

Python scripts work great as a canvas for writing and running code, but many times it helps to work out your ideas in an interactive environment. It can often be helpful to see immediate feedback from your code, see how it reacts to changing specific variables, or quickly display data in visualizations. And it's nice for all this functionality to be in one place. All of this isn't really possible when you are working inside a static *.py* script file.

Excel, on the other hand, does it beautifully. Since all your data, formulas, and charts are in one place, self-contained within the Excel application, immediate feedback is a built-in advantage. If you make a change to one cell, any formulas that depend on that cell automatically update, and charts adjust in real time. Modifications you make to data in spreadsheets are seamless,

whereas updating a Python script requires you to rerun the entire script to see the result.

Wouldn't it be nice if you could have the speed, modularity, reliability, and capacity of Python while also keeping everything in one place and available for interactive development with instant results? That's exactly what Jupyter Notebooks are for. Their immediate feedback is especially useful for data analysis, the focus of Part II of this book.

When you're looking to analyze, summarize, and visualize data, you frequently don't know exactly what you'll want your code to do from the outset, and you'll often make small changes, the results of which will impact what you decide to do next. The entire point of the process is experimentation. After all, if you knew the conclusion, you wouldn't be doing any data analysis in the first place. Compare this to writing a Python script to automate a process, as we did in Chapter 3. In that case, you knew exactly what the script needed to do, and the point of the script was to get you there.

What Are Jupyter Notebooks?

A *Jupyter Notebook* is a type of Python file that enables interactivity, allowing you to write and run your code, its output, and annotations and explanations of what it's doing all in one document. These notebooks are organized into *cells*, each of which can contain one of three kinds of content: Python code, Markdown text, or raw text.

A notebook's cells can be executed in any order, and you can also split them, merge them, or rearrange them however you like within the notebook. This allows for a flexible, organic workflow, where you can restructure and recompute your document as needed. You don't necessarily need to start a document with a clear idea about where it will end up, and you can do as little or as much planning as you wish.

Code Cells

Code cells allow you to write and execute Python code, and see its results. Figure 5-1 shows an example code cell in a Jupyter Notebook.



```
[1]: from datetime import date
      year = 1987
      month = 11
      day = 19
      start_date = date(year, month, day)
      start_date.strftime('%m/%d/%y')
[1]: '11/19/87'
```

Figure 5-1: The input and output of a code cell

In this example, the code cell imports the `date` class from Python's `datetime` module and defines variables for a specific year, month, and day.

The `start_date` variable is assigned a date object created using these values. The final line uses the `strftime()` method to format the `start_date` into a string.

Notice that the formatted date ('11/19/87') is displayed as output directly underneath the cell. Code cells always display their output (if any) after the code is executed. This includes anything that would print into the console while the code inside the cell is running, like `print()` statements or errors. It also includes the value of the last expression within the cell, even if it's not a `print()` statement, as the example in Figure 5-1 illustrates. The `start_date.strftime("%m/%d/%y")` expression evaluates to a string-formatted version of the date object, so that string is printed underneath the cell as its output. This automatic feature of Jupyter Notebook makes it easy to inspect the results of a cell without needing to write extra code.

Code cells build on one another: You can use the variables and functions that you've defined in one cell in subsequent cells. For example, say you define a function like this:

```
def reformat_date(dt):  
    return dt.strftime("%m/%d/%y")
```

After you execute this cell, the `reformat_date()` function will be available to use in the rest of the cells you execute, no matter their location in the Jupyter Notebook. For example, you could create another cell with code like this:

```
from datetime import date  
  
start_date = date(1987, 11, 19)  
reformat_date(start_date)
```

There's no limit to the number of times you can reuse the same function in other cells, so you could have yet another cell that formats a different date:

```
from datetime import timedelta  
  
end_date = start_date + timedelta(days=365)  
reformat_date(end_date)
```

Even though these cells are separate, the `reformat_date()` function, which you defined in a cell that was executed earlier, can be used across the notebook. This flexibility helps you break your work into smaller, more manageable steps while still being able to reuse code. It also means that the order in which you run your cells does matter, even if the order in which they appear in the document doesn't. If you haven't yet executed a cell where you define a variable or function, you'll get an error if you try to execute another cell that references that variable or function.

Markdown Cells

Markdown cells in Jupyter Notebooks allow you to write and style text by using Markdown, a formatting language used to specify which lines of your

text are headers, bullet points, numbered lists, code, or links. Markdown cells are useful for adding explanations or notes alongside your code cells. Figure 5-2 gives an example of a Markdown cell.

Example: Using `date` to Format a Date

How to create and format a start date:

1. Import `date` from the `datetime` module
2. Assign the year, month, and day
3. Create a `date` object for the date
4. Format the date

Figure 5-2: A Markdown cell before execution

In the example cell, the `###` and ``` (backtick) Markdown symbols indicate that text should be formatted as a heading and as code, respectively. The numbers also create a numbered list. When you execute the cell, the plain-text gets converted into a styled output based on the Markdown notation, as shown in Figure 5-3.

Example: Using `date` to Format a Date

How to create and format a start date:

1. Import `date` from the `datetime` module
2. Assign the year, month, and day
3. Create a `date` object for the date
4. Format the date

Figure 5-3: The same Markdown cell after execution

The rendered view presents the content in a more structured and visually appealing way, making it easier to read and understand.

Raw Text Cells

Unlike Markdown text cells, raw cells display their content exactly as written, without any formatting or execution. No output is attached to them, as shown in Figure 5-4.

The date should be "11/19/87".

Figure 5-4: A raw cell in a Jupyter Notebook

Plaintext is stored in the cell, and nothing else. If you execute the cell, nothing will change.

Raw cells are useful for adding notes or comments within your notebook. Because they don't include any additional formatting or output, they work well for content that you may want to store temporarily or keep for reference, like reminders, drafts of other cells, or comments.

The JupyterLab Code Editor

Because their format is different, Jupyter Notebooks benefit from a different code-editing environment than scripts. JupyterLab is a code editor designed to work with Jupyter Notebook. Like VS Code, JupyterLab is open source, so it's free to download and install.

While our focus in this chapter is on JupyterLab, a VS Code extension called Jupyter is also available that allows you to run Jupyter Notebooks within VS Code. This can be convenient when the rest of your project contains scripts that you edit in VS Code. You can find this extension by searching for *Jupyter* inside the Extensions tab located in VS Code's left sidebar.

Installing JupyterLab

If your Python environment is set up correctly, as discussed in Chapter 1, installing JupyterLab is simple. Just open your terminal or command prompt and run the following:

```
pip install jupyterlab
```

As usual, you may need to use `pip3` instead of `pip`, depending on your setup. This command downloads and installs JupyterLab, as well as all the dependencies it requires.

To test whether JupyterLab was installed correctly, run this command:

```
jupyterlab --version
```

If the console prints a version of JupyterLab, you've succeeded. If you run into any issues in this step, your environment variables might not be set up correctly. To remedy that, see Chapter 1.

Launching JupyterLab

To create a Jupyter Notebook, you first need to launch JupyterLab. Instead of being a stand-alone application like Excel, JupyterLab runs in a web browser such as Google Chrome, Apple Safari, or Microsoft Edge, and you need to start it up from the command line.

To do this, open a console window and type `cd` to navigate to the directory that you want to contain your code. For example, if you plan to store your files in a directory called *project* that lives at the path `C:\Users\YourUsername\Projects`, you would run this in the console to navigate there:

```
cd C:\Users\YourUsername\Projects\project
```

Once your console is in the intended folder, run the following command to open the JupyterLab code editor in your computer's default web browser:

```
jupyter-lab
```

This command references a script called *jupyter-lab* that likely lives in the *bin* directory of your Python installation. This is the directory where Python stores executable files for various packages and tools.

JupyterLab should open automatically in your browser, but if it doesn't, the console will print a URL that you can enter into your browser's address bar to access the code editor. This will generally start with *http://localhost:8888/lab* and then be followed by a long alphanumeric token.

Because JupyterLab runs from the console, in order to keep your session running, you also need to keep the console window open. If you close it, JupyterLab will close as well.

Creating a Jupyter Notebook

Once JupyterLab opens, you should see a Launcher tab where you'll be able to create a new Jupyter Notebook, as Figure 5-5 illustrates.

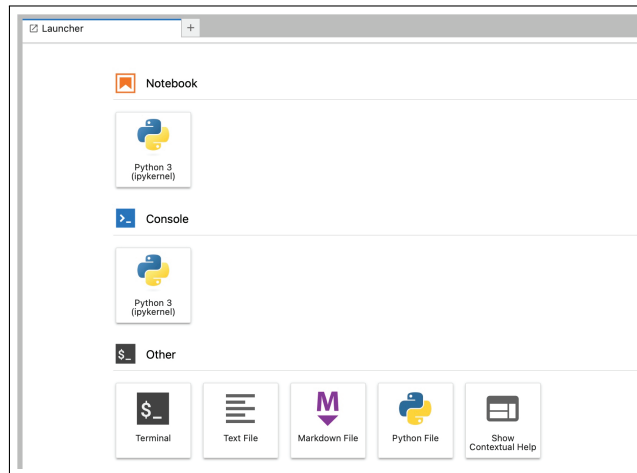


Figure 5-5: The JupyterLab launcher

Look for the Notebook section and select the option corresponding to the Python interpreter you'll use for the new Jupyter Notebook. Depending on your Python installation, you may see only one option. When you select your option, a new notebook will be created and will open as a tab in the window. It will be called *Untitled.ipynb* by default, but you can change the filename to something that better reflects its purpose. The *.ipynb* extension indicates this is a Jupyter Notebook and not a regular Python file.

You can see the newly created notebook in the Jupyter file sidebar, as shown in Figure 5-6. The dot next to the filename indicates that the notebook is currently running.

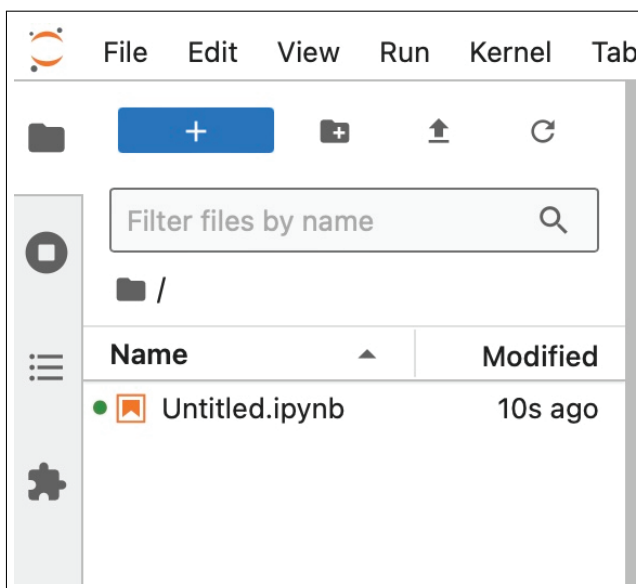


Figure 5-6: A new notebook file in the Jupyter sidebar

To rename the notebook, right-click the file in the sidebar. The file will live inside the directory where you started Jupyter. In our example, the full filepath to the notebook would be `C:\Users\YourUsername\Projects\project\Untitled.ipynb`.

Closing a Jupyter Notebook

When you're done working on your notebook, go back to the command line terminal and press `CTRL-C` to close JupyterLab. You'll be prompted to type `Y` or `N` to shut down the Jupyter server. If you don't respond quickly, the prompt will go away. In that case, you can shut down the server by closing the console window.

JupyterLab automatically saves your work periodically, but it's a good idea to manually save your notebook by clicking the Save icon or pressing `CTRL-S` (or `⌘-S` on macOS) before closing.

Building a Jupyter Notebook

To get a feel for how Jupyter Notebooks work, let's build a simple one called *Mortgage-Calculator.ipynb* that allows you to calculate the monthly payment for a 30-year mortgage by using the loan amount and the annual interest rate. We'll use Markdown cells intertwined with code cells to illustrate how you might use a Jupyter Notebook not only to write and run code but also to add readable explanations that will help the notebook make sense to others.

First, open a new notebook from the JupyterLab Launcher tab and rename the file to *Mortgage-Calculator.ipynb* in the left sidebar. It will already have one cell by default. Click into the cell and convert it to a Markdown cell

by selecting **Markdown** in the drop-down menu on the toolbar. Within the cell, add some Markdown text that explains the purpose of the notebook:

```
# Mortgage Monthly Payment Calculator (30-Year Fixed)
```

In this example, we will calculate the monthly payment for a 30-year fixed mortgage using:

- * The loan amount (principal)
- * The annual interest rate

To execute this Markdown cell, press SHIFT-ENTER, or click the ► button in the toolbar. This will render the text with Markdown formatting. The # line should render as a heading, and the * lines should render as bullet points.

To add another cell to your notebook, use the + button in the Jupyter-Lab toolbar. The default type of cell contains Python code. Add the following code to the cell to define a `calculate_mortgage()` function:

```
def calculate_mortgage(principal, annual_interest_rate, years=30):
    monthly_interest_rate = annual_interest_rate / 100 / 12
    total_payments = years * 12

    rate_factor = (1 + monthly_interest_rate) ** total_payments
    numerator = principal * monthly_interest_rate * rate_factor
    denominator = rate_factor - 1

    monthly_payment = numerator / denominator

    return monthly_payment
```

After you run this cell, you'll be able to use the `calculate_mortgage()` function in subsequent cells.

Now let's create a cell that gives an example of using `calculate_mortgage()` to compute a monthly mortgage payment:

```
loan_amount = 200000
interest_rate = 4

monthly_payment = calculate_mortgage(loan_amount, interest_rate)

print(f"The monthly payment for a loan of ${loan_amount}" +
      f" at an interest rate of {interest_rate}% is" +
      f" ${monthly_payment:.2f} per month.")
```

When you run this cell, the following output should display:

```
The monthly payment for a loan of $200000 at an interest rate of 4% is $954.83
per month.
```

You might also want to experiment with different interest rates to see how they affect the monthly payment. To do this, add the following code to a new cell:

```
interest_rates = [2, 3, 4, 5, 6, 7]

for rate in interest_rates:
    monthly_payment = calculate_mortgage(loan_amount, rate)
    print(f"At {rate}% interest," +
          f"the monthly payment is ${monthly_payment:.2f}")
```

Running this code should produce the following output:

```
At 2% interest, the monthly payment is $739.24
At 3% interest, the monthly payment is $843.21
At 4% interest, the monthly payment is $954.83
At 5% interest, the monthly payment is $1073.64
At 6% interest, the monthly payment is $1199.10
At 7% interest, the monthly payment is $1330.60
```

As you continue building the notebook, you can weave in Markdown cells as needed to explain what's going on in each part of the process.

In all, this notebook serves two purposes. It takes advantage of the interactivity and immediate feedback of Jupyter Notebook by allowing you to test and see results in real time. It also creates a document that integrates code, output, and explanations in one cohesive space, making it easy for others to follow your work.

Scripts and Notebooks Working Together

In Chapter 3, we explored writing scripts, or *.py* files, using VS Code. These scripts are an excellent place for code that you plan to automate or reuse elsewhere. Those use cases cover a lot of the reasons you would want to turn to Python, but not all of them. Most notably, scripts aren't ideal for interactive data analysis. By contrast, Jupyter Notebooks, or *.ipynb* files, let you run Python code in a way that's conducive to exploring and visualizing data, since they enable interactivity and include embedded output. Here's a closer look at the kinds of code that are better suited for scripts versus notebooks:

Code you should put in scripts

- Code you plan to import into other files
- Code you want to manage line-by-line with Git
- Code you want to run automatically
- Code that needs to be robustly tested

Code you should put in notebooks

- Code that includes unstructured exploration of datasets
- Code used to create ad hoc or flexible charts and visualizations
- Code that contains examples of output to be shared with others
- Code that will never be imported into other files

This choice isn't always an either/or situation: In many Python projects, using a mixture of both *.py* and *.ipynb* files is appropriate. This way, you can benefit from the structure and simplicity of scripts as well as the interactivity and fluidity of notebooks.

When you're working in JupyterLab and write code that might be useful outside of the notebook (like functions you anticipate reusing in the future), you can put that code inside a script and import it into the notebook rather than writing it in a notebook in the first place, where it will live in isolation, inaccessible from elsewhere. Here's how you might structure a basic project that stores functionalized code in a *.py* file and nonfunctionalized code used for exploration and visualization in a *.ipynb* file:

```
Project/  
├── util.py  
└── Notebook.ipynb
```

In this setup, *util.py* contains reusable functions. For example, it might have functions for loading and cleaning data, like this:

```
# util.py  
  
def load_data(data):  
    # Function to load data  
    return data  
  
def clean_data(data):  
    # Function to clean data  
    return data
```

You might then use *Notebook.ipynb* for interactive exploration and visualization of the data. Just as in a script, you can import functions from *util.py* into a notebook and then call those functions within the notebook cells:

```
from util import load_data, clean_data  
  
data = load_data(data)  
cleaned_data = clean_data(data)
```

This separation keeps your code clean and organized. The nitty-gritty of the functions lives in the script, while the interactive exploration of the data lives in the notebook. This promotes modularity and testability, since you

can import the functions and run them in a controlled environment, but it also allows you to benefit from the advantages of a notebook.

Version Control with Jupyter Notebooks

Because they include so many additional features, Jupyter Notebooks aren't as simple as scripts to manage with version control. With a script, you can easily track changes line by line, making it straightforward to see what was added, removed, or modified. However, notebooks, like Excel files, are complex in their raw form and need to be processed to be useful. If you were to look at the change log of a version-controlled *.ipynb* file, it would be a jumbled mess of difficult-to-decipher text.

This isn't to say that it's not useful to include Jupyter Notebooks in your Git repositories. Despite the difficulty in tracking changes, some of Git's features are useful for notebooks. For example, GitHub renders notebooks directly in the browser, allowing you to view the notebook with its output as you would in JupyterLab. As such, if the notebook is part of a public or private repository, you can share it by sending a simple link to collaborators, who can then view the notebook without needing Jupyter installed locally. And of course, since those notebooks are stored within the repository, you'll always be able to access their current state.

The issue with version control is another reason it's valuable to separate some parts of your code into scripts outside of your notebooks. Since scripts allow you to more precisely collaborate and manage the histories of the code inside them, version control will be more effective. In the previous section's example, Git could give you insight into the exact changes to *util.py*, showing a clear diff of which functions were added, removed, or altered. However, for *Notebook.ipynb*, the version-control history might be cluttered, but you would have a readily available rendered version of the file with a shareable link.

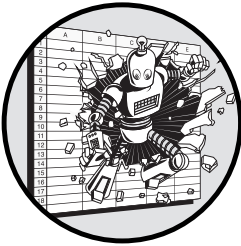
Conclusion

This chapter covered the basics of Jupyter Notebooks, an important tool for making Python practical for data analysis. We first explained how Jupyter Notebooks are structured and how to use them. Then we discussed JupyterLab, a code editor specifically designed for working with notebooks. Finally, we covered when and how to create a workflow that integrates both scripts and notebooks in order to take advantage of the maintainability of scripts and retain the interactivity of notebooks.

Now that you've learned the fundamentals of using Jupyter Notebooks, Python will become a more practical—maybe even an essential—tool for data analysis. You'll see this in the next few chapters, which cover other topics related to working with data in Python.

6

ANALYZING AND TRANSFORMING DATA



In this chapter, we'll dive into pandas, a library that simplifies data analysis in Python. We'll focus on working with the tabular data structure in pandas called a `DataFrame`, which you'll learn to create, manipulate, and transform. You'll see how to conduct the same kinds of analyses you're used to doing in Excel by using Python, and you'll learn a few more advanced techniques that Excel makes unnecessarily difficult. With a Jupyter Notebook as your canvas and pandas as your medium, you'll find that Python makes the art of data analysis accurate, efficient, and user-friendly.

Many times in Excel, you're forced to work with individual spreadsheet cells even when operations should apply to entire rows or columns all at once. For example, every cell in column D might be the sum of the cells in the same row in columns B and C. Excel will make you put a formula into each cell in column D when really you want to treat the whole column as a

single unit. Wouldn't it be easier to apply the sum operation to the entire column at once, instead of handling each cell individually?

The pandas library is built around this concept of performing operations efficiently on entire rows or columns of data, without needing to manually adjust the data points individually. Bundling operations together like this opens up a myriad of new ways to wrangle data, and this can make data analysis much more intuitive and efficient.

Through a series of simple examples, this chapter illustrates some of what you can do with pandas DataFrames. Although you may have a bit of a learning curve when it comes to thinking in terms of entire datasets rather than individual cells, this curve is much less steep when you are already familiar with Excel, since you already have experience working with tabular data. Using DataFrames should therefore be one of the most accessible and beneficial ways of incorporating Python into your existing workflow.

Efficient Data Analysis with pandas

The pandas library expands upon the basic elements in Python to make it more useful for data analysis. In other words, the data types in pandas build upon native Python data types like lists and dicts to make common data operations such as sorting, filtering, summarizing, and pivoting simpler and more efficient.

The library pulls much of its functionality from NumPy, another Python library that contains code related to data analysis and scientific computing. This means you can often use NumPy functions directly on pandas objects and that pandas structures can easily be converted to their NumPy counterparts.

Series and DataFrames

In pandas, data is organized into two primary data types: Series and DataFrames. A *Series* is like a single column of an Excel spreadsheet. It's made up of rows, each having a unique name (called an *index*), a value, and a specific position within the Series. A *DataFrame* is like a whole Excel sheet, consisting of multiple Series joined together by rows into a tabular structure. Internally, the DataFrame contains code that keeps track of the relationships among the individual data points, like the row and column labels, and the order of the rows and columns. These are examples of *properties*, attributes tied to the DataFrame that describe its characteristics. Because DataFrames maintain all this information, you don't need to explicitly state the relationships between data points when you work with them. This makes operations like summarizing and aggregating data and manipulating the shape of a DataFrame more intuitive.

Given the DataFrame's tabular structure, working with pandas isn't terribly different from working in Excel. Data lives in rows and columns, and you perform operations that manipulate the data in those rows and columns. However, DataFrames are far more flexible and efficient than spreadsheets, enabling you to analyze more data, automate your work more easily, and

better ensure its accuracy. You can feasibly work with a nearly infinite number of DataFrames at the same time, and they can also be extremely large since they're limited only by your computer's memory, not by the constraints of a visual interface.

Working with entire datasets in DataFrames is inherently safer than working with data stored in individual cells, as you do in Excel. DataFrames allow you to perform the same operations across entire columns or datasets. You can sort, transform, summarize, and otherwise manipulate data with just a line or two of code, and you can feel confident that each element in each row and column will be treated entirely the same way. This spares you from common Excel mistakes like typing the wrong column name or selecting a cell from the wrong row in a formula. And you won't accidentally click into a cell and edit a single value or forget to drag a formula all the way down a column. By design, pandas promotes standardization and therefore fewer errors.

pandas and Jupyter Notebooks

Jupyter Notebooks are a great way to experiment with pandas DataFrames. They let you run code one cell at a time, facilitating step-by-step exploration of your data. As you perform an operation on a DataFrame in a notebook, you can immediately see the results of each cell and keep everything in one place. On top of this, Jupyter Notebooks automatically apply additional formatting to DataFrames when they're the output of a cell. So instead of viewing them as plaintext, as you would if you printed a DataFrame to the console from a regular Python script, you can view a much more readable, tabular format.

For the examples in this chapter, I recommend following along in a Jupyter Notebook. You'll become familiar with DataFrames much more quickly if you can take advantage of a notebook's ability to visualize the results of each operation and transformation right away. You can choose to practice in scripts instead, but you might miss the visualization and interactive features, which can make the learning process much more intuitive and also more similar to what you're used to in Excel. That said, DataFrames work exactly the same way in scripts as they do in Jupyter Notebooks. You can always transfer code from a notebook into a script if needed, especially if you're functionalizing parts of your analysis or want to completely automate your workflow.

Getting Started with pandas

Since pandas is a third-party package, you need to install it with pip (or pip3) before you can use it:

```
pip install pandas
```

Then you need to import pandas by adding this line to your code:

```
import pandas as pd
```

This import statement uses `as pd` to create an *alias* for pandas. This way, when you need to use code from the package, you can prefix it with just `pd` instead of `pandas`. This isn't entirely necessary, but using the shortened alias is a common convention in the Python community to make code more concise. In a Jupyter Notebook, you can add the import statement to the first cell. After you execute that cell, pandas will be available to use in the rest of the notebook.

Working with pandas typically starts with creating a `DataFrame` to hold some data. This will store the data in a tabular structure, as well as other information like row names, column names, the order of rows and columns, and the data types of each column. You can create a `DataFrame` in many ways. We'll go over two common methods: starting from scratch and reading in data from a file.

Creating a DataFrame from Scratch

Say you want to create a simple `DataFrame` from scratch with only a few rows of information in a couple of columns. Table 6-1 shows the data you'll put into the `DataFrame`.

Table 6-1: Data for the *farmers_market_df* `DataFrame`

Product	Type	Price	In season
Apples	Fruit	0.99	True
Strawberries	Fruit	2.20	False
Broccoli	Vegetable	1.50	True
Carrots	Vegetable	0.85	True

You can create this `DataFrame` in multiple ways by using lists and dicts in Python. Which is most convenient depends on the way you store the information going into the `DataFrame`. The first method is to create the `DataFrame` from a list made up of lists, where each of the inner lists represents a row in the table:

```
import pandas as pd

items = [
    ["Apples", "Fruit", 0.99, True],
    ["Strawberries", "Fruit", 2.20, False],
    ["Broccoli", "Vegetable", 1.50, True],
    ["Carrots", "Vegetable", 0.85, True],
]

columns = [
    "product", "type", "price", "in_season"
]

farmers_market_df = pd.DataFrame(items, columns=columns)
```

You create the list of lists in a variable called `items` and then use the `pd.DataFrame()` function from `pandas` to turn the list into a `DataFrame` called `farmers_market_df`. You also pass a list of column names (called `columns`) to the function. By default, `pandas` uses numbers starting from 0 as indices to identify the rows. However, using one of the `DataFrame`'s columns as an index instead is common, provided the values in that column are unique. Here, for example, you can use the `set_index()` method to designate the `product` column of our `DataFrame` as the index:

```
farmers_market_df = farmers_market_df.set_index("product")
```

Since the values in the `product` column are unique, they can function as identifiers for the individual rows. When you try to access specific values by their index and column, Python will know exactly where to look.

When you're working with `pandas` in a Jupyter Notebook, it's common to end a cell with the variable name corresponding to your `DataFrame`, like this:

```
farmers_market_df
```

When you execute the cell, JupyterLab will see this and know to print a clean, formatted version of the `DataFrame` as the cell's output (see Figure 6-1).

	type	price	in_season
product			
Apples	Fruit	0.99	True
Strawberries	Fruit	2.20	False
Broccoli	Vegetable	1.50	True
Carrots	Vegetable	0.85	True

Figure 6-1: The `farmers_market_df` `DataFrame` displayed as the output of a Jupyter Notebook cell

The second option is to create a `DataFrame` by using a dict of lists. Each key in the dict is a column name, and its value is a list of all the values in that column:

```
items = {
    "product": ["Apples", "Strawberries", "Broccoli", "Carrots"],
    "type": ["Fruit", "Fruit", "Vegetable", "Vegetable"],
    "price": [0.99, 2.20, 1.50, 0.85],
    "in_season": [True, False, True, True]
}

farmers_market_df = pd.DataFrame(items)
farmers_market_df = farmers_market_df.set_index("product")
```

This code produces exactly the same result as the previous example, but since the column names come from the keys in the dict, you don't need to specify them separately. You still need to use `set_index()` to turn `product` into the index column, however.

And third, you can create the `DataFrame` by using a dict made up of dicts:

```
items = {
    "product": {
        0: "Apples", 1: "Strawberries", 2: "Broccoli", 3: "Carrots"
    },
    "type": {
        0: "Fruit", 1: "Fruit", 2: "Vegetable", 3: "Vegetable"
    },
    "price": {
        0: 0.99, 1: 2.20, 2: 1.50, 3: 0.85
    },
    "in_season": {
        0: True, 1: False, 2: True, 3: True
    }
}

farmers_market_df = pd.DataFrame(items)
farmers_market_df = farmers_market_df.set_index("product")
```

As before, each key in the outer dict represents a column name, but this time we use the keys of the inner dicts to explicitly specify the row number for each value in the column. Therefore, this method is typically more appropriate when you want to specify and use row names.

Reading Data from a File into a DataFrame

You can also create `DataFrames` by reading data from an external file, like a CSV file. To show this, we'll use an example dataset of winning New York Lottery numbers. You can download the CSV version of this dataset at <https://catalog.data.gov/dataset/lottery-powerball-winning-numbers-beginning-2010>. Save the file as *winning_lottery_numbers.csv* and move it to the same directory as your Jupyter Notebook. If both files are in the same directory, you won't need to specify the full filepath when you reference the CSV file in your code. Here's how to read the data from the file:

```
import pandas as pd

file_name = "winning_lottery_numbers.csv"
lottery_df = pd.read_csv(file_name, index_col=0)
```

You use the `pd.read_csv()` function to read the data from the CSV file specified with `file_name` and store it in a `DataFrame` called `lottery_df`. The named argument `index_col` sets which column to use for the index, or row

names, of the DataFrame. If this argument isn't provided, the DataFrame defaults to using sequential numbers, starting from 0, as the index.

Just as with `farmers_market_df`, when you end the Jupyter cell with the name of the variable containing the DataFrame, the DataFrame will be displayed as the cell's output. However, since `lottery_df` contains too many rows to reasonably show in the notebook, the result would be cut off. You can instead use the `head()` method to print only the first five rows:

```
lottery_df.head()
```

Figure 6-2 illustrates what the result should look like in your notebook.

	Winning Numbers	Multiplier
Draw Date		
09/26/2020	11 21 27 36 62 24	3.0
09/30/2020	14 18 36 49 67 18	2.0
10/03/2020	18 31 36 43 47 20	2.0
10/07/2020	06 24 30 53 56 19	2.0
10/10/2020	05 18 23 40 50 18	3.0

Figure 6-2: The first five rows of the `lottery_df` DataFrame

The `pd.read_csv()` function isn't the only one available for reading data from a file into a DataFrame. For example, there's also `pd.read_excel()`, `pd.read_json()`, and so on. For a list of all these functions, as well as all their possible arguments, see the pandas documentation at <https://pandas.pydata.org>.

Working with pandas Data Types

Each column in a DataFrame is represented by a pandas Series, and unlike lists or dicts, a Series can hold only values of a single data type. To see the data types of every column in the DataFrame, retrieve its `dtypes` property:

```
lottery_df.dtypes
```

This returns each column and its corresponding type:

```
Winning Numbers    object
Multiplier         float64
dtype: object
```

The output indicates that the `Winning Numbers` column is of type `object`, while the `Multiplier` column is of type `float64`, meaning it contains decimal numbers. The `dtype: object` at the end of the output refers to the data type of the `dtypes` output itself.

For numeric data, pandas skips the `int` and `float` types from native Python and uses `int64` and `float64` instead. These are types from the NumPy library that support larger datasets and provide more precision (the 64 refers to the 64 bits of computer memory each number takes up).

One data type specific to pandas is `category`. It represents *categorical data*, where the entries in a column can take on a limited list of possible values. For example, in `farmers_market_df`, you have two options for the `type` column: `Fruit` and `Vegetable`. The `category` type would be appropriate for this column.

Sometimes when you create a `DataFrame`, pandas may not immediately know which type to use for a given column, especially if it contains both text and numbers or comes from a source like a CSV file, where all data is initially read as text. In that case, pandas defaults to using the generic object data type, which can encompass any type of data, including strings, numbers, and more-complex objects. This was the case with the `Winning Numbers` column in the lottery `DataFrame`: It appears to contain numeric data, but because of the mix of digits and spaces, pandas assigns it the object type.

When pandas defaults to the object type, you may want to convert the column to a more specific data type so you can perform operations particular to that data type with the column. To convert a column's data type, use the `astype()` method. For example, here's how to change the `type` column of the `farmers_market_df` `DataFrame` from the object data type to `category`:

```
farmers_market_df["type"] = farmers_market_df["type"].astype("category")
```

Notice that you access the column by putting its name in square brackets, just like accessing a value from a list or a dict.

For temporal data, pandas defaults to the `datetime64` type for dates and the `timedelta64[ns]` type to represent units of time, instead of the `datetime` and `timedelta` types from the `datetime` module. The `datetime64` and `timedelta64[ns]` types are higher-precision versions of `datetime` and `timesdelta`. Like `int64`, `datetime64` uses 64 bits to store date and time information, while the `[ns]` in `timedelta64[ns]` refers to nanoseconds, indicating the precision level for measuring time differences.

When you read temporal data from a file into a `DataFrame`, pandas doesn't automatically understand that it should be formatted as a date. For example, the `index` column of our `lottery_df` `DataFrame` consists of dates in the form `MM/DD/YYYY`, which pandas interprets as text. To convert the column to the `datetime64` type, you need to use the `pd.to_datetime()` function:

```
lottery_df.index = pd.to_datetime(lottery_df.index, format="%m/%d/%Y")
```

The `format` argument here, `%m/%d/%Y`, uses the same standard formatting system as the `datetime` package, which we discussed in Chapter 2.

Moving Between Excel and pandas

Since `DataFrames` are organized in rows and columns, they work well in a workflow that combines Python and Excel. Knowing how to move data back

and forth between Excel and Python makes the process of incorporating Python into your real-world workflow much simpler.

Using the Clipboard

The simplest way to move data from a DataFrame to Excel is to use the handy `to_clipboard()` function in pandas, which puts your DataFrame into your computer's clipboard in tabular format so it can be pasted virtually anywhere, including Excel. Try using this to move the `farmers_market_df` DataFrame out of pandas and into Excel. First, run this code from your Jupyter Notebook to transfer the data from `farmers_market_df` to the clipboard:

```
farmers_market_df.to_clipboard()
```

Now open Excel and paste the contents of the clipboard into a blank sheet. You should see the values from the DataFrame organized into the rows and columns of the Excel sheet.

To go from Excel to Python, copy a range of cells in Excel, then use the `pd.read_clipboard()` function in pandas. With this function, pandas will attempt to read the data from the clipboard and convert it into a DataFrame. To test it, copy the cells you just pasted into Excel to save them to the clipboard. Then run the following Python code:

```
df_from_clipboard = pd.read_clipboard(index_col=0)
```

This inserts the data from the clipboard into a new DataFrame called `df_from_clipboard`. The `index_col=0` argument specifies that the first column should be used as the index of the DataFrame.

The pandas clipboard functions aren't always successful because they depend on your environment and the format of the data copied. They also require you to move manually between Excel and Python, which is more tedious than a more automated method. However, the functions can still be convenient for quickly moving data between Python and Excel in more ad hoc situations, which tend to come up often in data analysis.

ESCAPING TO EXCEL

Pythonizing a process that involves data analysis doesn't have to be all-or-nothing, especially when you're still becoming familiar with Python. Before you're fully comfortable working with data via code, looking up the syntax for every function or method you use may sometimes feel tedious. Thankfully, an easy escape route exists if you need it: the `to_clipboard()` method.

If you ever feel overwhelmed learning pandas, you can simply take your DataFrame as it is, export it to your clipboard, paste it directly into an empty spreadsheet, and finish up your data wrangling in a more familiar environment. This way, you can ease into Python at your own pace. Over time, as you gain confidence and proficiency, you'll find yourself relying on this workaround less.

Using openpyxl

A more robust method for moving data between Python and Excel is to use the `read_excel()` and `to_excel()` functions in pandas. These, in turn, harness `openpyxl`, a Python library with code to read and write Excel files. You don't need to have Excel installed to use `openpyxl`, since it interacts with only *.xlsx* files, not the Excel application itself. You do need to install `openpyxl` with `pip` (or `pip3`), however:

```
pip install openpyxl
```

Let's save `farmers_market_df` to an Excel file called *farmers_market.xlsx* with the `to_excel()` function:

```
file_name = "farmers_market.xlsx"
farmers_market_df.to_excel(file_name, engine="openpyxl")
```

This uses `openpyxl` to create a new Excel file called *farmers_market.xlsx* with the contents of `farmers_market_df` starting in cell A1 on the first sheet.

On the flip side, to read data from an Excel file via `openpyxl`, use the `pd.read_excel()` function, which works similarly to the `pd.read_csv()` function we discussed earlier:

```
file_name = "farmers_market.xlsx"
df_from_excel = pd.read_excel(file_name, index_col=0)
```

Here, we read in the content of the first sheet of *farmers_market.xlsx* and store it in a DataFrame called `df_from_excel`. As with `pd.read_csv()`, the `index_col=0` argument specifies that the first column of the Excel sheet should be used as the DataFrame index.

Manipulating and Transforming Data

In this section, we'll discuss common operations that can be applied to pandas DataFrames, such as accessing and updating values, sorting rows and columns, manipulating data, handling missing values, combining DataFrames, and performing aggregations.

Accessing Values

In Excel, rows are identified by numbers, columns are identified by letters, and cell addresses are the combination of the two. If you refer to cell A5, you're referencing the fifth cell in the first column. In a DataFrame, since the rows and columns have both names and an order, you can refer to the values within them by either technique: You can use the names and refer to values by *label*, or you can use the location and refer to values by *position*.

To refer to a cell by label, use `loc[]` to specify the element's row label and column label as strings. For instance, to access the value at row `row_label` in

column `column_name` in a DataFrame called `df`, you would use `df.loc["row_label", "column_name"]`. To refer to a cell by position, use `iloc[]` instead. For example, `df.iloc[0, 1]` accesses the element at the first row and second column of the `df` DataFrame. In pandas, selecting one or more values from a DataFrame is called *slicing*.

With both `loc[]` and `iloc[]`, you can use a list of row or column labels instead of a single label to select multiple rows and columns. For example, here's how to get the prices of just the Apples and Broccoli rows from the `farmers_market_df` DataFrame we created earlier:

```
item_list = ["Apples", "Broccoli"]
farmers_market_df.loc[item_list, "price"]
```

We create a list of the row labels we want, then pass it to `loc[]` along with the desired column label. Notice that the two rows we're selecting aren't adjacent to each other (the Strawberries row is between them). This is fine in pandas, whereas Excel requires that ranges in formulas be continuous (for example, A1:B2).

To select all rows or columns in a DataFrame, use a colon (:). For example, `farmers_market_df.loc[:, "price"]` selects all rows for the price column, and `farmers_market_df.iloc[:, :]` selects the entire DataFrame. With `iloc[]`, you can also use negative numbers to specify positions from the end of the DataFrame, similar to Python's list indexing. For example, `df.iloc[-1]` selects the last row of the DataFrame, and `df.iloc[:, -2]` selects the second-to-last column of the DataFrame.

Although `loc[]` and `iloc[]` are the recommended ways of indexing into DataFrames, they aren't the only ones. You can also retrieve specific columns in a DataFrame or values in a Series by using brackets only. For example, here we access the "price" column of `farmers_market_df`:

```
farmers_market_df["price"]
```

Once you have the column name, you can retrieve a specific element in the column by using another set of square brackets:

```
farmers_market_df["price"]["Apples"]
```

In addition to indexing based on specific row or column labels, pandas allows for *Boolean indexing*, where you filter rows based on the values in one or more columns. For example, here's how to select all products at the farmers market that cost more than \$1:

```
farmers_market_df[farmers_market_df["price"] > 1]
```

In this example, the expression `farmers_market_df["price"] > 1` evaluates to True if the price column is greater than 1, and False if not. By putting that expression inside square brackets, we select only the rows of `farmers_market_df` that meet the criteria.

Updating Values

As we discussed in the previous section, you can use a slice of a DataFrame to retrieve a value, but you can also use slicing as part of an assignment operation to set a value. For example, here we create a new column called `quantity` in `farmers_market_df` and set the entire column to the value 1:

```
farmers_market_df["quantity"] = 1
```

When you modify DataFrame values, you need to keep in mind that there are two types of subsets, or *slices*, of DataFrames: views and copies. *Views* are direct references to the original DataFrame, meaning that any changes made to the view will be reflected in the original DataFrame. *Copies*, on the other hand, are separate objects with their own data, independent of the original DataFrame. Modifying a copy will change only the copy, not the original.

Simple slices (for example, when you apply `iloc[]` or `loc[]` to cells or lists of cells) generally return views. This allows for *in-place* operations that modify the original DataFrame. Here, we update the price of apples in `farmers_market_df`:

```
farmers_market_df.loc["Apples", "price"] = 1.50
```

This is an in-place operation, since the assignment of 1.50 as the value at row Apples and column price modifies the value in the original `farmers_market_df` DataFrame. Other, more complex indexing operations aren't as straightforward. For example, indexing that involves chained assignments or conditional selections might yield copies instead of views. This means that your operation may not be directly modifying your original DataFrame but a copy of it instead. For example, consider this operation that sets the price of all fruits in `farmers_market_df` to \$2:

```
farmers_market_df[farmers_market_df["type"] == "Fruit"]["price"] = 2
```

When you run this code, you'll likely receive a warning like this:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

This means that you probably need to edit your code to modify the original DataFrame rather than the copy. You might change it as follows:

```
farmers_market_df.loc[farmers_market_df["type"] == "Fruit", "price"] = 2
```

Now we're using `loc()` to select rows containing the Fruit type of product and to directly assign a new value of 2 to the price column for those rows. Since there was no `SettingWithCopyWarning`, we can be sure that the operation affected the original DataFrame rather than a copy.

Sorting Datasets

Sorting datasets in pandas is noticeably simpler than it is in Excel. In Excel, in addition to specifying the column(s) you intend to sort by and the direction, you also need to specify the area that you want included in the sort. Cells can be unintentionally included or excluded from the sort, which can cause your data to misalign. On top of this, there's no easy way to sort columns rather than rows without copying and transposing your entire dataset.

In pandas, the values in a DataFrame are tied together by their rows and columns, so sorting is much safer. The library provides two primary functions for sorting: `sort_values()`, which allows you to sort by the values of one or more columns, and `sort_index()`, which allows you to sort by the row or column names.

Here, we use `sort_values()` to sort the products at the farmers market by price:

```
farmers_market_df.sort_values("price")
```

This code returns `farmers_market_df` with the rows ordered by their prices in ascending order. To instead sort them in descending order, add the `ascending=False` argument:

```
farmers_market_df.sort_values("price", ascending=False)
```

To sort by multiple columns, pass a list of column names to `sort_values()`. For example, here we sort `farmers_market_df` by "in_season" and then by price:

```
farmers_market_df.sort_values(["in_season", "price"])
```

To sort `farmers_market_df` by its row names in alphabetical order, use `sort_index()`:

```
farmers_market_df.sort_index()
```

To do the same but with the column names instead of the row names, add the `axis=1` argument:

```
farmers_market_df.sort_index(axis=1)
```

These methods all retain the row relationships in the DataFrame, meaning that the values within any given row won't change after the sort. Only the order changes.

Manipulating Data

The same operations that you can apply to individual integers, floats, strings, and Booleans can be performed on whole columns of DataFrames, and in some cases on entire DataFrames, through single lines of code. In this section, we'll discuss how to operate on numbers, strings, and dates within DataFrames.

Arithmetic

To perform basic arithmetic operations on DataFrame columns, use the standard `+`, `-`, `*`, and `/` operators. To illustrate, we'll create a new simple DataFrame called `numbers_df` like this:

```
numbers_df = pd.DataFrame({
    "a": [1, 2],
    "b": [3, 4],
})
numbers_df
```

As usual, the `numbers_df` at the end of the notebook cell prints a tabular view of the DataFrame:

	a	b
0	1	3
1	2	4

Let's create a new column in the DataFrame that's the sum of two other columns:

```
numbers_df["c"] = numbers_df["a"] + numbers_df["b"]
```

We access columns `a` and `b` and use the `+` operator to add them together. Now `numbers_df` includes the new column `c`:

	a	b	c
0	1	3	4
1	2	4	6

In the first example, both operands were DataFrame columns, but you can also do arithmetic with one column and a single value. For example, here we use `*` to multiply the values in column `a` by 10:

```
numbers_df["d"] = numbers_df["a"] * 10
```

This creates a new column `d` whose values are 10 times the values in the original `a` column.

The pandas library also has function versions of the standard arithmetic operators, including `add()`, `sub()`, `mul()`, and `div()`. Here's how to sum `a` and `b` by using `add()` instead of the `+` operator:

```
numbers_df["c"] = numbers_df["a"].add(numbers_df["b"])
```

These arithmetic functions may seem redundant since Python already has the `+`, `-`, `*`, and `/` operators. However, these functions are used primarily when an operator on its own doesn't give enough information about the way the operation should be applied. Adding function arguments clarifies the operation.

For example, when you're performing operations between a two-dimensional DataFrame and a one-dimensional object like a list, it's not

clear whether you want to perform the operation on the rows or the columns of the two-dimensional DataFrame. You need to specify whether you want the operation to be performed along the vertical axis (column-wise) or the horizontal axis (row-wise), using the arithmetic functions' axis argument. To illustrate, let's redefine the `numbers_df` DataFrame with its original two rows and two columns, and also create a list with two values:

```
numbers_df = pd.DataFrame({
    "a": [1, 2],
    "b": [3, 4],
})
numbers_list = [100, 200]
```

To add `ls` to `df` column-wise, we use `axis=0`:

```
numbers_df.add(numbers_list, axis=0)
```

This operation adds 100 to each element in the first row (1 and 3) and 200 to each element in the second row (2 and 4), giving the following result:

	a	b
0	101	103
1	202	204

Conversely, to add `numbers_list` to `numbers_df` row-wise, we use `axis=1`:

```
numbers_df.add(numbers_list, axis=1)
```

This operation adds 100 to each element in the first column (1 and 2) and 200 to each element in the second column (3 and 4), yielding this DataFrame:

	a	b
0	101	203
1	102	204

The same `axis` argument works for the other arithmetic functions, `sub()`, `mul()`, and `div()`.

String and Date Operations

When your columns are made up of strings or dates, you can apply functions available specifically to those data types to the entire column at once. For example, in `farmers_market_df`, we can make the values in the `type` column (Fruit and Vegetable) lowercase by using the `lower()` string operation we discussed in Chapter 2:

```
farmers_market_df["type"].str.lower()
```

This returns a new Series that has fruit and vegetable all in lowercase:

product	fruit
Apples	

Strawberries	fruit
Carrots	vegetable
Carrots	vegetable

Notice that we can't use `lower()` directly but rather have to call `str.lower()`. The `str` is an *accessor property* for the column that specifies that the values need to be treated as instances of the string data type.

In `lottery_df`, we can similarly use the `split()` string operation to split each element in the column of winning numbers into its own column. Once again, we need the `str` accessor property to be able to call `split()`:

```
lottery_df["Winning Numbers"].str.split(expand=True).astype(int)
```

We use `str.split(expand=True)` to split the string elements of the `Winning Numbers` column into separate columns, and then `astype(int)` to convert the resulting columns of strings into integers to make them more useful.

To manipulate columns holding dates, use the `dt` accessor property, followed by the function you want to apply, such as `dt.strftime()`. For example:

```
from datetime import datetime

date_series = pd.Series([
    datetime(2001, 4, 7),
    datetime(2003, 5, 8),
    datetime(2004, 6, 9)
])
date_series.dt.strftime("%Y-%m-%d")
```

We create a `Series` called `date_series` with three dates. Then we use the `dt` accessor to convert the entire column into a string with the format `%Y-%m-%d`. The result looks like this:

0	2001-04-07
1	2003-05-08
2	2004-06-09

Because we use the `dt` accessor, pandas knows to apply the method to all the values in the series at once.

Handling Missing Data

Missing values are a common problem in datasets. In pandas `DataFrames`, missing values in columns of any data type are indicated with an `NA` (not available) marker, which you can create like this:

```
pd.NA
```

The `NA` marker acts as a placeholder, similar to `#N/A` in Excel. It explicitly represents missing or null values in the data so you can handle them as

needed. The library also has several built-in functions for detecting and handling missing values, summarized in Table 6-2.

Table 6-2: Missing Data Operations in pandas

Operation	Description	Example
<code>isna()</code>	Detects missing values in the DataFrame or Series	<code>df.isna()</code>
<code>notna()</code>	Identifies nonmissing values in the DataFrame or Series	<code>df.notna()</code>
<code>dropna()</code>	Removes rows or columns containing missing values	<code>df.dropna()</code>
<code>fillna()</code>	Replaces missing values with a specified value or method	<code>df.fillna(0)</code>
<code>replace()</code>	Replaces any value, including NA, in the DataFrame	<code>df.replace(pd.NA, 0)</code>
<code>ffill()</code>	Replaces missing values with the previous value	<code>df.ffill()</code>

To see how some of these functions work, let's replace the price of strawberries in `farmers_market_df` with NA:

```
farmers_market_df.loc["Strawberries", "price"] = pd.NA
farmers_market_df
```

This changes only the value in row `Strawberries` and column `price` to be NA, as shown here:

	type	price	in_season
product			
Apples	Fruit	0.99	True
Strawberries	Fruit	NaN	False
Carrots	Vegetable	1.50	True
Carrots	Vegetable	0.85	True

When you see the DataFrame printed out, the value that you set by using `pd.NA` is shown as `NaN`. This is short for *not a number* and is a standardized way of expressing missing values that's common in coding. The actual value of the cell is NA, but `NaN` is how that value is represented when it's printed.

Now let's test the `dropna()` function, which removes the row with the missing value:

```
print(farmers_market_df.dropna())
```

This should output a DataFrame with the `Strawberries` row removed. Alternatively, we can use `fillna()` to fill in a 0 for the missing value:

```
(farmers_market_df.fillna(0))
```

Whether you drop rows with missing data or attempt to fill in the missing values somehow will depend on your specific goals for the analysis.

Combining DataFrames

There are two common ways of combining DataFrames in pandas: concatenation and merging. *Concatenation* combines DataFrames by stacking them

either vertically or horizontally, while *merging* combines DataFrames based on common values in specified columns.

Concatenation

Concatenation is useful when you have multiple DataFrames with the same columns and you want to simply append the rows from one after the rows of the other (vertical concatenation). Or perhaps you have two DataFrames with the same indices, and you want to stick the columns from one alongside the columns of the other (horizontal concatenation). In either case, you can use the `pd.concat()` function, with the `axis` argument to specify whether you want to join the rows (`axis=0`) or columns (`axis=1`).

As an example, say we have a second DataFrame of products from the farmers market called `additional_df`:

```
additional_items = {
    "product": ["Tomatoes", "Blueberries"],
    "type": ["Vegetable", "Fruit"],
    "price": [1.20, 3.00],
    "in_Season": [True, True]
}
```

```
additional_df = pd.DataFrame(additional_items).set_index("product")
```

Now we can use `concat()` to append `additional_df` onto `farmers_market_df`:

```
pd.concat([farmers_market_df, additional_df], axis=0)
```

This returns a new DataFrame with all six products included: four rows from `farmers_market_df` and two more from `additional_df`.

Alternatively, we can use `concat()` to add a column to `farmers_market_df`. To do this, let's create a Series called `suppliers` that gives the suppliers for each item in `farmers_market_df`:

```
suppliers = pd.Series({
    "Carrots": "Rainbow Meadow Farms",
    "Apples": "Rainbow Meadow Farms",
    "Broccoli": "Rainbow Meadow Farms",
    "Strawberries": "Berry Bliss Orchards"
})
```

Now we'll append the `suppliers` column to the end of `farmers_market_df`:

```
pd.concat([farmers_market_df, suppliers], axis=1)
```

Notice that in the definition of `suppliers`, the product names aren't in the same order as they were in `farmers_market_df`. That's okay. Since the index names of the DataFrame and Series being concatenated are the same, `pd.concat()` understands which values should go where, regardless of order.

Merging

Merging can be one of the most cumbersome operations in Excel because it generally requires you to store both datasets in sheets and perform an operation like a VLOOKUP or INDEX/MATCH across multiple columns to get all the data in a workable format. In pandas, merges can be done in a single line of code by using the `pd.merge()` function. To illustrate how to perform a merge with the `farmers_market_df` DataFrame, we'll first create the following extra DataFrame that gives characteristics of fruits and vegetables:

```
type_details_df = pd.DataFrame({
    "type": ["Fruit", "Vegetable"],
    "sweet": [True, False],
    "edible": [True, True]
})
```

To perform the merge, we need to use a column that's common to both DataFrames. In this case, that's the `type` column:

```
pd.merge(
    left=farmers_market_df.reset_index(),
    right=type_details_df,
    on="type",
    how="left"
).set_index("product")
```

We call the `pd.merge()` function, using several named arguments. The `left` argument specifies the DataFrame on the left side of the merge, and `right` specifies the DataFrame on the right side. The `on` argument specifies the common column to use for the merge, and `how` indicates how we want to fit the two DataFrames together. Here are some of the common types of merges set with the `how` argument:

- inner** Keeps only rows that have matching values in both DataFrames
- outer** Includes all rows from both DataFrames, filling in NaNs for missing matches
- left** Includes all rows from the left DataFrame and matching rows from the right DataFrame, filling in NaNs for missing matches
- right** Includes all rows from the right DataFrame and matching rows from the left DataFrame, filling in NaNs for missing matches

The `pd.merge()` function automatically removes the index from the merging DataFrames, which is why we apply the `reset_index()` function to `farmers_market_df` when we pass it as the left argument. This converts the `product` column from an index back to a regular column. Then we use `set_index("product")` after the merge to use `product` as the index of the new, merged DataFrame.

You can customize your merges in pandas in many ways. The library's documentation provides more information on all the available options.

Aggregating Data

Aggregations are operations that compute metrics across sections of a dataset, such as averages, sums, counts, and medians. The pandas library provides many aggregation functions for use on Series and DataFrames. For example, the `mean()` function calculates the average of a selection of values. Here's how to use it to find the average price of products in the `farmers_market_df` DataFrame:

```
farmers_market_df["price"].mean()
```

Table 6-3 lists some of the most common aggregation functions that can be applied to pandas Series and DataFrames. As you'll notice, these functions look and feel a lot like their equivalents in Excel.

Table 6-3: Common Aggregation Operations in pandas and Excel

Aggregation operation	pandas function	Excel equivalent
Sum	<code>df[column].sum()</code>	<code>SUM(range)</code>
Average	<code>df[column].mean()</code>	<code>AVERAGE(range)</code>
Maximum	<code>df[column].max()</code>	<code>MAX(range)</code>
Minimum	<code>df[column].min()</code>	<code>MIN(range)</code>
Count (non-NA)	<code>df[column].count()</code>	<code>COUNT(range)</code>
Median	<code>df[column].median()</code>	<code>MEDIAN(range)</code>
Standard deviation	<code>df[column].std()</code>	<code>STDEV.S(range)</code>
Count (unique values)	<code>df[column].nunique()</code>	<code>COUNT(UNIQUE(range))</code>
Product	<code>df[column].prod()</code>	<code>PRODUCT(range)</code>

Each of these pandas functions can apply across entire DataFrames instead of single columns. To do this, however, you need to specify whether you want the operation to be executed row-wise or column-wise by using the `axis` argument. For example, a column-wise sum, which returns the totals for each column, would use `axis=0`:

```
column_sums = numbers_df.sum(axis=0)
```

A row-wise sum, which returns the totals for each row, would use `axis=1`:

```
row_sums = numbers_df.sum(axis=1)
```

Using the expanded version of the `Winning Numbers` column in `lottery_df` that we created in “Manipulating Data” on page 121, we can find the median winning number for each lottery drawing by applying the `median()` function row-wise:

```
lottery_df["Winning Numbers"].str.split(expand=True).astype(int).median(axis=1)
```

This creates a Series with the same index as `lottery_df`, and values giving the median of each row.

Easily Reshaping Data with pandas

You'll often need to transform an entire DataFrame all at once by grouping and aggregating specific data points together, or pivoting rows into columns, or vice versa, so you can work with your data from all different angles. In Excel, these kinds of transformations can be clunky, manual, and consequently error prone, whereas pandas offers methods for these operations that are simple to read, write, and understand. This gives you greater flexibility to automate and scale up your calculations.

If you've used pivot tables in Excel, you'll know they work by selecting the range of cells you'd like to pivot and then selecting which column names you'd like to use as your pivot table's rows, columns, values, and filters. This enables you to transform, group, and filter your data as required. In pandas, this functionality is contained in two kinds of operations: grouping and pivoting.

Grouping

Grouping data means separating it by category so you can perform analysis on each group individually and compare them. In Excel, this is typically done by adding the category column to the rows in a pivot table, or by using formulas like SUMIFS and COUNTIFS, which aggregate data based on one or many conditions.

In pandas, grouping data is far simpler and more intuitive, and it can generally be done in a single, straightforward line of code. As an example, we can use the `groupby()` function with the types in `farmers_market_df` to find the average price of both fruits and vegetables:

```
farmers_market_df.groupby(by="type")["price"].mean()
```

We can break this line of code into three parts:

- `groupby(by="type")` tells pandas to group the `farmers_market_df` DataFrame by the `type` column. This means the fruits and vegetables will be treated as separate groups.
- `["price"]` specifies that we're interested in the `price` column of the grouped data. It indicates that the aggregation function that follows should apply to just this column.
- `mean()` finds the average of the values within each group. This aggregation function will return a new Series (or DataFrame if multiple columns are involved) with the average price by type.

The output is a Series that shows the average price across all products for each type:

type	
Fruit	1.595
Vegetable	1.175

Similarly, we can pass a list of multiple elements to the `by` argument of the `groupby()` method. For example, here we group the items in `farmers_market_df` first by `type` and then by `in_season`:

```
avg_price_srs = farmers_market_df.groupby(by=["type", "in_season"])["price"].mean()
avg_price_srs
```

Since the grouping includes two columns, the result is a Series with a multi-level index:

type	in_season	
Fruit	False	2.200
	True	0.990
Vegetable	True	1.175

With a multilevel index, multiple columns are used to uniquely identify each row—in this case, the `type` and `in_season` columns. Multilevel indexes can be difficult to work with, so it's often easiest to convert the index into a separate numeric column by using the `reset_index()` method:

```
avg_price_srs.reset_index()
```

Now the result is a much simpler DataFrame with three columns, `type`, `in_season`, and `price`:

	type	in_season	price
0	Fruit	False	2.200
1	Fruit	True	0.990
2	Vegetable	True	1.175

The rows are indexed numerically, from 0 to 2.

Pivoting

Pivoting means to view a dataset from a different angle in order to highlight or summarize specific relationships. In Excel, this can be done using a pivot table by dragging column names into the fields called Columns, Rows, and Values.

The `pivot()` method in pandas works similarly. Using `farmers_market_df`, we can create a DataFrame of Booleans that shows which fruits and vegetables are in season by specifying the product names as the index (rows), the type of product as the columns, and whether they're in season as the values:

```
avg_price_srs = farmers_market_df.groupby(by=["type", "in_season"])["price"].mean()
avg_price_df = avg_price_srs.reset_index()

avg_price_df.pivot(index="type", columns="in_season", values="price")
```

The three arguments to the `pivot()` method (`index`, `columns`, and `values`) are similar to the Rows, Columns, and Values fields, respectively, when creating a

pivot table in Excel. The result is a DataFrame that summarizes the products by type and by season and uses the product names as the index:

in_season	False	True
type		
Fruit	2.2	0.990
Vegetable	NaN	1.175

You can apply the `pivot()` method to any subset of columns in a DataFrame as long as one column provides the index, another defines the new columns, and the third provides the values.

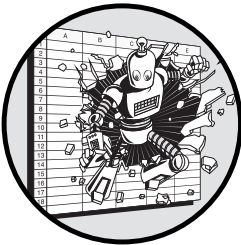
Conclusion

This chapter introduced pandas DataFrames. We started by creating DataFrames, both from scratch and by loading data from external files. Then we explored two ways to transfer data between pandas and Excel so you can easily integrate pandas into your current workflow. After that, we walked through several basic DataFrame operations, like accessing values, sorting, performing arithmetic, handling missing data, and merging. Last, we covered more advanced transformations like grouping and pivoting.

This introduced a lot of new concepts, but I hope you found most of them fairly intuitive thanks to your Excel experience. This content may seem like a lot at first, but just as you've mastered the many features (and quirks) of Excel, with a little practice you'll get comfortable with pandas. Once you do, you'll be much more nimble with your data than ever before.

7

WORKING WITH DATABASES



In this chapter, we'll look at how to use databases to store, organize, and extract data. Compared to using Excel for the same tasks, databases can make your workflow more efficient and reduce the chance of errors. First, we'll cover databases generally and why they're helpful. Next, we'll create an example database and interact with it using Structured Query Language (SQL), the primary programming language for working with databases. Finally, we'll integrate SQL commands into Python scripts.

Our emphasis in this chapter is on working with database infrastructure in a hands-on, useful way. SQL itself is pretty intuitive for most people, and it has a lot of similarities with Excel, so you already have a head start. The more difficult part is becoming comfortable with databases more generally—how to set them up, gain access to them, and make the most of them—so instead of attempting to teach you SQL in a vacuum, we'll demonstrate all of this here.

Why Use a Database?

Excel is designed around the idea that data can be organized, stored, cleaned, updated, analyzed, and visualized all in one place. When you're doing a little bit of all those things, that's exactly what you want, but most people who use Excel professionally eventually need more than a little bit of data storage and organization. And you may reach that point sooner than you think.

Poor data management sneaks up on you. It's like tooth decay, or credit card debt, or the famous frog that didn't realize it was boiling to death in the pot of water. It starts small, like when your coworker discovers that they made their report using an old Excel file with stale data, or when you struggle to reconcile multiple versions of a spreadsheet with suspiciously different conclusions. You may notice more and more situations where you can't keep quite as good a handle on the data you're working with, but you may not be able to pinpoint the foundational issue causing the cracks in the wall. In fact, the root cause is usually a lack of robust centralized data storage, but while this goes unnoticed, the resulting issues can continue to grow in number and impact and become more painful and expensive to fix.

What are some of the data-related issues that tend to come up in Excel-based workflows? One obvious limitation is size: Excel limits you to 1,048,576 rows and 16,384 columns of data. Excel also makes it all too easy to store data in inaccessible, inconsistent, or nondurable ways (like in multiple `.xlsx` files), which can lead to inefficiencies and data integrity issues. By contrast, databases are virtually limitless data stores, and they enforce structure around your data that maintains its integrity and enables you to work with it efficiently. With a database, everything is more organized, more secure, and more reliable.

Here are several ways that databases add value to an Excel workflow:

Size Databases aren't inherently tied to a visual interface, so they can handle much larger datasets much more efficiently.

Consistency Databases use validation rules and constraints to keep data consistent and reliable, so human error isn't as much of a risk as it is in Excel.

Concurrency Collaboration in Excel can be awkward and error prone, while databases support multiple users accessing and editing data simultaneously, eliminating conflicts and ensuring data integrity.

Flexibility Databases allow you to be far more nimble with your data wrangling than you can be with Excel.

Security It's far easier to enforce both broad and specific security features like user authentication and encryption with databases than it is with Excel.

Automation Like Python, databases can easily integrate with other applications, streamlining the process of automation.

You'll notice that many of these benefits are similar to those offered by Python. The similarity between Python and databases is that, unlike Excel,

neither binds you to a visual interface. This opens up a host of more powerful options for customization and scalability.

How Databases Work

Databases are designed around a set of four ideals: atomicity, consistency, isolation, and durability, or *ACID* for short. These four principles are meant to safeguard database transactions and prevent the kinds of issues we just discussed that plague Excel. In this context, a *transaction* is one or more SQL commands (like inserting data, updating data, or deleting data) that are executed as a single operation. Here's what the four ideals of ACID mean:

Atomicity

Each change to a database should be treated as a single, indivisible unit of work. If this principle is met, a transaction can't only partially finish. Either all operations within the transaction are completed successfully, or none of them are. If any part of the transaction fails, the entire transaction is rolled back, leaving the database in its original state.

Consistency

Any transaction will maintain the predefined rules and constraints placed upon the database. After a transaction, the database will be in a consistent state, and all data will adhere to integrity constraints.

Isolation

Changes to a database remain independent of one another. Any intermediate state of a transaction is invisible to other transactions. In this way, multiple transactions can take place concurrently without interfering with one another, ensuring that the database remains consistent even when many operations are happening simultaneously.

Durability

After a transaction has been committed, it will remain so, even in the event of a system failure. All committed transactions are saved permanently, typically through the use of transaction logs or backups. This ensures that data isn't lost and remains intact after a crash or hardware failure.

Incorporating Databases into Your Workflow

For Excel users who haven't ventured into database infrastructure, the jargon, the plethora of options, and the many nuances involved may seem foreign and intimidating. Once you understand the lay of the land, however, integrating a database into your workflow should become easier.

Where Databases Live

Databases can live locally on your computer, just like other files. In most corporate environments, however, they live in a place accessible to more users. This could be physical onsite servers that the company hosts itself, a

setup that offers complete control over the infrastructure but requires significant investment in hardware, maintenance, and security. Alternatively, the database can be hosted in the cloud by a third-party provider like AWS, Microsoft Azure, or Google Cloud. This allows companies to pay for what they use and easily scale resources up or down as needed.

How to Access Corporate Databases

Many people working in corporate environments who aren't software engineers face challenges when trying to gain direct access to their company's database. Obstacles tend to arise, sometimes technical and sometimes organizational. The engineering and IT teams are often hesitant to grant non-engineers access to databases because of valid security concerns. On top of this, non-engineers may not be familiar enough with the way databases work to articulate what exactly they need and the type of access they require. The upshot is that without real-world experience working with databases, it's difficult for non-engineers to become proficient in them.

To explain this more clearly, here's a short anecdote. Alice is a business intelligence analyst at Global Corp, an international business firm. She frequently works with a database that stores data she uses. However, she doesn't have direct access to the database. Instead, when she needs to access data, she has to ask Bob, a data engineer at Global Corp, to run SQL commands for her. Bob doesn't love having to do this extra work, but he is worried about giving Alice access to the database for security reasons. Alice, on the other hand, doesn't know how to ask for what she specifically needs.

Recognizing the inefficiency of the situation, Alice researches a bit about how databases work and thinks through what exactly she needs to ask for. Then she proposes to Bob a specific request: that she be given read-only access to one specific table within the database. She explains that she'll be running simple SELECT commands with limits to extract data from the table only a few times per day. By specifying the exact SQL statements she intends to run, Alice alleviates Bob's security concerns, and he feels much more comfortable granting the access she needs. This allows them both to work more efficiently. Alice can now directly create her reports, and Bob no longer needs to retrieve data for her. It's a win-win.

Sometimes the solution to a chasm like Alice and Bob's is a bit of shared knowledge in both directions. In practice, you need to do two things. First, you should determine specifically what you need to do with the database, when you need to do it, and why. Second, you should provide a detailed explanation of your needs while showing that you understand basic database concepts. With the right communication and a certain level of empathy for the database administrator's responsibility to maintain security, you too can find a mutually beneficial solution, just like Alice and Bob.

The Costs of a Database Connection

Adding a connection to a database comes with costs other than the human time and energy needed to create the connection. Requesting large amounts

of data or requesting data inefficiently can lock up computational resources for other database users. On top of this, database administrators have meaningful security risks to consider when allowing new users to connect to a database. Improper handling of database credentials, unencrypted connections, and a lack of input validation can expose the system to vulnerabilities like data breaches. It's important for you to communicate that you understand and can follow the protocols aimed at preventing these kinds of issues.

Relational Databases and SQL Workflows

Databases can take on two forms: relational and nonrelational. In *relational* databases, data is organized into tables with a predefined structure of columns. This structure governs the data types of the columns as well as the relationships between tables. Because of this structure, relational databases can be altered and explored using SQL.

By contrast, *nonrelational*, or *NoSQL*, databases don't use the same table-column structure as relational databases, so they can handle data formatted in unknown or irregular ways. For example, Redis, a popular nonrelational database system known for its speed, uses a key-value structure, where each element in the database is stored under a unique key instead of in columns. Another NoSQL format is the *document-oriented database*, which allows for more complexity and flexibility, like nested structures. MongoDB is one example of a nonrelational database system that works this way.

In this chapter, we'll focus exclusively on relational databases, which allow you to use SQL and are most common in corporate environments.

Database Management Systems

To take full advantage of a relational database, you'll need to use a *relational database management system (RDBMS)*. An RDBMS is software that provides the tools necessary to interact with relational databases. Table 7-1 lists several popular RDBMSs, all of which are compliant with ACID.

Table 7-1: Common Relational Database Management Systems

RDBMS	Development and governance team	License
PostgreSQL	PostgreSQL Global Development Group	Open source
MySQL	Oracle Corporation	Open source
SQL Server	Microsoft	Commercial (proprietary)
Oracle	Oracle Corporation	Commercial (proprietary)
MariaDB	MariaDB Corporation	Open source
Amazon Aurora	Amazon Web Services	Commercial (proprietary)
SQLite	D. Richard Hipp	Open source (public domain)
IBM Db2	IBM	Commercial (proprietary)

All these systems use SQL to manage, store, and access data, but each may have specific extensions and features that differentiate it. We'll technically be focusing on the PostgreSQL dialect and implementation of SQL in this chapter, but the basics discussed here will work in other RDBMSs as well, since basic SQL commands like `SELECT`, `INSERT`, `UPDATE`, and `DELETE` are consistent across systems.

That said, other details, including the way you define and manipulate data structures, handle date and time functions, and manage transactions, can vary. For example, PostgreSQL uses the `SERIAL` keyword to create auto-incrementing columns, whereas MySQL uses `AUTO_INCREMENT`. And sometimes it's not just the keywords that are different, but the behavior of the functions themselves, especially ones that deal with aggregations and more-complicated data types like dates. When in doubt, consult the documentation for the specific RDBMS you're using.

Sandbox Environments

SQL syntax isn't hard to learn, especially if you understand Excel, but many people don't know it because they've never encountered a database in the wild. Instead, many people try to learn using online "sandbox" tools like LeetCode or SQL Fiddle that allow you to practice SQL commands on simulated databases. Much of the real-world experience of coding in SQL centers around the process of working within the database environment itself, however, and sandbox environments don't come close to that experience. Therefore, no matter how much time you spend writing practice commands, it can be difficult to put your skills into practice and feel like you truly understand how to use SQL.

With Python scripts, the practice versions are close to the real thing. With SQL, that's not the case. Therefore, to practice SQL, I suggest you go through the process of creating a real database. To do that here, we'll use PostgreSQL.

Creating a Practice Database

PostgreSQL, or Postgres for short, is an excellent choice for learning and practicing SQL for several reasons. First, it's an open source database, meaning it's freely available and supported by a large community of developers. This makes it easy to access and find help or documentation. Second, PostgreSQL isn't just a toy database; it's powerful enough to be used in real-world applications, from small projects to large enterprise systems, and it's growing in popularity. By learning PostgreSQL, you'll know how to use a tool that can be deployed in a real business environment.

Installing PostgreSQL

Installing PostgreSQL on your system and initializing a database takes a few steps, which differ a bit by operating system.

macOS

The easiest way to install PostgreSQL on macOS is through Homebrew, a macOS package manager. Homebrew simplifies software installation by automatically handling details like dependencies and configurations so you don't have to. If you don't already have Homebrew, you can find installation instructions at <https://brew.sh>.

Once Homebrew is installed, you can install PostgreSQL:

```
brew install postgresql
```

Next, run this command to start the PostgreSQL server:

```
brew services start postgresql
```

If this is your first time starting the server, this command will also automatically set up the directories and configurations required for PostgreSQL to operate.

While the macOS installation process through Homebrew might not explicitly prompt you to set a password during installation, you can manually connect and set a password for the superuser account afterward. To do this, connect to PostgreSQL with this Terminal command:

```
psql -d postgres
```

Once inside the psql prompt (you'll see something like `postgres=#`), create the postgres role as follows (be sure to fill in a secure password):

```
CREATE ROLE postgres WITH SUPERUSER LOGIN PASSWORD 'mypassword';
```

Now exit the psql editor:

```
\q
```

To test the connection, run the following:

```
psql -U postgres
```

Now you'll be connected as the postgres superuser.

Windows

To install PostgreSQL on Windows, you'll first need to download the latest version of the PostgreSQL software from <https://www.postgresql.org/download/>. Run the downloaded installer and follow the steps outlined in the installation wizard. During the installation, be sure to set a password for the postgres user. This is the default administrative account, which has full *superuser* privileges, meaning it can perform any task within the database system, including creating and managing databases.

After you finish the installation process, the PostgreSQL server will start automatically. You can verify the installation by opening the SQL Shell (psql) from the Start menu and connecting to the default database, called postgres.

To do this, first navigate with `cd` into the PostgreSQL *bin* directory, which is typically named with the version of PostgreSQL you've installed:

```
cd "C:\Program Files\PostgreSQL\version\bin"
```

Now that you're in the *bin* directory, you can connect to the PostgreSQL server by using the `psql` command line tool:

```
psql -U postgres
```

If your installation is correct, you'll be prompted to enter the password for the `postgres` user, and then you should see the PostgreSQL prompt, which starts with `postgres=#`.

Managing Users and Databases

Once PostgreSQL is installed and running, you can execute SQL commands to create roles, users, and databases. *Roles* are a collection of privileges that define what a user can and can't do within PostgreSQL. Roles can have various permissions, such as the ability to create databases, read data, or write data. In PostgreSQL, *users* are a specific type of role that can connect to a database, but they can also take on the permissions of other roles. In other words, you can assign other roles with different permissions to a user. Roles with appropriate privileges manage *databases*, which are separate entities within PostgreSQL where your data is stored.

To manage all this, you'll first need to access the PostgreSQL command line interface (`psql`) within your shell:

```
psql postgres
```

Now you can create a user account for yourself and give it permission to create databases:

```
CREATE USER myuser WITH PASSWORD 'mypassword';  
ALTER USER myuser CREATEDB;
```

We use `CREATE` to create a user called `myuser` with a password of `mypassword` (you can choose your own more secure username and password instead). Then we use `ALTER` to update the account settings so the user can create new databases. You can also get to this same result, with the added ability to manage multiple users at the same time, by creating a role called `myrole` and assigning it to `myuser`:

```
CREATE ROLE myrole;  
GRANT myrole TO myuser;  
ALTER ROLE myrole CREATEDB;
```

This way, you have a user `myuser` with the ability to create databases through the assigned role `myrole`.

Now we'll create a new database and assign full access rights to myuser:

```
CREATE DATABASE mydatabase;  
GRANT ALL PRIVILEGES ON DATABASE mydatabase TO myuser;
```

We call the new database mydatabase, but you can use whatever name you like. With the database created, you can exit psql:

```
\q
```

This brings you back to your regular shell.

Notice that all the commands in the PostgreSQL shell end in a semicolon (;). Python takes line breaks and whitespace into account to recognize where one command ends and the next begins, but SQL doesn't. You need to mark the end of each statement with a semicolon so SQL knows to move on to the next command.

Another difference between the SQL commands and Python is the approach to capitalization. Python recognizes uppercase letters as different from their lowercase counterparts. SQL, on the other hand, doesn't differentiate, so CREATE and create mean the same thing. That said, the standard convention is to use all caps for SQL keywords (like CREATE, SELECT, and so on) and lowercase for identifiers such as users, database names, and table and column names. This helps anyone reading the code to distinguish between each element.

Using PostgreSQL in VS Code

While you can write SQL at the command line via psql, doing your database work within a code editor is generally simpler. The group that developed PostgreSQL maintains its own graphical interface and PostgreSQL code editor called pgAdmin, but rather than install a separate application, I recommend using the PostgreSQL extension in VS Code. This extension integrates PostgreSQL functionality directly into VS Code so you can write, edit, and run SQL in the same place that you write, edit, and run Python code.

To install the extension, launch VS Code, then click the Extensions icon in the far-left sidebar. Enter **PostgreSQL** into the search bar, find the PostgreSQL extension maintained by Microsoft, and install it.

Once you have the extension installed and enabled, you should see the PostgreSQL logo (a cylinder with an elephant) in the left sidebar. Click the elephant to open the extension, then click the + at the top of the extension window to add a new database connection. You'll then be prompted to input information about the connection. For the database we created in the previous section, the information should be as follows:

Hostname: **localhost**
User: **myuser**
Password: **mypassword**
Port: **5432**
Database: **mydatabase**

Once all the information is inserted, you should see your connection appear in the extension window, as shown in Figure 7-1.



Figure 7-1: A PostgreSQL database connection in VS Code

Right-click **mydatabase** and select **New Query** to open up the SQL command editor. In this window you can write SQL statements.

CONNECTING TO A REMOTE DATABASE

Most corporate databases are stored in a remote location and are accessible over the internet. Accessing these is similar to connecting to a local database, but with a few modifications. Instead of *localhost*, the hostname will be the address of the database. This could be an IP address (like 192.0.2.123) or a URL to connect to a cloud service (like *mydb-instance1.abc123xyz789.us-east-1.rds.amazonaws.com* or *mydb-instance1:us-central1:mydb*).

A corporate firewall or VPN may also come into play here, in which case the specifics on how to incorporate security permissions into the connection process will differ. You might need to configure your firewall to allow traffic to and from the database server or use a VPN to securely connect to your corporate network. Additionally, when you connect to remote databases, it's important to use secure connection methods, such as SSL/TLS, to encrypt the data transmitted between your client and the remote database. These details are specific to each organization, so it's best to discuss the requirements with your company's IT department.

Creating and Editing Tables

Relational databases are made up of *tables*, or structured sets of data organized into rows and columns. Within each table, each data point belongs to a *record*, or row, and each record has the characteristics represented by the table's columns. Each column in a table therefore represents a specific attribute of the data. For example, a table might contain data about cities, with columns for the city name, country, population, and GDP per capita. Each record in the table would represent a specific city with a particular name, country, population, and GDP.

SQL is designed around row-based operations, meaning it assumes that the data itself may grow or shrink over time but that the attributes you're interested in won't change. As such, it's much simpler to add, remove, or

edit the rows than the columns of a table with SQL. Therefore, you ideally want to structure your data in a way that will limit the number of times you edit the columns or their types.

In Excel, the structure around data isn't as rigid, so column-based operations come nearly as easily as row-based operations. Adding an additional column takes just as much work as adding an additional row. By comparison, SQL's rigidity may sound like a frustrating limitation, but it yields benefits in terms of performance and data integrity.

Creating a Table

To start learning SQL, let's use the CREATE command to create a table in our example PostgreSQL database with data representing the characteristics of some cities:

```
CREATE TABLE cities (  
    city_id SERIAL PRIMARY KEY,  
    city_name VARCHAR NOT NULL,  
    country VARCHAR NOT NULL,  
    population INTEGER NOT NULL,  
    gdp_per_capita NUMERIC NOT NULL  
);
```

Running this command creates a new table in the database called `cities` with columns called `city_id`, `city_name`, `country`, `population`, and `gdp_per_capita`. Beyond the table and column names, the command sets important details: the primary key and the data types of each column.

Primary Keys

A *primary key* is a unique identifier for each record in a database table. It can consist of a single column or a combination of columns. Primary keys by definition create a couple of constraints to help them serve their purpose of uniquely identifying rows. First, each individual record must have a primary key. And second, the primary keys must all be unique from one another.

In our case, we've designated the `city_id` column as the primary key, and we've used `SERIAL` to make it an auto-incrementing column. This way, the first entry in the table will automatically be given an ID of 1, the second entry an ID of 2, and so on. This scheme ensures that each row will have a unique ID, fulfilling the requirements of a primary key.

Data Types

Similar to Python, data stored in relational databases can be of different types, and as in pandas DataFrames, each column in a table has a specified type, such that all data within that column must be of the same type. The primary SQL data types we'll practice using in this chapter are `INTEGER`, `NUMERIC`, `VARCHAR`, `BOOLEAN`, and `NULL`. Table 7-2 summarizes these data types and shows how they compare to Python.

Table 7-2: SQL Data Types and Their Python Equivalents

SQL data type	Python equivalent	Description	Variations across RDBMSs
INTEGER	int	A whole number	TINYINT, SMALLINT, MEDIUMINT, BIGINT in some systems.
NUMERIC	float	A number that includes decimals	DECIMAL is often used interchangeably, as well as FLOAT or DOUBLE for floating-point numbers in some systems.
VARCHAR	str	A string of characters with a variable length	VARCHAR2 in Oracle.
BOOLEAN	bool	Either TRUE or FALSE	TINYINT(1) in MySQL, and BIT in SQL Server.
NULL	None	A missing or undefined value	Universally supported, but handling and default behavior can vary across different RDBMSs.

In our `cities` table, we've used the `VARCHAR` type for the `city_name` and `country` columns; `INTEGER` for `population`, since this column will always be a whole number; and `numeric` for `gdp_per_capita`, since this column may include decimals. We've also included `NOT NULL` for each column, a *constraint* ensuring that each column must be given a value when a new record is added to the table.

You may encounter other, more complex SQL data types in the wild, including date and time objects and arrays. These tend to have more variations in behavior across different RDBMSs. We won't cover them in detail here, but look to the documentation for PostgreSQL or the other RDBMSs you're using for more information.

Altering a Table's Structure

The structure of SQL tables isn't meant to be changed much after they're created, but sometimes this becomes necessary. To modify the structure of an existing table, such as adding a new column, use the `ALTER TABLE` command. For example, here we add a column for the average temperature to our `cities` table:

```
ALTER TABLE cities
ADD COLUMN avg_temp NUMERIC;
```

Another way to alter a table is by changing the constraints put on columns with the `ADD` or `DROP` keyword, followed by the constraint. For example, here's how to remove the `NOT NULL` constraint from the `gdp_per_capita` column:

```
ALTER TABLE cities
ALTER COLUMN gdp_per_capita
DROP NOT NULL;
```

With this change, it will be possible to omit the GDP value when adding a new record to the table. The column will allow a value of NULL to be inserted instead of requiring a numeric value.

Dropping a Table

Sometimes you might need to delete a table entirely, such as when it's no longer needed or when you want to start fresh. Deleting a table removes the table and all its data from the database. To delete a table, use the DROP TABLE command. For example, you could delete the cities table as follows:

```
DROP TABLE cities;
```

Once a table is dropped, its data can't be recovered, so don't use DROP TABLE lightly.

Adding Rows

The INSERT INTO command allows you to add records to a table line by line. Let's use it to add data to the cities table. If you dropped the table, create it again, then execute the following command:

```
INSERT INTO cities (city_name, country, population, gdp_per_capita)
VALUES
    ('New York', 'USA', 8419000, 150000),
    ('Los Angeles', 'USA', 3980400, 85000),
    ('Tokyo', 'Japan', 37400068, 120000),
    ('London', 'UK', 8982000, 90000),
    ('Paris', 'France', 2148000, 140000);
```

This statement consists of four parts. First, INSERT INTO cities specifies what we're doing and the table we are adding the data to. Next, (city_name, country, population, gdp_per_capita) lists the columns that will receive the values. Then the VALUES keyword introduces the actual data being inserted.

Each row's data is enclosed in parentheses. For example, ('New York', 'USA', 8419000, 150000) corresponds to the columns in the same order they were listed at the start of the command, adding a record for New York. The following lines add records for four other cities.

After running this statement, the table should be populated with five records in a tabular structure.

Updating Records

To update existing data in a table, use the UPDATE statement. This command allows you to modify the data in one or more columns for records that meet specific criteria.

For example, to update the population of Los Angeles in the cities table, you would use the following:

```
UPDATE cities
SET population = 4000000
WHERE city_name = 'Los Angeles';
```

In this example, `UPDATE cities` specifies the table to update, `SET population = 4000000` specifies the new value for the population column, and `WHERE city_name = 'Los Angeles'` identifies which record(s) to update. Without the `WHERE` clause, all rows in the table would be updated to have the same population.

Deleting Data

To remove data from a table, use a `DELETE` statement. This command allows you to delete one or more records from a table based on specific criteria. For example, here's how to delete the record for Paris from the cities table:

```
DELETE FROM cities
WHERE city_name = 'Paris';
```

Similarly to the `UPDATE` example, `DELETE FROM cities` specifies the table from which to delete the record, while `WHERE city_name = 'Paris'` identifies which record(s) to delete. Without the `WHERE` clause, all rows in the table would be deleted.

Retrieving Database Data

In Excel, all the data in your spreadsheet is right in front of you all the time. When you're using databases, however, the datasets can often be infinitely large and complex, and they're mostly hidden from view. As such, it's generally best to work with only a few small slices or aggregations of the data at a time by using *SQL queries*. These are requests made to the database to retrieve the specific data you need. Queries allow you to filter, sort, and aggregate data according to your requirements. This means you can focus on just the relevant information without being overwhelmed by the entire dataset.

Selections

The SQL keyword to query data is `SELECT`. To customize and narrow the query, follow up the `SELECT` keyword with *qualifiers*, which we'll explore in the next few sections. Like any SQL statement, the query should end with a semicolon so SQL knows to view the statements that follow as a separate transaction.

Since we've populated the cities table with sample data, we can run `SELECT` queries as practice. For example, to fetch all the records from the cities table, use the following:

```
SELECT * FROM cities;
```

In this statement, `SELECT` specifies that you're looking to retrieve data from a table, `FROM cities` specifies which table you're interested in, and the `*` indicates that you'd like to return all the columns in the table. The result should be all the records from all the columns in `cities`, as shown in Figure 7-2.

	city_id integer	city_name character varying	country character varying	population integer	gdp_per_capita numeric
1	1	New York	USA	8419000	1500000.00
2	2	Los Angeles	USA	3980400	1045000.00
3	3	Tokyo	Japan	37400068	1900000.00
4	4	London	UK	8982000	650000.00
5	5	Paris	France	2148000	800000.00

Figure 7-2: The result of a `SELECT` query

To instead specify a subset of columns that you'd like to retrieve, list those columns separated by commas after `SELECT`:

```
SELECT city_name, population FROM cities;
```

This returns only the `city_name` and `population` columns for all the records in the `cities` table.

LIMIT

More often than not, you want to be careful to not retrieve too many records when using `SELECT`. Queries that are too broad and return too much data take a lot of time and computational resources to run, so it's good practice to always specify a maximum number of rows to return, especially if you don't have a good idea of what the results of your query will look like. To limit the number of rows returned, use the `LIMIT` clause, like so:

```
SELECT * FROM cities LIMIT 2;
```

This statement fetches all columns from the `cities` table but returns only the first two rows of the result set.

AS

In `SQL`, you can use the `AS` keyword to assign aliases to the columns in your query. This is useful when the original column names in the database aren't the most suitable for your work and it would be helpful to see the results with different column names. For example, to rename the `city_name` column to `Name` and the `population` column to `Pop` in the result set, use the following:

```
SELECT city_name AS Name, population AS Pop FROM cities;
```

The query now returns results with the column names `Name` and `Pop` instead of `city_name` and `population`, respectively. This doesn't change the names

of the columns themselves; it changes only the way they're labeled when you view the results.

SELECT DISTINCT

You can use `DISTINCT` with `SELECT` to create a list of unique values within one or more columns in a table. This is similar to the Remove Duplicates feature in Excel. Here, we get a list of unique countries from the `cities` table:

```
SELECT DISTINCT country FROM cities;
```

This query returns each unique country name only once, removing any duplicates.

ORDER BY

To sort the results of a query, use an `ORDER BY` clause. You can arrange the results in ascending order (default) or descending order (the `DESC` keyword). For example, this query retrieves cities ordered by population in descending order:

```
SELECT city_name, population FROM cities ORDER BY population DESC;
```

This fetches the city names and populations from the `cities` table and orders the results from highest to lowest population.

Filters

You're probably familiar with the *filter* feature in Excel, where you can specify conditions to display only the rows that meet certain criteria. This allows you to easily view and analyze subsets of your data without altering the original dataset. Similarly, in `SQL` you can use a `WHERE` clause to filter database records. We already used `WHERE` in "Creating and Editing Tables" on page 142 to specify which records from a table to update or delete, but `WHERE` clauses also work in `SELECT` queries to set conditions specifying which rows should be included in the results. For example, to retrieve only cities that are in the United States, use the following:

```
SELECT * FROM cities WHERE country = 'USA';
```

This query returns all columns from the `cities` table but only for the rows where the `country` column has the value `USA`. We use `=` in the `WHERE` clause to create the comparison, but this is just one of the comparison operators that you can use to filter results. Table 7-3 lists other common comparison operators in `SQL`.

Table 7-3: SQL Comparison Operators for WHERE Clauses

Operator	Description	Example
<> or !=	Not equal to	SELECT * FROM cities WHERE country <> 'USA';
>	Greater than	SELECT * FROM cities WHERE population > 100000;
<	Less than	SELECT * FROM cities WHERE population < 100000;
>=	Greater than or equal to	SELECT * FROM cities WHERE population >= 100000;
<=	Less than or equal to	SELECT * FROM cities WHERE population <= 100000;
BETWEEN	Within a range	SELECT * FROM cities WHERE population BETWEEN 50000 AND 100000;
LIKE	String pattern matching	SELECT * FROM cities WHERE city_name LIKE 'New%';
IN	Contained in a list	SELECT * FROM cities WHERE city_name IN ('New York', 'Tokyo');
IS NULL	Is NULL	SELECT * FROM cities WHERE population IS NULL;
IS NOT NULL	Is not NULL	SELECT * FROM cities WHERE population IS NOT NULL;

Much like creating more-complex conditions in Python, you can use **AND** and **OR** in SQL to combine multiple conditions for filtering. For example, to retrieve cities that aren't in the United States *and* have a population greater than 5 million, use the following:

```
SELECT * FROM cities WHERE country <> 'USA' AND population > 5000000;
```

This query returns rows where both conditions are met. Alternatively, here's how to retrieve cities that aren't in the United States *or* have a population greater than 5 million:

```
SELECT * FROM cities WHERE country = 'USA' OR population > 5000000;
```

This query returns rows where at least one of the conditions is met.

WILDCARD CHARACTERS

In SQL, the **LIKE** operator allows you to search for patterns within string values in a **WHERE** clause by using *wildcard characters*. These are placeholders for an undefined part of the string.

The two most common wildcard characters are a percent sign (%) and an underscore (_). A percent refers to any number of unspecified characters. Usually, you use this to look for specific text at the beginning or end of a string. For example, `New%` matches any string that starts with *New*, such as *New York*, *New Delhi*, or *Newark*. An underscore refers to only one unspecified character. For example, `F_ji` matches both *Fiji* and *Fuji*.

Joins

In Excel, the VLOOKUP function allows you to merge data from multiple sheets or tables. This action is effectively a *join*, or a combination of two datasets based on a common column. In SQL, you can retrieve and combine data from multiple tables by using the JOIN keyword. This is similar to performing a VLOOKUP in Excel but with more flexibility.

In Chapter 6, we discussed the same concept in terms of pandas DataFrames when we looked at merging. SQL joins operate in much the same way. Just as the *how* argument in pandas allowed you to specify the kind of merge you wanted to perform between two DataFrames, SQL has several types of joins, including inner joins, left joins, right joins, and full outer joins, with each treating the common column in a slightly different manner. In pandas DataFrames, you also used the *on* argument to specify the common column or columns between the two DataFrames to be used for the merge. Similarly, joins in SQL use the ON keyword to specify the common column(s) that define how the rows from two or more tables are related and should be combined.

To illustrate how the types of SQL joins work, let's create a second table in mydatabase called attractions that holds information about attractions in different cities:

```
CREATE TABLE attractions (  
    attraction VARCHAR(255) PRIMARY KEY,  
    popularity INT,  
    city VARCHAR(100)  
);  
  
INSERT INTO attractions (attraction, popularity, city) VALUES  
    ('Statue of Liberty', 95, 'New York'),  
    ('Central Park', 90, 'New York'),  
    ('Tokyo Tower', 85, 'Tokyo'),  
    ('Senso-ji Temple', 88, 'Tokyo'),  
    ('Eiffel Tower', 98, 'Paris'),  
    ('Louvre Museum', 96, 'Paris'),  
    ('Sydney Opera House', 94, 'Sydney');
```

These two transactions create a new attractions table and populate it with data. Notice that the new table has a city column with some values that match the city_name column in the cities table. We'll use this connection as we try out some joins.

Inner

An *inner join* returns only the rows that have matching values in both tables. This join excludes any rows that don't have a corresponding match in both tables. For example, we could apply an inner join to the cities and attractions tables to get just the attractions in cities that are present in both tables, along with the associated popularity score (from the attractions table) and country (from the cities table):

```

SELECT
    cities.city_name AS name,
    cities.country,
    attractions.attraction,
    attractions.popularity
FROM
    cities
INNER JOIN
    attractions ON cities.city_name = attractions.city;

```

In the query, `SELECT` specifies that we’re looking to retrieve data, and the columns listed after it (`cities.city_name`, `cities.country`, and so on) indicate the specific information we want to include in the result and the tables they come from. The `FROM` clause refers not just to `cities` but to the entire combination of tables we’re querying data from. The `INNER JOIN` connects the `cities` and `attractions` tables based on the condition specified after `ON`, ensuring that only rows with matching city names in both tables are returned. Note that the join happens before the `SELECT` statement actually returns any data, so the order of operations doesn’t follow the order of the keywords in the SQL statement. When you write SQL queries, you’ll find that you often won’t write the clauses in order.

This query returns six rows in total, as shown in Figure 7-3.

	name character varying	country character varying	attraction character varying	popularity integer
1	New York	USA	Central Park	90
2	New York	USA	Statue of Liberty	95
3	Tokyo	Japan	Senso-ji Temple	88
4	Tokyo	Japan	Tokyo Tower	85
5	Paris	France	Louvre Museum	96
6	Paris	France	Eiffel Tower	98

Figure 7-3: The result of an inner join

Only those cities that have corresponding entries in both the `cities` and `attractions` tables are included in the results, along with the attractions and their popularity ratings.

Left and Right

When you join two tables, the table before the `JOIN` keyword is considered the *left* table, and the table after `JOIN` is considered the *right* table. A *left join* returns all rows from the left table and only the matched rows from the right table. If no match is found, the result is `NULL` in the columns coming from the right table. For example, this left join retrieves all cities from `cities` along

with their attractions, even if some cities don't have matching entries in the attractions table:

```
SELECT
    cities.city_name AS name,
    cities.country,
    attractions.attraction,
    attractions.popularity
FROM
    cities
LEFT JOIN
    attractions ON cities.city_name = attractions.city;
```

This query returns eight rows in total, all cities from the cities table and their attractions from the attractions table, as shown in Figure 7-4.

	name character varying	country character varying	attraction character varying	popularity integer
1	New York	USA	Central Park	90
2	New York	USA	Statue of Liberty	95
3	Los Angeles	USA	<i>null</i>	<i>null</i>
4	Tokyo	Japan	Senso-ji Temple	88
5	Tokyo	Japan	Tokyo Tower	85
6	London	UK	<i>null</i>	<i>null</i>
7	Paris	France	Louvre Museum	96
8	Paris	France	Eiffel Tower	98

Figure 7-4: The result of a left join

The attraction and popularity columns for Los Angeles and London contain NULL, since those cities don't have matching entries in the attractions table.

A *right join* is the opposite of a left join: It returns all rows from the right table and only the matched rows from the left table. If no match is found, the result is NULL in the columns coming from the left table. For example, this right join gets all attractions along with their cities, even if some attractions don't have matching entries in the cities table:

```
SELECT
    cities.city_name AS name,
    cities.country,
    attractions.attraction,
    attractions.popularity
```

```

FROM
    cities
RIGHT JOIN
    attractions ON cities.city_name = attractions.city;

```

This query returns seven rows, all attractions from the attractions table, along with their respective cities and countries from the cities table, as shown in Figure 7-5.

	name character varying	country character varying	attraction character varying	popularity integer
1	New York	USA	Central Park	90
2	New York	USA	Statue of Liberty	95
3	Tokyo	Japan	Senso-ji Temple	88
4	Tokyo	Japan	Tokyo Tower	85
5	Paris	France	Louvre Museum	96
6	Paris	France	Eiffel Tower	98
7	<i>null</i>	<i>null</i>	Sydney Opera House	94

Figure 7-5: The result of a right join

The name and country columns are NULL for the Sydney Opera House, since this attraction doesn't have a match in the cities table.

Full Outer

A *full outer join* returns all rows from both tables. If no match is found between the tables, the result is NULL for the nonmatching side. For example, to get all cities and all attractions, including those that don't have matching entries in either table, use the following:

```

SELECT
    cities.city_name AS name,
    cities.country,
    attractions.attraction,
    attractions.popularity
FROM
    cities
FULL OUTER JOIN
    attractions ON cities.city_name = attractions.city;

```

This query returns nine rows, all cities from the cities table and all attractions from the attractions table, as shown in Figure 7-6.

	name character varying	country character varying	attraction character varying	popularity integer
1	New York	USA	Central Park	90
2	New York	USA	Statue of Liberty	95
3	Los Angeles	USA	<i>null</i>	<i>null</i>
4	Tokyo	Japan	Senso-ji Temple	88
5	Tokyo	Japan	Tokyo Tower	85
6	London	UK	<i>null</i>	<i>null</i>
7	Paris	France	Louvre Museum	96
8	Paris	France	Eiffel Tower	98
9	<i>null</i>	<i>null</i>	Sydney Opera House	94

Figure 7-6: The result of a full outer join

Notice the NULL values on both sides of the table, where one table or the other doesn't have a matching entry.

Column Transformations

In SQL, you can apply standard operators such as `*`, `/`, `+`, and `-` to perform arithmetic operations on columns. This allows you to create new columns based on transformations of existing data. Consider this transformation of data in the cities table:

```
SELECT city_name, gdp_per_capita * population AS gdp_total FROM cities;
```

In this SELECT statement, we use the multiplication operator (`*`) to multiply the values in the `gdp_per_capita` and `population` columns, yielding the total GDP for each city. We display the results in a column called `gdp_total`. Note that we aren't actually adding this column to the database table. We're only creating the column temporarily in our view of the query results.

Aggregations

On top of basic arithmetic, you can create *aggregations* of your data, which are transformations that give a single result based on multiple records, such as a total or an average. Aggregations are useful for analysis because they allow you to create summaries of the data. For example, to retrieve the total and average population from all the cities in the table, you can apply the `SUM` and `AVG` aggregation functions:

```
SELECT
SUM(population) AS total_population,
AVG(population) AS average_population
FROM
cities;
```

The aggregation functions are pretty similar to what you’ve seen in Excel and what you saw in pandas in Chapter 6. Table 7-4 lists some of the most common ones.

Table 7-4: SQL Aggregation Functions and Their Equivalents

SQL function	Description	pandas equivalent	Excel equivalent
COUNT	Counts the number of rows	<code>df[column].count()</code>	<code>COUNTA(range)</code>
SUM	Sums the values in a column	<code>df[column].sum()</code>	<code>SUM(range)</code>
AVG	Calculates the average	<code>df[column].mean()</code>	<code>AVERAGE(range)</code>
MAX	Finds the maximum	<code>df[column].max()</code>	<code>MAX(range)</code>
MIN	Finds the minimum	<code>df[column].min()</code>	<code>MIN(range)</code>

When you run each of these aggregation functions, the result will be a single value for each function. Since aggregation functions operate across multiple rows, your result will be contained in only one row, unless you perform the aggregation over multiple groups of rows.

Groupings

With Excel pivot tables, you can group and summarize your data based on certain criteria, which allows you to quickly aggregate and analyze data in order to come up with conclusions. In SQL, you can similarly use the `GROUP BY` clause to group records in a table based on one or more columns and perform aggregation functions on these groups. For example, here we group the attractions in the attractions table by city and calculate the average popularity for each city’s attractions:

```
SELECT
    city,
    AVG(popularity) AS average_popularity
FROM
    attractions
GROUP BY
    city;
```

In this query, we group the attractions table by the city column, then apply the `AVG` aggregation function to the popularity column for each of the groups. The results show one record per city represented in the table, with columns for the city name and the average popularity of attractions within that city, as Figure 7-7 illustrates.

	city character varying	average_popularity numeric
1	Paris	97.0000000000000000
2	Sydney	94.0000000000000000
3	New York	92.5000000000000000
4	Tokyo	86.5000000000000000

Figure 7-7: The result of a GROUP BY query

The HAVING keyword allows you to filter the groups within a GROUP BY clause. This is similar to the way WHERE operates, but it applies to groups instead of rows. For example, suppose you want to find the cities with an average popularity of attractions greater than 90. You can use HAVING with GROUP BY:

```

SELECT
    city,
    AVG(attractions.popularity) AS average_popularity
FROM
    attractions
GROUP BY
    city
HAVING
    AVG(attractions.popularity) > 90;

```

Figure 7-8 shows the result.

	city character varying	average_popularity numeric
1	Paris	97.0000000000000000
2	Sydney	94.0000000000000000
3	New York	92.5000000000000000

Figure 7-8: The result of a GROUP BY query with a HAVING clause

We now get only the average popularity of cities when it's greater than 90. Because of the HAVING clause, Tokyo is excluded from the results since the average popularity of attractions there is only 86.5.

Structuring Complex Queries

Since SQL queries are run in single self-contained statements, they have the potential to quickly grow too complex for their own good, becoming difficult to read, debug, and maintain. Excel formulas, which are also contained in single statements, are vulnerable to the same problem.

To keep track of what you're doing, it helps to approach SQL queries piece by piece. Break your complex queries into smaller, more manageable parts, similar to the way you would handle complex Excel formulas. Understanding exactly how SQL approaches a complex query is also helpful.

SQL queries are executed in a specific sequence, which might not always align with your intuition or the order in which the clauses appear in the query. Here's the typical order of execution for a SQL query:

1. Identify the tables with `FROM`.
2. Filter the rows with `WHERE`.
3. Group the rows with `GROUP BY`.
4. Filter the groups with `HAVING`.
5. Choose the columns with `SELECT`.
6. Sort the results with `ORDER BY`.
7. Limit the number of rows with `LIMIT`.

As an example, say we have a query that includes both a `WHERE` and a `HAVING` clause:

```
SELECT
    cities.country,
    AVG(attractions.popularity) AS average_popularity
FROM
    cities
JOIN
    attractions ON cities.city_name = attractions.city
WHERE
    cities.population > 3000000
GROUP BY
    cities.country
HAVING
    AVG(attractions.popularity) > 90;
```

Because SQL executes `WHERE` clauses before `GROUP BY` and `HAVING` clauses, the `WHERE` clause filters the rows before any grouping or aggregation occurs. This means that only cities with a population greater than 3 million are considered in the subsequent grouping and aggregation steps.

Automating Database Tasks with Python and SQL

The final piece necessary to integrate SQL into your workflow is to learn how to incorporate it into Python scripts. This will allow you to programmatically manipulate database inputs and outputs, making analysis more efficient and automation more seamless.

Several Python libraries are available for working with PostgreSQL databases. We'll focus on one called `Psycopg2`. It provides the missing pieces that allow you to establish a database connection, execute SQL queries, and

handle the results using Python code. Under the hood, Psycopg 2 uses code written in a more primitive language called C to interact with PostgreSQL, but it separates you as the user from all that complexity. Instead, you interact with much simpler, more readable Python code, which makes the process easier.

This is a good example of how the availability and accessibility of Python's external libraries simplifies your work. Using Python, you don't necessarily need to understand how to write C code in order to interact with a database. Other coders have already done that work, and you can reap the benefits by using an established library.

NOTE

Other Python libraries for working with PostgreSQL include SQLAlchemy, pg8000, and pyodbc. Many of these use the same underlying C code to interact with the PostgreSQL server, but they have slightly different syntax and features in Python to reach it.

Installing Psycopg 2

As usual, you can install Psycopg 2 with pip (or pip3):

```
pip install psycopg2
```

Once Psycopg 2 is installed, you need to import it into your Python script:

```
import psycopg2
```

Now you can use the functions within Psycopg 2 to interact with a PostgreSQL database.

Connecting to PostgreSQL from Python

The first step in working with a database via Python is to establish a connection to the database from your script. You need to pass all the information required for making the connection as arguments to the `psycopg2.connect()` function. This is the same information that the VS Code PostgreSQL extension needed to connect to the database (hostname, database, user, password, port). Here's an example of how to do this with the database we created earlier in the chapter:

```
conn = psycopg2.connect(  
    host="localhost",  
    database="mydatabase",  
    user="myuser",  
    password="mypassword",  
    port=5432  
)
```

This creates a connection called `conn` between your Python script and the database. Next, you need to obtain a cursor object:

```
cur = conn.cursor()
```

The *cursor* manages the input and output through your connection. It keeps track of where you are in the database, what you're doing, and what data is going in and out. The cursor can send SQL commands to the database, such as SELECT, INSERT, UPDATE, or DELETE. It also retrieves results from queries, like the output from a SELECT statement, and allows you to navigate through them.

Running a Query from Python

To run SQL commands in Python, pass them as strings to the `execute()` method of your cursor object. For example, here we run a SELECT query on the cities table:

```
cur.execute("SELECT * FROM cities")
```

We don't need a semicolon at the end of the SQL statement because the `execute()` method processes one SQL command at a time. That said, you can still add one if you like.

After you run a query, you won't immediately see the output in your console. To retrieve the results, you need to *fetch* them from the cursor. To do this, you can use one of a few options, depending on your intent:

`fetchone()` Retrieves the next row of the result set in a single tuple, which is useful when you expect only one result or when the result is large and you want to process it one row at a time.

`fetchall()` Retrieves all the rows from the result set at once as a list of tuples. This is useful when you want to process all the results at once.

`fetchmany()` Retrieves a fixed number of rows, specified as an argument to the method. For instance, `fetchmany(5)` will retrieve five rows. This is useful when you want to process the results in chunks.

Here's an example of how to fetch the data from a query:

```
rows = cur.fetchall()
for row in rows:
    print(row)
```

Calling `fetchall()` returns all the rows from the query as a list of tuples. We then loop through the list and print each row.

One common transformation after a query is to convert the results into a pandas DataFrame for further analysis. Here's how to do it:

```
import pandas as pd

df = pd.DataFrame(
    rows,
    columns=["city_id", "city_name", "country", "population",
            "gdp_per_capita"]
)
```

We use the `rows` object, which is the output we fetched in the previous listing, and transform it into a DataFrame by using `pd.DataFrame()`.

Closing the Database Connection

After you're finished with database operations in your Python script, it's good practice to close both the cursor and the connection:

```
cur.close()
conn.close()
```

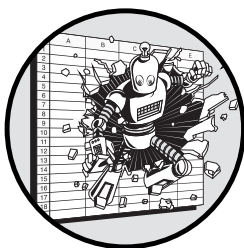
Most of the time, your operating system will automatically close open connections to a database when your Python script stops running, but this isn't a given. Leaving connections open uses computational resources that can take up memory and slow your computer as well as the database, so it's best to close your connections explicitly.

Conclusion

This chapter established where and why databases add value in an Excel-based workflow. We went through the basics and some of the nuances of database infrastructure. To get you started working with databases in the absence of access to an existing database, we covered how to set up a database by using PostgreSQL. Then we walked through all the basic SQL queries that allow you to populate, maintain, and query from that database. We also integrated those SQL queries into Python scripts with the Psycopg 2 library.

8

RETRIEVING DATA FROM THE INTERNET



The chapter focuses on APIs, which allow you to write scripts to retrieve data from external sources across the internet. We'll cover the basics of making API requests and processing the responses, and we'll consider some of the complexities that come up when working with APIs, like pagination, error handling, and authentication. In the end, we'll show how to write code for making API requests in clean, organized, and maintainable functions that you can use and reuse in whatever capacity you may need.

In Excel, your work is siloed; it lives only within the Excel application and the Microsoft applications that tie to it. Connections to the non-Microsoft world often don't exist, and that leaves you needing to manually bridge the gap. That might mean copying and pasting data from a website or clicking the Download button to extract data from an application. By contrast, APIs let you make these connections through code and bring external data right into your

workflow. And since Python allows you to automate these connections, APIs become one of the most essential tools for boosting productivity with code.

All kinds of data are retrievable through APIs, some of which you may not expect. Maybe you often send documents for e-signature; you can automate that with Python and the DocuSign API. Maybe you work in HR and use Workday; you can use the Workday API to automate all sorts of tasks. Or maybe you run a Shopify store selling T-shirts with relatable Excel-themed sayings like *CTRL+S and No Regrets*; you can keep track of your inventory and orders by using the Shopify API.

With an API, you can pull data from a vast array of external sources programmatically—that is, automatically from within a Python script. From there, you can pass the data along to a library like pandas for analysis. This allows you to leverage data from far and wide while streamlining your workflow, meaning you can do much more in much less time.

What Is an API?

An *application programming interface (API)* is a set of standards governing applications' interactions with one another. APIs describe how commands translate into actions, enabling different types of software to work together through code.

This definition is as nonspecific as it sounds. APIs can be any set of commands allowing pieces of software to exchange information with one another. There are lots of types of software that need to interact in lots of ways, so there are a lot of kinds of APIs.

APIs work by connecting a client and a server. The *client* is the entity that makes requests to access data or services from the API. This could be your Python script or any other application seeking information. The *server* is the entity that hosts the API, processes requests, and sends back the required data.

REST APIs

The primary API that we'll focus on in this chapter is the *Representational State Transfer (REST) API*. The name refers to the process of transferring representations of resources (data) between the client and the server. I say *representations* because the data the client receives may not be in the same format as it's stored on the server's side. The word *state* indicates that the request is for the data as it exists at a specific point in time. In the most direct terms, a REST API facilitates the transfer of a representation of a state.

The most salient advantage of REST APIs is their simple user experience. The interface that you, as the client, interact with, which defines how you structure requests and receive responses, is standardized. Therefore, once you know how to work with one REST API, applying those skills to other REST APIs is easy.

REST APIs also allow the API provider to entirely separate the client from all the complicated intermediate infrastructure that supports the API. This

means you ordinarily won't be able to tell whether you're connected directly to the end server or to an intermediary in the middle. This separation also allows the server to introduce multiple layers of infrastructure, such as load balancers, proxies, and gateways, to improve the API's performance. These additions can improve scalability, balance the workload more evenly, and enhance security without requiring the user to change anything on their side.

Other Types of APIs

REST is the most common type of API and serves the widest variety of applications and use cases, but several other relatively common types also exist:

Simple Object Access Protocol (SOAP) APIs Operate like REST APIs but provide more rigidity in protocol and messaging format, ensuring stricter standards and error handling. These are used in contexts like banking that require greater security and reliability.

WebSocket APIs Operate by opening a channel of communication and leaving it in place over a period of time. These are common for applications like real-time messaging or trading platforms and financial exchanges, where maintaining an open connection allows for instant data transmission and low latency.

GraphQL APIs Allow for more-complex querying than REST APIs. Developed by Meta (formerly Facebook), they're used when the user needs precise control over the data they fetch, like social media platforms with a wide variety of complex user data.

If you're not a software engineer, you likely won't encounter these other kinds of APIs as often in your everyday work, but it's helpful to know that they exist and their typical uses in case you do.

Making Requests and Handling Responses

Messages that go through REST APIs take the form of requests and responses, which are made over the internet via URLs. A client sends a *request* to a server, and the server returns a *response* with the requested data or an appropriate status message. These interactions typically involve one of a few standard methods established by the REST architecture.

REST APIs rely on *Hypertext Transfer Protocol (HTTP)*, a standard of communication used for transmitting web pages over the internet. You're familiar with this protocol if you've ever visited a website; it's the *http://* at the start of a URL. Making REST API requests also involves working with URLs, just like visiting web pages in your browser. To understand what that means exactly, it helps to talk a little about how the internet works.

Internet Communication Protocols

On the most basic level, the internet is just a network of connected computers around the world. Some of these computers are connected physically with

wires, and some are connected wirelessly, but since every computer in the network is connected somehow to at least one other computer in the network, it's possible to send a message between any two computers in the network no matter where they're located or what they're connected to. This system of connections and the rules that govern it is called the *Internet Protocol (IP)*. You might already be familiar with IP in the context of IP addresses, which are the identifiers devices use to locate each other on the network.

IP alone isn't enough to move data reliably between computers, since IP doesn't guarantee that information sent over the network will be delivered. It also doesn't ensure that information will arrive in any specific order, and it lacks any mechanism to adjust the flow of information when the network is congested. This is where the *Transmission Control Protocol (TCP)* comes into play.

TCP establishes a connection between the source and destination computers before data is sent over the internet. It systematically breaks information into smaller pieces, delivers each piece reliably and in the correct order, and applies checks to ensure everything works as intended. All in all, TCP adds necessary improvements on top of the IP network that make transmitting information through it more practical.

While TCP and IP work together to provide reliable data transfer between computers, neither specifies how data should be structured or interpreted. Enter HTTP. This protocol builds on top of TCP by providing a common language for clients and servers operating within the system. This common language standardizes how requests and responses should be formatted, how resources should be identified, and how data should be interpreted. Without HTTP, accessing and interacting with web content in a consistent and meaningful way would be impossible.

To explain these three protocols in simpler terms, if the World Wide Web were a restaurant, HTTP would be the menu, TCP would be the wait-staff, and IP would be the floor plan. HTTP defines what the clients (customers) can request (order) and how the server (kitchen) should respond. TCP handles the reliability and sequencing of the requests (dishes) being delivered. And IP lays out the routes and addresses, determining how to get from one computer (table) to another.

HTTPS

You'll notice that we use `https://` instead of `http://` at the start of the URL in all the example API requests in this chapter. *Hypertext Transfer Protocol Secure (HTTPS)* extends HTTP by adding extra security measures. Specifically, HTTPS encrypts the data exchanged between the client and the server. This encryption ensures that any sensitive data transmitted (such as API keys, login credentials, or personal information) is protected from any eavesdroppers who might attempt to listen in and that it can't be read if intercepted.

Request Types

Several types of requests can be transferred through a REST API. We'll focus on two of the most common: GET and POST. In "Simplifying Requests with the Requests Library" on page 169, we'll demonstrate how to make these requests through Python, but for these first few examples, we'll use `curl`, a simple tool for sending API requests directly from the command line. You don't need to do anything to install it, since `curl` is already built into both Windows and macOS, making it an excellent tool for practicing API interactions.

GET

A GET request retrieves data from a server. When you use a GET request, you're asking the server to send you information about a specific resource. When you visit a web page in your browser, the browser sends a GET request to the server to fetch the content of that page, which returns the necessary elements to display the page.

You can use `curl` in your console to make a GET request. As an example, we'll make a request to the Random User Generator API, an open source API that creates random user data for testing purposes:

```
curl -X GET "https://randomuser.me/api/?inc=name"
```

This request returns a long string of data including a randomly generated name and information about the response:

```
{
  "results": [
    {
      "name": {
        "title": "Mr",
        "first": "Paul",
        "last": "Allen"
      }
    }
  ],
  "info": {
    "seed": "abc123xyz789",
    "results": 1,
    "page": 1,
    "version": "1.4"
  }
}
```

In "Simplifying Requests with the Requests Library" on page 169, we'll discuss how to process this data in Python.

POST

A POST request sends data to a server. When you use a POST request, you're typically submitting data to the server for processing. For example, if you

were using an API to post a comment to a social media site, that would require a POST request. So would sending payment details to process a transaction or registering as a new user on a website. All these situations require sending data to the server, so POST is appropriate.

Say you want to send the following data about a cat to a server for storage in a database:

```
{
  "name": "whiskers",
  "age": 3,
  "color": "gray",
  "breed": "persian"
}
```

This data is in JSON format, which we'll discuss shortly. To illustrate how to send this data to an API, we'll use an API called JSONPlaceholder, a free service that allows you to test API calls. The curl command to send the cat data to JSONPlaceholder is shown here:

```
curl -X POST "https://jsonplaceholder.typicode.com/posts" \
-H "Content-Type: application/json" \
-d '{
  "name": "whiskers",
  "age": 3,
  "color": "gray",
  "breed": "persian"
}'
```

This command uses the POST method to send the JSON data to the specified URL. The server processes the data and responds with a confirmation, including the data you sent along with a new ID assigned to it. The response would look something like this:

```
{
  "name": "whiskers",
  "age": 3,
  "color": "gray",
  "breed": "persian",
  "id": 101
}
```

This indicates that the server successfully received and processed the data.

Response Types

REST APIs give responses in plaintext, but that text can come in many formats. Often a REST API may let you choose the format for your response. Some common choices are CSV and JSON.

CSV Files

Comma-separated values (CSV) is a simple text format where each line corresponds to a single record, and each record consists of fields separated by commas. The first line typically contains headers that indicate the names of the columns. Here's an example of some data in CSV format:

```
name,age,color,breed
whiskers,3,gray,persian
mittens,2,black,siamese
tiger,4,orange,bengal
sparkline,7,brown tabby,domestic shorthair
```

The first line contains the headers: name, age, color, and breed. Each subsequent line contains the data for one cat, with the individual fields separated by commas.

JSON Files

JSON is a format for text-based data that uses key-value pairs. Each piece of data is designated with a key, and this key is paired with a value that represents the data. Like CSV, the JSON format is easy for humans to read and write and easy for machines to parse and generate.

JSON stands for *JavaScript Object Notation*; the format was derived from the way objects are defined in the JavaScript programming language. JSON itself is language agnostic, however, and doesn't have much to do with JavaScript specifically. The format can handle keys and values that are strings (with quotes); numbers (without quotes); arrays, which are ordered lists of values; or other, nested JSON objects containing their own key-value combinations. Here's the same cat data packaged into a JSON object:

```
{
  "metadata": {
    "total_cats": 4,
    "source": "example dataset"
  },
  "cats": [
    {
      "name": "whiskers",
      "age": 3,
      "color": "gray",
      "breed": "persian"
    },
    {
      "name": "mittens",
      "age": 2,
      "color": "black",
      "breed": "siamese"
    }
  ]
}
```

```

        "name": "tiger",
        "age": 4,
        "color": "orange",
        "breed": "bengal"
    },
    {
        "name": "sparkline",
        "age": 2,
        "color": "brown tabby",
        "breed": "domestic shorthair"
    }
]
}

```

At the top level, this object consists of two keys, `metadata` and `cats`. The value for `metadata` is a nested object (enclosed in curly brackets) with two keys of its own, while the value for `cats` is an array (enclosed in square brackets) of objects, each representing an individual cat.

Although CSV responses are commonly available for REST APIs, they don't allow for as much flexibility in terms of what you can put in a response as JSON does. CSV responses are limited to tabular data, where each row has values for the same set of attributes (columns). They can't easily represent nested or hierarchical data. By contrast, JSON allows for each element in the response to have an entirely different set of attributes associated with it, and it can easily include complex nested structures.

JSON isn't a native data type in Python, like a list or a dictionary. To work with JSON data in your scripts, you need to use the functions in the `json` module. It's part of the Python Standard Library, so there's no need to separately install `json` if you already have Python installed. Some of the module's functions for working with JSON data include the following:

- `json.load()`** Reads JSON data from a file and parses it into a Python object, like a dictionary, list, or string
- `json.loads()`** Parses a JSON string (not from a file) into a Python object
- `json.dump()`** Writes a Python object as JSON data to a file
- `json.dumps()`** Converts a Python object to a JSON string without writing it to a file

As an example, to save the ages of our four cats as a JSON file, you can use the `json.dump()` function:

```

import json

cats = {
    "whiskers": 3,
    "mittens": 2,
    "tiger": 4,
    "sparkline": 2,

```

```
}
```

```
with open("cats.json", "w") as file:  
    json.dump(cats, file, indent=4)
```

We declare the cats and their ages as a Python dictionary, whose key-value pairs already have a JSON-like structure. Then we write the data to a new file called *cats.json*. Notice that we call `json.dump()` inside a `with` block to ensure that the file is properly closed after writing, even if an error occurs.

To read that data back into a Python script from the JSON file, you can use the `json.load()` function:

```
with open("cats.json", "r") as file:  
    cats = json.load(file)
```

This code reads the contents of the *cats.json* file and loads it into a Python dictionary named `cats`. Once again, notice that we use a `with` block to ensure that the file is properly managed.

Simplifying Requests with the Requests Library

To work with APIs programmatically from Python scripts, we'll use the Requests library. It includes functions that simplify the process of making requests over the internet through HTTP. If you didn't already install Requests in Chapter 1 to run our first sample script, use `pip` (or `pip3`) to install it now:

```
pip install requests
```

You'll need to import the library with `import requests` to use it in your scripts.

Making an API Request

Let's try making an API request with Python and the Requests library. We'll send a request to the Random User Generator API introduced earlier.

When working with a new API, it's always helpful to look at the API's documentation so you can understand the kind of data the API provides and how to get it. In this case, the Random User Generator documentation (<https://randomuser.me/documentation>) indicates that a generic GET request to the API will yield data for one random user in JSON format. Here's how to make that request:

```
import requests  
  
url = "https://randomuser.me/api/"  
response = requests.get(url)
```

We use the Requests library's `get()` method to make a GET request to the specified URL, storing the server's response in the `response` variable. The

response object includes a status code and the retrieved data. If you just print the variable with `print(response)`, you'll simply see the status code, like this:

```
<Response [200]>
```

The status code alone doesn't tell us much, but `response` includes within it properties that give more information. For example, `response.elapsed` gives the amount of time taken between sending the request and receiving the response from the server, and `response.text` gives the content of the response as a string.

To make the data useful, we can use the object's `json()` method to convert the JSON data into a Python dictionary:

```
result = response.json()
print(result)
```

This prints the JSON data retrieved from the API, which includes details about a randomly generated user, such as their name, location, email, and other attributes, as well as metadata about the request. The result is a nested dictionary, like this:

```
{
  "results": [
    {
      "gender": "male",
      "name": {
        "title": "Mr",
        "first": "Satya",
        "last": "Nadella"
      },
      "location": {
        --snip--
      }
    }
  ]
}
```

Since the data is now in dictionary format, it's much easier to use however you like in the rest of your Python script.

Adding Arguments

You can often customize API requests by adding arguments to the URL that alter the API's behavior or response. For example, the Random User Generator API takes various arguments to specify the number of users you want to generate, their nationality, and their gender, among other details.

The arguments are specified as key-value pairs at the end of the URL in the form *argument_name=value*. They're separated from the rest of the URL by a question mark. For example, to get multiple random users instead of one, you can add the `results` argument to the Random User Generator URL:

```
import requests

url = "https://randomuser.me/api/?results=5"
```

```
response = requests.get(url)
result = response.json()
print(len(result))
```

This returns the number of users, which is five. You can also attach multiple arguments to a single request by separating the key-value pairs with ampersands (&). For example, here we generate three female users from the United Kingdom (nat=gb):

```
url = "https://randomuser.me/api/?results=3&gender=female&nat=gb"
response = requests.get(url)
result = response.json()
```

To find out what other arguments are accepted by any given API and the options available for each, see the API documentation.

Perhaps you can see how these API URLs can get out of hand as you customize your requests with more and more arguments. To make your code cleaner and more manageable, you can use the `params` argument of the `get()` method to separate the arguments from the URL. This allows you to put all your arguments in a Python dictionary and then let the Requests library do the work of tacking them onto the URL for you:

```
url = "https://randomuser.me/api/"
params = {"results": 3, "gender": "female", "nat": "gb"}

response = requests.get(url, params=params)
result = response.json()
```

This version of the request has exactly the same result as the previous version, where all the arguments were coded directly into the URL, but now the `params` dictionary contains the query arguments instead. These are automatically appended to the URL by the `get()` method when it sends the request. With this approach, it's easy to edit the code in the future to make different requests by simply adding, changing, or removing arguments from the `params` dictionary.

Building in Error Handling

API requests add a level of uncertainty to a Python script that you don't typically encounter when you're working with local files or in-memory data. This uncertainty stems from all the possible new kinds of errors that may arise. These errors may be entirely out of your control and can include internet connectivity issues, rate limits, unexpected response formats, invalid credentials, and more.

With this added room for potential problems, handling errors and validating results is a much more important factor when you're writing code that interacts with APIs. In this section, we'll look at Python's built-in mechanisms for handling errors generally, and then we'll see how to apply them to API calls.

try...except Blocks

In Python, you can explicitly tell your script how to handle errors by using `try...except` blocks. This technique allows you to manage and respond to errors as they occur during the execution of your program rather than letting them crash your program unexpectedly. With `try...except`, you define two possible paths for your code: the path you want to take assuming everything works and the path to take if an error occurs along the first path. The basic structure is as follows:

```
try:
    # Code that might raise an exception
except:
    # Code that says what to do if there is an exception
```

The `try` block contains the code that you want to execute, and the `except` block defines what to do in case of an error (called an *exception*) during the `try` block. If the `try` block code runs without any exceptions, the `except` block is skipped. However, if an error does occur within the `try` block, the rest of the `try` block is skipped and the code in the `except` block is executed. This is called *catching* the exception. If an error occurs in code that isn't contained within the `try` block, that error won't be caught by the `except` block and will propagate up, potentially crashing the program.

Many kinds of errors can arise in a Python program, each represented by a different kind of Exception object (we'll discuss some of these errors shortly). By default, an `except` block will catch all kinds of exceptions, but you can also specify a particular kind of exception to catch with the following syntax:

```
try:
    # Code that might raise an exception
except SomeException as e:
    # Code that says what to do if there is an exception
```

Here, *SomeException* represents the particular kind of exception you want the `except` block to handle, and as `e` makes the associated Exception object available within the `except` block as the `e` variable (usually for the purpose of printing out a custom error message). If the type of error encountered in the `try` block doesn't match the error specified by the `except` block, the error won't be caught, and again the program could crash.

It's good practice to explicitly define which error you think you might encounter and would like to handle with your `except` block, or to plan for multiple types of errors by adding multiple `except` blocks after the `try` block. If you don't, your code may hit a kind of error that you didn't anticipate without you noticing, and this could cause unexpected results. Here's an example:

```
try:
    value = int("abc")
except ValueError as e:
    print(f"ValueError occurred: {e}")
```

We try to convert the string `abc` into an integer, which obviously isn't possible, so the code will run into a `ValueError`. When you run this, the interpreter executes the code inside the `except` block and prints the following to the console:

```
ValueError occurred: invalid literal for int() with base 10: 'abc'
```

By specifying this error in particular, we handle only that specific exception. This approach makes the code more predictable and easier to debug than using a “naked” `except` and handling all errors generally. If you catch all exceptions without distinction, you might inadvertently catch exceptions you didn't intend to handle, masking other issues in your code.

Common Errors

As mentioned earlier, you may encounter many kinds of errors in Python. Some are built into the language, while others come from libraries and external sources. Python's built-in errors are automatically recognized by the interpreter, regardless of which version of the language you've installed or which libraries you've imported. You can see a full list of Python's built-in errors in the Python documentation at <https://docs.python.org/3/library/exceptions.html>.

We've already seen an example of `ValueError`, which occurs when a function receives an argument with an inappropriate value. Another built-in error is `KeyError`, which occurs when you attempt to access a key that doesn't exist in a dictionary. Because working with APIs in Python typically involves dictionaries, `KeyError` tends to come up often. Here's an example:

```
my_dict = {"a": 7, "b": 5}
try:
    print(my_dict["c"])
except KeyError:
    my_dict["c"] = 1
    print("Added the value for c.")
```

We try to access the nonexistent key `c` in the dictionary, causing a `KeyError`. The `except` block catches the error and adds a new key-value pair to the dictionary, and a message is printed.

Several errors that are unique to the Requests library often get triggered when working with APIs. These are found in `requests.exceptions`, which is a submodule of the library:

ConnectionError A network problem exists, like a faulty internet connection.

Timeout The server takes too long to respond.

HTTPError HTTP request returns an unsuccessful status code like 404.

InvalidURL provided URL is malformed or contains invalid characters.

Another common error, called `JSONDecodeError`, comes from the `json` module and occurs when decoding a JSON response fails because the data isn't valid JSON.

Here's an example of how to handle `ConnectionError` when making an API request. To see this in action, try running the code with your Wi-Fi turned off:

```
import requests

try:
    url = "https://randomuser.me/api/"
    response = requests.get(url)
    result = response.json()
    print(result)
except requests.exceptions.ConnectionError:
    print("Connection error: Please check your internet connection.")
```

If there's no internet connection, calling the `get()` method within the `try` block will trigger a connection error. Since the `except` block is designed to handle this type of error (`requests.exceptions.ConnectionError`), the exception is caught instead of allowing the program to crash. The `except` block then prints a message to check the internet connection.

Breaking Large Requests into Smaller Parts

APIs often place limits on the size of requests and responses, or on the number of requests you can send within a certain time period, so they aren't inundated and can maintain consistent performance. One way to work within these limits, or to just make a response more feasible to process, is *pagination*. In this technique, you divide a request into smaller, more manageable requests called *batches*. This breaks the desired dataset into individual chunks that can be reconstituted on the receiving end, just as the text of a long novel is divided across the individual pages of a book.

As an example, the Random User Generator API sets an upper limit of 5,000 users per request. To retrieve a larger number of users (say, 10,050), we'll need to paginate through the API. The common pattern is to use a

while loop to repeatedly make smaller requests until the desired number is reached, like so:

```
import requests

url = "https://randomuser.me/api/"

total_users = 10050
users_per_request = 5000
retrieved_users = []

while len(retrieved_users) < total_users:
    remaining_users = total_users - len(retrieved_users)
    current_batch_size = min(users_per_request, remaining_users)

    params = {"results": current_batch_size}

    response = requests.get(url, params=params)
    if "error" in response.json():
        print(response.json())
        break

    data = response.json()
    retrieved_users.extend(data["results"])
```

We define three variables to help manage the pagination: `total_users` is the total number of users we'd like to retrieve, `users_per_request` is the batch size, and `retrieved_users` is an empty list for accumulating the results as we receive them. The condition for the `while` loop, which needs to be `True` at the start of each repetition for the loop to keep iterating, is `len(retrieved_users) < total_users`, meaning that we'll start a new iteration only if we haven't retrieved enough users yet. Within the `while` loop, we identify the number of users left to retrieve (`remaining_users`) and the number of results we need to ask the API for this time (the lesser of `users_per_request` or `remaining_users`). Then we make the request and append the results to the `retrieved_users` list. Just in case any of our requests results in an error from the API, we add an `if` statement to print the error.

Using a more straightforward `for` loop instead of a `while` loop would be technically possible in this example, since we know the exact number of users that will be included in each API response. However, often you don't know how many acceptable results will be retrieved from the API and therefore how many iterations you'll need to make. In those cases, a `while` loop is your only option, since it doesn't require you to specify the number of iterations you'll be making up front.

For example, say we'd like to retrieve 20 users who specifically live in Texas. The Random User Generator API doesn't offer an argument to request a particular state, only a country, so there's no way to know how many

users we'll get from Texas in any given set of results. Therefore, we'll have to use `while` to make an indefinite number of API requests and filter the results on our end until we've accumulated 20 users from Texas:

```
url = "https://randomuser.me/api/"

total_users = 20
users_per_request = 5000
retrieved_users = []

while len(retrieved_users) < total_users:
    remaining_users = total_users - len(retrieved_users)
    current_batch_size = min(users_per_request, remaining_users)

    params = {"results": current_batch_size, "nat": "us"}

    response = requests.get(url, params=params)
    if "error" in response.json():
        print(response.json())
        break

    data = response.json()
    users = data["results"]

    texas_users = [user for user in users if user["location"]["state"].lower() == "texas"] ❶
    retrieved_users.extend(texas_users)
```

This `while` loop will continue to fetch users in batches of 5000 until the total number of users from Texas reaches `total_users`, which is 20. Each time through the loop, we use a list comprehension to keep only the users from the correct state ❶. The total number of iterations the code makes will depend on how many users from Texas randomly appear in the results of each request. Once the target number of users from Texas is reached, the `while` loop stops iterating, and the code moves on.

Handling Authentication

Many APIs, even publicly accessible ones, require users to authenticate before interacting with them. Authentication allows the API to identify who's making requests, what requests they're making, and how often they're making them. With this information, the API provider can impose restrictions on who can submit and receive data, as well as how much data they can process. Authentication also lets providers customize the data and services delivered to each user, and log and audit access to meet legal and regulatory requirements.

APIs use two primary mechanisms to authenticate users: API keys and Open Authorization (OAuth). Let's look at what's involved in each.

API Keys

An *API key* is a unique identifier, typically a random alphanumeric string, used to authenticate a client making requests to an API. You obtain a key by registering with an API, and then you include that key when you make requests to that API. This security measure lets API providers know who's accessing their API, how often they're doing it, and what information they can and can't receive.

API keys are sensitive information. An API key getting into the hands of a bad actor, or just someone who doesn't know what they're doing, opens the door for all sorts of nefarious activity. Depending on the API's purpose, this could mean significant financial costs or data getting into the wrong hands. At the very least, a compromised API key can mean rate limits are hit faster and more often, so it's important to keep API keys in a place that's out of view from everyone else.

A place that's *not* out of view from everyone else is in your Python scripts. If your scripts are version controlled (and they should be), they'll be tracked and likely saved in repositories that multiple people can access. Even if a repository is private, or your scripts aren't version controlled at all, they can still live on; you never know when files might be shared or content copied and pasted into places you don't intend it to be. A much safer approach is to store your API keys in environment variables.

Storing Keys in Environment Variables

The environment variables stored locally on your computer are out of view from everyone else. They meet the two necessary qualifications for safely storing an API key:

- They aren't stored within your Python scripts.
- They can be easily accessed by your Python script when it's executed.

To illustrate how to authenticate with an API key stored in an environment variable, we'll use a REST API that provides weather data maintained by a company called OpenWeather. The API provides weather data and uses API keys for authentication. To start, go to <https://openweathermap.org/api> and sign up to create an API key. There's no cost for signing up and creating a key, but the free tier limits you to 1,000 calls per day. That should be plenty for practice.

Once you've received your API key, save it as an environment variable called `WEATHER_API_KEY`. On macOS, add the following line to your shell configuration file:

```
export WEATHER_API_KEY=your_api_key_here
```

On Windows, use the Environment Variables section of Advanced System Settings to create the new environment variable. To verify that your API key has been stored, run `echo $WEATHER_API_KEY` (macOS) or `echo %WEATHER_API_KEY%` (Windows) at the command line. (See Chapter 1 for a more detailed review of how to set environment variables for your operating system.)

Now that you've saved your key as an environment variable, you'll need to retrieve it from within your Python script. To do that, use the `getenv()` function from the `os` module, which is part of the Python Standard Library:

```
import os

api_key = os.getenv("WEATHER_API_KEY")
```

This makes your API key available in your Python script as the `api_key` variable but only when the script is run on your local computer where the environment variable is stored.

Making Requests with API Keys

With most REST APIs that use keys for authentication, you can apply those keys to requests in two ways. One is to include your key as an argument in the URL, just like any other argument. The second way is to add your key to the header of the HTTP request. *Headers* are key-value pairs that provide metadata or additional context or information about the request.

For the OpenWeatherMap API, both techniques are an option, so we'll try both. First, here's a script that authenticates with the API by including the key as an argument in the URL:

```
import os
import requests

api_key = os.getenv('WEATHER_API_KEY')

url = "https://api.openweathermap.org/data/2.5/weather"
params = {"lat": 47.6458, "lon": 122.1320, "appid": api_key}

response = requests.get(url, params=params)
result = response.json()
humidity = result["main"]["humidity"]
```

We build a request for current weather data with three arguments: `lat` and `lon`, which set the latitude and longitude of the desired location, and `appid` for the API key, as specified in the API's documentation. Then we attach those arguments to the request URL by using `get()`. After that, we store the JSON version of the response into a new variable called `result`. From there, we can use any part of the response that we like. In this example, we extracted the humidity by running `result["main"]["humidity"]`.

To see the results, simply run the following:

```
print(humidity)
```

This prints the relative humidity at the specified location that was retrieved from the API.

This approach is straightforward but opens up security concerns. If your API key is included in the URL, it can be logged by any server handling your

request, including your own server, proxy servers, and the API provider's server. Anyone with access to those servers can view the logs and discover your key. URLs are also often shared in error messages that include debugging output, and anyone who sees that information could access the keys. Because of these security concerns, and in order to keep your code more organized, it's generally a better practice to send your API key as part of the HTTP header rather than in the URL, as shown here:

```
api_key = os.getenv('WEATHER_API_KEY')

url = "https://api.openweathermap.org/data/2.5/weather"
headers = {"x-api-key": api_key}
params = {"lat": 47.6458, "lon": 122.1320}

response = requests.get(url, headers=headers, params=params)
result = response.json()
humidity = result["main"]["humidity"]
```

In this example, the API key is paired with the `x-api-key` header, which is added to the request by using the `headers` argument of the `get()` function.

OAuth

Instead of API keys, many major APIs that you might encounter in your day-to-day work, like GitHub, LinkedIn, and Slack, use *Open Authorization* (OAuth). This authorization standard allows third-party applications to grant limited access to a user without exposing their credentials. OAuth provides a secure way for users to authorize applications (like Python scripts) to interact with APIs on the user's behalf.

In simple terms, OAuth provides a mechanism that allows you to input credentials on the server side in order to receive a temporary key that you can store on the client side. Here's a general overview of the typical steps involved:

1. Register with an API provider and establish a client ID and password.
2. Visit a server-side authorization URL to authenticate. Here, you can log in with your credentials and grant permission for your Python script to send requests.
3. After server-side authentication is complete, the authorization server automatically redirects back to the application you're looking to authenticate with an authorization code.
4. Exchange the authorization code for an access token by making a POST request to the API.
5. Use the temporary access key to make an authorized GET request to the API from your Python script.

The exact process, and the Python code you'll need to complete the process, will differ by API. This may seem like a daunting and complex task, but

major API providers typically have up-to-date documentation that details exactly how the authentication process works.

Streamlining Your Requests with Reusable Functions

The code to run API calls can quickly become complex as you apply features like additional arguments, error handling, pagination, and authentication. By encapsulating the code to make requests into functions, you can separate all that complexity from the bread and butter of your script—specifying the data you need and retrieving it. In this section, we'll revisit the Random User Generator and OpenWeather APIs to illustrate two examples of functionalizing code to make API requests.

First, let's create a function to fetch a list of random users from the Random User Generator API:

```
import requests

def fetch_random_users(total_users, params={}):
    url = "https://randomuser.me/api/"
    users_per_request = 5000

    retrieved_users = []
    while len(retrieved_users) < total_users:
        remaining_users = total_users - len(retrieved_users)
        current_batch_size = min(users_per_request, remaining_users)
        params["results"] = current_batch_size
        try:
            response = requests.get(url, params=params)
            data = response.json()
            users = data["results"]
            retrieved_users.extend(users)
        except requests.exceptions.HTTPError as http_err:
            print(f"HTTP error occurred: {http_err}")
            break
        except requests.exceptions.RequestException as req_err:
            print(f"Request error occurred: {req_err}")
            break
        except ValueError as json_err:
            print(f"JSON decode error occurred: {json_err}")
            break

    return retrieved_users
```

This `fetch_random_users()` function takes in two arguments: one that's required, called `total_users`, that specifies the number of users you'd like to retrieve, and one that's optional, called `params`, for any additional request arguments. All the API-specific information like `url` and `users_per_request` is contained within the function, meaning it's abstracted away from the code to

call the function. The while loop for pagination and the try...except blocks for error handling are also encapsulated within the function, so all this complex functionality will unfold seamlessly when the function is called. If one of the errors covered by the except blocks is caught, a statement to that effect will be printed in the console.

To call the function, you need to define the values you'd like to use as the `total_users` and `params` arguments:

```
total_users = 105
params = {"nat": "us", "gender": "female"}

all_users = fetch_random_users(total_users, params=params)
```

After running this code, `all_users` should contain a list of users outputted from the API request. Notice how concise this main script is now that all the nitty-gritty code for making the request is delegated to the `fetch_random_users()` function.

Next, we'll functionalize the code for fetching weather forecast data from the OpenWeather API:

```
import os
import requests

def fetch_weather_forecast(params={}):
    url = "https://api.openweathermap.org/data/2.5/forecast"
    api_key = os.getenv("WEATHER_API_KEY")

    headers = {"x-api-key": api_key}

    try:
        response = requests.get(url, headers=headers, params=params)
        data = response.json()
        forecasts = data["list"] # Assume "list" contains the forecast data
        return forecasts
    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.RequestException as req_err:
        print(f"Request error occurred: {req_err}")
    except ValueError as json_err:
        print(f"JSON decode error occurred: {json_err}")
    return []
```

Similar to `fetch_random_users()`, this `fetch_weather_forecast()` function encapsulates all the logic for making an OpenWeather API request and handling potential errors. This includes the `os.getenv()` function to retrieve the API key from the environment variable. The function uses two return statements, one in the try block to return the retrieved weather data if the request was a success, and one at the end to return an empty list if the request failed and one of the except blocks was triggered.

To call the function, define a dictionary containing the API request arguments and pass it through the `params` argument of `fetch_weather_forecast()`:

```
params = {"lat": 47.6458, "lon": 122.1320, "cnt": 50, "units": "metric"}
```

```
all_forecasts = fetch_weather_forecast(params=params)
```

If all goes well, `all_forecasts` should contain the weather forecast data from the OpenWeather API.

To make these two functions easy to find, reuse, and maintain, let's create a Python file called *api_utils.py* and place both function definitions within it. This will enable us to easily import and use these functions in other scripts:

```
import os
import requests
```

```
def fetch_random_users(params={}):
    # Code inside the fetch_random_users function
```

```
def fetch_weather_forecast(params={}):
    # Code inside the fetch_weather_forecast function
```

Don't forget to include an *__init__.py* file in the same directory as *api_utils.py* if you plan to import the functions to a file in another directory or if the directory containing the functions isn't included in the `PATH` environment variable.

To use these functions in another script, you can import them and call them from within that script's main block, as we discussed in Chapter 3. For example, you may have *random_users.py* that imports `fetch_random_users()`, makes a request, and saves the results to a JSON file:

```
from api_utils import fetch_random_users
import json
```

```
if __name__ == "__main__":
    total_users = 105
    params = {"nat": "us", "gender": "female"}

    all_users = fetch_random_users(total_users, params=params)
    with open("users.json", "w") as f:
        json.dump(all_users, f)
```

Similarly, you may also have a *weather_forecast.py* file that saves some weather data to a JSON file:

```
from api_utils import fetch_weather_forecast
import json

if __name__ == "__main__":
    params = {"lat": 47.6458, "lon": 122.1320, "cnt": 50, "units": "metric"}

    all_forecasts = fetch_weather_forecast(params=params)
    with open("forecast.json", "w") as f:
        json.dump(all_forecasts, f)
```

Packaging your API request code into reusable functions like this provides several advantages. First, it makes your code easier to maintain. You can import these functions into as many other scripts as you like, but anytime you fix an error or add another feature to the function, you'll need to make the change in only the original *api_utils.py*. This approach also makes the *random_users.py* and *weather_forecast.py* files much shorter and easier to read and understand. If you want to change the arguments of either API call, locating and updating the `params` argument is quick and easy. Without functionalizing the API request part of the code, the code would be far more cumbersome and chaotic.

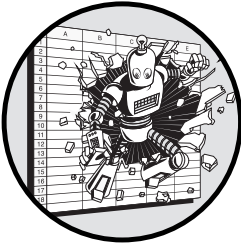
Conclusion

This chapter covered the basics and benefits of working with REST APIs. We explored the architecture of a REST API request, including the role of HTTP and its integration into the broader structure of the internet. We discussed the essential components of a robust GET request using the Requests library, such as adding arguments, handling errors, implementing pagination, and managing authentication. Finally, we demonstrated how to functionalize these requests to promote simple, readable, and maintainable code.

Knowing how to retrieve data from APIs will open up opportunities for automating the work you currently do in Excel. With simple Python scripts like the ones we've explored in this chapter, you can eliminate much of the manual data collection that an Excel-based workflow tends to rely on. Once you have that data in hand, you can analyze, visualize, or process it—whether in Excel or with the Python-based analytical tools covered in this book.

9

CREATING CHARTS AND VISUALS



This chapter covers charting in Python. We'll examine the situations where creating charts with Python code is superior to creating them in Excel. Then we'll explore how to work with Plotly, a powerful and relatively simple charting library. We'll start with basic line and bar charts, then move on to more complex, multidimensional charts with custom styling, reusable themes, and interactive elements.

No matter the goal of your data analysis—spotting trends, seeing relationships, making decisions, or communicating ideas—visualizations are one of the most powerful tools you have. Creating these visualizations more efficiently can be a real boost to productivity.

Before we begin, it's worth acknowledging that charting is one of Excel's best features. Making aesthetically pleasing charts in common styles in Excel is fairly intuitive. Since you already know Excel, you may therefore wonder why you would want to start making charts in Python instead. It all comes down to what exactly you want your chart to look like and the context in which you're creating it.

Charting with Excel vs. Python

Charting is an example of how Excel's visual, interactive nature can sometimes make your work easier. In Excel, you create charts by selecting a set of data and then choosing one of a few simple charting templates, like a bar chart or a scatter plot, to illustrate it. When you want to change a specific element in the chart, you can simply click it, and a menu emerges with formatting options for that specific element. The process is quick and intuitive, and creating charts that are perfectly suitable for many situations doesn't require much technical know-how.

In Python, because you can't simply click visual elements to edit them, charting isn't nearly as intuitive (at least not at first). To change an element, you need to know how to reference it in code, an extra step that takes time when you're new to the process (but that can become faster with practice). Because of this time factor and learning curve, Excel really shines when you're creating a simple, one-time visualization that you don't think you'll ever reuse, which is often the case for many of the charts that most of us create in our day-to-day work.

However, often situations arise that require more from your charts. You might need more complexity, scalability, automation, customization, or a more robust workflow. We'll walk through a few situations where each of these factors makes Python a superior charting tool to Excel.

Integration into a Broader Workflow

When you create a chart, you must first acquire the data that informs it. Often the source of this data is a database, API, or just a local CSV or Excel file. As we've seen so far in this book, all these sources are easily (and possibly most easily) accessible through Python code. When you're deciding whether it's worth the time to create a chart in Python, an appropriate first question to ask yourself is, Where is the data coming from? If the answer is Python, then why add the additional effort of moving the data into Excel?

Similarly, after you create a chart, you often need to export it somewhere. Sometimes you'll attach it to an email or instant message and send it to a colleague. Other times you'll publish the chart to the web or include it in a report. And sometimes you'll just save it for future reference. Exporting a chart to any of these places can be handled from within a Python script. That leads to a second question: What are you doing with the chart? If it's something that could be automated through a Python script, then why not?

Ultimately, if the rest of your workflow is in Python, creating the chart in Python probably makes sense as well. This way, you can streamline and simplify the entire process that the chart is part of.

Reusability

Sometimes you'll need to create the same chart repeatedly with different datasets. For example, you may generate a regular sales report showing the same set of charts with new data every month, or you might perform the

same analysis for each department in a company. These situations involve charting code that you create once and reuse multiple times.

Charting in Python allows you to leverage the reusability of Python code. You can write functions and scripts that are easily adaptable to different datasets, saving time in the long run, even if it requires a bit more time in the short run.

Version Control

Whether you're creating a chart in Excel or Python, the initial process usually involves adding complexity over time. Typically, you start with something simple like a basic plot with your data, then you add and edit other elements in the chart, like titles and labels, and finally you apply styling elements like colors and fonts. As you refine and develop your chart, you may want to revisit previous versions of your work to compare changes or revert to an earlier state. Version control allows you to see who made changes to the charts, what those changes were, and when they were made. This helps manage contributions from others and ensures that you can revert to earlier versions if necessary.

Charting in Python enables you to take advantage of all the benefits of version control in ways that Excel can't. With tools like Git, you can maintain a complete history of your charting code, track modifications, collaborate, and experiment with different visualizations without losing track of your progress. This is helpful anytime you're gradually adding elements and making refinements over time.

Customization

Charting templates in both Excel and Python begin with a standard set of elements. These templates have default options for colors, fonts, font sizes, line widths, axis labels, and every other element shown in a chart. Excel and Python differ in how specifically you can deviate from this standard set of options, and how easy it is to do so.

For example, you may want all the data labels for negative values to be red and positive values to be green. That's difficult to do in Excel without a lot of manual effort or VBA code. In Python, you can do this with two lines of code. You also might want to use custom shapes for markers for different data points. This is fairly straightforward in most Python charting libraries. These sorts of customizations are relatively painless because of the granularity of control that charting in Python enables. It simply has more options for every element, and you have more granular control over these options.

Standardization

The more you customize your charts, the more you may want to standardize your customization choices across all the charts you make. In other words, if you've put a lot of work into creating charts that look a specific way, you likely want to reuse those choices again and again. And when you're able to

maintain this uniform look and feel across all your charts, your reports and presentations will look much more professional.

While Excel allows you to create custom reusable themes, your options are limited. You can specify the colors and fonts for your charts, but that's as far as it goes. Once you deviate from the basic template of a given chart, you often end up needing to manually apply similar customizations to every chart you create. For every Excel chart you make, you'll need to click into every element you want to edit and make all your changes by hand, over and over again.

In Python, you can create boilerplate styling code for charts that defines your own reusable set of standard design choices for every element of the chart—essentially like creating your own custom theme in Excel but with much greater specificity. This makes standardization much simpler and more efficient in Python.

Charting Libraries in Python

Our focus in this chapter is on the Plotly library, which supports a wide range of chart types, allows for publication-quality formatting and styling, and accommodates interactivity. However, you can use several other libraries for creating visualizations with Python, each with its own advantages and disadvantages. Most will allow you to just as easily create the example visualizations shown in this chapter. Here are a few of the most popular:

Matplotlib Was initially created with scientists and engineers in mind, but it's useful for all kinds of charting. Matplotlib charts don't require as much processing power as Plotly charts, but at a cost of less interactivity and often less modern styling.

Seaborn Is built on top of Matplotlib to provide a high-level interface for drawing attractive and informative statistical graphics, like heatmaps. It's particularly good for visualizing data distributions and relationships.

Bokeh Is built to work well with the updated features of modern web browsers. It excels at highly interactive visualizations where you intend to have users explore on their own.

Altair Is more lightweight than Plotly. It emphasizes simplicity and user-friendliness but doesn't have as many customization options.

pandas Includes basic charting functionality for creating simple plots directly from DataFrames. Pandas charts use Matplotlib behind the scenes.

Plotly's advantage is that it can accommodate a wide variety of visualizations and customizations. It also provides higher-level features like animation and interactivity while still maintaining a relatively straightforward syntax. Additionally, Plotly integrates well with Jupyter Notebooks and Dash dashboards. We'll focus on the latter in Chapter 10.

Using Plotly in Jupyter Notebooks

As we've discussed, Jupyter Notebooks allow you to write and run small sections of code interactively to get immediate feedback about the results. This is especially useful for visual experimentation, as when you're creating different types of plots and data transformations with Plotly. Jupyter Notebooks also allow for *in-line visualizations*, meaning you can render plots directly within the notebook cells. This makes it easy to visually inspect your data and the results of your code in real time.

With a Jupyter Notebook, you can also display multiple Plotly plots in sequence in a single, cohesive document. This feature is particularly useful for creating reports and presentations that involve multiple charts based on the same dataset. We'll mimic this process with the example dataset used in this chapter. Work through each example within a Jupyter Notebook, with each chart in its own notebook cell. This will enable you to immediately see the results of your code and make iterative adjustments quickly.

Installing and Importing Plotly

Before you can start charting in Plotly, you first need to install it. You can do this using pip (or pip3) as follows:

```
pip install plotly
```

You also need to import Plotly into your *.py* file or *.ipynb* Jupyter Notebook before using it:

```
import plotly
```

This will allow you to access Plotly's functionality for creating visualizations. However, for the examples in this chapter, most of the functionality is contained within the library's `graph_objects` submodule, often abbreviated with the alias `go` to simplify the syntax. You can import that submodule as follows:

```
import plotly.graph_objects as go
```

Now you can use `go` to create your charts.

Creating Simple Charts

In the most abstract sense, making a chart with Plotly isn't terribly different from making one with Excel. In Excel, you select the data you want to visualize, go to the Insert tab, and choose one of the Chart options. Excel generates the chart, and then you customize it however you like. Similarly, in Plotly you first prepare your data in a structured format, such as a pandas DataFrame. Then instead of clicking a button, you call a function from the Plotly library. You'll see how this works in this section as we create some simple visualizations.

An Example Dataset

For the examples in this section, we'll use the Diamonds dataset from the PyDataset library. This library contains a collection of datasets that are commonly used to practice manipulating and visualizing data. Install PyDataset with `pip` or `pip3` as follows:

```
pip install pydataset
```

Then import the `pydataset.data()` function into your Jupyter Notebook:

```
from pydataset import data
```

Now you can load the Diamonds dataset, like so:

```
diamonds = data("diamonds")
```

This saves the dataset, which includes information on the characteristics and prices of 53,940 diamonds, into a pandas DataFrame called `diamonds`. To see what the dataset looks like, use the `head()` method:

```
diamonds.head()
```

This prints the first five lines of the DataFrame, as shown in Figure 9-1.

	carat	cut	color	clarity	depth	table	price	x	y	z
1	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
2	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
3	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
4	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
5	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

Figure 9-1: The first five rows of the Diamonds dataset

To see the details of what each column refers to, use the `show_docs=True` parameter:

```
data("diamonds", show_doc=True)
```

This displays the documentation for the dataset in Markdown text that looks like this:

```
## Prices of 50,000 round cut diamonds
```

```
### Description
```

A dataset containing the prices and other attributes of almost 54,000 diamonds. The variables are as follows:


```
### Usage
data(diamonds)
--snip--
```

The output also gives the size of the dataset (53,940 rows and 10 variables) and a description of each column.

A Basic Line Chart

The first chart we'll create is a line chart showing the average price of diamonds by carat weight. To create the dataset for this chart, we'll have to do a simple transformation on the dataset to get an average across all diamonds within the same carat weight:

```
avg_px_by_carat = diamonds.groupby("carat")["price"].mean()
```

We group the diamonds DataFrame by the carat column and calculate the mean price for each carat value. This gives us a pandas Series called `avg_px_by_carat` containing the average price for each carat value. To turn that Series into a chart, we first use `go.Figure()` to create a new Figure object:

```
fig = go.Figure()
```

A Figure object contains all the information needed to create and display a plot with Plotly. It's composed of two main components: data and layout. The data component holds the actual data to be plotted, such as *traces*, which are sets of data stored in key-value pairs, similar to dictionaries. The layout component defines the appearance of the plot, including axis settings, titles, legend placement, and overall styling.

You make changes to a chart in Plotly by adding or updating these two components of the Figure object. In this case, we'll add a trace to `fig` by using the `add_trace()` method with the `go.Scatter()` function, which creates traces for scatter plots and line charts:

```
fig.add_trace(go.Scatter(
    x=avg_px_by_carat.index,
    y=avg_px_by_carat,
    mode="lines",
    name="Price by Carat"
))
```

In the trace, the x-coordinates are the carat values, and the y-coordinates are the average prices. The argument `mode="lines"` specifies that we want to draw a line chart, and `name="Price by Carat"` gives the trace a name that will appear in the chart's legend, if we have one.

Now we'll add a chart title and axis titles, using the `update_layout()` method:

```
fig.update_layout(
    title_text="Price of Diamonds by Carat",
```

```
    xaxis_title_text="Carat",  
    yaxis_title_text="Price",  
)
```

Each argument here is a characteristic of the chart that we want to change: `title_text` sets the title of the chart, and `xaxis_title_text` and `yaxis_title_text` set the axis titles. After this method is executed, the `fig` object will internally implement these changes.

Finally, to display the chart in your Jupyter Notebook, run `fig.show()`:

```
fig.show()
```

The `show()` method renders and displays the figure. In a Jupyter Notebook, calling `fig.show()` will embed the chart directly within the notebook cell. Your result should look like Figure 9-2.

Price of Diamonds by Carat

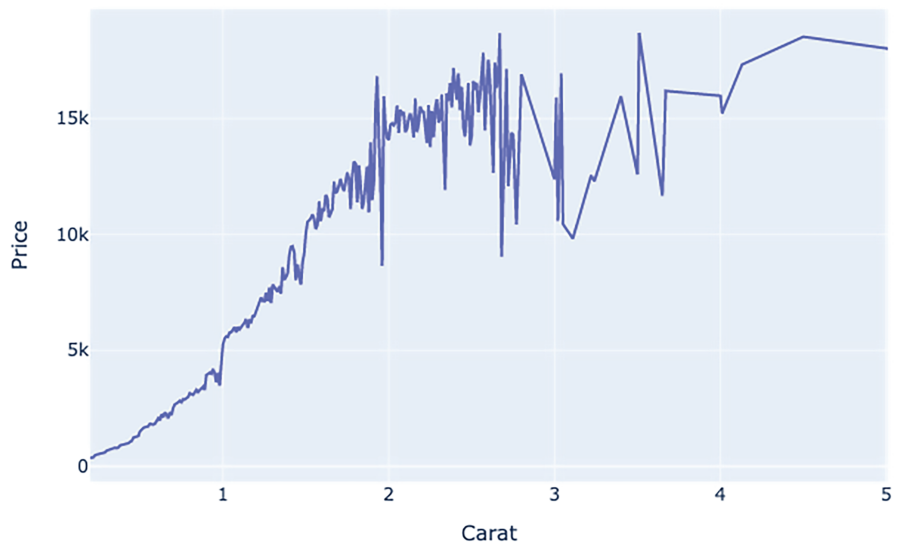


Figure 9-2: A simple Plotly line chart

In the code to create this chart, you needed to specify only the data used for the x- and y-values of each data point, that it was a line chart, the name of the trace, and the chart and axis titles. Plotly determined on its own what the range and interval of the axes should be, and it applied some default colors and fonts.

A Multi-Trace Line Chart

Now let's create a line chart with separate traces to show two related series from the dataset. We'll plot the average price of diamonds by carat for the two most extreme diamond color classifications, labeled as D and J in the dataset. These classifications are specified in the color column in the diamonds DataFrame. First, we need to transform the dataset to include the average prices by carat and color:

```
avg_px_by_carat_color = diamonds.groupby(["carat", "color"])["price"].mean()
avg_px_by_carat_color = avg_px_by_carat_color.reset_index()

d_color_data = avg_px_by_carat_color[avg_px_by_carat_color["color"] == "D"]
j_color_data = avg_px_by_carat_color[avg_px_by_carat_color["color"] == "J"]
```

After transforming the data, we extract two Series objects containing the average price by carat for each color, `d_color_data` and `j_color_data`. Next, we need to create and modify a Figure object:

```
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=d_color_data["carat"],
    y=d_color_data["price"],
    name="D"
))

fig.add_trace(go.Scatter(
    x=j_color_data["carat"],
    y=j_color_data["price"],
    name="J"
))

fig.update_layout(
    title_text="Average Price of Diamonds by Carat and Color",
    xaxis_title_text="Carat",
    yaxis_title_text="Average Price",
)

fig.show()
```

We add two separate traces to `fig`, one for each color, using the `add_trace()` method as before. We assign each trace a name attribute, which will be used to identify the trace in the chart's legend. Once again, we also set the chart title and axis titles by using `fig.update_layout()`. This produces a line chart with two lines, as shown in Figure 9-3.

Average Price of Diamonds by Carat and Color

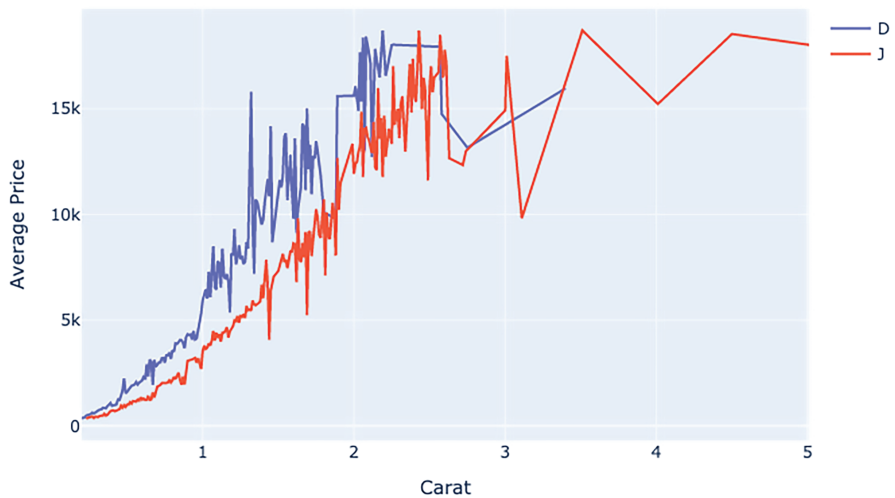


Figure 9-3: A line chart with two traces

By default, Plotly assigns a different color to each trace in the chart—blue for the first trace and red for the second (not shown here, as the book is in grayscale). These colors follow a predefined sequence in the default color palette, similar to the one used when you make charts in Excel.

A Basic Bar Chart

Let's try creating a different type of chart: a bar chart. As you'll see, the process is largely the same, differing primarily in that we need to use a different function when creating the trace. Our bar chart will show the average price per carat of diamonds by their clarity.

The Diamonds dataset doesn't already have a column for price per carat, so we'll add one:

```
diamonds["price_per_carat"] = diamonds["price"] / diamonds["carat"]
```

Next, we group the diamonds by their clarity and calculate the average price per carat for each category:

```
avg_px_per_carat_by_clarity = diamonds.groupby("clarity")["price_per_carat"]  
                                .mean()
```

The diamond clarity grades aren't already in an order that Plotly can understand, like alphabetical or numeric. To ensure that the clarity categories are displayed in a logical order in the bar chart, we need to sort the data by the common clarity grading order:

```
clarity_order = ["I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"]
avg_px_per_carat_by_clarity = avg_px_per_carat_by_clarity.loc[
    clarity_order
]
```

This step is necessary because the default order Plotly uses might not reflect the typical clarity grading scale used in the diamond industry. By specifying the order, we make sure the bar chart is easier to understand and correctly represents the clarity categories from lowest to highest quality.

Next, we create a new Figure object and give it a trace:

```
fig = go.Figure()

fig.add_trace(go.Bar(
    x=avg_px_per_carat_by_clarity.index,
    y=avg_px_per_carat_by_clarity,
))
```

We pass the `go.Bar()` function to `add_trace()` instead of `go.Scatter()` to make this a bar chart instead of a line chart. As usual, we finish by adding a chart title and axis titles with `update_layout()` and displaying the chart:

```
fig.update_layout(
    title_text="Average Price of Diamonds by Clarity",
    xaxis_title_text="Diamond Clarity",
    yaxis_title_text="Average Price",
)

fig.show()
```

Your result should look like Figure 9-4.

Average Price of Diamonds by Clarity

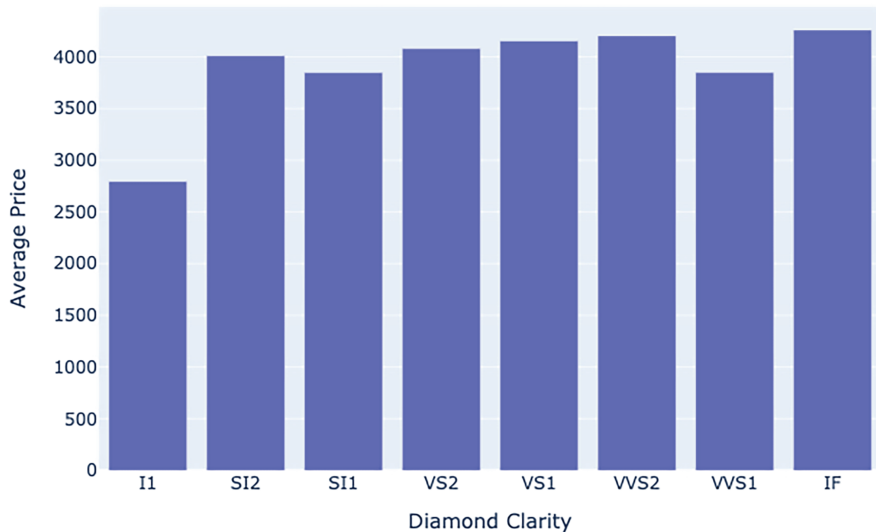


Figure 9-4: A simple Plotly bar chart

With bar charts like this, Plotly automatically adjusts the width of the bars based on the number of bars and the available space, and the y-axis automatically adjusts to fit the range of bar sizes.

A Multi-Trace Bar Chart

Bar charts, too, can have multiple traces, meaning there will be multiple bars for each category shown along the x-axis. To illustrate, we'll create a bar chart showing the average price per carat of diamonds by their clarity and cut classifications. This will be a grouped bar chart; each color of bar will represent a cut, and each group of bars a clarity.

First, using the `diamonds` DataFrame, we group the diamonds by their clarity and cut and calculate the average price per carat for each group:

```
avg_px_per_carat_by_clarity_cut = diamonds.groupby(  
    ["clarity", "cut"]  
)[ "price_per_carat"].mean().unstack()
```

Similar to the way we ordered the clarity ratings in the previous section, we'll order the cuts and apply both the cut and the clarity orders to the dataset:

```
cut_order = ["Fair", "Good", "Very Good", "Premium", "Ideal"]  
avg_px_per_carat_by_clarity_cut = avg_px_per_carat_by_clarity_cut.loc[  
    clarity_order, cut_order  
]
```

Now we create the Figure object and declare the traces:

```
fig = go.Figure()

for cut in cut_order:
    fig.add_trace(go.Bar(
        x=avg_px_per_carat_by_clarity_cut.index,
        y=avg_px_per_carat_by_clarity_cut[cut],
        name=cut
    ))
```

We use a for loop to add a trace for each cut in the `cut_order` list. As in the multi-trace line chart example, we give each trace a `name` property so the traces will be labeled in the chart's legend.

Finally, we update the layout to add title and axis labels, and we show the chart:

```
fig.update_layout(
    title_text="Average Price Per Carat of Diamonds by Clarity and Cut",
    xaxis_title_text="Diamond Clarity",
    yaxis_title_text="Average Price",
)

fig.show()
```

Your result should look like Figure 9-5.

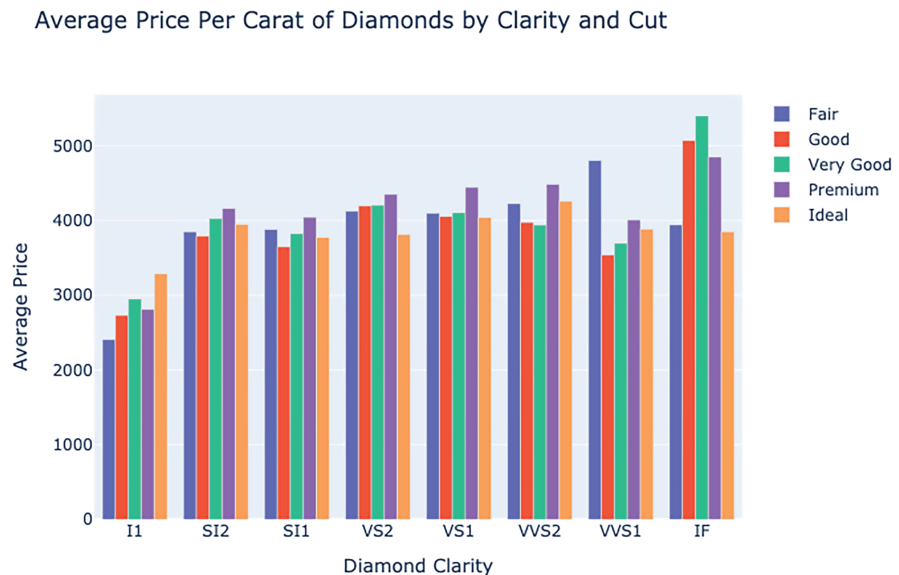


Figure 9-5: A multi-trace bar chart

In this chart, each trace of bars has its own unique color, specified by the default palette. For each level of diamond clarity, there's a group of bars, one of each color.

Saving Charts as Images

To use your Plotly charts outside the Jupyter Notebooks where you create them, you need to save them as image files, like PNGs or JPEGs. That requires another Python library called Kaleido, which you can install using pip or pip3 as follows:

```
pip install -U kaleido
```

The Kaleido library is just a dependency for Plotly, meaning you don't need to import or work with it directly. Instead, Plotly works with Kaleido behind the scenes through the code in its `plotly.io` submodule to save your charts. To import the submodule, run this code (`pio` is a common alias for `plotly.io`):

```
import plotly.io as pio
```

After creating your figure and customizing it as needed, you can save the figure as an image file by using the `pio.write_image()` function. For example:

```
pio.write_image(fig, "chart.png")
```

This command saves the current figure (`fig`) to a file named *chart.png* in the current working directory. To specify a different directory, include the path in the filename. Other file formats you can use include JPEG and PDF.

Charting Multidimensional Data

With Plotly, you can adjust the size and color of markers to represent additional dimensions of data. This approach provides a richer understanding of the dataset compared to basic line or bar charts.

To illustrate, let's create a scatter plot that conveys multiple dimensions of the Diamonds dataset (carat, price, clarity, and color) in a single visualization. Each diamond in the dataset will be represented with a circular marker on the plot. We'll use the x-axis to plot carats and the y-axis to plot price, as in our line charts, but we'll also use the size and color of the markers to convey the diamonds' clarity and color, respectively.

First, we use two dictionaries to define the order for the clarity and color designations. Plotly will use these orders to create the appropriate size and color gradations for the markers in the scatter plot:

```
clarity_order = {  
    "I1": 1,  
    "SI2": 2,  
    "SI1": 3,  
    "VS2": 4,  
    "VS1": 5,  
    "VVS2": 6,  
    "VVS1": 7,  
    "IF": 8  
}  
color_order = {"J": 1, "I": 2, "H": 3, "G": 4, "F": 5, "E": 6, "D": 7}
```

Then we generate two new columns in the diamonds DataFrame, `clarity_num` and `color_num`, by mapping the orders we defined to the values in the `clarity` and `color` columns:

```
diamonds["clarity_num"] = diamonds["clarity"].map(clarity_order)  
diamonds["color_num"] = diamonds["color"].map(color_order)
```

Next, we create the Plotly figure and add the scatter plot trace:

```
fig = go.Figure()  
  
fig.add_trace(go.Scatter(  
    x=diamonds["carat"],  
    y=diamonds["price"],  
    mode="markers",  
    marker=dict(  
        size=diamonds["clarity_num"],  
        color=diamonds["color_num"],  
    ))))
```

We use the `carat` column for the x-axis and the `price` column for the y-axis. The other arguments to the `go.Scatter()` function help define the additional dimensions of the size and color of the dots. With `mode="markers"`, we specify that we want to create a scatter plot with markers representing each data point (as opposed to our earlier line plots, which used lines to show the changes between data points without actually marking the data points themselves).

The `marker` argument is a dictionary that defines the appearance of the markers. The `size=diamonds["clarity_num"]` argument sets the size of each marker based on the `clarity_num` column, so diamonds with different clarity levels will be represented by markers of different sizes. Likewise, `color=diamonds["color_num"]` sets the color of each marker based on the `color_num` column, so diamonds with different colors will be represented by markers of different colors.

As usual, we finish by adding a title and axis labels and displaying the chart:

```
fig.update_layout(  
    title_text="Scatter Plot of Price vs Carat with Clarity and Color Gradients",  
    xaxis_title_text="Carat",  
    yaxis_title_text="Price"  
)  
  
fig.show()
```

The result should look like Figure 9-6.

Scatter Plot of Price vs Carat with Clarity and Color Gradients

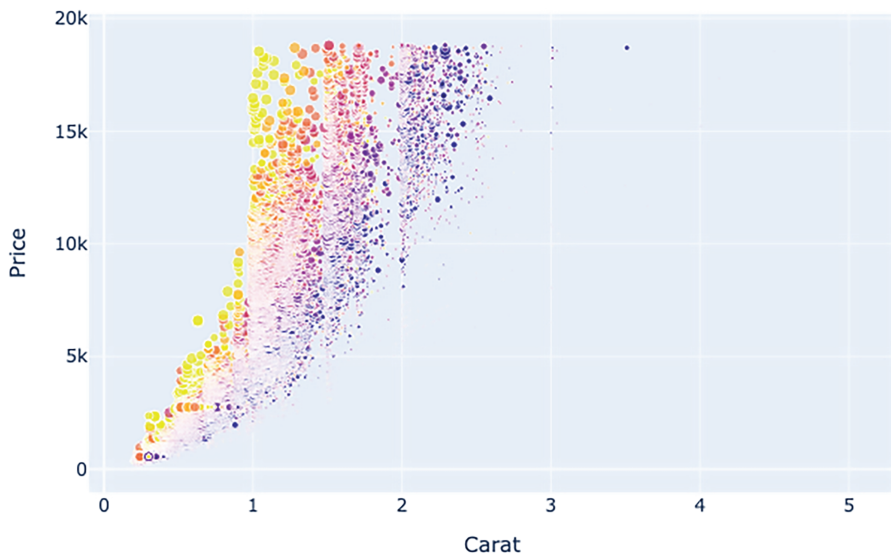


Figure 9-6: A multidimensional scatter plot

This is an example of a chart that's straightforward to create in Python but would be more challenging to create in Excel because of the sheer number of data points and the color and size gradients.

Elevating Charts with Custom Styling

A chart's *styling* encompasses all the aspects of the chart other than the raw data, such as colors, fonts, line thickness, and background shading. Styling is often an afterthought, viewed as purely aesthetic, but it's an integral part of making visualizations effective.

The person consuming your chart probably has only a finite amount of time to devote to understanding it. Think of their attention as a ticking time bomb. If an axis that represents percentages is formatted as decimals rather

than whole numbers (such as 0.9 instead of 90%), it takes the viewer time to make the conversion. If data labels are too small, the viewer will need to squint or zoom in, and that also takes time. Or if the colors used in your chart are too similar, the viewer might have difficulty quickly differentiating between data points. All these marginal challenges for the viewer may add up, to the point where you lose their attention altogether.

Contextually appropriate and aesthetically pleasing styling buys you more time and helps keep your audience engaged. Well-designed charts are easier to read and interpret, making it more likely that your audience will understand and retain the information you're presenting without losing their focus along the way.

Another aspect of styling is the cohesion between charts that are presented together. Part of keeping people engaged is ensuring that your charts aren't visually jarring. For example, if a relatively unremarkable data point is highlighted bright red for no apparent reason, or if the labels on your x-axis are inexplicably in a different font than the rest of the chart, those sorts of details can distract the viewer and make it harder to process the information. The surprising aspect of your analysis should be its conclusions, not the strange background color or tiny font you used.

Layout Properties

The range of options for customizing the layout of a Plotly chart can be overwhelming. While you can always check Plotly's online documentation for the details about a particular property you want to adjust, Plotly's layout properties are structured in a way that can make it easier to manage custom styling.

Plotly follows a hierarchy of properties and subproperties, and the underscores in property names communicate that hierarchy. For example, the underscore in `xaxis_title` indicates that we're referring to the title sub-property of the main `xaxis` property. Meanwhile, `xaxis` also has other sub-properties like `tickformat` and `linecolor` that you can access in the same manner.

If you're setting a lot of subproperties of the same main property, you can set them all within a dictionary and then assign that dictionary to the main property. This keeps your code more organized. For example, say you have the following code setting lots of properties via the `update_layout()` method:

```
fig.update_layout(  
    title_text="Bar Chart",  
    xaxis_title="Category",  
    yaxis_title="Values",  
    xaxis_tickformat="%b-%d",  
    yaxis_tickformat=".2f",  
    xaxis_linecolor="black",  
    yaxis_linecolor="black",  
    xaxis_linewidth=2,  
    yaxis_linewidth=2  
)
```

This code is less than ideal because it lists all the properties directly as arguments to the `update_layout()` method, making it harder to read and maintain. To make it cleaner, you can group the properties hierarchically into dictionaries, like so:

```
xaxis_properties = {
    "title": "Category",
    "tickformat": "%b-%d",
    "linecolor": "black",
}

yaxis_properties = {
    "title": "Values",
    "tickformat": ".2f",
    "linecolor": "black",
}

fig.update_layout(
    title_text="Bar Chart",
    xaxis=xaxis_properties,
    yaxis=yaxis_properties
)
```

The subproperties related to the x-axis are set within `xaxis_properties`, and the subproperties related to the y-axis are set within `yaxis_properties`. Then those dictionaries are assigned to the main `xaxis` and `yaxis` properties when `update_layout()` is called.

This approach makes it much easier to understand and maintain your code. If, for example, you need to make a change to `yaxis_linecolor`, you can easily locate and edit the value of the `linecolor` key within the `yaxis_properties` dictionary. The more customization options you're juggling, the more this hierarchical approach will help you keep your code organized.

Common Properties

To save you from wading through the Plotly documentation every time you make a chart, this section breaks down some of the most common properties you can update with `update_layout()`.

The title is the main title of the chart. Here are some of its properties:

- font** Gives font settings for the title
- x** Indicates the horizontal position of the title (0 to 1)
- y** Indicates the vertical position of the title (0 to 1)

In Plotly, title can also be a subproperty of another property. For example, `xaxis`, `yaxis`, and `legend` all have a `title` setting. Meanwhile, `font` has many subproperties of its own, including these:

family Specifies the font family—for example, Arial, Courier New, or Times New Roman (see “Fonts and Font Families” on page 204 for more information)

size Sets the font size in pixels

color Sets the font color—for example, black or #1f77b4 (see “Colors” on page 205 for more information)

Putting these pieces together, you can adjust the font settings of a given title with a nested dictionary, like so:

```
title={
  "font": {
    "size": 18,
    "family": "Arial"
  }
}
```

The `xaxis` and `yaxis` properties control the scale and appearance of a chart’s two axes. They have the same set of subproperties, including the following:

title Sets the title of the axis

showgrid Shows or hides grid lines

gridcolor Sets the color of the grid lines

gridwidth Sets the width of the grid lines

zeroline Shows or hides the zero line

zerolinecolor Sets the color of the zero line

zerolinewidth Sets the width of the zero line

showline Shows or hides the axis line

linecolor Sets the color of the axis line

linewidth Sets the width of the axis line

showticklabels Shows or hides the tick labels

tickcolor Sets the color of the ticks

tickwidth Sets the width of the ticks

ticklen Sets the length of the ticks

tickfont Sets the font for the tick labels (size, color, family)

tickangle Sets the angle of the tick labels

range Sets the range of the axis (min, max)

type Sets the type of the axis (linear, log, date, category)

The `legend` property controls the chart's legend, which holds labels for the chart's traces. Its subproperties include the following:

- x** Sets the horizontal position of the legend (0 to 1, relative to the plot area)
- y** Sets the vertical position of the legend (0 to 1, relative to the plot area)
- orientation** Sets the orientation of the legend (v for vertical or h for horizontal)
- bgcolor** Sets the background color of the legend
- bordercolor** Sets the border color of the legend
- borderwidth** Sets the border width of the legend
- font** Sets the font for the legend text (size, color, family)
- title** Sets the title of the legend

The `margin` property sets the spacing between the plot area (where the actual chart is drawn) and the edges of the entire plot (the boundary of the overall figure). It has the following subproperties:

- l** Sets the left margin
- r** Sets the right margin
- t** Sets the top margin
- b** Sets the bottom margin
- pad** Sets the padding between the plot area and the axis lines

In addition, some properties refer to the entire figure:

- showlegend** Indicates whether to show or hide the legend
- plot_bgcolor** Sets the background color of the plot area
- paper_bgcolor** Sets the background color of the entire chart

This list may seem long, but as you become familiar with these properties, the process of creating charts in Plotly will become quicker, and your charts will begin to look more bespoke.

Fonts and Font Families

Plotly doesn't have its own built-in fonts. Instead, it uses the fonts that are available on the system where the chart is being rendered. Not everyone has the same fonts on their system, so it's important to provide fallback options when setting the fonts for your plots.

Typically, you specify more than one font through the `font_family` property, plus a generic type of font (like serif or sans-serif) as a last resort. This way, you have fallback options in case the primary font you want isn't available. For example, the property `font_family="Helvetica, Arial, sans-serif"`

specifies a primary font (Helvetica), a more widely available fallback font (Arial), and a generic font family as a last resort (sans-serif). This ensures that the text will at least be displayed in a preferred style, even if the specific preferred fonts aren't available.

Colors

You can refer to specific colors in Plotly in several ways. If you don't have an exact shade in mind, you can use Plotly's named colors, which include the usual suspects like red, green, and yellow, as well as a wider range of less common colors like darkmagenta and peachpuff.

If you need to achieve a specific shade, perhaps because you're following a certain design standard or palette, you can use *hexadecimal codes* like #022d16, which is a shade of dark green. After the hash mark, these codes consist of six digits, two each for the red, green, and blue (RGB) components of a color. They're written in *hexadecimal*, a base-16 number system that uses the digits 0 through 9 plus the letters A through F to represent values from 0 to 15 (A is 10, B is 11, and so on).

If hexadecimal isn't your thing, you can also use the `rgb()` function to define the RGB components of a color as regular base-10 integers ranging from 0 to 255. However, in order for Plotly to integrate it, you'll need to specify the color as a string. For example, `"rgb(214, 164, 192)"` (with the quotes) produces a shade of pink. Lots of online tools let you pick a color and see its hexadecimal and RGB codes.

Themes

A *theme* is a predefined set of styling options that can be applied to multiple charts. Using a theme enforces consistency across all your charts. You can use one of Plotly's built-in themes, or you can define your own custom theme. You need to create the theme only once regardless of the number of charts you apply it to.

Maintaining a consistent look with a theme is especially important when you're creating charts on behalf of an organization that has a distinct brand to uphold. If every other presentation is blue and yellow, your charts should also be blue and yellow. If your company uses a specific font, you should use that font. This kind of consistent branding projects an air of professionalism and credibility around your visualizations.

Built-in Themes

If you don't have a specific set of preferences for each layout option in your chart or a brand standard you need to adhere to, Plotly offers several built-in themes that you can use to change multiple properties of the chart at once. You can find a list of built-in templates and view examples of them at <https://plotly.com/python/templates/>.

To apply one of these themes to a figure, call the `update_layout()` method again to change the `template` attribute. For example, here we apply the built-in `simple_white` theme to the grouped bar chart created in “A Multi-Trace Bar Chart” on page 196:

```
fig.update_layout(template="simple_white")
```

```
fig.show()
```

This code creates a bar chart with the `simple_white` theme applied, ensuring that all elements of the chart follow that theme’s styling conventions. The result should look like Figure 9-7.

Average Price Per Carat of Diamonds by Clarity and Cut

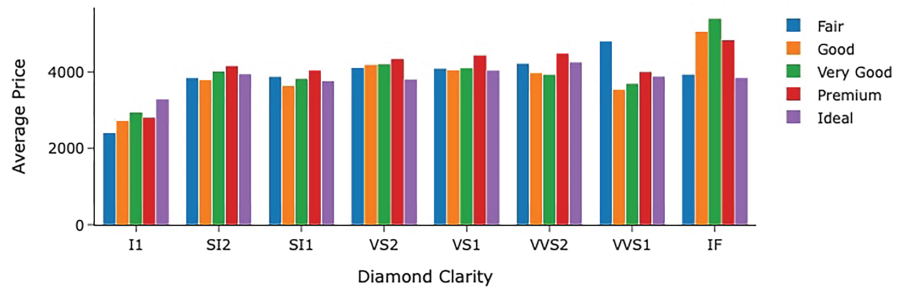


Figure 9-7: A chart styled with Plotly’s `simple_white` theme

Using built-in templates like this, you can quickly and easily apply consistent styling across your visualizations, making them more cohesive and professional.

Custom Themes

To create your own custom theme within Plotly, define a set of styling properties and save them in a separate Python file. Then import the code from that file into the Python script or Jupyter Notebook where you’re creating a chart to apply the properties as the chart’s theme.

Say the company you work for likes a specific set of colors, prefers dark gray instead of black lines and text, and uses a particular font. You can define all these preferences in a separate `themes.py` file, which you can import like any other Python module. Within that file, you first define the `colorway`, which is a list of colors that will be used for the traces in your charts. This can be done through a function called `my_colorway()`:

```
def my_colorway():  
    colors = ["#1f77b4", "#17becf", "#8c564b", "#7f7f7f", "#bcbd22", "#2ca02c"]  
    return colors
```

You can define the rest of the theme's settings as a dictionary, like this:

```
my_theme = {
    "font": {
        "family": "Helvetica, Arial, sans-serif",
        "size": 14,
        "color": "#333333"
    },
    "paper_bgcolor": "#FFFFFF",
    "plot_bgcolor": "#E5E5E5",
    "title": {
        "font": {
            "size": 20,
        },
        "xanchor": "center",
        "x": 0.5
    },
    "xaxis": {
        "linecolor": "#777777",
        "linewidth": 2,
        "title": {
            "font": {
                "size": 16,
            }
        }
    },
    "yaxis": {
        "linecolor": "#777777",
        "linewidth": 2,
        "title": {
            "font": {
                "size": 16,
            }
        }
    },
    "colorway": my_colorway()
}
```

Here, we use nested dictionaries to specify the fonts used in the chart, the background color, and the title and axis formats. We also specify that we'll use the `my_colorway()` function as the colorway for the traces.

To apply `my_theme` to a chart, first import it from *themes.py*:

```
from themes import my_theme
```

Next, after you create your Figure object, use the `**` operator, which unpacks the dictionary, to apply all the properties within the dictionary at once through `update_layout()`:

```
fig.update_layout(**my_theme)
```

Returning once more to the grouped bar chart from “A Multi-Trace Bar Chart” on page 196, here’s how to apply the custom `my_theme` to that chart:

```
from themes import my_theme
```

```
fig = go.Figure()
for cut in cut_order:
    fig.add_trace(go.Bar(
        x=avg_px_per_carat_by_clarity_cut.index,
        y=avg_px_per_carat_by_clarity_cut[cut],
        name=cut
    ))
fig.update_layout(
    title_text="Average Price Per Carat of Diamonds by Clarity and Cut",
    xaxis_title_text="Diamond Clarity",
    yaxis_title_text="Average Price",
)
fig.update_layout(**my_theme)

fig.show()
```

Notice that we call `update_layout()` twice, once for the settings particular to this chart (the main title and axis titles) and a second time with `**my_theme` to set all the more generic properties via the custom theme. Now the chart should look like Figure 9-8.

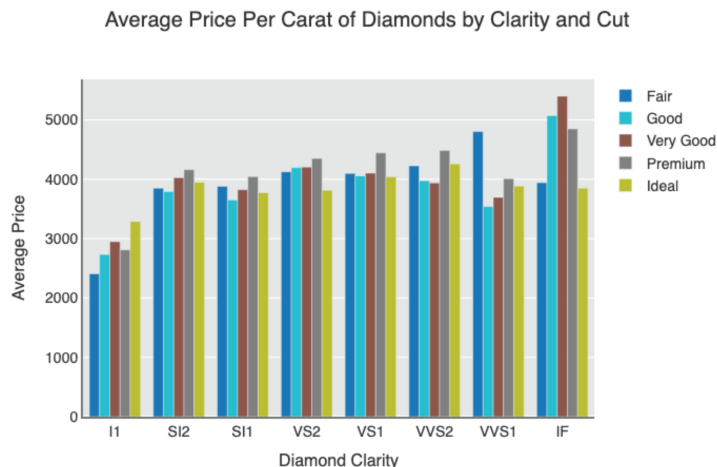


Figure 9-8: Styling a chart with a custom theme

When you apply this custom theme across all your charts, you can keep your visualizations consistent without having to write the same styling code over and over again. Besides being more efficient, this approach helps you avoid errors and makes it easier to change settings as needed.

Using Plotly's Interactivity Features

Plotly provides basic built-in interactivity features for every chart by default. These include simple and intuitive features like zooming into specific x- and y-axis ranges, panning across the chart, and hovering over data points to display tooltips with additional information. These interactive elements can make your charts more engaging and useful for exploring the data in more detail.

When you render your charts inline inside a Jupyter Notebook, you can take full advantage of these interactive features right out of the box. You can zoom in on specific areas of interest, pan across sections of the chart, and hover over data points to see detailed information. In each of the visualizations we created earlier in the chapter, these interactive features are enabled by default, making it easier to analyze the data. For example, in the line charts, you can zoom into a specific date range to see trends more clearly, and in the bar charts, you can hover over each bar to see the exact values for different categories.

For more customized interactivity than the default settings provide, you can set interactivity properties through the `add_trace()` and `update_layout()` methods, much like setting other layout properties. Customizing these interactivity properties can make your charts more user-friendly and tailored to the specific needs of your audience, enhancing their ability to explore and understand the data. For instance, the `hovermode` property controls the display of hover interactions. You can set this option to display tooltips for the closest data point, display tooltips for all data points at the same x-coordinate, or disable hover interactions entirely. Here's an example:

```
fig.update_layout(hovermode="x unified")
```

Here, `hovermode="x unified"` specifies that tooltips will display information for all data points at the same x-coordinate, making it easier to compare values across multiple traces.

Another interactivity property you can change is `dragmode`, which controls how users can interact with the chart via drag actions. You can set this property to enable zooming, enable panning, or disable dragging entirely.

Let's return to the grouped bar chart we created in "A Multi-Trace Bar Chart" on page 196 and add some custom interactivity. Because some of the settings need to be made through the `add_trace()` method, we need to re-create the entire Figure object from scratch:

```
fig = go.Figure()
for cut in cut_order:
    fig.add_trace(go.Bar(
```

```

        x=avg_px_per_carat_by_clarity_cut.index,
        y=avg_px_per_carat_by_clarity_cut[cut],
        name=cut,
        hovertemplate="${y:.2f}"
    ))

fig.update_layout(
    title_text="Average Price Per Carat of Diamonds by Clarity and Cut",
    xaxis_title_text="Diamond Clarity",
    yaxis_title_text="Average Price",
    hovermode="x unified",
    dragmode="zoom"
)

fig.show()

```

The `hovertemplate` argument to `go.Bar` within `add_trace()` customizes the tooltip text. We also set `hovermode` to `x unified` and `dragmode` to `zoom`.

For the `hovertemplate` argument, the string `${y:.2f}` specifies how the tooltip should display the y-values. The `%` and the `y` refer to the y-value of the trace, which is the average price per carat, and `:.2f` indicates that we want to format the price as a number rounded to two decimal places. We also add a dollar sign before the value, so for each level of clarity, the tooltip should show the average prices formatted like \$2408.68.

When you render the chart again, you should see that hovering over the bars will show tooltips for all data points at the same x-coordinate, and you can zoom into specific areas by dragging across the chart.

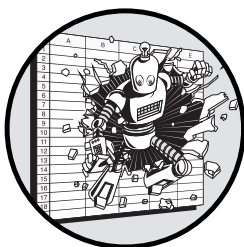
Conclusion

This chapter covered how to create charts in Python by using the Plotly library. We looked at the basics of constructing a chart from a dataset, rendering it, and saving it. Then we customized the layout, applied a theme for consistent styling, and used interactivity to make the chart more engaging and user-friendly.

Knowing these skills not only will allow you to create more effective, aesthetically pleasing, and consistent charts but will also allow you to automate and scale the creation process. You will be able to say more than you ever could with Excel, and you can be far clearer and more professional doing it.

10

BUILDING INTERACTIVE REPORTS



In this chapter, we'll explore turning your charts into interactive reports with a Python library called Dash. First, we'll outline the basics of Dash and where it fits in among other tools for creating interactive reports, including Microsoft Power BI. Then we'll cover the basics of two programming concepts necessary to create applications with Dash: HTML and CSS. Finally, we'll put this all together in an application that allows a user to customize the data behind one of the Plotly charts we created in the preceding chapter.

By the end of this chapter, you should understand how to use the Dash framework to build interactive reports with Python—making data analysis more scalable, engaging, and collaborative than a spreadsheet ever could.

Why Use Interactive Reports

One of Excel's most obvious advantages is its interactivity, but it isn't really designed in a way for that interactivity to be shared. If you'd like to share

a chart with someone else for them to explore, you could always just send them a spreadsheet. However, the lack of structure around this interactivity means that often the user doesn't have a clear path for navigating the data. This makes Excel a poor tool when you need to curate the way the user explores the data.

By contrast, interactive reports, like the examples in this chapter, impose structure on the user's experience. This means that you can implicitly guide your audience through your report while still enabling exploration. Wielding this power well and in the right circumstances can open up new methods of communication and collaboration that will make a far greater impact than what Excel can offer.

Creating Interactive Reports with Dash

Dash is a tool to create interactive reports with Python. These reports can include visualizations like Plotly charts, as well as tables, text, images, and various input controls, like sliders, drop-down menus, and buttons. When you create a Dash application, you build the interface so the user doesn't have to think about that part. They can instead focus entirely on the information at hand within the structure you've provided for them.

A common theme within this book is that Excel's lack of structure can hold you back, whereas Python allows you to apply structure to your work that enables you to take it further than you could otherwise. For example, we've seen how Git allows you to apply structure around the history of your files, which enables you to collaborate more effectively. Likewise, pandas allows you to apply structure around your data, which enables you to perform operations with it more efficiently. And similarly, databases allow you to apply structure to the way you store your data so you can more easily query and retrieve precise information via SQL. By the same logic, Dash lets you strategically apply structure to make your charts and visualizations more effective.

When Interactive Reports Make Sense

Not every situation demands an interactive report, so whether you need to use Dash depends on the context. When you find yourself sharing slightly different versions of the same chart over and over again, that's when Dash may be the best tool for the job.

For example, if you send financial forecasts based on specific scenarios to an internal team at your company, you might want to create an interactive report so users can select different assumptions and instantly see how those changes impact the overall forecast. This way, instead of creating and sharing multiple versions of the same chart, you provide a single, dynamic tool that updates in real time based on the user's input, saving you and your audience time.

However, you may not always need or want to give your audience so many choices. When too many dimensions or variables are included, interactive reports lose their effectiveness. A user who faces too many selections

or ones that aren't clearly labeled might become overwhelmed. When the audience needs to sift through clutter, you could lose their attention.

Alternatively, if your audience does want broad, uninhibited exploration, as is common in corporate contexts, a Dash report's specific set of choices could lead to more questions than answers. For example, say you're in charge of providing your organization's marketing team insight into an internal dataset on customer behavior. At first you think that it might be most efficient to provide the team with an interactive dashboard displaying a few common relationships in the dataset, like how discounts affect sales or how different marketing campaigns influence customer purchases. Maybe if you allow the team members to adjust the chart's parameters on their own, they won't need your assistance as much.

When you start building the dashboard, however, you realize that the team will have follow-up questions about the variables in the report, like customer demographics, regional preferences, or time-of-day patterns. For your dashboard to truly save time, it would need to include all this information, which probably isn't feasible. In these kinds of cases that have a strong propensity for rabbit holes, sticking to pandas and Plotly within a Jupyter Notebook might be a better option.

How to Choose the Right Framework

Numerous dashboarding tools are available that cater to various needs and skill levels. *No-code* and *low-code* tools aim to make the process available to even those with little to no coding knowledge. *Pro-code* tools, on the other hand, require some understanding of programming but allow for more control over the result.

Some of the most popular low-code dashboarding tools include Power BI, Tableau, Qlik, and Looker. These tools are proprietary, commercial products, often with subscription-based pricing models. Offering low-code dashboarding software for free is a difficult proposition for a few reasons. First, the infrastructure costs associated with hosting and scaling a low-code platform can be substantial. Second, these tools don't rely on the user to handle security, data privacy, and other critical features; those features must be handled by the service provider, which adds even more cost. Third, with less technical users comes the need for more comprehensive technical support, which increases overhead.

Pro-code tools, by contrast, are often open source. Many pro-code platforms offer special features through premium subscriptions or enterprise solutions, but their baseline product is typically free to use so that new coders can get accustomed to it. Some of the most common ones that use Python are Dash, Streamlit, and Shiny. (The latter is R-based but also supports Python.) We'll focus on Dash for several reasons: It integrates well with Plotly, it's popular enough to have lots of online resources available if you get stuck, and it can accommodate lots of customization as you grow in your dashboarding abilities.

Compared to no- and low-code tools, pro-code tools allow incremental improvements down the line, both by turning your dashboard into a more

production-quality application that could handle a large number of users and by reusing its parts. In terms of the former, tools like Dash allow you to increase the complexity of the sources of input and output the dashboard uses or how it's deployed. If you want to transition your dashboard toward more advanced, scalable infrastructure, that's a much more doable proposition than it would be with a low-code platform. In terms of reusability, when you create a dashboard with Dash, you've also created the code behind each aspect, which can be extracted and reused on its own or for another project. The Plotly chart that you built, the styling that you add, the basic structure of the application—all these aspects can be packaged and used in other contexts. As you create more dashboards, the process becomes easier, and your library of reusable code to draw from becomes more extensive and efficient.

The closest alternative to Dash within the Microsoft ecosystem is Power BI, an analytics tool for creating data visualizations, dashboards, and interactive reports from a wide range of data sources. Power BI is low-code, so you, or the people who maintain your application, don't necessarily need to understand complex programming languages to create and manage reports. It has features like a drag-and-drop interface, prebuilt connectors to data sources, and templates for visualizations, all of which allow you to quickly and without much background develop applications that you can share with others. And Power BI integrates well with other Microsoft products like Azure and Excel. It even allows you to use Python scripts as a data source and to create visualizations.

Like work you do in Excel, however, interactive reports that you build within Power BI will always be constrained by the platform's interface, predefined functionalities, and the capacity limits of your subscription to the service. If you need some sort of customization that isn't already available as an option in the application, you're out of luck. By contrast, Dash gives you complete control over the structure and components of your application, including all aspects of the data processing, visualization, and user interface. You can independently test, maintain, reuse, and scale each of these individual components, separate from the application itself. This enables you to use your interactive report as a jumping-off point for further development.

Dash Basics

In this section, we'll cover the various components of a basic interactive Dash application. The end goal is to create a simple application that allows the user to customize the Plotly bar chart of diamond prices by clarity that we made in the preceding chapter based on the cut and color of the diamonds used to calculate it. Figure 10-1 shows what we're aiming to create.

Diamonds Dashboard

Filter by Color

☒D☒E☒F☒G☒H☒I☒J

Filter by Cut

☒Fair☒Good☒Very Good☒Premium☒Ideal

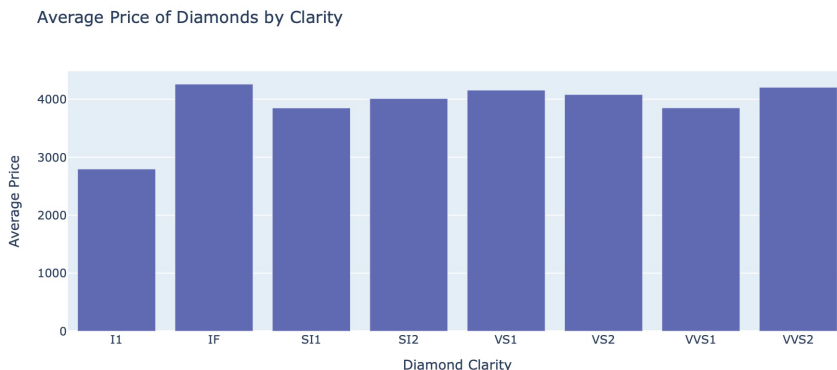


Figure 10-1: A sample Dash dashboard

Other than code used to create the Plotly chart from the Diamonds dataset, which we covered in the previous chapter, this dashboard contains new concepts like interactive components that can accept user input, HTML components, CSS styling, and a dynamically updating graph. We'll address each concept individually, but first you need to get set up to use Dash.

Installing Dash

To get started making dashboards, you need to install the Dash library. You can do this easily using `pip` or `pip3`:

```
pip install dash
```

Now you're ready to start creating dashboards using Dash.

Setting Up a Dashboard

In this section, we'll set up the basic structure for creating any Dash application, a blank framework ready to be expanded with content and interactivity. Create an empty `.py` file named something like `app.py`. Then add the following:

```
from dash import Dash, html

app = Dash()
app.layout = html.Div(children=['This is a Dashboard!'])

if __name__ == '__main__':
    app.run(debug=True)
```

This is only five lines of code, but a lot is happening here. First, we import two elements from the Dash library: `Dash`, which is the object that holds all the components of the dashboard, and `html`, a submodule of `dash` that includes HTML components you can use in your layout. Next, we initialize the dashboard with `app = Dash()`. This creates a new empty object with the type `Dash` and stores it in a variable called `app`.

You can change various aspects of the dashboard by changing the properties of the `app` object. One of those properties is `app.layout`, which we update in the next line. This property contains all the components of the dashboard. They're represented in the code as a list of elements, which can be almost anything, including text, images, and charts. In this case, we have a single element, an `html.Div()` component containing the text `This is a Dashboard!`. We'll discuss what this element is doing in more detail in "HTML Basics" on page 217.

Finally, the line `app.run(debug=True)` within the `if __name__ == '__main__':` block of the script will launch the Dash server and the dashboard when the script runs. When you include the `debug=True` argument, the application will automatically reload every time you make a change to the source code. This argument isn't necessary for the dashboard to run, but it's helpful for development and experimentation.

Deploying a Dashboard Locally

To run a dashboard on your local machine, execute the Python script that contains your Dash application. This needs to be done from the command line, not from within the Python interpreter. To do this, navigate to the directory where your `app.py` file is located and run the following command:

```
python app.py
```

You may need to use `python3` rather than `python`, depending on your setup.

After you execute the script, the output will include a URL, typically something like `localhost:8050`. Open this URL in your web browser, and you'll be able to interact with your Dash application locally.

Web Development Concepts and Dash

Dash dashboards are web applications in that they run in your web browser and provide an interactive interface. Dash allows you to build these applications with Python, letting you focus on your data and visualizations without having to think too much about the intricacies of what's going on in the front-end architecture. However, to build even the simplest Dash applications, you need to know a little about some aspects of web development.

Web applications work by combining three essential components: HTML, CSS, and JavaScript. HTML, or Hypertext Markup Language, provides the overall structure of a web page, including what's shown and what order it's

shown in. CSS, or Cascading Style Sheets, controls the appearance of the HTML elements, such as their color, font, and layout. Finally, JavaScript dictates the dynamic behavior of the page, including how content is updated and how the page reacts to user actions. Think of HTML as your bones; CSS as your skin, hair, and eye color; and JavaScript as the muscles that make everything move and interact.

You don't need to know any JavaScript to create an application with Dash, but it helps to know some basic HTML and CSS. We'll cover the fundamentals in this section so you can make sense of your dashboard code syntax at a high level.

HTML Basics

HTML documents contain *tags*, which define the elements of a web page. Tags designate headings, body text, user input fields, hyperlinks, and more. HTML tags can be nested within each other to establish hierarchical relationships between the various page elements.

HTML has two primary types of elements: block-level elements and in-line elements. *Block-level elements* start on a new line by default, meaning they appear as distinct blocks within the layout. These elements also take up the full width that's available in an application, so they're commonly used for structuring the main layout of the page. Table 10-1 lists some common block-level HTML elements.

Table 10-1: Common Block-Level Dash Components and Corresponding HTML Tags

Dash component	HTML tag	Explanation
html.Div()	<div>	A generic container to group elements
html.H1() to html.H6()	<h1> to <h6>	A text-based heading or subheading
html.P()	<p>	A paragraph of text
html.Ul() and html.Ol()	and 	An unordered and ordered (numbered) list
html.Section()	<section>	A section of related content
html.Header()	<header>	A header that can contain multiple components, like logos, headings, or menus

Of the elements in the table, perhaps the most versatile is `div`, which is a container element used to group other elements together. It's one of the most commonly used tags for layout purposes. We already saw `div` in the basic Dash framework code discussed earlier. Here's another example:

```
from dash import html

html.Div(children='This is a div element')
```

Here, `html.Div()` creates a `div` HTML tag in Dash. The `children` argument defines the nested elements or text content within an HTML component. In pure HTML, this Dash code translates as follows:

```
<div>This is a div element</div>
```

Notice that HTML tags come in pairs: `<div>` marks the start of the element, and `</div>` marks its end.

You'll also likely use HTML headings in your dashboards, which range from `h1` to `h6`. These block-level elements define a hierarchy of headings and subheadings within your content, with `h1` being the highest level and `h6` the lowest:

```
html.H1(children='Main Header')
html.H2(children='Subheader')
html.H3(children='Section Header')
html.H4(children='Subsection Header')
html.H5(children='Smaller Subsection Header')
html.H6(children='Smallest Subsection Header')
```

This translates to the following HTML:

```
<h1>Main Header</h1>
<h2>Subheader</h2>
<h3>Section Header</h3>
<h4>Subsection Header</h4>
<h5>Smaller Subsection Header</h5>
<h6>Smallest Subsection Header</h6>
```

Another useful block-level element for dashboard work is `p`, which defines paragraphs of text. In Dash, you can access this element with `html.P()`:

```
html.P(children='This is a paragraph')
```

This translates to the following HTML:

```
<p>This is a paragraph</p>
```

Whereas block-level HTML elements define a new line in the layout, *inline elements* don't need to take up the full line, so they can live between other elements within the same line of text or content. These elements are typically used for styling or linking small portions of text or images. Table 10-2 lists examples of inline elements.

Table 10-2: Common Inline Dash Components and Corresponding HTML Tags

Dash component	HTML tag	Explanation
html.Span()		A generic inline container
html.A()	<a>	A hyperlink
html.Img()		An image
html.Button()	<button>	A clickable button
html.Strong()		Inline text with strong emphasis (bold)
html.Em()		Inline text with emphasis (italic)
html.Br()	 	A line break within text
html.Label()	<label>	A text label for another element

While block elements build the layout of your page, inline elements add content and styling to the block elements that exist within the layout. Unlike block elements, they don't take up the entire width of the page. For example:

```
html.P(children=[  
    "This is a paragraph with ",  
    html.Strong("bold text")  
])
```

Here, the inline element `html.Strong("bold text")` makes the words *bold text* bold. It lives inside the `html.P()` block element, which defines the overall paragraph structure.

HTML in Dash

Within the `app.layout` of your Dash application, you can add both block-level and inline elements to structure your dashboard. The order in which you add these elements determines the way they're rendered on the page. Block-level elements will stack vertically, and inline elements will flow horizontally within the block-level elements or within other inline elements, taking up only as much space as necessary. The flexibility of using these elements together allows you to customize your layouts to whatever suits your application best.

To harness the real power of HTML, you can nest elements. For instance, you can group several elements inside a `div` so they can be treated as a unit. To nest HTML elements in a Dash application, use an element's `children` parameter to place other elements inside it. You can include as many levels of nesting as needed. Here's an example:

```
from dash import html

app.layout = html.Div(children=[
    html.H1(children='Main Header'),
    html.H2(children='Subheader 1'),
    html.H3(children='Subheader 2'),
    html.Div(children=[
        html.H4(children='Nested Header 1'),
        html.P(children='This is a paragraph inside a nested div.')
    ])
])
```

Notice that the value of the outer `div` element's `children` parameter is a list of other HTML elements, including three levels of headings and another `div`, which itself contains another heading and some text. Now let's see how to use this code in context to create a complete Dash application:

```
from dash import Dash, html

app = Dash(__name__)

app.layout = html.Div(children=[
    html.H1(children='Main Header'),
    html.H2(children='Subheader 1'),
    html.H3(children='Subheader 2'),
    html.Div(children=[
        html.H4(children='Nested Header 1'),
        html.P(children='This is a paragraph inside a nested div.')
    ])
])

if __name__ == '__main__':
    app.run_server(debug=True)
```

When you run this dashboard, your code will be converted into HTML that a web browser can understand and render:

```
<div>
  <h1>Main Header</h1>
  <h2>Subheader 1</h2>
  <h3>Subheader 2</h3>
  <div>
```

```
<h4>Nested Header 1</h4>
<p>This is a paragraph inside a nested div.</p>
</div>
</div>
```

Here, just as the `H1()`, `H2()`, `H3()`, and second `Div()` functions were nested within the first `Div()` function in the Python code, the corresponding HTML elements are nested within the outer `<div>` and `</div>` tags.

CSS Basics

CSS controls the visual styling of HTML elements within an application. This separation of a page's style (the CSS) from its structure (the HTML) allows for easier maintenance and more flexible design. To create styling, CSS files contain *selectors* that target specific HTML elements, and *properties* that define how those elements should be styled.

A few common selectors and properties are helpful to know when styling Dash dashboards. These styles can be contained within a CSS file and linked to your dashboard to enhance its appearance. The first CSS selector to be aware of is the *element selector*, which targets all instances of a type of HTML element. For example, here's some CSS that styles all `div` elements:

```
div {
    background-color: #f0f0f0;
    padding: 20px;
}
```

This `div` selector targets all `div` elements and applies the specified properties to them. The properties are listed as key-value pairs inside curly brackets. The `background-color` property sets the background color, and the `padding` property adds space inside each `div` element so that any nested elements will be offset from the edge.

In CSS, you can also group multiple HTML elements under a common identifier, called a *class*. You can then use a *class selector* to apply the same styles to all elements of the same class, without needing to specify each one individually. This is useful when you want to apply the same style to different kinds of HTML elements or if you want some, but not all, instances of an HTML element to be styled a certain way. In both of those cases, an element selector wouldn't work.

You assign a class in HTML by using an element's `class` attribute, and you refer to classes in CSS with a period (`.`), like so:

```
.container {
    border: 1px solid #ccc;
    margin: 10px;
    padding: 20px;
}
```

In this example, the `.container` selector applies a border, margin, and padding to any HTML element that has the `class="container"` attribute. You can then use this class in your HTML elements within Dash:

```
from dash import html
```

```
html.Div(className='container', children='This is a styled container')
```

Another tool in CSS is the *ID selector*, which targets a single, unique element with a specific ID attribute. IDs are used when you need to style a specific element that should be unique on the page. An ID is defined in HTML by using the `id` attribute and is referenced in CSS with a hash mark (`#`):

```
#main-header {  
    font-size: 24px;  
    color: #333;  
    text-align: center;  
}
```

The `#main-header` selector applies specific styles to the element with `id="main-header"`. Here's how to assign that ID to an element with Dash:

```
html.H1(id="main-header", children="Main Header")
```

This will make the Main Header text a 24-pixel dark gray font and center-aligned.

CSS in Dash

To apply CSS to your Dash application, you typically place your CSS file inside a directory called *assets* within your project folder. For example, you might have a file structure like this:

```
parent_directory/  
├── app.py  
└── assets/  
    └── styles.css
```

In the *styles.css* file, you define your CSS selectors and properties. When you run your Dash app, Dash automatically detects and applies any CSS files in the *assets* directory to your application, styling the relevant HTML elements. This enhances the visual appearance and layout of your dashboard.

Returning to the skeletal Dash app discussed in “Setting Up a Dashboard” on page 215, let's add some styling to the This is a Dashboard! text. Create an *assets/styles.css* file and add some CSS rules, such as the following:

```
.dashboard-text {  
    font-size: 64px;  
    color: #FF0000;  
    padding: 20px;  
}
```

We use a class selector, so we need to apply the `dashboard-text` class to the `div` element in the Dash code:

```
from dash import html

html.Div(
    children=["This is a Dashboard!"],
    className="dashboard-text"
)
```

Now, when you run your Dash application, the text `This is a Dashboard!` will be styled according to the rules defined in the `styles.css` file. The text will appear in a large font size (64 px), with red color (`#FF0000`), and with padding around it to give it some space from the edges of the container.

Integrating Plotly and Dash

You can easily integrate Plotly charts into a Dash application. When you embed a Plotly chart in a Dash application, you can still take advantage of Plotly's default interactivity features, like zooming and panning, while also adding your own Dash-based interactivity features, like filtering the data in real time based on user inputs. We'll demonstrate how this works in this section by building a dashboard around one of our charts from Chapter 9.

Functionalizing Plotly Charts

To add charts to your Dash applications, you need to package the charting code into a function. This allows you to dynamically generate (and regenerate) the chart based on changing inputs set by the dashboard user.

In the preceding chapter, we built a Plotly bar chart showing the average price per carat of different clarities of diamond in the `Diamonds` dataset from the `PyDataset` library. Let's put the code to create that chart into an `update_bar_chart()` function:

```
import plotly.graph_objects as go
from pydataset import data

def update_bar_chart():
    diamonds = data("diamonds")

    diamonds["price_per_carat"] = diamonds["price"] / diamonds["carat"]
    avg_px_per_carat_by_clarity = diamonds.groupby("clarity")["price_per_carat"].mean()

    clarity_order = ["I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"]
    avg_px_per_carat_by_clarity = avg_px_per_carat_by_clarity.loc[
        clarity_order
```

```

]

fig = go.Figure()

fig.add_trace(go.Bar(
    x=avg_px_per_carat_by_clarity.index,
    y=avg_px_per_carat_by_clarity,
))
fig.update_layout(
    title_text="Average Price of Diamonds by Clarity",
    xaxis_title_text="Diamond Clarity",
    yaxis_title_text="Average Price",
)
return fig

```

To recap, this code adds a `price_per_carat` column to the `diamonds` DataFrame, then calculates the average price per carat, grouped by the values in the `clarity` column. After that, we build the chart in a Plotly Figure object and use `go.Bar()` to add a trace, followed by `fig.update_layout()` to set the chart's title and axis labels. Because the code is now packaged in a function, we end by returning the Figure object.

We've encapsulated the logic for creating the bar chart into a function, making it easy to reuse and modify. However, to make the chart interactive, we need to allow the inputs to the chart to change in response to user selections. As an example, we'll allow the user to select specific diamond colors and cuts and to update the chart to reflect only the selected options. Let's say, for instance, the user selects the following colors and cuts:

```

selected_colors = ["H", "I", "J"]
selected_cuts = ["Fair", "Ideal"]

```

Here's how to filter the DataFrame to include only those colors and cuts:

```

color_filter = diamonds["color"].isin(selected_colors)
cut_filter = diamonds["cut"].isin(selected_cuts)

selected_rows = color_filter & cut_filter
filtered_df = diamonds[selected_rows]

```

To determine which rows meet the criteria, we use the `isin()` method, which tells us which rows have a value for `color` that's in `selected_colors` and separately which rows have a value for `cut` that's in `selected_cuts`. This gives us two Boolean Series, `color_filter` and `cut_filter`, which we pass to the `&` (AND) logical operator to create a new `filtered_df` DataFrame with just the desired rows. We also need to update the calculation of the average price per carat by clarity to use this filtered DataFrame:

```
avg_px_per_carat_by_clarity = filtered_df.groupby("clarity")["price_per_carat"].mean()
```

Let's integrate those changes into `update_bar_chart()` and modify the function to accept `selected_colors` and `selected_cuts` as arguments:

```
def update_bar_chart(selected_colors, selected_cuts):

    color_filter = diamonds["color"].isin(selected_colors)
    cut_filter = diamonds["cut"].isin(selected_cuts)
    selected_rows = color_filter & cut_filter
    filtered_df = diamonds[selected_rows]

    avg_px_per_carat_by_clarity = filtered_df.groupby("clarity")["price_per_carat"].mean()

    fig = go.Figure()
    fig.add_trace(go.Bar(
        x=avg_px_per_carat_by_clarity.index,
        y=avg_px_per_carat_by_clarity,
    ))
    fig.update_layout(
        title_text="Average Price of Diamonds by Clarity",
        xaxis_title_text="Diamond Clarity",
        yaxis_title_text="Average Price",
    )
    return fig
```

This function now dynamically generates the bar chart based on the selected colors and cuts, creating the potential for interactivity. For the interactivity to work, though, we need to give the user a way to make selections, and then we need a way for Dash to connect those selections with the `update_bar_chart()` function. But first, let's embed our bar chart in a Dash application.

Embedding a Chart in a Dashboard

In Dash, a Plotly chart is represented using a `dcc.Graph` component. Dash's `dcc` submodule contains components such as drop-down menus, sliders, and graphs that have interactive elements. Here's how to include our example:

```
from dash import dcc

html.Div(children=[dcc.Graph(id="bar-chart")])
```

We declare the `dcc.Graph()` component as a child of a `div`. This is where the bar chart will be rendered. We assign the component an id of `bar-chart`.

This gives us a way to select the chart with CSS selector syntax, which will be essential for enabling interactivity.

Enabling User Selections

To capture user input for filtering the data, we'll use `dcc.Checklist()` components. These components create checklists of options that users can select. Since the user will be able to filter the dataset by both color and cut, we need two checklists. First, here's the checklist for filtering by color:

```
from dash import dcc

dcc.Checklist(
    id="color-filter",
    options=[{"label": color, "value": color} for color in colors],
    value=colors,
    inline=True
)
```

This `dcc.Checklist()` component will display a list of colors, each with a checkbox that the user can select or deselect. The `id` attribute is set to `color-filter`. The `options` attribute defines the available choices, and `value` specifies the default selected options, which in this case includes all the colors. The `inline=True` parameter ensures that the checkboxes will be displayed in a horizontal line.

Here's the other checklist for filtering by diamond cuts:

```
dcc.Checklist(
    id="cut-filter",
    options=[{"label": cut, "value": cut} for cut in cuts],
    value=cuts,
    inline=True
)
```

This checklist works similarly to the color filter, but it allows the user to select from the diamond cuts available in the dataset. The `id` attribute is set to `cut-filter`, and the `options` and `value` parameters are defined based on the available cuts.

Updating Charts with Callbacks

Now that we have both the charting and the user input components of the application, we need to somehow tie the user's responses in the checklist to the arguments of the `update_bar_chart()` function. To do that, we'll use a callback.

In this context, a *callback* is a function that updates the output of a Dash component in response to user actions, such as changes to an input component. A callback in Dash is defined using `@app.callback`. The at sign (`@`) indicates that the callback is a *decorator*, a special type of function in Python

that modifies the behavior of another function. When you see the @ symbol, it means that the function that follows will have extra functionality, which is defined in this case by the code inside the parentheses in `@app.callback()`.

Although `@app.callback()` looks like a function, since it takes arguments inside the parentheses, it technically isn't one. Decorators in Python are *higher-order functions*, which is one way of saying they're functions that operate on other functions. Because of this, they take functions instead of objects as arguments and return functions instead of objects as results.

The callback takes two parameters, `Output()` and `Input()`, both of which are functions in the `dependencies` submodule of `dash`. The `Output()` parameter specifies the Dash component that will be updated, and the `Input()` parameter specifies the input components and properties that trigger the callback. The `Output()` function for our example will be the bar chart created in the `dcc.Graph(id='bar-chart')` component of the application:

```
from dash import dependencies

dependencies.Output("bar-chart", "figure")
```

Here, we pass the id of the bar chart as the `component_id` and `figure` as the `component_property` positional arguments to the function. Next, we need to provide the callback with the inputs:

```
[dependencies.Input("color-filter", "value"), dependencies.Input("cut-filter", "value")]
```

Instead of just one `Input()` function, we make a list of two, one for each checklist. This tells Dash that the callback should be triggered whenever the `value` property of either the `color-filter` or the `cut-filter` component changes. These inputs will then be passed as arguments to the charting function.

Now to create the callback:

```
@app.callback(
    dependencies.Output("bar-chart", "figure"),
    [dependencies.Input("color-filter", "value"),
     dependencies.Input("cut-filter", "value")]
)
def update_bar_chart(selected_colors, selected_cuts):
    # Functionalized chart code
    return fig
```

This callback listens for changes in the `color-filter` and `cut-filter` inputs. When a change is detected in either input, the callback reacts by rerunning `update_bar_chart()` to change the bar chart in the application.

To summarize, three parts of the code are affecting one another here. First, the checklists take in user input. Second, the `update_bar_chart()` function regenerates the chart when needed. And third, the chart component in the application displays the chart to the user. The callback facilitates the dynamic relationships among each of these parts of the code.

Putting It All Together

We've seen how the components of the diamond price dashboard work in isolation. Now let's put everything together and create a complete Dash application that incorporates the interactive chart using the functionalized Plotly charting code, the callback function, the graph component, and the checklists. First, we import the necessary libraries and initialize the Dash application:

```
from dash import Dash, html, dcc, dependencies
import plotly.graph_objects as go
from pydataset import data
import pandas as pd

app = Dash(__name__)
```

Next, we define the available options for colors, clarities, and cuts that the user can filter by:

```
colors = ["D", "E", "F", "G", "H", "I", "J"]
clarities = ["I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"]
cuts = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
```

Now we set up the layout of the application by using the `dcc.Checklist()` and `dcc.Graph()` components. We add HTML components like `html.H1()` and `html.Label()` to make the dashboard more readable:

```
app.layout = html.Div([
    html.H1("Diamonds Dashboard"),
    html.Div(children=[
        html.Label("Filter by Color"),
        dcc.Checklist(
            id="color-filter",
            options=[{"label": color, "value": color} for color in colors],
            value=colors,
            inline=True
        )
    ]),
    html.Div(children=[
        html.Label("Filter by Cut"),
        dcc.Checklist(
            id="cut-filter",
            options=[{"label": cut, "value": cut} for cut in cuts],
            value=cuts,
            inline=True
        )
    ])
])
```

```

    html.Div(children=[
        dcc.Graph(id="bar-chart")
    ])
])

```

In this layout, we create a title for the dashboard and two checklists for filtering the diamond data by color and cut, each contained within a `div` element. The `dcc.Graph()` component, also inside its own `div`, is where the bar chart will be rendered.

Since we have all the components of the application defined in `app.layout`, we can now style them using CSS in an external `assets/styles.css` file:

```

h1 {
    text-align: center;
    font-size: 36px;
    font-family: "Open Sans", sans-serif;
}

label {
    font-weight: bold;
    font-size: 18px;
    margin-bottom: 10px;
    font-family: "Open Sans", sans-serif;
}

```

In this example `assets/styles.css` file, we define styling for two of the HTML components: `h1`, which is the main title, and `label`, which includes the titles of the checklists and the labels of each box. You can add styling to most of the layout components from this file, but keep in mind that Plotly directly controls the styling of elements inside the chart, such as axis labels and grid lines. To change the styling of those elements, you'll need to change the charting code itself.

Next, we define the callback function that will update the chart based on the user's selections:

```

@app.callback(
    dependencies.Output("bar-chart", "figure"),
    [dependencies.Input("color-filter", "value"),
     dependencies.Input("cut-filter", "value")]
)
def update_bar_chart(selected_colors, selected_cuts):
    diamonds = data("diamonds")
    diamonds["price_per_carat"] = diamonds["price"] / diamonds["carat"]

    color_filter = diamonds["color"].isin(selected_colors)
    cut_filter = diamonds["cut"].isin(selected_cuts)
    selected_rows = color_filter & cut_filter
    filtered_df = diamonds[selected_rows]

```

```

avg_px_per_carat_by_clarity = filtered_df.groupby("clarity")[
    "price_per_carat"
].mean()

fig = go.Figure()
fig.add_trace(go.Bar(
    x=avg_px_per_carat_by_clarity.index,
    y=avg_px_per_carat_by_clarity,
))
fig.update_layout(
    title_text="Average Price of Diamonds by Clarity",
    xaxis_title_text="Diamond Clarity",
    yaxis_title_text="Average Price",
)
return fig

```

This callback function filters the diamond dataset based on the selected colors and cuts, calculates the average price per carat for each clarity level in the filtered data, and then updates the bar chart with the results.

Finally, we run the Dash application:

```

if __name__ == "__main__":
    app.run(debug=True)

```

To launch the dashboard, simply execute the Python script containing all this code. Once the server is running, you can open the provided URL in your web browser to interact with the dashboard. Try selecting different diamond colors and cuts from the checklists. As you make the selections, the bar chart should automatically update to reflect the average price per carat for the selected criteria. In this way, the dashboard allows for interactive and dynamic exploration of the diamond dataset.

Sharing Your Reports with Others

Deploying a Dash application on your local machine, as we've done so far, is great for development and testing, but it's not so great if you want to share your dashboard with others, which is usually the point of creating an interactive report in the first place. To make your dashboard available to a wider audience, you'll need to deploy it to a server that others can access.

The way you deploy your dashboard depends on several factors, including the intended audience, the complexity of your application, and your preferred hosting environment. The choice of deployment will also be influenced by considerations such as security requirements, scalability, and who will need to maintain it going forward.

For example, you may plan for your application to be used only internally by a few people within your organization. In this case, you could coordinate with your internal IT team to deploy the dashboard on an on-premises server.

This approach gives you full control over the environment, allowing you to adhere to internal security policies and keep sensitive data within your network.

On the other hand, if your application needs to be accessed by a broader audience or shared with external stakeholders, you have a few options. For instance, you might choose to deploy your dashboard by using a cloud-based service like Heroku or AWS Elastic Beanstalk. These platforms make it easier to get your application online quickly, without the need to manage the underlying infrastructure. They're well suited for public-facing dashboards where you anticipate a moderate number of users and want the flexibility to scale as needed.

When your audience is larger, or your application's complexity increases, you might look into more-sophisticated deployment solutions that can handle higher traffic and more-complex workloads. These solutions would require a deeper level of technical expertise to set up and maintain. At that point, you may also consider converting your Dash application into a full-fledged web application.

Conclusion

This chapter explored the when and the how of creating interactive dashboard applications with Python. We described situations that warrant the creation of a dashboard. Then we outlined some of the no-, low-, and pro-code frameworks that are available for creating interactive dashboards. In particular, we compared Dash with Power BI, the most similar tool within the Microsoft universe.

Next, we walked through creating an interactive application with Dash, including all the elements necessary to turn a basic Plotly bar chart into a custom interactive visualization. These components included HTML tags, CSS selectors, functionalized charting code, and interactive elements for accepting user input. And finally, we mapped out how these individual components work together in a Dash application.

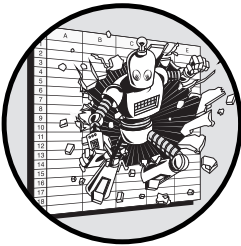
Now that you know what Dash can do and how it works, hopefully you can identify the situations where embedding your data visualizations in a dashboard can create an interactive user experience that facilitates collaborative and efficient data analysis.

PART III

WRITING BETTER CODE

11

ORGANIZING YOUR CODE WITH CLASSES



This chapter introduces the basics of *object-oriented programming (OOP)*, a style of coding that revolves around creating and manipulating custom data types called *classes*. We'll start with a description of classes and how to create them. Then we'll look at some of the benefits of OOP through a series of both theoretical and practical examples. By the end of the chapter, you should be able to recognize when using classes in your code will improve both your scripts and your coding workflow more broadly.

Much of this book has been about applying structure to your Python workflow so that the tools do most of the heavy lifting. Each of the concepts we've discussed frees you in some way from lower-level details so you can think about the big picture. Git manages the various histories of your work so you don't need to worry about which files have changed and how. The pandas library simplifies data wrangling so you don't need to think about exactly what happens to each individual data point along the way. Plotly

takes care of the intricacies of creating visualizations so you don't need to handle the details of rendering a chart. Classes and OOP perform a similar function: They allow you to abstract away lower-level operations so you can structure your code around higher-level concepts.

Many tutorials dwell on the theoretical aspects of OOP, concepts that might be interesting to computer scientists but aren't all that helpful when you just want to get things done, especially if coding isn't your full-time job. In this chapter, we'll focus only on aspects of OOP that are practical, showing you how to use classes in a way that directly benefits your day-to-day work.

From Rows and Columns to Classes

A Python *class* is a data type that you can create and customize to fit your needs. Each *object* made from that class is called an *instance* of the class. Because you create the class, you're free to determine the kinds of information that instances of the class should store and how they should behave. To do this, you define *methods*, which are functions tied to the class, as well as *attributes*, which are variables that hold the data specific to the class.

As an example, say we want to define a class representing bank accounts. The class as a whole would lay out the attributes and methods that any bank account should have. There might be attributes like an account number and a balance, and methods for depositing and withdrawing money. Each instance of the class would represent a particular bank account with a particular account number and balance.

Think about how you would store this same information in Excel. You'd probably have each row of a spreadsheet represent a specific bank account and each column represent a specific attribute. This might look something like Figure 11-1.

	A	B	C	
1	Account Number	Holder	Balance	
2	425	Bill Gates	\$ 3.10	
3	408	Steve Jobs	\$ 3.43	
4	206	Jeff Bezos	\$ 1.91	

Figure 11-1: Representing bank accounts in Excel in an OOP-like way

It may not look like it on the surface, but Excel's grid is doing two things to help you keep track of the data inside it. First, it enforces a consistent structure of three variables, Account Number, Holder, and Balance. Thanks to the grid, each row includes these three, and only these three, pieces of data. If you weren't storing this data in Excel's grid, you might not follow the same pattern. For example, you might be inclined to give one account an "ID" instead of an "Account Number." Second, the grid implies relationships among the pieces of information, maintaining the link between any

given account number and its holder name and balance. Without this, it would be easy to mix up which balance goes with which account, causing your data to become disorganized.

Python classes provide the same kind of structure and links as the Excel grid in this example. The individual rows in the spreadsheet are like instances of a bank account class, and the values held in each column are like class attributes. But, of course, you can fit only so much in a spreadsheet cell. What if you wanted to store an entire history of all transactions made by each account? Or what if you wanted to create a customized report of account activity? The grid isn't flexible enough for that, but as you'll see, Python classes very much are.

Examples of Classes in Common Python Libraries

You've already seen many examples of classes so far in the book. In fact, nearly all popular Python libraries use classes in some way. For example, the pandas library defines the `DataFrame` class, a structure designed to facilitate the process of analyzing datasets. The `DataFrame` class is built by adding additional attributes and methods to the `ndarray` class from NumPy, another Python library. It has attributes that store information related to the dataset represented by the object, like `columns` and `index`. It also has methods that allow you to manipulate and transform the data, like `head()`, `merge()`, and `sum()`.

Another example of a class you've already seen is the `date` class from the `datetime` module, which represents dates in Python. It includes attributes like `year`, `month`, and `day`, which give insight into all the parts of the date, and methods like `strftime()`, which allows you to format dates as strings. Because this information is contained within the `date` class, much of the logic required to manipulate dates is already handled for you, making it easier to work with dates and times in your programs.

Custom Classes

While lots of classes are already defined in external libraries, the real benefit of OOP comes from defining your own custom classes. To create a new class, start with the `class` keyword, then the name of the class, and then a colon (`:`). To illustrate, we'll define the `BankAccount` class described earlier (by convention, class names start with a capital letter):

```
class BankAccount:
    # Code block defining the BankAccount class
```

All the code that you add indented within the class block is contained within the scope of the class and is part of the class definition. Any attributes or methods defined in this block will belong to the class.

Constructors

Typically, the first thing in a class block is an `__init__()` method, also known as a *constructor*. This is a special method that gets called when you create a

new instance of the class. The constructor always contains the `self` argument, which represents the new instance of the class being created. The other arguments you pass to the constructor are values for the class's attributes. These can be positional arguments, which you must pass in when you create an instance of the class, or keyword arguments, which can have default values and therefore be omitted if desired. For the `BankAccount` class, our constructor might look like this:

```
class BankAccount:
    def __init__(self, number, holder, balance=0):
        self.number = number
        self.holder = holder
        self.balance = balance
```

In this example, we give the `BankAccount` constructor three arguments besides `self`: `number`, `holder`, and `balance`. The `balance` argument has a default value of 0, meaning that if you don't specify a balance when creating a `BankAccount` object, it will start with a balance of zero. Inside the body of the constructor, we take the values of the three arguments and assign them to class attributes of the same names. Remember, `self` refers to the current instance of the `BankAccount` class being created, so `self.number = number` means to take the `number` argument and use it as the value for the instance's `number` attribute.

To create an instance of a class, reference the class name, followed in parentheses by any arguments needed for the constructor. You don't need to specify anything for the `self` argument, which Python already knows how to handle. This will automatically invoke the constructor and return a new object with the specified attribute values. For example, here's how to create an instance of the `BankAccount` class:

```
account = BankAccount(425, "Bill Gates", balance=3.10)
```

The `account` variable now refers to an instance of `BankAccount` with the account number 425, a holder named `Bill Gates`, and a balance of 3.10. We can access and print the attributes of this instance via dot notation:

```
print(account.holder)
```

This prints the value we passed into the constructor for the `holder` attribute, which in this case is `Bill Gates`. Alternatively, we can check the balance like so:

```
print(account.balance)
```

This prints 3.10, because we specified this value when creating the `BankAccount` instance.

Methods

After the constructor, the rest of a class body is typically devoted to defining methods for the class. These are defined with the `def` keyword, just like any

other function. As an example, let's add `deposit()` and `withdraw()` methods to the `BankAccount` class:

```
class BankAccount:
    def __init__(number, holder, balance=0):
        self.number = number
        self.holder = holder
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance = self.balance - amount
```

The `deposit()` method takes an `amount` argument, representing the amount of the deposit, and updates the `balance` attribute by adding the value of `amount`. Similarly, the `withdraw` method takes an `amount` argument, verifies that the account has enough money in it, and updates the account's balance to reflect the withdrawal. Like the constructor, both methods also have a first positional argument, `self`, which allows the method to access the current instance of the class.

To use one of these new methods, create an instance of `BankAccount` and call the method on that instance by using dot notation, like this:

```
account = BankAccount(425, "Bill Gates", balance=3.10)
account.deposit(10)
print(account.balance)
```

This outputs 13.10, since the original balance was 3.10 and we added 10 by using the `deposit()` method.

Programming with Objects to Stay Organized

The object-oriented style of coding relies heavily on objects created from user-defined classes. When your code is object oriented, your script will create and modify these objects succinctly, often making your main code look quite simple. In fact, the bulk of the logic lies inside the methods and attributes in the class definitions.

The opposite of OOP is *functional programming*, in which the bulk of the logic is centered around composing functions and transforming data through a series of function calls. This means that you won't often modify a single object over and over again, but rather create new data structures with each step as you pass data through individual functions.

To illustrate the difference, let's revisit our `BankAccount` class. Once the class is defined, we can create a main code block within a script that manipulates an instance of the class:

```
if __name__ == "__main__":
    account = BankAccount(425, "Bill Gates", balance=3.10)
    account.deposit(10)
    account.withdraw(30)
    print(account.balance)
```

In this case, we perform each action on the same account variable, which retains its information between each step. First, we create the account, then we deposit to it, then we withdraw from it, and then we print the balance. Every time, it's the same account.

By contrast, here's a functional version of the `BankAccount` class:

```
def create_account(number, holder, balance=0):
    return {
        "number": number,
        "holder": holder,
        "balance": balance
    }

def deposit(account, amount):
    new_account = {
        "number": account["number"],
        "holder": account["holder"],
        "balance": account["balance"] + amount
    }
    return new_account

def withdraw(account, amount):
    if amount > account["balance"]:
        raise ValueError("Insufficient funds")
    new_account = {
        "number": account["number"],
        "holder": account["holder"],
        "balance": account["balance"] - amount
    }
    return new_account
```

We define `deposit()` and `withdraw()` as stand-alone functions instead of methods of a class, along with a `create_account()` function in lieu of a constructor. Notice that each function creates and returns a new dictionary containing the data about a bank account. To use these functions as we used the `BankAccount` class, we can write the following:

```
if __name__ == "__main__":
    account = create_account(425, "Bill Gates", balance=3.10)
```

```
)  
account = deposit(account, 10)  
account = withdraw(account, 30)  
print(account["balance"])
```

In this version, we also start by creating an account, but when we deposit and withdraw, we create a new dictionary object rather than modify the original, so each function call is part of an assignment statement. We end up with the same result as the object-oriented version, but we take a different path to get there.

At first glance, the OOP version of the script may look a bit more concise, but someone without much Python experience may find it more difficult to follow. As with any decision in programming, trade-offs exist. When you're deciding whether to use functions or a class in your script, consider the objects you're manipulating and how you intend to use them. If each step in the process is mostly independent from the others and you won't need to retain much information between steps, then relying more on functions might be a better choice. On the other hand, if you need to keep track of multiple variables related to a single entity, using functions might hold you back. With classes, you can group all this information together, which keeps your code simpler and more organized.

In the rest of this chapter, we'll explore some of the specific benefits of OOP through a concrete example of working with US unemployment data obtained from the Bureau of Labor Statistics (BLS) REST API. The BLS API organizes data by series IDs and allows you to specify parameters like the time period for which you'd like to retrieve data. You can find the documentation for it at <https://www.bls.gov>.

So that we have a more familiar example to compare to, let's first look at a functional approach to downloading data from the BLS API:

```
import pandas as pd  
import requests  
  
def download_bls_data(series_id, start_year, end_year):  
    url = f"https://api.bls.gov/publicAPI/v2/timeseries/data/{series_id}"  
    params = {"start_year": start_year, "end_year": end_year}  
    response = requests.get(url, params=params)  
    json_data = response.json()  
    data = json_data["Results"]["series"][0]["data"]  
    return pd.DataFrame(data)
```

In this functional approach, we define a `download_bls_data()` function to handle downloading and returning the data. We pass the necessary arguments (`series_id`, `start_year`, and `end_year`) into the function. Then, much as in Chapter 8, we use `requests.get()` to make an HTTP GET request to the BLS API. We process the JSON response and store it in a pandas `DataFrame`, which the function returns.

To use this function, we call it with the three positional arguments that are required. We provide a series ID of LNS14000000, which corresponds to national unemployment data:

```
series_id="LNS14000000"
start_year=2000
end_year=2009

unemployment = download_bls_data(series_id, start_year, end_year)
print(unemployment.head())
```

This prints the first few rows of the downloaded data from the BLS API.

To illustrate the benefits of OOP, let's rewrite this code by using a class called `BLSDataset`. We'll start with the class's constructor:

```
import pandas as pd
import requests

class BLSDataset:
    def __init__(self, series_id, series_name, start_year, end_year):
        self.series_id = series_id
        self.series_name = series_name
        self.start_year = start_year
        self.end_year = end_year

        self.data = None
```

Within the class's `__init__()` method, we pass in the series ID, series name, start year, and end year as arguments. These are stored as class attributes, which makes them available to other methods of the class. Next, we'll add a method to the class called `download()` that allows you to download data from the BLS API:

```
class BLSDataset:
    def __init__(self, series_id, series_name, start_year, end_year):
        # Constructor code here

    def download(self):
        url = f"https://api.bls.gov/publicAPI/v2/timeseries/data/{self.series_id}"
        params = {
            "start_year": self.start_year,
            "end_year": self.end_year
        }
        response = requests.get(url, params=params)
        json_data = response.json()
        data = json_data['Results']['series'][0]['data']

        self.data = pd.DataFrame(data)
```

The `download()` method mostly follows the same approach as the original `download_bls_data()` function, but instead of using `return` to return the data `DataFrame` from the function, we store the `DataFrame` in the class's `self.data` attribute.

After we've defined the class, we can create an instance of it called `unemployment` by using the same parameters as the functional version:

```
unemployment = BLSDataset(  
    series_id="LNS14000000",  
    series_name="Unemployment",  
    start_year=2000,  
    end_year=2009  
)  
unemployment.download()  
print(unemployment.data.head())
```

In the next four sections, we'll use our `BLSDataset` class to illustrate how OOP promotes reusability, extendability, encapsulation, and inheritance, and how each of those principles can improve your code.

Reusability

Classes promote *reusability* by storing specific aspects of your code inside the objects they relate to. If you had a `Triangle` class, for example, you might have a property called `is_isosceles`, or if you had a `Circle` class, you might have a `calculate_circumference()` method. Since you can tie these components to the objects they relate to, your code becomes cleaner and more modular. Every time you create a new `Triangle` object, you don't need to also save and maintain a separate variable that indicates whether it's isosceles. And every time you create a new `Circle`, you don't need to also import the `calculate_circumference()` function from somewhere else in your code. Keeping these elements within the objects themselves makes reusing and maintaining your code easier.

As an instance of the `BLSDataset` class, the `unemployment` object we created in the preceding section retains all the properties and methods we created for the class. If we wanted to print the start year, for example, we could write the following:

```
print(unemployment.start_year)
```

This would print the year 2000, which we passed in when creating the instance and stored in `self.start_year` within the class definition.

The beauty of reusable classes is that you can create further instances of a class without having to define the class all over again. You just have to call the constructor with different arguments:

```
non_farm_payrolls = BLSDataset(  
    series_id="CES0500000001",  
    series_name="Non-Farm Payrolls",
```

```
        start_year=2018,  
        end_year=2022  
    )  
    non_farm_payrolls.download()  
    print(non_farm_payrolls.data.head())
```

This creates a second instance of the `BLSDataset` class with different properties and a different dataset, but all the methods and logic stored within the class remain the same. In short, anytime you need a different dataset from the BLS, you can simply create a new instance of the `BLSDataset` class. The class contains all the code it needs to download and process the data, and all those aspects will automatically come along when you create instances of the class elsewhere.

Extendability

With classes, you can add new methods and properties without affecting what's already in place, so there's no need to modify other parts of your code. This makes your code *extendable*, meaning you have the flexibility to build on it gradually as your needs change without worrying about breaking what you already have. There's no need to rethink every detail as you make enhancements. To illustrate, let's add a `yearly_summary()` method to the `BLSDataset` class that calculates yearly statistics for the dataset:

```
class BLSDataset:  
    # Constructor and other methods  
  
    def yearly_summary(self):  
        data = self.data  
        data["value"] = data["value"].astype(float)  
  
        grouper = data.groupby(data["year"])["value"]  
        summary = pd.DataFrame({  
            "first": grouper.first(),  
            "last": grouper.last(),  
            "min": grouper.min(),  
            "max": grouper.max(),  
            "mean": grouper.mean()  
        })  
        return summary
```

Within the `yearly_summary()` method, we perform everything needed to create the summary. First, we convert the year and period columns into a proper format, then we group the data by year and calculate various metrics for each year, such as the first and last values, minimum, maximum, and mean. We return the results in a `DataFrame` called `summary`. This differs from the `download()` method, which saved the results as an attribute of the class instead of returning them. In that case, we knew we would likely use `self.data` again in the life of the object, so retaining it made sense. With

`yearly_summary()`, that seems less likely, so we return the `DataFrame` directly instead of storing it.

To call `yearly_summary()` on the `unemployment` object, run the following:

```
unemployment_summary = unemployment.yearly_summary()
print(unemployment_summary)
```

This prints a `DataFrame` containing the yearly summary statistics for the unemployment data.

As your needs evolve, you can further extend the `BLSDataset` class by adding more methods, all while keeping the original functionality intact.

Encapsulation

Another benefit of using classes is *encapsulation*, which means you can separate code that specifically relates to the class from the rest of your script. This allows you to enforce rules for accessing and modifying data within objects of the class.

You might think that this concept sounds like it's more about security than anything else. If you primarily write scripts to automate and improve processes that you run on your own or with a small team, and you aren't a software engineer writing code within a large-scale enterprise environment, you might wonder whether a seemingly abstract concept like encapsulation really matters to you. As you'll see, however, some practical uses of encapsulation make your code more efficient, even on a small scale.

Properties

Encapsulation is often enforced through the use of *properties*, internal attributes that you can edit only through specially defined methods within the class. Say we have a `Rectangle` class with attributes called `length` and `width`. We might also want to know the rectangle's area, but rather than create a regular area attribute, we can make `area` a property. This way, we can make sure that the area of the rectangle always remains the length times the width, and that we can't assign any other value to `area`. To make this happen, the first step is to specify that `area` is an internal attribute when declaring the `Rectangle` class constructor:

```
class Rectangle:
    def __init__(self, width, length):
        self.width = width
        self.length = length
        self._area = None
```

In Python, it's common practice to indicate attributes and methods that are private to the class by prefixing their names with an underscore (`_`). This convention signals to you and anyone else who might use your code that these elements are intended for internal use within the class and shouldn't be directly accessed or modified from outside the class. Just using the underscore doesn't on its own change the rules around how we can update the

attribute, however. We can still technically change the value of `_area` (even though this wouldn't be good practice):

```
rectangle = Rectangle(width=2, length=3)
self._area = 1000000
```

If you run this, you'll end up with a rectangle with a width of 2, a length of 3, and an area of 1,000,000, which doesn't make any sense. To prevent this from happening, we need to give `_area` a getter method.

Getters

A *getter* is a method that defines custom logic to execute whenever a particular attribute is accessed. It's what turns a regular attribute into a property. The getter should have the same name as the attribute associated with it—in this case, `area()`—and it's preceded by the `@property` tag, as shown here:

```
class Rectangle:
    def __init__(self, width, length):
        self.width = width
        self.length = length
        self._area = None

    @property
    def area(self):
        self._area = self.width * self.length
        return self._area
```

The function definition under `@property` becomes the getter for the `area` property. Every time you retrieve the `area` attribute of a `Rectangle` object, the function automatically runs and calculates the area by multiplying the object's length and width. The `@property` is a *decorator*. As we mentioned in Chapter 10, decorators, which are indicated by the `@` sign, are a way to apply specific behaviors to functions. Other types of decorators exist, besides `@property`, like the `@app.callback` decorator from Chapter 10.

Here's how to access the new `area` property:

```
rectangle = Rectangle(width=2, length=3)
print(rectangle.area)
```

Running `rectangle.area` invokes the `area()` getter, which calculates and returns the area of the rectangle (6 in this case). Notice that we don't include parentheses after `area` as we would with a normal function; instead, the syntax treats `area` as if it were a regular attribute.

The `area()` getter will run every time you access the `area` property and dynamically calculate the rectangle's area. If you were to change the width or length, the area would therefore automatically reflect that change the next time you access it:

```
rectangle = Rectangle(width=2, length=3)
print(rectangle.area)
rectangle.width = 5
print(rectangle.area)
```

The first print statement should output 6, while the second should output 15, indicating that the area is recalculated after changing the width.

Setters

The counterparts of getters are *setters*, special methods that control how properties can be modified. Setters are useful for enforcing rules when updating an attribute. For example, it doesn't make much sense to have a rectangle with a negative width. We can turn width into a property and use a setter to prevent negative values:

```
class Rectangle:
    def __init__(self, width, length):
        self.width = width
        self.length = length
        self._area = None

    @property
    def area(self):
        self._area = self.width * self.length
        return self._area

    @property
    def width(self):
        return self._width

    @width.setter
    def width(self, value):
        if value <= 0:
            raise ValueError("Width must be greater than zero.")
        self._width = value
```

In this version of the Rectangle class, the width setter, designated with `@width.setter`, checks whether the provided value is positive before allowing it to be assigned to the `_width` attribute. If the value is zero or negative, the setter raises a `ValueError`. This setter will be invoked anytime we try to assign a value to `_width`, including as part of the constructor. For example, say we try to create the following Rectangle object:

```
rectangle = Rectangle(-1, 4)
```

This will raise the error because the width setter won't allow a value of -1. Similarly, say we try to update the width after creating a Rectangle object:

```
rectangle = Rectangle(width=2, length=3)
rectangle.width = -1
```

The setter will raise an error here too, preventing the change to a negative width value.

Lazy Loading

Class properties with getters and setters can be extremely handy when classes have attributes that are resource intensive to compute or retrieve. An example is an attribute representing a large dataset from a database, API, or other external source, such as the data attribute of our BLSDataset class. We can use a property to ensure that the data is downloaded and processed only when it's needed rather than immediately upon creating an instance of the class.

This is called *lazy loading*, which means to delay an expensive operation or prevent it from running until it's necessary. For the BLSDataset class, lazy loading allows us to make the call to the API to retrieve the data only the first time the data attribute is accessed rather than doing so up front when the object is created or every other subsequent time we use it.

Let's modify the BLSDataset class to implement lazy loading. First, we'll assign `self._data` the value `None` in the constructor:

```
class BLSDataset:
    def __init__(self, series_id, series_name, start_year, end_year):
        --snip--
        self._data = None
```

Next, we'll create getter and setter methods for data:

```
class BLSDataset:
    --snip--

    @property
    def data(self):
        if self._data is None:
            self.download()
        return self._data

    @data.setter
    def data(self, value):
        if not isinstance(value, pd.DataFrame):
            raise TypeError("Data must be a pandas DataFrame.")
        self._data = value
```

The getter method first checks whether a value is stored in the internal `self._data` property. If not, we run `self.download()` to populate it. The setter

method ensures that any value assigned to `self._data` is a valid `DataFrame`, raising a `TypeError` if it's not.

Now let's create an instance of the `BLSDataset` class:

```
unemployment = BLSDataset(  
    series_id="LNS14000000",  
    series_name="Unemployment",  
    start_year=2000,  
    end_year=2009  
)
```

We no longer need to explicitly call the `download()` method to retrieve the data. Instead, the first time we reference the `data` property, the getter method will run `download()` automatically:

```
print(unemployment.data.head())
```

If we access the `data` property again, the data will already be populated, so there's no need for the getter to invoke `download()` again:

```
print(unemployment.data.tail())
```

Using encapsulation and lazy loading here saves time and makes the code cleaner. We don't need to worry about when to tell the code to download the unemployment data, and we don't need to worry about unnecessarily downloading the data multiple times. The class just knows to download it on its own when the data is needed.

Inheritance

Classes have the ability to build on, or *inherit* from, one another, much as a child inherits traits from a parent. When you create a class, you can also create a *subclass* that directly inherits all the properties and methods of its parent class, while still being able to add its own additional properties and methods. Once you've made a subclass, the original parent class is often called a *superclass*.

In practice, inheritance is helpful when you need to create multiple classes that share similarities. You can place the common attributes and methods in the parent class, which keeps your code tidy, consistent, and more efficient. To explain this in theoretical terms, say we have three classes called `Circle`, `Triangle`, and `Rectangle`. Each shape has different attributes—perhaps `Circle` has a `radius` attribute, while `Rectangle` has `length` and `width` attributes—but the classes also have aspects in common. For example, all these types of shapes have a `perimeter` and an `area`, although they have different formulas to calculate them. Because of these common characteristics, we can make our code more efficient with inheritance.

To start, we'll define a parent class called `Shape` that includes getter methods for the area and perimeter properties:

```
class Shape:
    def __init__(self):
        self._area = None
        self._perimeter = None

    @property
    def area(self):
        if self._area is None:
            self.get_area()
        return self._area

    def get_area(self):
        raise NotImplementedError("Subclasses must implement this method")

    @property
    def perimeter(self):
        if self._perimeter is None:
            self.get_perimeter()
        return self._perimeter

    def get_perimeter(self):
        raise NotImplementedError("Subclasses must implement this method")
```

The `Shape` class doesn't have any of the features that don't apply to all the child classes. The `radius` attribute isn't there because we're going to include it in `Circle` and because it doesn't apply to `Rectangle`. The `length` and `width` attributes aren't there because we'll include them in `Rectangle`, and they don't apply to `Circle`. The parent class includes just the logic that should be shared among all the child classes.

Notice that we include the `get_area()` and `get_perimeter()` methods, which are invoked from the area and perimeter getters, but we have them raise a `NotImplementedError`. We do this because when we create the child classes, we'll need to *override* these methods with new definitions specifying how each child class will handle area and perimeter. We want the code to tell us with an error if we forget to override these methods.

Now we can create the `Rectangle` child class that inherits from `Shape`:

```
class Rectangle(Shape):
    def __init__(self, length, width):
        super().__init__()
        self.length = length
        self.width = width

    def get_area(self):
        self._area = self.length * self.width
```

```
def get_perimeter(self):
    self._perimeter = 2 * (self.length + self.width)
```

In the parentheses after the class name, we provide the name of the parent class that `Rectangle` should inherit from. Then, within the constructor of `Rectangle`, we run the line `super().__init__()`, which calls the constructor of the parent `Shape` class. This sets up the `area` and `perimeter` properties that will be inherited from parent. If the parent constructor takes in arguments, which it doesn't in this case, you could pass those to `super().__init__()` as well.

The `Rectangle` class is much more concise than it would be otherwise, thanks to inheritance. It only has to set the `length` and `width` parameters in the constructor and fill in the definitions for `get_area()` and `get_perimeter()`. Without inheritance, `Rectangle` would also need to define its own `area` and `perimeter` properties from scratch, and we'd have to do the same for `Circle` and `Triangle` as well, repeating the same code that applies to all shapes over and over again.

This approach carries another advantage besides avoiding repetition: If you want to make changes to all types of shapes at once, even though they're separate classes, you can still do that in the parent class. For example, if you want to add a `describe()` method that prints the `area` and `perimeter` of the shape, you can do that within the `Shape` class, and the method will be passed down to all shapes.

Making Parent Classes Useful

Our `Shape` example was theoretical, but let's apply what we've discussed to the `BLSDataSet` class to illustrate how inheritance can be useful in your day-to-day work. In all likelihood, you aren't using just a single source for datasets. You're probably working with various datasets from different sources. While each source will have its own method for downloading data, all the sources will have some common characteristics.

For example, you probably want to use lazy loading in all cases, but your methods for cleaning and summarizing the data may be different. Let's therefore create a parent class called `DownloadedDataset` that contains everything necessary to make `self.data` a property and have it load only when needed:

```
import pandas as pd

class DownloadedDataset:
    def __init__(self):
        self._data = None

    def download(self):
        raise(NotImplementedError)
        self._data = None
```

```

@property
def data(self):
    if self._data is None:
        self.download()
    return self._data

@data.setter
def data(self, value):
    if not isinstance(value, pd.DataFrame):
        raise ValueError("Data must be a pandas DataFrame")
    self._data = value

```

This DownloadedDataset class doesn't contain the specifics necessary to download data from any one source, but it does contain the extra steps necessary to trigger the `download()` method only when `self._data` is empty. The class also contains the setter for the data property that restricts it to a DataFrame. With this general code moved to the parent class, we can simplify BLSDataset to focus only on the specifics for retrieving and processing data from the BLS API:

import requests

```

class BLSDataset(DownloadedDataset):
    def __init__(self, series_id, series_name, start_year, end_year):
        super().__init__()
        self.series_id = series_id
        self.series_name = series_name
        self.start_year = start_year
        self.end_year = end_year

    def download(self):
        url = f"https://api.bls.gov/publicAPI/v2/timeseries/data/{self.series_id}"
        params = {
            "start_year": self.start_year,
            "end_year": self.end_year
        }
        response = requests.get(url, params=params)
        json_data = response.json()
        data = json_data['Results']['series'][0]['data']

        self.data = pd.DataFrame(data)

    def yearly_summary(self):
        # Rest of the yearly summary method

```

In the `BLSDataset` constructor, we once again use `super().__init__()` to invoke the parent constructor. Other than this, the class definition is simpler and more focused on the unique aspects of working with BLS data. Having the superclass also means that if we wanted to create classes that download data from other APIs, those classes could also inherit from `DownloadedDataset`. This means you wouldn't need to create the data property again or its getters or setters.

Taking Inheritance Too Far

When used appropriately, inheritance can make your code cleaner, more concise, and easier to follow. However, you can overdo it. Properties and methods that live inside your parent class are separated from where they're used in subclasses, making them harder to see or track for you or anyone else reading your code. Consequently, adding multiple levels of inheritance or making the parent class too complex can start to hurt more than it helps.

In our `Shape` example, imagine that instead of two levels to get to `Rectangle` (`Shape` ► `Rectangle`), you have four (perhaps `Shape` ► `Polygon` ► `Quadrilateral` ► `Rectangle`). In this case, you'd spread the functionality across all four levels, which would be really difficult to follow. You might have defined the function to calculate the perimeter of the shape in the `Polygon` class, but someone reading your code would need to scour through many layers of classes to determine that. To avoid this, it's generally best to use no more than two levels of inheritance. Sometimes additional levels of parent classes are intuitive, such as having `Rectangle` inherit from `Shape` and having `Square` inherit from `Rectangle`, but these situations are the exception, not the rule.

Instead of having a chain of inheritance like `Shape` ► `Polygon` ► `Rectangle`, it might make more sense to cut out `Polygon` and recast some of its features as stand-alone functions. This will keep the inheritance chain simple while also leaving the parent `Shape` class relatively sparse. One of the motivations to include a separate level of inheritance for `Polygon` might have been that it could contain the code to calculate the perimeter of a shape by summing up the lengths of its sides. Rather than add a whole extra class for that, you could accomplish the same thing by creating a separate function:

```
def calculate_perimeter_polygon(sides):  
    return sum(sides)
```

Within the Shape class, you'll want to make sure you allow for different perimeter functions to apply to different kinds of shapes (a circle's perimeter is calculated differently than a polygon's, for example). To do this, you can create an attribute called `perimeter_function` to act as a placeholder for the name of the perimeter function to be used for that kind of shape:

```
class Shape:
    def __init__(self, perimeter_function=None, perimeter_function_args={}):
        self._perimeter = None
        self.perimeter_function = perimeter_function
        self.perimeter_function_args = perimeter_function_args
    @property
    def perimeter(self):
        if self._perimeter is None:
            self.get_perimeter()
        return self._perimeter

    def get_perimeter(self):
        if self.perimeter_function is None:
            raise NotImplementedError("A perimeter_function must be provided.")
        self._perimeter = self.perimeter_function(**self.perimeter_function_args)
```

Any class that inherits from Shape will set its perimeter property by using whatever function is specified in the `perimeter_function` attribute. For Rectangle, when you use `super().__init__()` to call the Shape constructor, you would specify that rectangles will use `calculate_perimeter_polygon()` to calculate the perimeter:

```
class Rectangle(Shape):
    def __init__(self, length, width):
        super().__init__(
            perimeter_function=calculate_perimeter_polygon,
            perimeter_function_args={"sides": [length, length, width, width]}
        )

        self.length = length
        self.width = width
```

Rather than having three levels of inheritance, you now have a simple, stand-alone function that's much easier to maintain and reuse.

Separating out logic like this is a technique called *composability*, and it can help keep your classes simpler while ensuring that each part of your code is focused on a single responsibility. Combining composability with a judicious use of inheritance can allow you to write code that's easier to read and maintain, more scalable, more efficient, and more concise. What's more, you can write that code much faster.

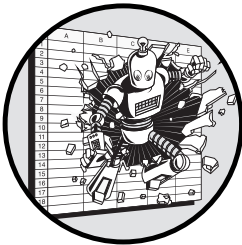
Conclusion

This chapter explored how to create classes in Python and the contexts in which object-oriented programming can make you more productive. Through the practical example of the `BLSDataset` class, we demonstrated that classes make your code easier to reuse and extend, and that the OOP principles of encapsulation and inheritance can make your code more efficient and scalable.

Once you become comfortable with writing classes, your approach to coding will shift dramatically. Instead of viewing coding as a strenuous exercise, it will become a more thoughtful and strategic process. As you start using classes, you'll quickly notice that you can manage much more complexity than before. Your thought process will evolve from simply asking, How do I get this script to run? to considering, What's the most effective way to structure this problem? At that point, you'll be reaching for Python rather than Excel as your default tool for more tasks than you ever thought you would.

12

FINDING AND FIXING ERRORS



This chapter focuses on making sure your code works as expected and knowing what to do when it doesn't. We'll walk through how to use the debugger in VS Code through a simple example. This includes setting breakpoints and using the debugger commands to step into and out of functions in order to isolate the source of an issue. After that, we'll cover a few basic examples of creating tests to make sure your code is resilient regardless of the conditions where you might run it.

The Art of Debugging

Debugging means to identify and resolve errors or issues in your code. It's one of the most essential skills in coding because, frankly, we all tend to get stuck sometimes. When you're writing code, you don't have visual cues, instant feedback, and contextual help to lean on, so it's easy to find yourself lost in the depths of a problem without any clue about which direction to start swimming.

Think about how you get out of this kind of situation in Excel. Formulas in Excel are often long and convoluted, and if you've spent much time with Excel, you've probably encountered some that you couldn't understand completely on first look. Say you come across a spreadsheet with an elaborate formula like this one:

```
=IF(SUM(IF(LEFT(C1:C1568, 1)=G1, 1, 0))>0, INDEX(D1:D1568, MATCH(1, (A1:A1568=G2)*(B1:B1568=G3)*(LEFT(C1:C1568, 1)=G1), 0)), NA())
```

Several nested functions are here, so it's hard to determine exactly what's going on. If this function wasn't giving you the result you expected, what would be your first step to identify the problem? You'd probably start by breaking it down. You might extract each part of the formula individually and see what it does on its own, and then build the formula back up after you understand how each piece works. For example, you could start with the most interior formula on the left side, `LEFT(C1:C1568, 1)`, and work your way outward. Or you could start by checking the output of the `MATCH` function separately to ensure it's finding the correct row. Either way, you'd need to work your way through each individual component until you found something that didn't do what you expected.

This is debugging, and in Python, the process isn't too dissimilar: You pick apart a script to determine what its components are doing and make sure they're working as expected. In Chapter 2, we discussed using `print()` statements to output values at intermediate stages of a script to the console. This is a simple way to find out what's happening when there's an issue with your code, but debugging takes this idea much further. As we'll discuss next, code editors like VS Code have built-in tools that make debugging streamlined and straightforward. When you use these tools efficiently, you may find that writing code and working through bugs becomes much more enjoyable.

Debugging Inside the VS Code Editor

The VS Code *debugger* is a tool that interacts with the Python interpreter and allows you to precisely control the flow of execution when you run a program. The debugger provides an interface for you to oversee the execution process and pinpoint where a script goes wrong. To demonstrate how the debugger works in VS Code, let's use a script called *greeting.py* that contains two functions:

```
from datetime import datetime
```

```
def time_based_greeting(hour):  
    if 5 <= hour < 12:  
        return "Good morning"
```

```

elif 12 <= hour < 18:
    return "Good afternoon"
else:
    return "Good evening"

def generate_greeting(name, hour):
    greeting = time_based_greeting(hour)
    return f"{greeting}, {name}!"

if __name__ == "__main__":
    person_name = "Bill Gates"
    current_hour = datetime.now().hour

    greeting = generate_greeting(person_name, current_hour)
    print(greeting)

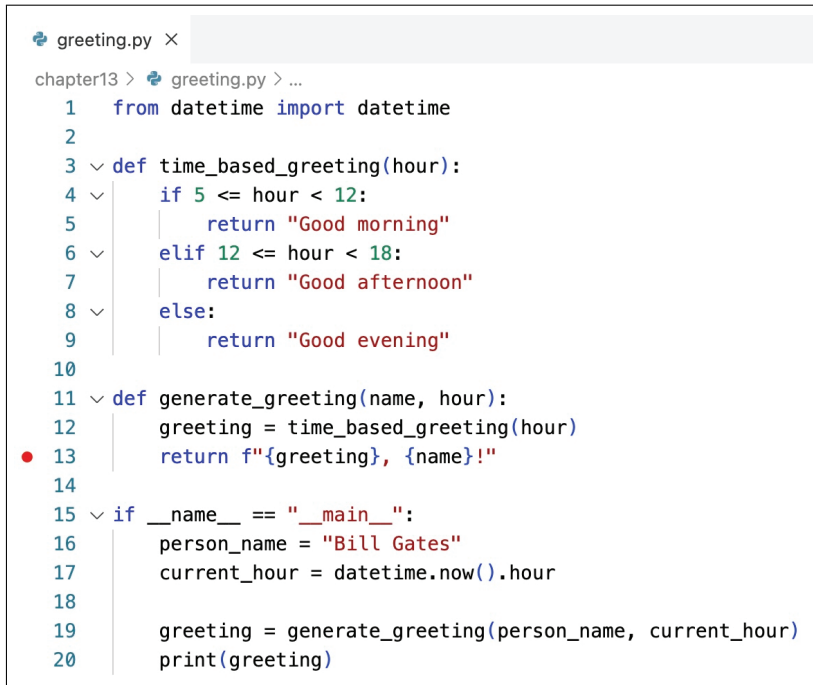
```

In the script, `time_based_greeting()` takes `hour` as an argument and returns a greeting based on the time of day. The `generate_greeting()` function takes in two arguments, `name` (a person's name) and `hour` (the current hour), and generates a greeting for that person with help from `time_based_greeting()`. When you execute the script directly, it sets the `person_name` and `current_hour` variables, generates a greeting, and prints the result.

Breakpoints

Debuggers work by allowing you to pause the execution of your script at specific points in the code, called *breakpoints*. You can investigate the state of your program at those particular moments in time and step through the code to observe how the underlying variables change as it executes. The breakpoints live in the debugging environment you're working in (in this case, VS Code), not in the script itself, so they aren't part of your `.py` files and don't affect how your code behaves when you run it outside of the debugger.

To place a breakpoint in VS Code, click in the left margin next to the line number where you want the program to pause. A dot will appear where you clicked, indicating that a breakpoint has been set. For example, try placing a breakpoint on line 13 of your *greeting.py* script, where the return statement for the `generate_greeting()` function is located. Figure 12-1 shows how this looks in VS Code.



```
greeting.py ×
chapter13 > greeting.py > ...
1  from datetime import datetime
2
3  def time_based_greeting(hour):
4      if 5 <= hour < 12:
5          return "Good morning"
6      elif 12 <= hour < 18:
7          return "Good afternoon"
8      else:
9          return "Good evening"
10
11 def generate_greeting(name, hour):
12     greeting = time_based_greeting(hour)
13     return f"{greeting}, {name}!"
14
15 if __name__ == "__main__":
16     person_name = "Bill Gates"
17     current_hour = datetime.now().hour
18
19     greeting = generate_greeting(person_name, current_hour)
20     print(greeting)
```

Figure 12-1: The dot next to line 13 indicates a breakpoint in VS Code.

This breakpoint will cause the debugger to pause execution of the program just before line 13 runs, which will give you time to inspect the values of the variables created up to that point in the script. You can add as many breakpoints as you like within a script. The more that you place, the more opportunities you'll have to see what the program is doing. The only potential problem with placing too many breakpoints is that it will slow you down. To remove a breakpoint, click the dot in the left margin next to the line number where the breakpoint is set, and it will disappear.

The Debug Panel

To run your script with the VS Code debugger, first make sure you have the Python extension installed, and then click the Run and Debug icon in the editor's left sidebar (it looks like a play button with a small bug next to it) to open the Debug Panel. You can also press CTRL-SHIFT-D (on Windows) or ⌘-SHIFT-D (on macOS) to open it. From there, you can start and control your debugging session.

In the drop-down menu at the top of the Debug Panel, make sure that Python: Current File is selected. This tells VS Code to run the currently open Python file in debug mode. When you run *greeting.py* through the debugger with a breakpoint placed on line 13, the execution will pause just before this line runs, allowing you to explore a snapshot of the variables that the Python interpreter sees at that specific point in execution (see Figure 12-2).



Figure 12-2: The Debug Panel at line 13 of *greeting.py*

The Debug Panel begins with a Variables section consisting of two subsections, Locals and Globals. The Locals subsection lists variables that fall within the current scope at the breakpoint—that is, variables that have been defined and are accessible within the current function or block of code where the debugger has paused. In this case, that includes the `greeting`, `hour`, and `name` variables.

The Globals subsection includes variables that are accessible throughout the entire script. These variables are divided into three categories:

Special variables Internal Python variables or automatically managed variables used by the Python interpreter. These might include things like the current module’s name or specific system variables, like `__name__` and `__file__`.

Function variables References to functions within your script. In Figure 12-2, the `generate_greeting()` and `time_based_greeting()` functions are listed here. This indicates that these functions are currently loaded and accessible globally.

Class variables Variables that are associated with a particular data type, like their attributes and methods. In this case, Figure 12-2 shows the `datetime` class, which we used to find the current hour.

Below these three categories, you’ll find the two global variables defined in the main code block of the *greeting.py* script: `current_hour` and `person_name`. And after all the variables, the Debug Panel has a Call Stack section (see Figure 12-3).



Figure 12-3: The call stack at line 13 of `greeting.py`

The *call stack* of a program shows the sequence of function calls that have been made up to that point in execution. This allows you to trace the flow of execution from the entry point of the program to its current state. In Figure 12-3, the call stack contains two entries: `generate_greeting` and `<module>`. This indicates that the `generate_greeting()` function is being executed. The number `13:1` next to it shows that the debugger is paused at line 13 in the `greeting.py` file. The `<module>` refers to the top-level script where `generate_greeting()` was called. This occurred at line 19 of the code.

The call stack in this example is simple, but having insight into the execution flow is more helpful when function calls become more complex—for example, with one function calling another function that in turn calls another function. The call stack becomes your guide through this convoluted path.

The Debug Console

Beyond simply inspecting values, the VS Code debugger allows you to modify the state of your program while your script's execution is paused, through the Debug Console at the bottom of the screen. The Debug Console, which appears as a separate tab next to the Terminal, provides an interactive environment for evaluating expressions, changing variables, and interacting with your code in real time. For example, while you have `greeting.py` paused at the breakpoint on line 13, try changing the value of the `name` variable within `generate_greeting()` from `Bill Gates` to `Steve Jobs` by entering the following in the Debug Console:

```
name = "Steve Jobs"
```

When you run this command, you'll notice that the `name` variable in the Debug Panel will update to reflect the new value. This doesn't change the script itself. It just alters the value of the variable for the current execution within the debugger, allowing you to see how the program would behave with the different value. The interpreter will use the updated value for any further operations, meaning it should output the following greeting when the script finishes execution:

```
Good afternoon, Steve Jobs!
```

Using the Debug Console allows you to test the effects of changes to your code without having to stop or restart your debugging session. You can experiment with different values and scenarios directly in the console, making it easier to identify issues or test fixes in your code on the spot.

Debugging Code with Errors

To get comfortable using the VS Code debugger to spot and resolve errors, let's add a pretend bug to our *greeting.py* script. We'll use the debugger to locate the issue, isolate its source, and then determine a solution.

Right now, line 17 of the script, part of the main code block, reads as follows:

```
current_hour = datetime.now().hour
```

Try changing the line so `current_hour` is set to a string rather than an integer:

```
current_hour = "7"
```

Because of this change, running the script triggers an error like this:

```
Exception has occurred: TypeError
'<=' not supported between instances of 'int' and 'str'
❶ File "/home/user/projects/greeting.py", line 4, in time_based_greeting
    if 5 <= hour < 12:
        ^^^^^^^^^^^^^^^^^
❷ File "/home/user/projects/greeting.py", line 12, in generate_greeting
    greeting = time_based_greeting(hour)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/user/projects/greeting.py", line 20, in <module>
    greeting = generate_greeting(person_name, current_hour)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: '<=' not supported between instances of 'int' and 'str'
```

The `TypeError` indicates that the script is trying to compare a string ("7") with an integer, which isn't possible. The error message indicates where the error occurred, as well as the path the code took to get there. It explains that the error happened at line 4, which is inside the `time_based_greeting()` function ❶, and that `time_based_greeting()` was itself called inside the `generate_greeting()` function ❷.

While this error message communicates a lot of information, it doesn't allow you to see the entire context behind the error. The VS Code debugger makes getting to the root of the problem more interactive and intuitive. To illustrate, we'll look at two methods of using the debugger to isolate the error, both of which reach the same conclusion but take different paths to understanding what's happening.

The Step Out Command

First, let's isolate the error by starting from the location where the error occurs and using the Step Out debugger command to exit the function currently in process and move back up the call stack to the higher-level code that called it. Place a breakpoint on the line where the script hits the error, which in this case is line 4 (`if 5 <= hour < 12:`), part of the `time_based_greeting()` function. Then run the script with the debugger. When it stops, VS Code will highlight

the line of code where the error occurs. From there, you can inspect the state of the program in the Debug Panel to determine the value of the variable that's causing the issue. You should see that the `hour` variable stores the string "7" instead of the integer 7, the source of the issue.

You know that `hour` is the wrong data type, but you may not know where in the code this problem was introduced. In other words, you need to locate exactly where `hour` is assigned a string. This is where the Step Out button (the up arrow in the debugger toolbar) comes in. Click it, and the debugger should move to line 12, where the `time_based_greeting()` function is called:

```
greeting = time_based_greeting(hour)
```

This line is part of the `generate_greeting()` function, and you should notice that the contents of the Debug Panel have changed to reflect the local variables within this function instead of `time_based_greeting()`. Here, too, `hour` contains a string, so we haven't yet found the source of the error.

Click **Step Out** again, and the debugger will take you back to the `__main__` block of code where `generate_greeting()` is called. When you look at the Debug Panel again, you should see that the `current_hour` variable is set to a string. When `current_hour` is passed through `generate_greeting()` and then through `time_based_greeting()`, it causes an invalid comparison between strings and integers.

The Step Into Command

Another method of debugging places a breakpoint closer to the entry of the script, where the initial function calls are made, and then uses the Step Into command to follow the flow of execution forward rather than backward. This helps you understand the structure of the program by diving into each function as it's called, examining how variables are passed and manipulated step-by-step.

To use this method, instead of placing the breakpoint directly on the line of the error, place it on the first significant function call after the entry point of the script. In this case, place the breakpoint on line 19:

```
greeting = generate_greeting(person_name, current_hour)
```

When the debugger stops at this line, use the **Step Into** button (the down arrow in the debugger toolbar) to dive into the `generate_greeting()` function. This will move you immediately to the first line of that function, where `current_hour` is passed as an argument. Here, you can see how the function begins to process this variable: The `hour` parameter in `generate_greeting()` receives the value "7" (a string), which is incorrect for the intended comparison operations.

Click **Step Into** again, and you'll end up on line 4, inside `time_based_greeting()`, where the invalid comparison occurs:

```
if 5 <= hour < 12:
```

At each stage, you can inspect the values of variables and observe how they're manipulated, which helps in understanding where things go wrong. With this method, you won't identify the source of the error as directly as using Step Out, but you'll gain a more holistic understanding of how the program flows and how data is processed throughout the script. This, of course, isn't entirely necessary in our simple example, but it comes in handy when the issue you're debugging is hidden behind several interconnected elements of code.

Pausing Code at the Right Moment

When you're debugging a file, you may want to pause the execution of your code only under certain conditions rather than every time a specific line is reached. VS Code allows for this through *conditional breakpoints*. These are most useful when your code is performing the same action multiple times, like within a for loop. Sometimes one particular input to the loop might cause an error or unexpected result, and a conditional breakpoint can help you isolate and investigate that specific case.

To show how conditional breakpoints work, we'll walk through an example. In the *greeting.py* script, replace the original `if __name__ == "__main__":` block with the following code:

```
if __name__ == "__main__":
    person_names = [
        "Bill Gates",
        "Steve Jobs",
        "Jeff Bezos",
        "Mark Zuckerberg"
    ]
    current_hour = datetime.now().hour

    for person_name in person_names:
        greeting = generate_greeting(person_name, current_hour)
        print(greeting)
```

Instead of processing a single name, this code iterates over a list of names with a for loop. We'll set a conditional breakpoint on line 25 (`greeting = generate_greeting(person_name, current_hour)`) that depends on the following condition:

```
person_name == "Jeff Bezos"
```

This will allow us to pause the execution only when the script is processing Jeff Bezos instead of every time it reaches that line.

Click in the left margin next to the desired line to initialize a breakpoint as usual. Then right-click the dot that appears, select **Edit Breakpoint**, and edit the Expression field by entering the condition `person_name == "Jeff Bezos"`. Now when you run the debugger, the execution will pause only on the Jeff Bezos iteration of the loop. At that point, you can inspect variables and use the

console to change aspects of the execution, just as you would with a regular breakpoint.

Another kind of condition you can apply to a breakpoint is *hit counts*, where the debugger pauses based on the number of times the breakpoint has been hit so far in the execution. In our example, we could use hit counts to achieve the same result of stopping on the Jeff Bezos iteration. To do this, set a new breakpoint on the same line, then right-click the breakpoint and select **Edit Breakpoint ► Hit Count**. Since Jeff Bezos is the third name in the list, set the hit count to **3**. This tells the debugger to ignore the first two hits and pause only on the third hit, which corresponds to the Jeff Bezos iteration.

This method is useful when the specific item you're interested in appears consistently at the same position in a loop or when you're debugging a repetitive process and want to focus on a particular iteration.

Debugging with Different Inputs

Just as you can pass in arguments when running a script from the command line, you can also pass arguments to your scripts when debugging in VS Code to test how the code behaves with a given set of inputs. As an example, let's change the `if __name__ == "__main__":` block in *greeting.py* to take in a command line argument for the name instead of using a hardcoded default value:

```
import sys

if __name__ == "__main__":
    if len(sys.argv) > 1:
        person_name = sys.argv[1]
    else:
        person_name = "Bill Gates"

    current_hour = datetime.now().hour
    greeting = generate_greeting(person_name, current_hour)
    print(greeting)
```

The `if...else` statement allows the script to take in a name as an argument when run, rather than always defaulting to Bill Gates. For example, you could enter the following at the command line to run the script with an argument of Steve Jobs:

```
python greeting.py "Steve Jobs"
```

In VS Code, you can mimic this behavior of accepting command line arguments during debugging through the debugger's settings. To change these settings, you need to edit a file within your VS Code workspace called *launch.json*, which configures how the debugger should start your application. The file lets you customize the environment where the Python debugger runs, including passing specific arguments to your script.

If you've never configured the settings in your *launch.json* file before, you'll need to create a new one. The first time you open the Debug Panel,

there should be a prompt to create a *launch.json* file, as illustrated in Figure 12-4. Click that, then select **Python Debugger** in the drop-down menu. Now you should have the *launch.json* file open to edit.

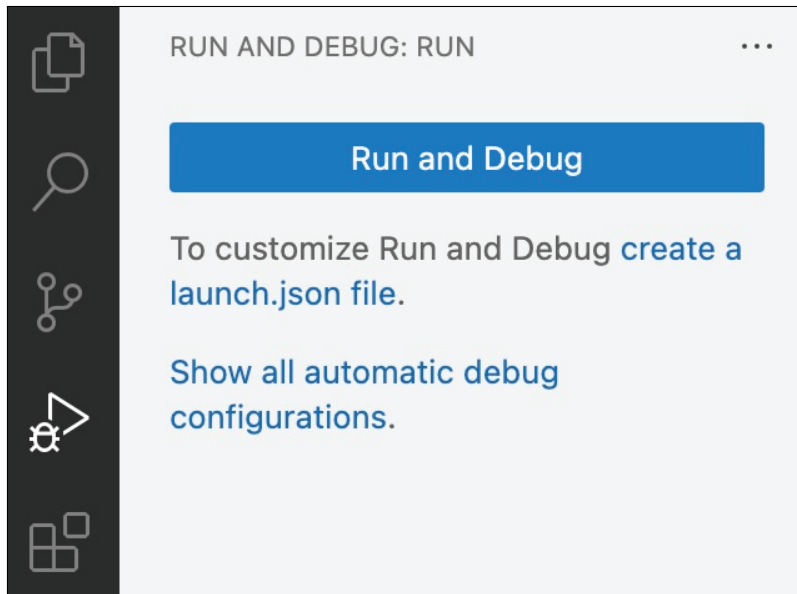


Figure 12-4: Creating a *launch.json* file from the Debug Panel in VS Code

If you've created a *launch.json* file for your VS Code workspace before, click the gear icon at the top of the Debug Panel to open that file. Alternatively, you can bring up *launch.json* by pressing CTRL-P on Windows (or ⌘P on macOS) to launch the Quick Open bar, then typing *launch.json* and selecting the first result.

Once you have the *launch.json* file open, it should look like a dictionary. Add the following inside the configurations section:

```
"args": ["Steve Jobs"]
```

With this addition, *Steve Jobs* will automatically be passed as an argument to the script when you start the debugger, allowing you to test and debug the way your script behaves with a particular input.

The Process of Testing Code

Debugging helps you pinpoint and resolve errors in your scripts, but before you can fix issues, you need to identify them by running your code. This is the *testing* process.

For a simple script that doesn't have much variability in its inputs and outputs, you probably don't need to do any testing other than running the script manually to verify that it produces the expected results. But as you lean into automation and begin to rely on scripts to work on their own

without oversight, you'll want to make sure they run as intended across all the kinds of conditions they might encounter.

In this section, we'll introduce two Python tools, assert statements and the unittest module, which both allow you to perform unit tests. In coding, *unit testing* means verifying that individual components of a script, like functions or methods, work as expected on their own. The idea with unit tests is that if you isolate a small piece of your code and test it on its own, you can ensure its correctness without interference from other parts of the program. This way, after all units are tested and working correctly, you can have greater confidence that the entire program will function as intended when the components are integrated.

Assertions

The simplest way to test a function in Python is to use an assert statement. This statement checks whether a given condition is True. If it is, the program continues executing. If the condition is False, the program raises an AssertionError and stops executing. This ensures that you'll notice when something isn't quite right.

As an example, let's use assert to test the time_based_greeting() function:

```
def time_based_greeting(hour):
    if 5 <= hour < 12:
        return "Good morning"
    elif 12 <= hour < 18:
        return "Good afternoon"
    else:
        return "Good evening"

# Simple tests
assert time_based_greeting(6) == "Good morning"
assert time_based_greeting(13) == "Give me an error!"
assert time_based_greeting(20) == "Good evening"
```

Each assert statement checks that time_based_greeting() returns the correct greeting based on a given input hour. If any of these assertions fail, Python will raise an AssertionError, helping you identify the issue quickly.

This is a basic example of a unit test. The assert commands test different sets of input conditions for your functions, ensuring that they behave as expected across various scenarios. Using assert has limitations that can make the process of testing code inefficient, however. For instance, when the code hits an error on line 11, the script will stop executing. You won't see the results of any subsequent tests, and you won't know how many of your tests fail. This is where the unittest module comes in handy.

The unittest Module

The `unittest` module helps make the process of running unit tests more structured and organized. It allows you to create multiple tests at once and automatically handle errors and exceptions, allowing the test suite to continue running even if one test fails. Since `unittest` is part of the Python Standard Library, it comes preinstalled with Python.

Working with `unittest` involves creating a new class that inherits from `unittest.TestCase`. Within the class, you define methods to test various aspects of your code. For example, let's create a file called `test_generate_greeting.py` to test the `generate_greeting()` function:

```
import unittest

class TestGenerateGreeting(unittest.TestCase):
    # Add the testing code here

if __name__ == "__main__":
    unittest.main()
```

This file can live anywhere in the project file structure that you like as long as you can import the functions from `greeting.py` into it. Inside the class, each test method you add needs a name that starts with `test_` to ensure that `unittest` will recognize it as a test that should be run. Each test method should look at one specific aspect of your function. Here's an example:

```
from greeting import generate_greeting

def test_generate_greeting_morning(self):
    self.assertEqual(generate_greeting("Alice", 7), "Good morning, Alice!")
```

This method uses the `unittest` module's `assertEqual()` method to test whether the `generate_greeting()` function returns the correct greeting for the inputs `Alice` and `7`. It's designed to make sure the morning branch of the conditional logic works as expected. Here's how that method looks in the context of the whole `test_generate_greeting.py` file:

```
import unittest
from greeting import generate_greeting

class TestGenerateGreeting(unittest.TestCase):

    def test_generate_greeting_morning(self):
        self.assertEqual(generate_greeting("Alice", 7), "Good morning, Alice!")

if __name__ == "__main__":
    unittest.main()
```

When you run this script, unittest will automatically detect and run the test method we've defined. If the test passes, it will proceed without any issues and print a summary of how many tests ran and how long they took:

```
Ran 1 test in 0.000s
```

```
OK
```

If the test doesn't pass, two outcomes are possible. First, if the function hits an error that isn't handled with an exception, you might see something like this:

```
Ran 1 test in 0.000s
```

```
FAILED (errors=1)
```

Second, if the test fails because the assertion didn't match the expected result, you might see the following:

```
Ran 1 test in 0.001s
```

```
FAILED (failures=1)
```

With the added insight from the unittest module, you can see the difference between tests that run successfully but don't output the expected results and tests that fail to run altogether.

By writing multiple tests, you can ensure that your function handles different scenarios correctly. For example, let's try three additional test cases:

```
import unittest
from greeting import generate_greeting

class TestGenerateGreeting(unittest.TestCase):

    def test_generate_greeting_morning(self):
        self.assertEqual(generate_greeting("Alice", 7), "Good morning, Alice!")

    def test_generate_greeting_afternoon(self):
        self.assertEqual(generate_greeting("Bob", 13), "Good afternoon, Bob!")

    def test_generate_greeting_evening(self):
        self.assertEqual(generate_greeting("Charlie", 7), "Good evening, Charlie!")

    def test_generate_greeting_early_morning(self):
        self.assertEqual(generate_greeting("Dave", 4), "Good morning, Dave!")

if __name__ == "__main__":
    unittest.main()
```

When you run this script, unittest will execute all the test methods in the class. You'll get a summary of how many tests ran, how many passed, and any errors or failures that occurred:

```
Ran 4 tests in 0.000s
```

```
FAILED (failures=1, errors=1)
```

Above the summary, there will be details of which tests failed or ran into errors. In this case, two of the test cases had issues, with one causing a failure and one causing an error. The third test case causes an error because "7" is a string instead of an integer:

```
=====
ERROR: test_generate_greeting_evening
(__main__.TestGenerateGreeting.test_generate_greeting_evening)
-----
```

And the fourth test case causes a failure because Good morning, Dave! isn't the correct output of the function when hour is set to 4:

```
=====
FAIL: test_generate_greeting_early_morning
(__main__.TestGenerateGreeting.test_generate_greeting_early_morning)
-----
```

In all, the framework the unittest module provides for testing your code allows you to run more tests more efficiently and catch a broader range of potential issues, making your code more robust and reliable.

Identifying Test Cases

Which test cases you choose to run greatly impacts whether your tests are able to find any bugs that might be present. Test cases should cover situations that are typical as well as less common *edge cases* where the program needs to handle extreme, unexpected, or error-prone inputs that could cause issues.

In the case of the *greeting.py* script, the test cases you choose for the two functions depend on what the functions do. With the `time_based_greeting()` function, you might choose your test cases based on the boundary conditions in the logic that determines which greeting to return. For example, you might choose two typical cases for the `hour` argument, such as 9 AM, which is in the middle of the morning, and 11 PM, which is in the middle of the evening. Then you might also choose two edge cases, such as 12 PM (the boundary between morning and afternoon) and 6 PM (the boundary between afternoon and evening).

For the `generate_greeting()` function, your test cases might target the format of the `name` argument as well as the correctness of the greeting based on the time of day. For example, you might have typical cases like using a common name such as Alice at 9 AM, expecting Good morning, Alice!, and using

Bob at 3 PM, expecting Good afternoon, Bob!. Then you might also have edge cases, such as passing a name with special characters, like John Doe, to see how the function handles spaces, or passing a nonstandard time input, like the string "7" instead of the integer 7, to test the function's robustness.

When an edge case causes a test failure or error, it might be worth adding a try...catch block to your code to handle the resulting exception. For example, to handle a string input for hour, you might implement a try...except block to manage ValueError exceptions:

```
def time_based_greeting(hour):
    try:
        hour = int(hour) # Convert the input to an integer
        if 5 <= hour < 12:
            return "Good morning"
        elif 12 <= hour < 18:
            return "Good afternoon"
        else:
            return "Good evening"
    except ValueError:
        return "Invalid input: hour must be an integer"
```

In this code, the try block attempts to convert the hour input to an integer by using int(hour). If the input is something that can't be converted to an integer, like a nonnumeric string, a ValueError is raised. The except block then catches this error and returns a message indicating that the input is invalid. These are the kinds of improvements that you can make to your code when you see how it reacts to edge cases.

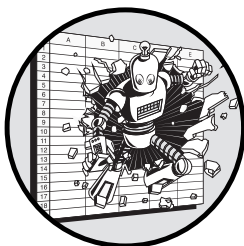
Conclusion

This chapter explored the essential skills and tools for ensuring that your code runs smoothly and correctly, as well as what to do when it doesn't. You used the debugging tools in VS Code to set breakpoints, step through code, and inspect variables to identify issues. You also explored unit testing, creating tests to verify that your code works as expected under various conditions. By leveraging tools like assert statements and the unittest module, you can systematically test your functions, ensuring that they handle both typical and edge cases robustly.

Throughout this chapter, you've seen that debugging and testing aren't just about fixing problems but also about building confidence in your code and your ability to write it. When you have a defined, straightforward process to identify and work through errors, you won't feel like you're walking into a dark abyss every time you try something new. These skills will not only help you develop more reliable and maintainable code but also ultimately make your coding experience more efficient and enjoyable.

13

THREE GOOD CODING HABITS



In this chapter, we'll cover three good habits that can improve the quality of your code. We'll focus on best practices that champion progress over perfection. In other words, these guidelines are more than just theoretically nice to have; they bring real value to every project you aim to tackle with Python, particularly the kinds of projects that evolve out of spreadsheets.

We'll also explore how Excel can encourage less-than-ideal practices in each of these areas, so as you shift more of your work to Python, you'll be better equipped to recognize and avoid them. These three habits for effective coding will allow you to not just increase your productivity but also promote collaboration and scalability like you never could in Excel.

The world of coding has some controversial topics that programmers will always love to debate: tabs versus spaces, functional versus OOP, light versus dark mode for code editors, and many more. Try asking a few software engineers for their opinions on these questions, and you'll find plenty of divergent ideas about what makes the best possible code. It's easy to get bogged down in the slurry of options, but not all these issues are relevant for the everyday work most of us do, especially when we don't plan to write

code in a large, high-impact production system that will be scrutinized by teams of software developers.

As such, this chapter isn't about perfection; it's about practicality. Each of the habits discussed here will allow you to make the most of your code as you transition some of your work out of Excel and into Python, even if you have no intention of becoming a professional software developer.

Simpler Code Is Better Code

Our first good coding habit is this: Always promote simplicity over complexity. Sure, complexity can have a certain *je ne sais quoi*. It's interesting and can look impressive, and it's always tempting to use complexity to illustrate that the work you do is difficult and sophisticated. But complexity only for the sake of complexity isn't actually sophisticated at all, and it often creates more problems than it solves. If you can accomplish the same task in a simpler manner without compromising on functionality, you should always choose the simpler approach.

Complexity in a process often arises when the interactions between elements multiply, making it harder to predict how changing one element will affect the overall system. Excel is notorious for this. Cells, formulas, visualizations, and sheets all tend to have multiple links to one another, and the number of links increases as your work grows, making it difficult to scale a process in Excel without adding complexity. The larger the web of connections, the more difficult it becomes to untangle and therefore to maintain. When you add a column to a sheet in Excel, formulas in other cells may need to change to reflect the new shape of the data. If you add a row, a chart may now reference the wrong range. If you add a sheet, you'll need to create entire new connections, and anytime you make a change to those, you could break something.

These links are all sources of complexity, but the way Excel is built to hide the connections behind cell references and formulas puts the true level of complexity and the impact of changes out of reach. Understanding and controlling the way parts of the process interact becomes difficult. This hidden complexity can lead to errors and make the process increasingly fragile as it scales.

Code, on the other hand, can be much more transparent about the way elements interact. For example, each function has clearly defined inputs (the arguments it takes) and outputs (what it returns). Still, code runs the risk of becoming complex when it's hard to follow step-by-step, difficult to understand at a glance, or challenging to maintain or modify. Here, we'll break down some of the ways complexity often arises.

Too Many Conditions

One common source of complexity is having too many nested conditions, which can make the code harder to follow. In Chapter 2, we used the example

of whether a person shopping at a store is eligible for a discount based on their age and membership status to explain nested conditionals in Python. If we functionalized that example, it would look like this:

```
def check_discount(age, is_member):
    if is_member:
        eligible_for_discount = True
    else:
        if age < 18 or age >= 65:
            eligible_for_discount = True
        else:
            if 25 <= age <= 30:
                eligible_for_discount = True
            else:
                eligible_for_discount = False
    return eligible_for_discount
```

Even though this function is significantly simpler than the equivalent Excel formula would be, it's still somewhat convoluted. A more streamlined approach would be to use a dictionary and the `any()` function to handle the discount criteria:

```
def check_discount(age, is_member):
    discount_criteria = {
        'under_18': age < 18,
        'senior': age >= 65,
        'member': is_member,
        'promo_age': 25 <= age <= 30
    }

    return any(discount_criteria.values())
```

In the second version of this function, it's much easier to follow what's happening step-by-step. We take in someone's age and membership status, then construct a dictionary with four entries, one for each possible discount condition. Then `any()` looks inside the dictionary and returns `True` if any entry in the dictionary is `True`, meaning the person is eligible for the discount. Using a dictionary in this way gives anyone reading the code a way to look up each discount criterion without having to sift through multiple layers of nested logic. The code clearly separates the conditions and clarifies how each one contributes to the final decision.

Nested conditionals don't always make your code unnecessarily complex. But when you have multiple layers of logic and multiple options for each layer, sometimes aggregating that information in a more structured format, like a dictionary, can make the code easier to understand. A good rule of thumb is to keep nested conditional statements to fewer than three levels.

Mixed Scopes

Your code can also become complex by intermingling variables that live in different scopes in the same code. This can be confusing to follow because it's harder to keep track of where each variable is defined and how it interacts with other parts of the program.

For example, say you need a function that calculates the total cost of an item including the tax rate. Since the tax rate probably won't change often, it might seem intuitive to set a global `TAX_RATE` variable at the top level of the script and then use that variable wherever it's needed, including within the `calculate_total_cost()` function:

```
TAX_RATE = 0.08

def calculate_total_cost(price):
    return price * (1 + TAX_RATE)

if __name__ == "__main__":
    price = 100
    final_cost = calculate_total_cost(price)
```

This design has an intermingling of scopes: Within `calculate_total_cost()`, a local `price` variable is interacting with the global `TAX_RATE` variable. This makes the function somewhat unclear and hard to manage because the source of the `TAX_RATE` variable isn't obvious from within the function. If someone else (or even you, later) looks at the function, they might not immediately know where `TAX_RATE` is coming from or whether it could be modified elsewhere in the code. It does help a little bit that `TAX_RATE` is written in all caps, a common convention for global variables, but what if it were written as `tax_rate` instead?

A clearer approach is to set the tax rate within the main function and pass it as an argument to `calculate_total_cost()`:

```
def calculate_total_cost(price, tax_rate):
    return price * (1 + tax_rate)

if __name__ == "__main__":

    price = 100
    tax_rate = 0.08
    final_cost = calculate_total_cost(price, tax_rate)

    print(f"The final cost: {final_cost}")
```

This approach is better because it clarifies the scope of variables. The `calculate_total_cost()` function is now completely self-contained and relies only on the `price` and `tax_rate` arguments passed to it.

One way to consider this is to think about the path you might take if you were to find a bug in the function and had to go through the process of debugging it. If you weren't getting the result you expected, you might

put a breakpoint inside `calculate_total_cost()` to investigate the values of the two variables, `price` and `tax_rate`. However, if `tax_rate` were a global variable, you might not see its value directly within the function. You'd have to do the extra work of stepping out of the function and tracing where `tax_rate` is defined and whether it's been modified elsewhere in the code.

For this reason, you should use global variables sparingly. They introduce hidden dependencies and can make your code less predictable. If you see variables within their appropriate scopes, your code will always be easier to understand and work with.

Inconsistency

When you're reading a book, staying engaged with the narrative is easiest when the writing style and language are consistent throughout. Imagine that a novel's author suddenly switches from English to French in the middle of a sentence, then quickly switches back to English. Even if you understand both languages, you might get a little confused, and comprehending what's happening in the story would require extra effort.

Inconsistency in your code has a similar effect. When the way your code is written varies throughout a project, following and understanding it becomes harder. For example, using different naming conventions, indentation styles, or ways of organizing code blocks in different parts of your code can confuse anyone trying to understand or modify it. On the other hand, consistent formatting reinforces the structure and logic of your code, making it easier to comprehend.

Let's say that instead of using a standard capitalization convention, you mix things up every time you name a function:

```
def totalCost(p, t):  
    return p * (1 + t)  
  
def DISCOUNT(x, d):  
    x = x * (1 - d)  
    return x
```

When someone comes across `totalCost()` and `DISCOUNT()` in your code, it may not be immediately obvious that they're functions. The inconsistent capitalization forces the reader to take extra steps to figure out what the identifiers represent, slowing down their understanding and making things harder than they need to be.

Confusing Variable Names

The variable names you choose can hurt the readability and maintainability of your code. Shorter isn't always simpler. Going with a longer, more descriptive variable name is better than using a cryptic abbreviation that leaves the reader guessing.

For example, instead of using a vague name like `tr`, it's clearer to use something like `tax_rate`, which immediately conveys the variable's purpose. You could use a comment to describe what `tr` means, but why do that when you can make the variable name clear enough in the first place?

Of course, descriptive variable names can also be taken too far:

```
percentage_rate_of_value_added_tax_on_goods_and_services = 0.08
```

This excessively long name is cumbersome and complicates the code without adding any additional value compared to something like `tax_rate`.

All in all, variable names should include enough information to tell you what they represent and how they're different from other variables—nothing more and nothing less.

Repetition

If you say the same thing twice, you've created not only twice as much code to maintain but also twice as many opportunities for mistakes. This is the reasoning behind the common programming principle *don't repeat yourself* (*DRY*). If you've already created logic to accomplish a specific task, finding a way to reuse that logic is better than duplicating it elsewhere in your code.

For example, if your script calculates prices with tax and with a discount, you might initially write separate functions to handle each task:

```
def calculate_price_with_tax(price, tax_rate):
    return price * (1 + tax_rate)

def calculate_discounted_price(price, discount):
    return price * (1 - discount)
```

Now suppose you need to calculate the final price, considering both membership discounts and tax. You might be tempted to do this:

```
def calculate_final_price(price, discount, is_member):
    if is_member:
        price = price * (1 - discount)
    tax_rate = 0.08
    return price * (1 + tax_rate)
```

This function re-creates the logic of calculating the discounted price and applying the tax. The logic to accomplish these tasks, `price = price * (1 + tax_rate)` and `price = price * (1 - discount)`, is already included elsewhere in the script, so writing it out again increases the chance of inconsistencies. What if you mix up the `+` and `-` signs, for example? Instead, you can use the previously defined functions to keep the code *DRY*:

```
def calculate_final_price(price, discount, is_member):
    if is_member:
        price = calculate_discounted_price(price, discount)
```

```
tax_rate = 0.08
return calculate_price_with_tax(price, tax_rate)
```

This version avoids repeating the logic for applying discounts and taxes by reusing the existing `calculate_discounted_price()` and `calculate_price_with_tax()` functions. Therefore, if you need to make a change to either of these functions, that change will also be reflected in `calculate_final_price()`. You don't need to modify the logic in multiple places.

Making Code Manageable

One of the best ways to make a task easier is to break it into smaller, more manageable steps. The clearer and more well-defined the division between steps, the smoother and more efficient the process becomes. This separation allows you to focus on only one specific aspect of the task at a time rather than dividing your attention, which can lead to mistakes, oversights, and inefficiencies. This brings us to our next good coding habit: Aim to have each component of your code do only one thing, but do it well.

Spreadsheets encourage the intermingling of responsibilities. Often your spreadsheet is your database, your analytical toolkit, and your visualization tool all at once. This naturally blurs the lines between individual tasks in your project, which may cause you to miss mistakes, inconsistencies, or opportunities to improve in your Excel-based workflow.

You might try to impose some structure to manage the tasks separately, but there's no way to enforce strict separation within a spreadsheet environment. For example, you might try to include the raw data for your process in its original form right from its source in a worksheet called Data. Then you have another sheet that cleans and transforms the data called Clean Data and, finally, a third sheet called Report where you create charts and summaries of the data. This seems a bit better than mingling all these tasks in a single sheet; when you want to make a change to a specific part of the process, you immediately have a good starting place for where to look.

However, this division of tasks is more conceptual than actual. There's nothing stopping you from doing data cleaning in the Data sheet since the sheet itself has exactly the same functionality as the others. Likewise, you might create a chart in Clean Data and forget to move it over to Report, and then when you need the same chart later, you won't be able to find it. When you don't have the ability to create clear boundaries between each piece of your process, it quickly becomes disorganized, ultimately undermining the reliability and efficiency of your workflow.

In Python, defining clear boundaries between tasks is much easier, ensuring that each component of your code is responsible for only one specific part of the task. You can make the most of this ability by modularizing your code. Each function, class, and method should serve its own specific, well-defined purpose, contributing to the overall task without overstepping into other areas.

The Right Amount of Modularization

Modularization and “doing only one thing” is all about maintaining the boundaries between tasks. You should ensure that each component fulfills a single, clearly defined purpose. You’ll know that a function’s purpose isn’t well-defined if you can’t specifically describe what it does without using the words *and* or *or*. For example, if you said something like, “This function loads the dataset from the CSV file and calculates the weighted average,” that’s probably too much. You’d likely be better off creating two functions, one that loads the dataset from the CSV file and another that calculates the weighted average.

To illustrate further, let’s create a function that doesn’t fully adhere to the modularization principle. This function converts a temperature from Fahrenheit to Celsius or from Celsius to Fahrenheit (notice the telltale *or*):

```
def temperature_converter(temp, scale):
    if scale == "Celsius":
        converted_temp = (temp * 9/5) + 32
        print(f"{temp}°C is {converted_temp}°F")
    elif scale == "Fahrenheit":
        converted_temp = (temp - 32) * 5/9
        print(f"{temp}°F is {converted_temp}°C")
    else:
        raise ValueError("Please enter a valid scale.")
    return scale
```

This function is relatively simple and not necessarily difficult to follow, but it handles both the temperature-conversion logic and the scale-detection logic within a single block of code. You couldn’t easily test or reuse just the conversion logic individually. Here’s a more modular version:

```
def celsius_to_fahrenheit(celsius):
    return (celsius * 9/5) + 32

def fahrenheit_to_celsius(fahrenheit):
    return (fahrenheit - 32) * 5/9

def temperature_converter(temp, scale):
    if scale == "Celsius":
        converted_temp = celsius_to_fahrenheit(temp)
        print(f"{temp}°C is {converted_temp}°F")
    elif scale == "Fahrenheit":
        converted_temp = fahrenheit_to_celsius(temp)
        print(f"{temp}°F is {converted_temp}°C")
    else:
        raise ValueError("Please enter a valid scale.")
```

This version uses more lines of code, but it separates the conversion logic for both scales into their own functions, `celsius_to_fahrenheit()` and

`fahrenheit_to_celsius()`. The `temperature_converter()` function then calls these smaller, more focused functions depending on the scale provided. You can still accomplish the same task as the first version, but with a few additional benefits.

First, if a problem occurs with the conversion logic, it's easier to isolate and fix the issue in the specific function responsible for it, without worrying about other parts of the code. You'll immediately know where to look, and you won't need to worry about code that serves an entirely different purpose clouding your vision. Second, you can reuse the conversion functions in other parts of your code or in different projects altogether. If you needed to apply temperature conversion in another script, you have the option of reusing any of the three functions here, rather than only everything at once.

Too Much Modularization

It's possible to take modularization too far by breaking your code into components that are smaller than necessary. As we already hinted in "Functionalizing Logic" on page 65, this can make your code overly fragmented. Let's explore this further with another exaggerated example. Say we decide to break `celsius_to_fahrenheit()` into its individual steps:

```
def multiply_by_nine(number):
    return number * 9

def divide_by_five(number):
    return number / 5

def add_thirty_two(number):
    return number + 32

def celsius_to_fahrenheit(celsius):
    result = multiply_by_nine(celsius)
    result = divide_by_five(result)
    result = add_thirty_two(result)
    return result
```

We split the conversion logic into three separate functions: `multiply_by_nine()`, `divide_by_five()`, and `add_thirty_two()`. The operations these functions perform are too specific to be useful in other contexts, and as a result the code is more convoluted than it needs to be. Instead of a clear and concise conversion function, we now have several small functions that make the code harder to follow and maintain.

A useful way to think about how far to break down your code is to consider how you would describe these functions. A function adds value only if it encapsulates a complete, meaningful task, so its description should be more than just the specific actions inside. For example, `divide_by_five()` might accurately describe a specific operation, but it doesn't convey any

broader purpose or context. It's really not that different from something even more redundant, like this:

```
def no_actually_divide_by_five(number):  
    return divide_by_five(number)
```

Hopefully, you can see how the `no_actually_divide_by_five()` function is a total waste of time. It simply takes in a number and passes it along to the `divide_by_five()` function. When you're deciding how far to break down your code, try to have each component perform a single, isolated, useful action. That's the point where you should stop.

First Make It Run, Then Make It Better

When you're tackling a new problem or project, your initial focus should be to make a functional solution, even if it isn't perfect. The goal at this stage is to get something working, something that does the job, even if it's not the most elegant or efficient solution. Once it works, you can refine and optimize your code.

The benefit of operating this way is that you can prevent *premature optimization*, or spending unnecessary time on details that might not end up mattering when you don't yet have a working solution. When you start to optimize your solution prematurely, you waste time, but you also limit your options in the future. All that optimization tends to lead you to add irrelevant features that increase complexity, and your choices may reduce flexibility down the road.

In Excel, tweaking and refining a solution is often more difficult than building it from scratch. Once you've built a spreadsheet, identifying the areas where improvements or optimizations are needed can be difficult because everything is so interconnected. To backtrack through your work, you need to weed through the connections, which takes nearly as much effort as constructing the entire spreadsheet in the first place. Because it's so taxing to make changes after the fact, you tend to devote too much attention to the details too early, which encourages premature optimization.

On the other hand, code is much more conducive to *iterative development*, which improves your solution incrementally. You don't need to worry about absolute perfection from the get-go. As long as you have something working, you can make it a little bit better each time you revisit your code. This is easiest to do when your code is simple and modular. If you notice that something could be improved, you can tweak just that part without worrying about breaking everything else. This is called *refactoring*: improving a piece of code, like a function or a script, without changing what it does.

Iterative development works especially well when you take full advantage of version control. If you're making many incremental updates, each commit can focus on a single, manageable change. Since each incremental change is well documented, you can track and review each change individually, and you have the option to go back to earlier versions of your code as needed. You're not stuck with one big, risky, poorly thought-out change; instead, you

have a series of well-documented steps that you can review, revisit, and learn from. Using version control in this way also makes collaboration easier. When each branch has a focused, specific purpose, less disruption occurs when contributors merge in changes, and if a new contributor to the code makes a mistake, you have more versions to go back to if needed.

In Python, this process of continuous improvement is natural and encouraged. You can start with something basic that works and then steadily make it better, one small step at a time, with gradual consistent improvement and gradual, consistent progress.

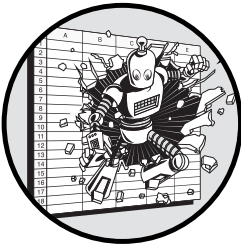
Conclusion

This chapter explained three habits that make for better code, why each is helpful, and how Excel inherently discourages their implementation. The first habit is to favor simplicity over complexity because complexity makes code more difficult to read, follow, and maintain. The second habit is to separate the distinct tasks in your scripts so that the code is as modular as possible, without taking this modularity too far. The third habit is to focus on reaching a working solution first and then to improve it incrementally.

All these habits do more than promote good code; they also promote collaboration. When your code is simple and easy to follow, it becomes accessible to others, allowing your team to understand and contribute without getting bogged down by unnecessary complexity. When your code is modular, delegating tasks is easier, allowing team members to work on different parts of the project independently but cohesively. When you make a point of reaching a fully fledged solution and then improving it incrementally, it encourages a continuous, shared process of refinement, where everyone can pitch in and make meaningful contributions.

The impact of these habits is therefore not just on the quality of your code but also on the culture of your team. By working in code, you and your team will begin to work together in a way that was never possible with Excel.

AFTERWORD



A common paradox, first articulated by Socrates in ancient Greece and revived in 1999 by two psychologists at Cornell University, is familiar to anyone who's ever tried to master a new skill: The more you know, the more you realize you don't know. Just as you take your first steps toward mastery, you also begin to understand exactly how much more there is to learn. As a result, confidence can be elusive, even though you've come a long way.

Coding is certainly one of those skills where this paradox applies. The range of concepts you can learn is practically infinite, and as your knowledge expands, so does your awareness of what's out there to know. With this book and your hard work, however, you've come a long way toward fluency in Python, so don't let your Socratic humility get in the way of a well-deserved sense of accomplishment.

Being comfortable with Excel, you weren't really standing at the bottom of a mountain before you started this book. You'd taken a few meaningful steps upward as you learned to think both visually and systematically through spreadsheets. But now, after taking in these chapters, you've reached a point where you can admire the view.

So let's take a look around. You can write scripts to automate processes that might have been manual before, saving time and reducing errors. And perhaps even more important, you can share and build upon that code in reusable modules and through Git repositories. You can process, transform, and visualize data programmatically and interactively. And you can write code that's organized, efficient, and effective.

All these concepts have built not just a sturdy foundation but a solid structure for you to be productive and continue growing in your coding abilities. Lucky for you, a nearly limitless and constantly expanding universe of resources can support your journey in Python and coding in general. For every topic in this book, you'll find countless other books, YouTube tutorials, Medium articles, and plenty of helpful strangers in online forums like Reddit and Stack Overflow ready to lend a hand as you move deeper into the weeds. And don't forget to experiment—every project, no matter how small, is progress.

INDEX

Note: *Page numbers in italics refer to figures or tables*

Symbols

- > (angle bracket), 8–9
- * (asterisk), 122–123, 154
- @ (at sign), 226–227
- \ (backslash), 6
- : (colon), 119, 289
- { } (curly brackets), 33
- \$ (dollar sign), 7
- = (equality operator), 35
- = (equal sign), 29
- / (forward slash), 6, 122–123
- > (greater than) operator, *149*
- >= (greater than or equal to) operator, *149*
- # (hash mark), 29
- < (less than) operator, *149*
- <= (less than or equal to) operator, *149*
- != (not equal to) operator, *149*
- < > (not equal to) operator, *149*
- operator, 122
- % (percent sign), 149
- + (plus sign), 122–123
- ; (semicolon), 141
- ~ (tilde), 7
- """ (triple quotes), 29
- _ (underscore), 149, 245

A

- accessor property, 124
- ACID (Atomicity, Consistency, Isolation, Durability), 135
- add() function, 122–123
- ADD keyword, 144
- add_trace() method, 193, 195, 209
- advanced navigation, 20–21
- aggregations, 128, 154–155
- AI (artificial intelligence), xxv

- alias, 112
- Altair, 188
- ALTER, 140
- ALTER TABLE command, 144
- Amazon Aurora, *137*
- Amazon Web Services (AWS),
 - xxx, 136
- American Standard Code for Information Interchange (ASCII), 33
- any() function, 275
- API (application programming interface)
 - authentication handling, 176–180
 - breaking large requests into smaller batches, 174–176
 - defined, 162
 - error handling, 171–174
 - errors, 173–174
 - GraphQL, 163
 - overview, 161–162
 - REST, 162–179
 - SOAP, 163
 - streamlining requests with reusable functions, 180–183
 - WebSocket, 163
- API keys
 - making requests with, 178–179
 - storing in environment variables, 177–178
- append() method, *40*
- application-specific environment variables, 10
- app.py file, 216
- app variable, 216
- arguments, 5, 39, 60–61
 - adding, 170–171
 - keyword, 61
 - positional, 61
 - system, 72–73

- arithmetic operators, 122–123.
 - See also* operators
- artificial intelligence (AI), xxv
- asana library, xxx
- ASCII (American Standard Code for Information Interchange), 33
- AS keyword, 147–148
- assembly language, 26
- assets, CSS, 222
- assets/styles.css* file, 229
- assignment statements, 29–30
- `astimezone()` method, 48
- `astype()` method, 116
- at sign (@), 226–227
- attributes, 27
- authentication. *See also* API
 - with API keys, 177–179
 - handling, 176–180
 - with OAuth, 179–180
- automation, xxviii–xxx
- AVG function, 155
- AWS (Amazon Web Services), xxx, 136
- Azure, 214

B

- backslash (\), 6
- bar charts. *See also* charts
 - basic, 194–196
 - multi-trace, 196–198
- bash, 5, 11, 12
- batches, 174
- BETWEEN operator, 149
- `bgcolor` subproperty, 204
- bin* subdirectory, 19
- Bitbucket, 78
- block elements, 217–218, 217*t*
- blocks, 28
- Bloomberg API, xxxi
- BLS API, 241
- `BLSDataset` class, 251–252
 - lazy loading, 248
 - unemployment object, 243
 - `yearly_summary()` method, 244
- blue screen of death, xxxi
- Bokeh, 188
- BOOLEAN data type, 144*t*
- Boolean indexing, 119

- Boolean variables, 31
- `bordercolor` subproperty, 204
- `borderwidth` subproperty, 204
- Boto3, xxx
- boundaries, 13
- branches, 87–88
- break keyword, 57–58
- breakpoints, 21, 258–259
 - conditional, 264
 - hit counts, 266
- `b` subproperty, 204

C

- callbacks, 226–227
- call stack, 262
- Cascading Style Sheets. *See* CSS
- catching exceptions, 172
- categorical data, 116
- `cd` (change directory) command, 6, 8
- `celsius_to_fahrenheit()` function, 280
- central processing unit (CPU), 26
- characters, 32–33
- charts
 - bar charts, 194–198, 196, 197
 - colors, 205
 - common properties, 202–204
 - Excel vs. Python, 186–187
 - fonts and font families, 202–205
 - layout properties, 201–205
 - line charts, multi-trace, 193–194, 194
 - multidimensional data, 198–200, 200
 - scatter plots, 198–200, 200
 - themes, 205
- Chinese characters, 33
- classes, 53, 236
 - constructors, 237–238
 - custom, 237–239
 - encapsulation with, 245
 - extendability with, 244–245
 - inheritance, 249–250, 253–254
 - methods, 238–239
 - parent class, 251–252
 - in Python libraries, 237
 - reusability with, 243–244
 - superclass, 249
- class selectors, 221
- class variables, 261
- CLI (command line interface), 4–5, 69

- clipboard, 117
 - cloning repositories, 83–84
 - co-authoring, xxiv
 - code, 26
 - completion, 20
 - pausing, 265–266
 - running, 17–18
 - testing, 266–272
 - assertions, 268
 - test cases, 271–272
 - unittest module, 269–271
 - code cells, 98–99
 - code editor, 19–21
 - coding habits, 273–283
 - to avoid, 274–279
 - improving code, 282–283
 - iterative development and, 282
 - managing code, 279–282
 - modularization and, 279–282
 - overview, 273–274
 - premature optimization and, 282
 - refactoring and, 282
 - simplicity over complexity, 274–278
 - collaboration, xxiv
 - colon (:), 119, 237
 - color property, 203
 - column transformations, 154
 - command line interface (CLI), 4–5, 69
 - Command Prompt (CMD), 5, 69
 - commands, 5
 - comma-separated values (CSV),
 - 67–69, 167
 - commits, 85
 - comparison operators, 35, 149.
 - See also* operators
 - component_id property, 227
 - component property, 227
 - composability, 254
 - composite data types, 36–38
 - concatenation, 126
 - concat() function, 126
 - conditional breakpoints, 264
 - conditionals, 41–45
 - elif clauses, 43–44
 - else clauses, 43
 - if statements, 42–43
 - nested, 44–45
 - conditions, 59
 - conflicts, 88–90
 - ConnectionError, 174
 - console, 5
 - constructors, 237–238
 - containers, 27
 - continue keyword, 59
 - control flow, 41
 - copies, 120
 - Copilot, xxv, 20
 - corporate IT policies, working within,
 - 13–14
 - COUNT function, 155
 - COUNTIFS, 129
 - COVID test results, xxi–xxii
 - C programming language, 32
 - CREATE command, 140, 142
 - CSS (Cascading Style Sheets)
 - assets, 222
 - basics, 221–222
 - classes, 221
 - in Dash, 222–223
 - ID selectors, 222
 - overview, 216–217
 - properties, 221
 - selectors, 221
 - CSV files, 67–69, 167
 - csv module, 66–73
- ## D
- Dash
 - basics of, 214–216
 - creating interactive reports with,
 - 212–214
 - dashboard, 215–216
 - dependencies submodule, 227
 - installing, 215
 - Plotly charts in, 223–227
 - vs. Power BI, 214
 - web development concepts and,
 - 215–223
 - CSS, 221–223
 - HTML, 217–220
 - overview, 216–217
 - data
 - aggregating, 128, 154–155
 - categorical, 116
 - grouping, 129–130
 - manipulating, 121–124

- data (*continued*)
 - missing, handling, 124–125
 - temporal, 116
- database management systems, 137–138
- databases
 - adding value to Excel workflow
 - with, 134
 - atomicity, 135
 - consistency, 135
 - corporate, accessing, 136
 - costs of connection, 136–137
 - creating, 138–142
 - document-oriented, 137
 - durability, 135
 - incorporating into workflow, 135
 - location of, 135–136
 - managing, 140–141
 - nonrelational, 137
 - NoSQL, 137
 - relational, 137
 - remote, connecting to, 142
 - retrieving data, 146–156
 - tables, 143–145
 - task automation with Python and SQL, 157–160
- DataFrames, 110–116
 - accessing values, 118–119
 - arithmetic operations, 122–123
 - combining, 125–129
 - creating, 112–114
 - date operations, 123–124
 - handling missing data, 124–125
 - moving data to Excel from, 117
 - reading data from files into, 114–115
 - string operations, 123–124
- datasets
 - for Plotly charts, 189–191
 - sorting, 121
- data types
 - basic, 31–34
 - composite, 36–38
 - importing multiple, 46
 - None, 34
 - SQL, 143–144, 144
- date object, 46
- date operations, 123–124
- dates, 45–47
- datetime library
 - format codes and Excel counterparts for, 47
 - library, 46–47
 - objects, converting to/extracting from string, 47–49
 - working with time zones, 49–50
- datetime64 type, 116
- DATEVALUE function, 45
- dcc.Checklist() component, 226, 228
- dcc.Graph() component, 224, 228
- Debug Console, 262
 - Step Into command debugger
 - command, 264–265
 - Step Out command debugger
 - command, 263–264
- debugger, 21
- debugging, 257–258
 - code with errors, 263–265
 - with different inputs, 266–267
 - integrated, 21
 - Step Into command, 264–265
 - Step Out command, 263–264
 - in VS Code, 258–267
- Debug Panel, 260–262
- decorator, 226–227
- def keyword, 60
- Desktop directory, 8–9
- dictionaries, 38
- dictionary methods, 41
- dir command, 8, 9, 10
- directed acyclic graph (DAG), 92–93
- directories, 5
 - creating, 8
 - deleting, 8–9
 - listing, 8
 - navigating, 6, 7–9
 - paths, 6
- DIV/0! error, xxiii–xxiv
- div element, 217–218
- div() function, 122–123
- docstrings, 29
- documentation, xxiii
- document-oriented databases, 137
- dollar sign (\$), 7
- dot notation, 39
- download() method, 239–243, 244–245, 249, 252

- dragmode property, 209
- Dropbox, 78
- dropbox-sdk-python, xxx
- DROP keyword, 144
- dropna() function, 125
- DROP TABLE command, 145
- dt.strftime() function, 124
- dtypes property, 116

E

- edge cases, 271
- element sectors, 221
- elif clauses, 43–44
- else clauses, 43
- emojis, 33
- encapsulation, 245
- encoding, 33
- encryption, 164
- environment variables, 10, 177–178
 - commands, 11
 - modifying on macOS, 11–12
- errors
 - common, 173–174
 - detecting, xxii–xxiii, 20
 - handling, xxii–xxiii, 171–174
- Excel
 - aggregation operations in, 128
 - charting in, 186–188
 - collaboration in, xxiv
 - comparison operations in, 35*t*
 - error handling in, xxii–xxiii
 - learning to code in Python with, xx
 - logical operations in, 35–36
 - moving between pandas and, 116–118
 - as productivity tool, xix
- excel_utils.py* file, 70
- except blocks, 181
- exceptions, 172
- execute() method, 159
- extendability, 244–245

F

- family property, 203
- feature branches, 87–88
- fetchall(), 159
- fetchmany(), 159
- fetchone(), 159

- ffill() function, 125
- fig.update_layout() function, 193, 224
- filesystem navigation commands, 6
- fillna() function, 125
- filters, 148
- FIND function, 45
- firewalls, 6, 14, 142
- floats, 31–32
- folders, 5
- font property, 202–203
- fonts, 202–205
- font subproperty, 204
- for loop, 55–56. *See also* loops
- FROM clause, 151
- f-strings, 33
- full outer joins, 153–154
- functional programming, 59–64, 239
- function definitions, 53
- functions, 38
 - components of, 60–63
 - higher-order, 227
 - modifier, 64
 - modularity of, 60, 65
 - in Python, 39
 - reusable, 60, 180–183
 - simplification with, 60
 - user-defined, 59–64
- function variables, 261

G

- get() method, 41, 171, 174
- GET request, 165
- getters, 246–247
- Git, 77–78
 - best practices, 90–91
 - branches, 87–88
 - commits, 85
 - configuring, 82–83
 - conflicts, 88–89
 - diagnostic commands, 91
 - directed acyclic graph (DAG)., 92–93
 - file structure of Python
 - project in, 80
 - vs. GitHub, 78
 - hashing, 93
 - installing, 82
 - integrating with VS Code, 92

Git (*continued*)

- managing histories in, 87–90
- repositories, 78–87
 - centralizing work in single folder, 78–80
 - cloning, 83–84
 - creating from scratch, 83
 - local, 80–81
 - maintaining *.gitignore* file, 81–82
 - remote, 80–81
 - setting up, 81–84
 - syncing, 86–87
- staging area, 84–85
- tracking changes to files, 84–85
- troubleshooting with status messages, 91–92
- `git blame` command, 91
- `git branch` command, 91
- `git commit` command, 85
- `git diff` command, 91
- `git fetch` command, 85
- .git* folder, 80
- GitHub, 78
- GitHub Copilot, xxv, 20
- .gitignore* file, 80, 81–82
- GitLab, 78
- `git log` command, 91
- `git pull` command, 85
- `git push` command, 85–86
- `git show` command, 91
- `git status` command, 91
- global variables, 54
- `go.Bar()` function, 195, 224
- `google-api-python-client`, xxx
- Google Colab, 14
- Google Drive, 78
- GraphQL APIs, 163
- greater than (>) operator, 149
- greater than or equal to (>=) operator, 149
- `gridcolor` subproperty, 203
- `gridwidth` subproperty, 203
- gross domestic product (GDP), xxi
- `GROUP BY` clause, 155–156
- `groupby()` function, 129–130
- grouping, 129–130, 155–156

H

- hashing, 93
- hash mark (#), 29
- hash tables, 38, 93
- `HAVING` clause, 156
- `HAVING` keyword, 155
- headers, 178–179
- `head()` method, 115, 190
- hexadecimal codes, 205
- higher-order functions, 227
- high-level languages, 26, 27
- hit counts, 266
- horizontal concatenation, 126
- `hovermode` property, 209
- `hovertemplate` argument, 210
- HTML (hypertext markup language)
 - basics, 217–219
 - block-level elements, 217–218, 217
 - in Dash, 219–220
 - inline level elements, 218–219, 219
 - overview, 216–217
 - tags, 217*t*
- HTTP (Hypertext Transfer Protocol), 163
- `HTTPError`, 174
- HTTPS (Hypertext Transfer Protocol Secure), 164
- `hubspot-api-client`, xxxi

I

- IBM Db2, 137
- iCloud, 81
- IDE (integrated development environment), 21
- ID selectors, 222
- `if...else` statement, 266–267
- `IF` function, 42–43
- `if` statements, 42–43
- `iloc[]`, 119
- `imports`, 53
- inconsistency in coding, 277
- `INDEX/MATCH`, 127
- indexes, 37, 110
- inheritance, 249–251
- `__init__()` method, 237, 242
- inline elements, 218–219, 219
- inner joins, 150–151
- `IN` operator, 149
- `Input()` parameter, 227

- INSERT INTO command, 145
- insert() method, 40
- installing
 - Dash, 215
 - JupyterLab, 101
 - pandas, 111–112
 - Plotly, 189
 - PostgreSQL, 138–140
 - Psycopg 2, 158
 - Python, 14–18
 - VS Code, 21–23
- instances, 236
- INTEGER data type, 144
- integers, 31–32
- integrated development environment (IDE), 21
- interactive applications, 27
- interactive reports
 - creating with Dash, 213–214
 - reasons for using, 212–213
 - sharing, 230–231
- intermediate-level languages, 26–27
- Internet Protocol (IP), 164
- interoperability, xxx–xxxi
- interpreter, 4, 22–23
- int() function, 39
- InvalidURL error, 174
- .ipynb file, 102, 106
- isin() method, 224
- isna() function, 125
- IS NOT NULL operator, 148
- IS NULL operator, 148
- items() method, 41
- iterations, 54, 282

J

- JavaScript, 216–217
- JavaScript Object Notation (JSON), 167–169
- JOIN keyword, 150
- joins, 148–149. *See also* databases
 - full outer, 153–154
 - inner, 150–151
 - left, 151–153
 - right, 151–153
- JPMorgan Chase, xxi
- JSONDecodeError, 174
- json.dump() function, 168

- json.dumps() function, 168
- JSON files, 167–169
- json.load() function, 168
- json.loads() function, 168
- json() method, 170
- JSONPlaceholder, 166
- JupyterLab, 101–102, 102
- Jupyter Notebooks
 - building, 103–105
 - closing, 103
 - code cells, 98–99
 - code to put in, 106
 - creating, 102–103
 - markdown cells, 99–100, 103–104
 - pandas and, 111
 - raw text cells, 100–101
 - renaming, 103
 - using Plotly in, 189
 - version control, 107

K

- Kaleido library, 197
- key, 10
- KeyError, 173
- keys() method, 41
- key-value pairs, 38
- keyword arguments, 61

L

- large language models (LLMs), xxv, 20
- launch.json file, 266–267
- lazy loading, 248–249
- LEFT function, 45
- left joins, 151–153
- legend property, 204
- len() function, 39
- less than (<) operator, 149
- less than or equal to (<=) operator, 149
- libraries, 18. *See also* pandas; Plotly
 - Altair, 188
 - asana, xxx
 - Bokeh, 188
 - charting, 188
 - csv, 66–67
 - Dash, 215, 216
 - datetime, 46–47
 - JSON, 168, 169
 - Kaleido, 197

- libraries (*continued*)
 - Matplotlib, 188
 - NumPy, 110, 116, 237
 - openai, xxx
 - os module, 178–179
 - Psychopg 2, 157, 158, 160
 - PyDataset, 190, 223, 228
 - Python Standard Library, 5
 - pytz, 48
 - Requests, 169–171
 - Seaborn, 188
 - sys module, 72
 - timezone, 49–50
 - unittest, xxiii, 268, 269–271, 272
 - xlwings, 67
- LIKE operator, 149
- LIMIT clause, 147
- line charts
 - basic, 192
 - multi-trace, 193–194
 - simple, 191–192
- linecolor subproperty, 201, 203
- linewidth subproperty, 203
- list comprehension, 56–57
- list methods, 40
- lists, 36–37, 55
- loc[], 119
- localize() method, 48
- logic, functionalizing, 65–66
- logical operators, 35–36.
 - See also* operators
- Looker, 213
- loops, 54–59
 - control keywords for, 57–59
 - for, 54–55
 - while, 59
- low-code tools, 213
- lower() method, 40
- low-level languages, 26
- ls command, 6, 9
- l subproperty, 204

M

- machine code, 26
- macOS. *See also* Windows
 - directory navigation on, 7–8
 - environment variable
 - commands for, 11

- file manipulation on, 9
- file paths, 6*t*
- filesystem navigation commands, 6
- installing PostgreSQL on, 139
- modifying environment variables
 - on, 11–12
- troubleshooting, 17
- main block, 71–72
- main branches, 87
- major versions, 15
- make directory (mkdir) command, 6, 8
- makedirs() function, 66
- manual execution time, xxviii–xxx
- margin property, 204
- MariaDB, 137
- markdown cells, 99–100, 103–104
- master branches, 87
- Matplotlib, 188
- max() function, 139
- MAX function, 155
- mean() function, 128, 129
- merging, 88, 127
- methods, 39–41, 236, 238–239.
 - See also specific methods*
- Microsoft SharePoint, 81
- micro versions, 15
- MID function, 45
- MIN function, 155
- minor versions, 15
- mixed scopes in coding, 276–277
- mkdir (make directory) command, 6, 8
- modifier functions, 64
- modularization, xxviii, 279–282
- modules, importing scripts as, 70–71
- MongoDB, 137
- mul() function, 122–123
- multiplication operator (*), 154
- multi-trace bar chart, 196–198, 197
- multi-trace line chart, 193–194
- mutable data structures, 64
- MySQL, 137

N

- NA marker, 124–125
- NaN marker, 125
- nano (file editor), 12
- negative indexing, 37
- nested conditionals, 44–45, 275

- no-code tools, 213
- None data type, 33
- non-English writing systems, 33
- None return value, 63
- nonrelational databases, 137
- NoSQL databases, 137
- not equal to (`!=`) operator, 148
- `NotImplementedError`, 250
- `notna()` function, 125
- NULL data type, 144
- NUMERIC data type, 144
- NumPy, 110

O

- O365, xxx
- OAuth, 179–180
- object-oriented programming (OOP)
 - encapsulation, 244
 - extendability, 244–245
 - vs. functional programming, 239
 - getters, 246–247
 - inheritance, 249–251
 - lazy loading, 248–249
 - overview, 235–236
 - properties, 245–246
 - reusability, 243–244
 - setters, 247–248
- objects, 27–28, 236
- OneDrive, 78
- ON keyword, 150–151
- OpenAI, xxx
- openai library, xxx
- Open Authorization (OAuth), 179–180
- openpyxl, 118
- OpenWeather API, 177–178, 181
- operators
 - arithmetic, 34–35
 - comparison, 35
 - defined, 33
 - logical, 35–36
- optimization, 282
- Oracle, 137*t*
- ORDER BY clause, 148
- orientation subproperty, 204
- os module, 66–73
- `Output()` parameter, 227

P

- pad subproperty, 204
- pagination, 174
- pandas library, 109–110
 - aggregation operations in, 128
 - charting functionality, 188
 - data analysis with, 110–111
 - DataFrames data types, 110–111, 115–116
 - installing, 111–112
 - Jupyter Notebooks and, 111
 - missing data operations in, 125
 - moving between Excel and, 116–118
 - pd as alias for, 112
 - reshaping data with, 129–131
 - Series data types, 110–111, 115–116
 - sorting datasets in, 121
- `paper_bgcolor` property, 204
- `params` argument, 171, 180–181
- parent class, 251–253
- PATH environment variable, 15, 17
- `path.exists()` function, 66
- paths, 6
- `pd.concat()` function, 126
- `pd.DataFrame()` function, 113
- `pd.merge()` function, 127
- `pd.read_clipboard()` function, 117
- `pd.read_csv()` function, 114–115, 118
- `pd.read_excel()` function, 115, 118
- `pd.read_json()` function, 115
- percent sign (%), 149
- pg8000, 158
- pip (pip installs packages), 18–19
- pip command, 18, 190, 215
- pivoting, 130–131
- `pivot()` method, 130–131
- `plot_bgcolor` property, 204
- Plotly, 188
 - charting multidimensional data with, 198–200, 200
 - creating simple charts with, 189–198
 - bar chart, 194–196, 196
 - datasets for, 189–191
 - line chart, 191–192, 192

Plotly (*continued*)

creating simple charts (*continued*)

multi-trace bar chart,

196–198, 197

multi-trace line chart, 193–194

in Dash, 223–227

importing, 189

installing, 189

integrating Dash and, 223–230

interactivity features, 209–210

layout properties, 202–205

saving charts as images, 197

styling charts in, 200–209

layout properties, 201–205

themes, 205–209

using in Jupyter Notebooks, 189

polymorphism, 35

pop() method, 41

positional arguments, 61

PostgreSQL, 137

installing, 138–140

managing users and databases,
140–141

using in VS Code, 141–142

POST request, 165–166

Power BI, 213, 214

PowerShell, 5

premature optimization, 282

primary keys, 143

print() function, 39

pro-code tools, 213

profile, 12

programming languages, 26

prompt, 5

psql, 139–140

Psycopg 2, 157

closing database connection, 160

installing, 158

running query from Python, 159

Public Health England, xxi–xxii

pwd command, 6

pydataset.data() function, 190

.py file, 106

pyodbc, 158

Python

alternative to, xxxii–xxxiii

automating database tasks with
SQL and, 157–160

charts, 186–188

code editors, 21–23

common functions in, 39

dictionary operations in, 41

installing, 3–4

directory's contents, 16

macOS troubleshooting, 17

running code, 17–18

running installer, 14–16

verifying installation, 16–17

Windows troubleshooting, 17

string manipulations in, 40

syntax, 28–30

version numbering, 15

Python in Excel, xxxiii–xxxiv

Python interpreter, 5

Python Package Index (PyPI), 19

Python Standard Library, 5

Python *vs.* Excel

automation, xxviii–xxx

interoperability, xxx–xxxii

modularity, xxviii

overview, xxv–xxvi

security, xxx–xxxiv

speed, xxvi–xxvii

py-trello, xxx

pytz library, 48

Q

qualifiers, 146

R

Random User Generator API, 165, 169,
170, 174, 180

random_users.py file, 182

range() function, 39, 55–56

ranges, creating for loop using, 55–56

range subproperty, 203

raw text cells, 100–101

read_excel() function, 118

README.txt file, 80

records, 142

refactoring, 282

#REF! error, xxiii

Reinhart, Carmen, xxi

relational database management system
(RDBMS), 137–138

relational databases, 137

- remote database, connecting to, 142
- remove() method, 40
- repetition in coding, 278–279
- replace() function, 40, 125
- repositories, 75, 77
 - centralizing work in single folder, 78–80
 - cloning, 83–84
 - creating from scratch, 83
 - local, 80–81
 - maintaining .gitignore file, 81–82
 - remote, 80–81
 - setting up, 81–84
 - syncing, 86–87
- requests
 - GET, 165
 - POST, 165–166
 - streamlining with reusable functions, 180–183
 - using API keys, 178–179
- requests.get(), 241
- reserved words, 57
- reset_index() function, 128, 129
- REST API, 162
 - internet communication protocols and, 163–164
 - Requests library, 169–171
 - request types, 165–166
 - response types, 166–168
- returns, 62–63
- reusability, 243–244
- reusable functions, 180–183
- RIGHT function, 45
- right joins, 151–153
- rmdir (remove directory) command, 8–9
- Rogoff, Kenneth, xxi
- roles, 140
- root directory, 6
- rows, adding, 145
- r subproperty, 204

S

- sandbox environments, 138
- scatter plots, 198–200, 200
- scope, 10, 62
- scripts, 51–54
 - building and running, 66–73

- code to put in, 105
- importing as modules, 70–71
- main block, 70–71
- parts of, 53–54
- running on command line, 69
- when to use, 52–53
- Scripts subdirectory, 19
- SE (Standard Edition), 146
- Seaborn, 188
- SELECT commands, 136
- SELECT DISTINCT, 148
- SELECT keyword, 146–147
- selectors, CSS, 221, 223
- semicolon (;), 141
- sequence, 55
- Series data types, 110–111, 115–116
- set_index() method, 113–114
- setters, 247–248
- shell, 5, 12
- SHELL environment variable, 12
- show_docs=True parameter, 190
- showgrid subproperty, 203
- showLegend property, 204
- showline subproperty, 203
- showticklabels subproperty, 203
- Simple Object Access Protocol (SOAP) APIs, 163
- simple-salesforce, xxx
- size property, 203
- slackclient, xxx
- slices, 119
- sort_index() function, 121
- sort_values() function, 121
- special variables, 261
- split() method, 40, 124
- spreadsheets
 - drawbacks of, xxi–xxii
 - errors in, xxii–xxiii
- SQL (Structured Query Language)
 - aggregation functions, 155
 - automating database tasks with Python and, 157–160
 - comparison operators for WHERE clauses, 149*t*
 - data types, 143–144, 144
 - overview, 133

SQL (*continued*)

queries

- aggregations, 154–155
- column transformations, 154
- complex, 156–157
- filters, 148–149
- groupings, 155–156
- joins, 148–149
- selections, 146–147

SQLAlchemy, 158

SQLite, 137

SQL Server, 137

SQL Shell (psql), 139

staging area, 84–85

Standard Edition (SE), 146

statements, 29–30

Step Into debugger command, 264–265

Step Out debugger command, 263–264

str accessor property, 124

strftime() method, 48

str() function, 39

string methods, 40

string operations, 123–124

strings, 32–33, 56

strip() method, 40

strptime() method, 48

Structured Query Language. *See* SQL

styles.css file, 222–223

sub() function, 122–123

sum() function, 39

SUM function, 155

SUMIFS, 129

super().__init__(), 252, 254

superclass, 249

superuser, 139

symbols, 33

syntax, Python

- assignments, 29–30
- case sensitivity, 28
- comments, 29
- indentation, 28–29
- statements, 29–30

system arguments, 72–73

system environment variables, 10

system settings, customizing and
storing, 10–13

T

Tableau, 213

tables. *See also* databases

- adding rows to, 145
- altering structure of, 144–145
- creating, 142–143
- deleting, 145
- deleting date from, 146
- primary keys, 143
- updating records in, 145–146

tags, HTML, 217, 219

TCP (Transmission Control
Protocol), 164

template attribute, 206

temporal data, 116

Terminal, 5, 69

testability, xxii–xxiii

text highlighting, 20

themes

- built-in, 205–206
- custom, 206–209

themes.py file, 206–207

tickangle subproperty, 203

tickcolor subproperty, 203

tickfont subproperty, 203

tickformat subproperty, 201

ticklen subproperty, 203

tickwidth subproperty, 203

tilde (~), 7

time commitment, 46

timedelta64[ns] type, 116

timedelta object, 47

time object, 46

Timeout error, 174

time zones, 49–50

title property, 202–203

title subproperty, 201, 203, 204

to_clipboard() function, 117

to_excel() function, 118

touch command, 9

Transmission Control Protocol
(TCP), 164

Trello, xxx

triple quotes ("""), 29

troubleshooting, 17

try...except blocks,
172–173

t subproperty, 204

- tuples, 37–38
- TypeError, 249, 263
- type() function, 39
- type subproperty, 203

U

- underscore (`_`), 149, 245
- Unicode, 33
- unique argument, 64
- unittest module, 269–271
- UPDATE command, 145
- update_layout() method, 195, 201–202, 206, 208, 209
- update() method, 41
- upper() method, 40
- user account, 6
- user-defined functions, 59–64
- user environment variables, 10
- user level, 6
- users
 - managing, 140–141
 - roles, 140
 - superuser privileges, 139

V

- VALUE, 11
- ValueError, 172, 247
- values, 10
 - accessing, 118
 - assigning to variables, 35
 - comparing, 35
 - labels, 118
 - position, 118
 - slicing, 119
 - updating, 120
- values() method, 41
- VARCHAR data type, 144
- variable names, 277–278
- variables, 27–28, 236
 - assigning values to, 35
 - class, 261
 - environment, 10, 177–178
 - function, 261
 - global, 54
 - names of, 277–278
 - special, 261
- VBA (Visual Basic for Application),
xxxii–xxxiii

- version control
 - charting with Excel *vs.* Python, 187
 - Jupyter Notebook, 107
 - overview, xxiv
 - reasons for using, 76
- version numbering, 15
- vertical concatenation, 126
- virtual private network (VPN), 142
- Visual Basic for Application (VBA),
xxxii–xxxiii
- VLOOKUP function, 127, 149–150
- VPN (virtual private network), 142
- VS Code, 21–23
 - debugger, 258–267
 - breakpoints, 258–259
 - Debug Console, 262
 - Debug Panel, 260–262
 - installing, 21–23
 - integrating with Git, 92
 - testing examples in, 30–31
 - using PostgreSQL in, 141–142

W

- WebSocket APIs, 163
- WHERE clause, 148, 156
- while loop, 59, 181. *See also* loops
- wildcard characters, 149
- Windows. *See also* macOS
 - directory navigation on, 8–9
 - environment variable commands
 - for, 11
 - file manipulation on, 10
 - file paths, 6*t*
 - filesystem navigation
 - commands, 6
 - installing PostgreSQL on, 139–140
 - modifying environment variables
 - on, 12–13
 - troubleshooting, 17
- withdraw() function, 240

X

- xaxis_properties, 201–202, 203
- xaxis_title, 201
- xbbg, xxxi
- xlwings library, xxix, 67
- x subproperty, 204

Y

yaxis_linecolor, 202
yaxis_properties, 201–202, 203
y subproperty, 204

Z

Zendesk API, xxxi
Zenpy, xxxi

zerolinecolor subproperty, 203
zeroline subproperty, 203
zerolinewidth subproperty, 203
Zoom, xxx
zoomus, xxx
Z shell (zsh), 5, 12

The fonts used in *Python for Excel Users* are New Baskerville, Futura, The Sans Mono Condensed, and Dogma. The book was typeset with L^AT_EX 2_ε package nostarch by Boris Veytsman with many additions by Alex Freed and other members of the *No Starch Press* team (2023/07/19 v2.4 *Typesetting books for No Starch Press*).

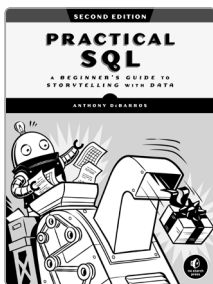
RESOURCES

Visit <https://nostarch.com/python-excel> for errata and more information.

More no-nonsense books from

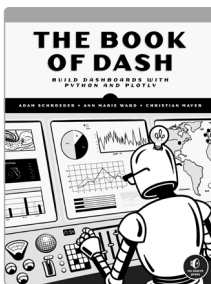


NO STARCH PRESS



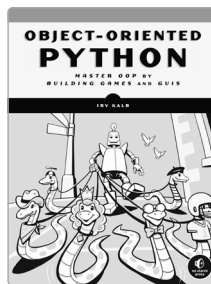
PRACTICAL SQL, 2ND EDITION A Beginner's Guide to Storytelling with Data

BY ANTHONY DEBARROS
464 pp., \$39.99
ISBN 978-1-7185-0106-5



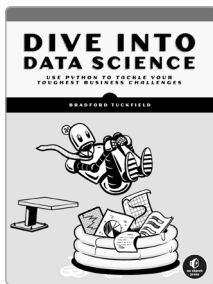
THE BOOK OF DASH Build Dashboards with Python and Plotly

BY ADAM SCHROEDER, ANN MARIE WARD, and CHRISTIAN MAYER
224 pp., \$34.99
ISBN 978-1-7185-0222-2



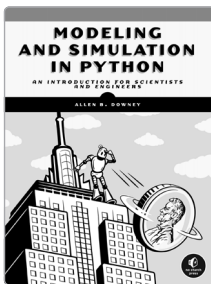
OBJECT-ORIENTED PYTHON Master OOP by Building Games and GUIs

BY IRV KALB
416 pp., \$44.99
ISBN 978-1-7185-0206-2



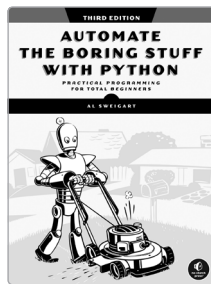
DIVE INTO DATA SCIENCE Use Python to Tackle Your Toughest Business Challenges

BY BRADFORD TUCKFIELD
288 pp., \$39.99
ISBN 978-1-7185-0288-8



MODELING AND SIMULATION IN PYTHON An Introduction for Scientists and Engineers

BY ALLEN B. DOWNEY
280 pp., \$39.99
ISBN 978-1-7185-0216-1



AUTOMATE THE BORING STUFF WITH PYTHON, 3RD EDITION Practical Programming for Total Beginners

BY AL SWEIGART
672 pp., \$59.99
ISBN 978-1-7185-0340-3

PHONE:

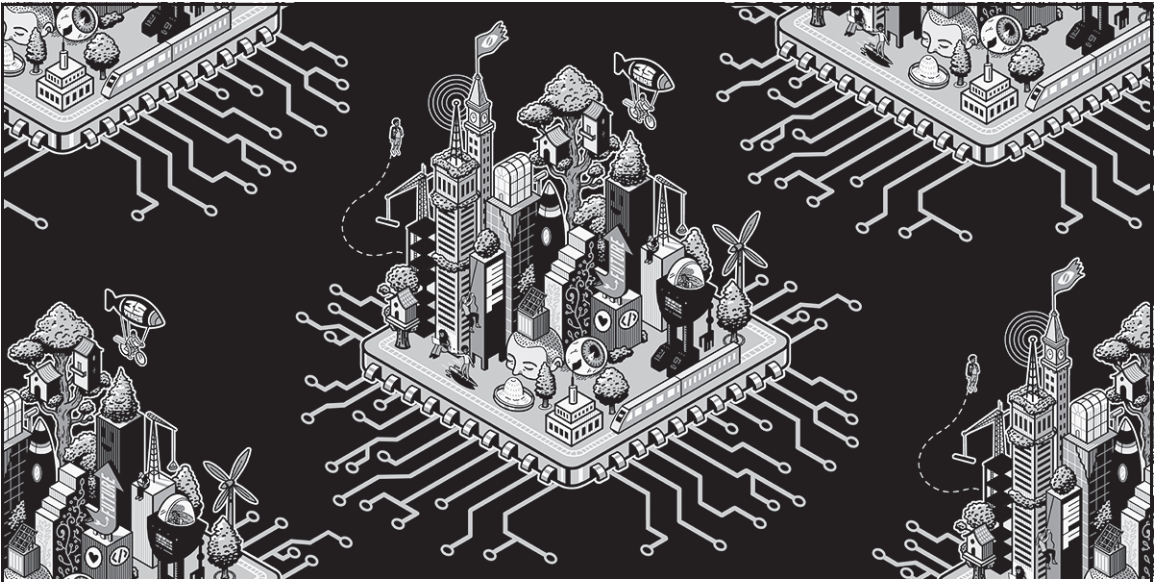
800.420.7240 OR
415.863.9900

EMAIL:

SALES@NOSTARCH.COM

WEB:

WWW.NOSTARCH.COM

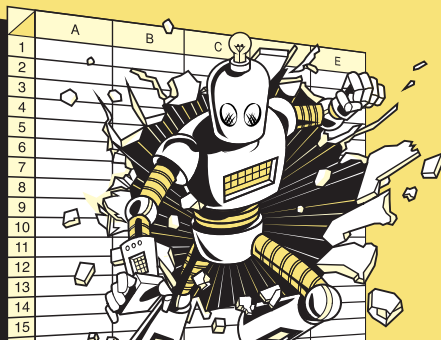


Never before has the world relied so heavily on the internet to stay connected and informed. That makes the Electronic Frontier Foundation's mission—to ensure that technology supports freedom, justice, and innovation for all people—more urgent than ever.

For over 35 years, EFF has fought for your rights through activism, in the courts, and by developing software because we believe in a better future—one where your device is truly yours, you can speak without being surveilled, and technology helps you connect with the people you care about. With your help, we can realize that vision for a brighter world together.



LEARN MORE AND JOIN EFF AT EFF.ORG/NOSTARCH



When Excel isn't enough, it's time to learn Python.

If you're comfortable in Excel, but you've hit a wall—slow files, broken formulas, hours spent on repetitive tasks—this book offers a way forward. It shows you how to take the work you already do in spreadsheets and make it faster, smarter, and more powerful with Python.

You'll start by setting up your environment and getting comfortable with Python through short, Excel-inspired exercises. From there, you'll gradually move into writing scripts that automate manual work, structure your data, and generate consistent results—no prior programming knowledge required.

You'll use your preexisting Excel skills to learn how to:

- Translate spreadsheet logic into Python code
- Use pandas to clean, reshape, and filter data
- Automate reports you'd normally build by hand
- Read and write Excel files directly from Python
- Connect to databases and APIs
- Create professional visualizations with Plotly and Dash
- Organize code into sharable modules and write simple tests

Throughout the book, you'll find practical examples that show why and how to move your work out of spreadsheets and into scripts, and how to resolve issues along the way.

Author Tracy Stephens has extensive practical experience with both Excel and Python. Her approach is grounded in real workflows, and she introduces each concept through tasks you've likely handled in Excel.

This book won't ask you to replace everything you do in spreadsheets, but it will help you use Python to work faster, more reliably, and with greater flexibility than you ever could with Excel.

ABOUT THE AUTHOR

Tracy Stephens is a quantitative developer based in New York City. Her experience includes building systematic trading strategies at some of the world's top financial institutions. A long-time Python evangelist, she focuses on designing quantitative infrastructure that's flexible, explainable, and efficient—when she can successfully keep her one-eyed tuxedo cat off her keyboard.



THE FINEST IN GEEK ENTERTAINMENT™

nostarch.com